1

INTRODUÇÃO

Um computador digital é uma máquina capaz de nos solucionar problemas através da execução de instruções que lhe são fornecidas. Denomina-se **programa** uma seqüência de instruções que descreve como executar uma determinada tarefa. Os circuitos eletrônicos de cada computador podem reconhecer e executar diretamente um conjunto limitado de instruções simples para as quais todos os programas devem ser convertidos antes que eles possam ser executados. Estas instruções básicas raramente são mais complicadas do que:

Some dois números.
Verifique se um número é zero.
Mova um dado de uma parte da memória do computador para outra.

Juntas, as instruções primitivas do computador formam uma linguagem que torna possível às pessoas se comunicarem com o computador. Tal linguagem é denominada **linguagem de máquina.** Ao se projetar um novo computador deve-se decidir que instruções devem estar incluídas nesta linguagem de máquina. Geralmente, tenta-se fazer as instruções primitivas tão simples quanto possível, consistentes com o uso pretendido e necessidades de desempenho, a fim de reduzir a complexidade e o custo da eletrônica empregada. Como a maioria das linguagens de máquina são muito simples, é difícil e tedioso utilizá-las.

Este problema pode ser atacado de duas maneiras principais, ambas envolvendo o projeto de um novo conjunto de instruções de uso mais conveniente para as pessoas do que o conjunto de instruções embutidas da máquina. Juntas, estas novas instruções formam uma linguagem que chamaremos de L2, exatamente como as instruções embutidas de máquima formam uma linguagem, que chamaremos de L1. Os dois conceitos diferem na maneira com que os programas escritos em L2 são executados pelo computador, que, afinal de contas, pode apenas executar programas escritos em sua linguagem de máquina, L1.

Um método de execução de um programa escrito em L2 consiste, em primeiro lugar, em substituir cada instrução nele por uma seqüência equivalente de instruções em L1. O programa resultante é composto inteiramente de instruções L1. O computador então executa o novo programa em L1 em vez do programa em L2. Denomina-se esta técnica **tradução**.

A outra técnica consiste em escrever um programa em L1 que receba os programas escritos em L2 como dados de entrada e efetue a execução examinando uma instrução de cada vez e executando a seqüência equivalente de instruções L1 diretamente. Esta técnica não requer a geração de um novo programa em L1. Denomina-se **interpretação**, e o programa que a executa denomina-se **interpretador**.

A tradução e a interpretação são similares. Em ambos os métodos, as instruções em L2 são executadas pelas seqüências equivalentes de instruções em L1. A diferença é que, na tradução, o programa inteiro em L2 é primeiramente convertido para um programa em L1, o programa L2 é abandonado, e então o novo programa em L1 é executado. Na interpretação, depois de cada instrução L2 ser examinada e decodificada, ela é executada imediatamente. Nenhum programa traduzido é gerado. Ambos os métodos são amplamente utilizados.

Em vez de pensar em termos de tradução e interpretação, é muitas vezes mais adequado imaginar a existência de um computador hipotético ou **máquina virtual** cuja linguagem de máquina seja L2. Se tal máquina pudesse ser construída a um custo suficientemente baixo, não haveria absolutamente nenhuma necessidade de se ter L1 ou uma máquina que executasse programas em L1. As pessoas poderiam simplesmente escrever seus programas em L2 e o computador os executaria diretamente. Até mesmo se a máquina virtual cuja linguagem é L2 fosse muito cara para ser construída com circuitos eletrônicos, poder-se-ia ainda escrever programas para ela. Estes programas podem ser interpretados ou traduzidos por um programa escrito em L1, linguagem esta que possa ser diretamente executada por um computador existente. Em outras palavras, podem-se escrever programas para máquinas virtuais como se elas realmente existissem.

Para tornar práticas a tradução e a interpretação, as linguagens L1 e L2 não devem ser "muito" diferentes. Esta restrição significa muitas vezes que L2, embora melhor que L1, estará ainda longe do ideal para a maioria das aplicações. Talvez isso seja desencorajante em vista do objetivo original de criar L2, ou seja, livrar o programador da carga de ter que expressar os algoritmos em uma linguagem mais adequada às máquinas do que às pessoas. Entretanto, a situação está longe de desesperante.

A solução óbvia é criar um outro conjunto de instruções que seja mais dirigido às pessoas e menos dirigido à máquina do que aquele de L2. Este terceiro conjunto também constitui uma linguagem, que chamaremos L3. Podem-se escrever programas em L3 como se uma máquina virtual cuja liguagem de máquina fosse L3 realmente existisse. Tais programas podem tanto ser traduzidos para L2 quanto executados por um interpretador escritos em L2.

A invenção de toda uma série de linguagens, cada uma delas mais conveniente do que suas predecessoras, pode continuar indefinidamente até que uma adequada seja finalmente conseguida. Cada linguagem utiliza sua predecessora como base, assim podemos visualizar um computador que utiliza esta técnica como uma série de **camadas** ou **níveis**, um sobre o outro, como está

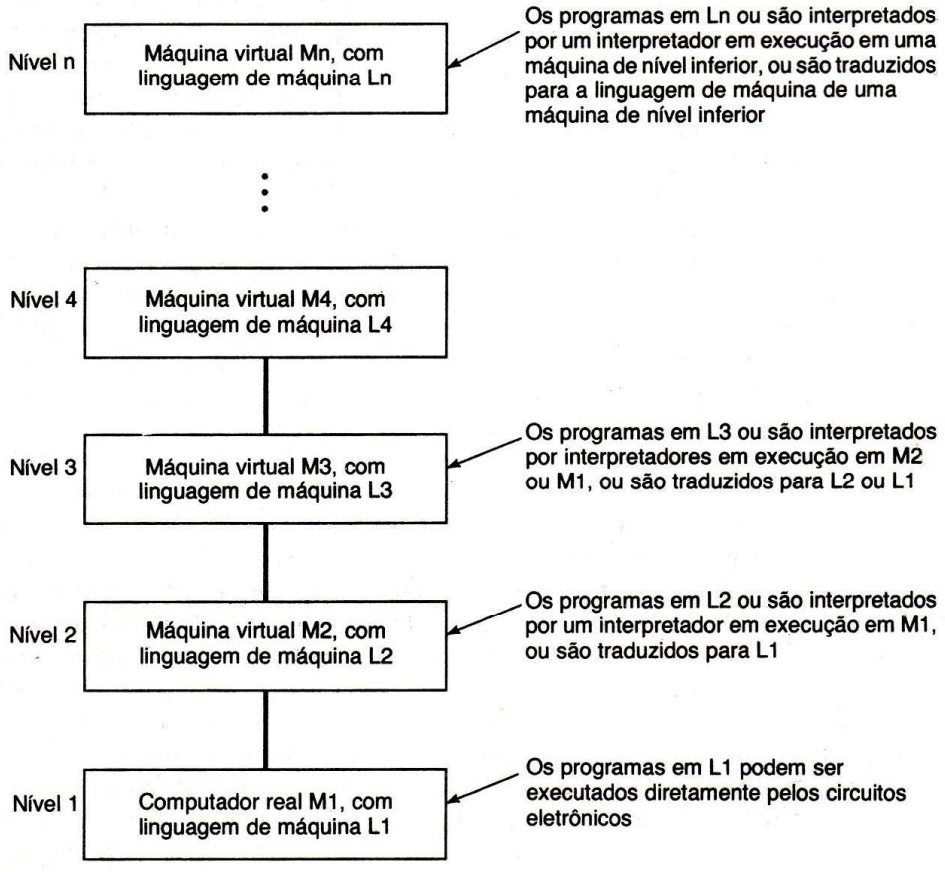


Fig. 1.1 Uma máquina multinível.

mostrado na Fig. 1.1. A linguagem ou nível mais baixo é a(o) mais simples e a linguagem ou nível mais alto é a(o) mais sofisticada(o).

1.1 LINGUAGENS, NÍVEIS E MÁQUINAS VIRTUAIS

Há uma relação importante entre uma linguagem e uma máquina virtual. Cada máquina tem a sua linguagem de máquina, que consiste em todas as instruções que a máquina pode executar. De fato, uma máquina define uma linguagem. Do mesmo modo, uma linguagem define uma máquina — ou seja, a máquina que pode executar todos os programas escritos nesta linguagem. Naturalmente, a máquina definida por uma determinada linguagem pode ser enormemente complicada e cara de construir diretamente a partir de circuitos eletrônicos, mas de qualquer modo podemos imaginá-la. Uma máquina com C, Pascal ou Cobol como sua linguagem de máquina seria complicadíssima, mas é certamente imaginável e, talvez dentro de poucos anos, tal máquina seja considerada de construção trivial.

Um computador com *n* níveis pode ser considerado como *n* diferentes máquinas virtuais, cada uma delas com uma diferente linguagem de máquina. Podemos utilizar os termos "nível" e "máquina virtual" indistintamente. Apenas os programas escritos em linguagem L1 podem ser executados diretamente pelos circuitos eletrônicos, sem a necessidade de intervenção da tradução ou interpretação. Os programas escritos em L2, L3,... *Ln*, devem ser interpretados por um interpretador em execução em um nível inferior ou ser traduzidos para uma outra linguagem correspondente no nível inferior.

Uma pessoa cujo trabalho fosse escrever programas para a máquina virtual de nível *n* não precisaria estar ciente dos interpretadores e tradutores subjacentes. A estrutura da máquina assegura que estes programas serão de algum modo executados. É de pouco interesse se eles são executados passo a passo por um interpretador, que por sua vez é também executado por um outro interpretador, ou se eles são executados diretamente pelos circuitos eletrônicos. O mesmo resultado surge em ambos os casos: os programas são executados.

A maioria dos programadores que utilizam uma máquina de nível *n* está apenas interessada no nível do topo, que lembra o mínimo possível a linguagem de máquina situada lá embaixo. Entretanto, os interessados em compreender como um computador realmente funciona devem estudar todos os níveis. As pessoas interessadas em projetar novos computadores ou novos níveis (isto é, novas máquinas virtuais) devem também estar familiarizadas com outros níveis além do nível do topo. Os conceitos e técnicas de construção de máquinas como uma série de níveis e os detalhes de alguns níveis importantes constituem por si só o assunto principal deste livro. O título *Organização Estrutural de Computadores* vem do fato de se visualizar o computador como um conjunto hierárquico de níveis que provê uma boa estrutura para compreender como os computadores estão organizados. Além disso, projetar computadores como uma série de níveis ajuda a assegurar um produto final bem estruturado.

1.2 MÁQUINAS MULTINÍVEL CONTEMPORÂNEAS

A maioria dos computadores modernos possui dois ou mais níveis. As máquinas de seis níveis são muito comuns, como está mostrado na Fig. 1.2. O nível 0, lá embaixo, é o *hardware* verdadeiro da máquina. Seus circuitos executam os programas em linguagem de máquina de nível 1. Para completar, deve-se mencionar ainda a existência de um outro nível abaixo do nível 0. Este nível, que não está mostrado na Fig. 1.2 porque se situa no campo da engenharia elétrica (e por isto está fora do escopo deste livro), é denominado **nível de dispositivos**. A este nível, o projetista enxerga transistores individuais, os quais são os primitivos de mais baixo nível para os projetistas de computadores. (Naturalmente, pode-se também indagar como funcionam os transistores por dentro, mas isto já pertence à física de estado sólido.)

No nível mais baixo que estudaremos, o **nível da lógica digital**, os objetos interessantes são denominados **portas**. Embora construídas a partir de componentes analógicos, como os transistores, as portas podem ser modeladas com precisão como dispositivos digitais. Cada porta possui uma ou mais entradas digitais (sinais representando 0 ou 1), e fornece como saída funções simples destas entradas, tais como AND ou OR. Cada porta é constituída no máximo por um punhado de transistores. Nós examinaremos o nível de lógica digital em detalhes no capítulo 3. Embora

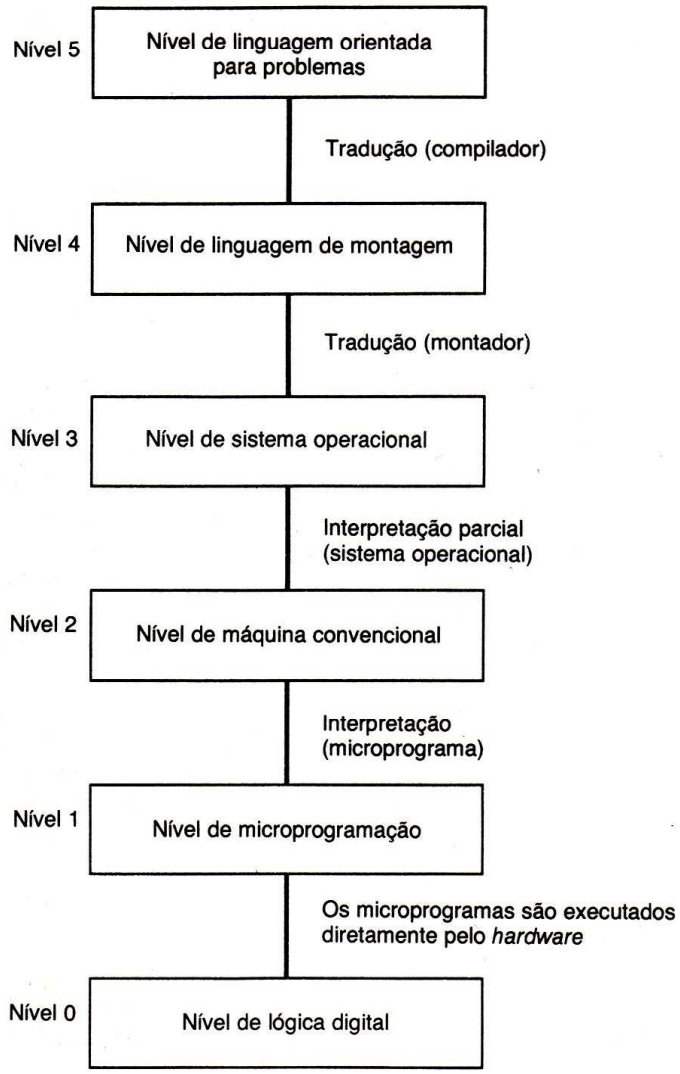


Fig. 1.2 Os seis níveis presentes na maioria dos computadores modernos. O método como cada nível é implementado está indicado abaixo dele, juntamente com o nome do programa que o implementa entre parênteses.

o conhecimento do nível de dispositivos seja um tanto especializado, com o advento dos microprocessadores e microcomputadores, mais e mais pessoas estão entrando em contato com o nível de lógica digital. Por este motivo, nós o incluímos em nosso modelo e dedicamos um capítulo inteiro do livro a este nível.

O nível superior seguinte é o nível 1, que é o verdadeiro nível de linguagem de máquina. Ao contrário do nível 0, onde não existe o conceito de programa como uma seqüência de instruções a serem executadas, em nível 1 há definitivamente um programa, denominado **microprograma**, cuja função é interpretar as instruções de nível 2. Chamamos o nível 1 de **nível de microprogramação**. Embora seja verdadeiro que não existem dois computadores com níveis de microprogramação idênticos, existem semelhanças suficientes que nos permitem extrair os aspectos essenciais do nível e discuti-lo como se ele fosse bem definido. Por exemplo, poucas são as máquinas que têm mais de 20 instruções neste nível, e a maior parte destas instruções envolve a movimentação de dados de uma parte da máquina para outra, ou a realização de alguns testes simples.

Cada máquina de nível 1 tem um ou mais microprogramas que são executados nela. Cada microprograma define implicitamente uma linguagem de nível 2 (e uma máquina virtual, cuja linguagem de máquina é essa linguagem). Estas máquinas de nível 2 têm muito em comum. Até mesmo máquinas de nível 2 de fabricantes diferentes têm mais semelhanças do que diferenças.

Neste livro, denominaremos este nível de **nível de máquina convencional**, por falta de um nome de aceitação geral.

Todo fabricante de computadores publica um manual para cada tipo de computador que ele comercializa, intitulado "Manual de Referência da Linguagem de Máquina" ou "Princípios de Operação do Computador Western Wombat Modelo 100X" ou algo semelhante. Estes manuais tratam, na realidade, da máquina virtual de nível 2, e não da máquina real de nível 1. Quando eles descrevem o conjunto de instruções de máquina, estão, de fato, descrevendo as instruções interpretadas pelo microprograma, e não as próprias instruções de *hardware*. Se o fabricante de computadores escrever para uma de suas máquinas dois interpretadores que interpretem duas máquinas diferentes de nível 2, ele precisará fornecer dois manuais de referência de "linguagem de máquina", um para cada interpretador.

Deve-se mencionar que alguns computadores não têm um nível de microprogramação. Nestas máquinas, as instruções do nível de máquina convencional são executadas diretamente pelos circuitos eletrônicos (nível 0), sem qualquer interpretador intermediário de nível 1. Como resultado, o nível 1, e não o nível 2, é o nível de máquina convencional. De qualquer modo, continuaremos a chamar o nível de máquina convencional de "nível 2", apesar destas exceções.

O terceiro nível é geralmente um nível híbrido. A maior parte das instruções desta linguagem está também na linguagem de nível 2. (Não há motivo para uma instrução que aparece em um nível não poder estar presente também em outros níveis.) Além disso, há um conjunto de novas instruções, uma organização diferente de memória, capacidade de execução de dois ou mais programas em paralelo, e várias outras características. Existem diferenças maiores entre máquinas de nível 3 do que entre máquinas de nível 1 ou de nível 2.

As novas facilidades adicionadas ao nível 3 são efetuadas por um interpretador em execução no nível 2, que historicamente tem sido denominado **sistema operacional**. Aquelas instruções de nível 3 idênticas às do nível 2 são executadas diretamente pelo microprograma, e não pelo sistema operacional. Em outras palavras, algumas das instruções de nível 3 são interpretadas pelo sistema operacional e outras são interpretadas diretamente pelo microprograma. Isto é o que se quer dizer por "híbrido". Denominaremos este nível de **nível de sistema operacional**.

Há uma diferença fundamental entre os níveis 3 e 4. Os três níveis inferiores não foram projetados para o uso direto pelo programador médio comum. Eles são voltados principalmente para a execução dos interpretadores e tradutores necessários para suportar os níveis superiores. Estes interpretadores e tradutores são escritos por pessoas chamadas de **programadores de sistema**, que são especialistas em projetar e implementar novas máquinas virtuais. Os níveis 4 e superiores são dirigidos aos programadores de aplicação com problemas a serem solucionados.

Uma outra mudança que ocorre no nível 4 é o método pelo qual os níveis superiores são suportados. Os níveis 2 e 3 são sempre interpretados. Os níveis 4, 5 e superiores são geralmente, mas nem sempre, suportados por tradução.

Uma outra diferença ainda entre os níveis 1, 2 e 3, de um lado, e os níveis 4, 5 e superiores, de outro, é a natureza da linguagem provida. As linguagens de máquina de níveis 1, 2 e 3 são numéricas. Os programas escritos nestas linguagens consistem em longas séries de números, os quais são ótimos para as máquinas e péssimos para as pessoas. A partir do nível 4, as linguagens contêm palavras e abreviaturas significativas para as pessoas.

O nível 4, o nível da linguagem de montagem, é realmente uma forma simbólica para uma linguagem subjacente. Este nível provê um método para as pessoas escreverem programas para os níveis 1, 2 e 3 de uma maneira não tão desagradável quanto as próprias linguagens de máquina virtual. Os programas em linguagem de montagem são primeiramente traduzidos para a linguagem do nível 1, 2 ou 3, e então são interpretados pela máquina virtual ou real apropriada. O programa que executa a tradução é denominado **montador**. A linguagem de montagem já foi importante, mas está perdendo a importância com o tempo.

O nível 5 consiste em linguagens projetadas para serem utilizadas por programadores de aplicação com problemas a serem resolvidos. Tais linguagens são denominadas **linguagens de alto nível.** Existem literalmente centenas delas. Algumas das mais conhecidas são BASIC, C, COBOL, FORTRAN, LISP, Modula 2 e Pascal. Os programas escritos nestas linguagens são geralmente traduzidos para o nível 3 ou nível 4 por tradutores conhecidos como **compiladores**, embora ocasionalmente eles sejam interpretados.

Os níveis 6 e superiores consistem em coleções de programas projetados para criar máquinas

especialmente adequadas para certas aplicações. Eles contêm grandes quantidades de informação acerca da aplicação. Podem-se imaginar máquinas virtuais voltadas a aplicações em administração, educação, projeto de computadores etc. Estes níveis estão no terreno da pesquisa atual.

Em resumo, o ponto-chave a ser lembrado é que os computadores são projetados como uma série de níveis, cada um deles construído sobre seu predecessor. Cada nível representa uma abstração distinta, com a presença de diferentes objetos e operações. Ao projetar e analisar computadores deste modo, podemos temporariamente suprimir detalhes irrelevantes e assim reduzir um assunto complexo para algo mais fácil de ser compreendido.

O conjunto de tipos de dados, operações e características de cada nível é denominado sua **arquitetura**. A arquitetura trata daqueles aspectos que são visíveis ao usuário daquele nível. Os aspectos que um programador vê, tais como quanta memória está disponível, fazem parte da arquitetura. Aspectos de implementação, tais como qual o tipo de tecnologia da pastilha utilizada na implementação da memória, não fazem parte da arquitetura. O estudo de como projetar as partes de um computador que são visíveis aos programadores denomina-se **arquitetura de computadores**. Na prática, arquitetura de computadores e organização de computadores significam essencialmente a mesma coisa.

1.3 EVOLUÇÃO DAS MÁQUINAS MULTINÍVEL

Para darmos uma visão geral das máquinas multinível, examinaremos resumidamente seu desenvolvimento histórico. Os primeiros computadores digitais, na década de 40, tinham apenas dois níveis: o nível de máquina convencional, no qual toda a programação era feita, e o nível de lógica digital, que executava estes programas. Os circuitos do nível de lógica digital eram complicados, difíceis de entender e de construir, e não eram confiáveis.

Em 1951, M. V. Wilkes, na Inglaterra, deu a idéia de se projetar um computador de três níveis, a fim de simplificar drasticamente o *hardware*. Esta máquina teria um interpretador embutido imutável, cuja função era executar interpretativamente os programas em linguagem de máquina convencional. Como agora o *hardware* tinha apenas que executar microprogramas, que possuem um repertório limitado de instruções, em vez de programas na linguagem de máquina convencional, uma quantidade menor de circuitos eletrônicos seria necessária. Como os circuitos eletrônicos, naquele tempo, eram construídos a partir de válvulas, tal simplificação prometia a redução do número de válvulas e, conseqüentemente, aumentava a confiabilidade. Umas poucas destas máquinas de três níveis foram construídas na década de 50. Outras mais foram construídas na década de 60. Por volta de 1970, a idéia de se usar interpretação no nível de máquina convencional através de um microprograma, em lugar de execução direta pelos circuitos eletrônicos, generalizou-se.

Os montadores e compiladores foram desenvolvidos durante a década de 50 para facilitar o trabalho do programador. Naqueles dias, a maioria dos computadores era do tipo "*open shop*"*, o que significava que o programador tinha que operar a máquina pessoalmente. Próximo a cada máquina havia uma folha de reservas. O programador que quisesse executar um programa reservaria um intervalo de tempo, digamos, quarta-feira de 3 às 5 da madrugada (muitos programadores gostavam de trabalhar quando a sala do computador estivesse tranqüila). Quando chegasse a sua vez, o programador se dirigia à sala portando um *deck* de cartões numa mão e um lápis apontado na outra. Ao chegar na sala, ele gentilmente expulsava o programador anterior e assumia o controle da máquina.

Se quisesse executar um programa FORTRAN, seguiria os seguintes passos:

1. Ele iria até o armário onde a biblioteca de programas estivesse guardada, pegaria o imenso *deck* verde de cartões etiquetados compilador FORTRAN, colocá-lo-ia na leitora de cartões e apertaria o botão *start* (partida).
2. Ele colocaria seu programa FORTRAN na leitora de cartões e apertaria o botão *continue*. O programa era então lido.

*N. da R.: Lugar de trabalho onde não é necessário pertencer a um sindicato, ou seja, qualquer um pode trabalhar lá.

3. Quando o computador parasse, ele leria o programa FORTRAN uma segunda vez. Embora alguns compiladores necessitassem de apenas um passo de entrada, muitos necessitavam de dois ou mais. Para cada passo, um grande *deck* de cartões tinha que ser lido.
4. Finalmente, a tradução se aproximava do término. O programador ficava, muitas vezes, nervoso perto do final, pois se o compilador encontrasse um erro no programa, ele teria de corrigi-lo e começar tudo de novo. Se não encontrasse mais erros, o compilador perfurava em cartões o programa traduzido em linguagem de máquina.
5. O programador colocava então o programa em linguagem de máquina na leitora de cartões, junto com o *deck* de cartões da biblioteca de sub-rotinas, e lia ambos.
6. O programa começava a ser executado. Muitas vezes, ele não dava certo e parava inesperadamente no meio. Geralmente, o programador mexia nas chaves da console e observava suas luzes durante um certo tempo. Se tivesse sorte, ele descobria o erro, corrigia-o e voltava ao armário que continha o imenso compilador FORTRAN verde para começar tudo novamente. Se não obtivesse êxito, ele tiraria uma listagem do conteúdo da memória e levaria para casa para estudar.

Este procedimento, com pequenas variações, foi normal em muitos centros de computação durante anos. Isto forçava os programadores a aprenderem a operar a máquina e saber o que fazer quando ela quebrava, o que era freqüente. Amiúde a máquina ficava ociosa enquanto se estavam carregando cartões pela sala ou quebrando a cabeça, tentando descobrir por que os programas não estavam se comportando da maneira esperada.

Por volta de 1960, tentou-se reduzir o tempo perdido através da automatização do trabalho do operador. Um programa chamado **sistema operacional** era mantido no computador durante todo o tempo. O programador fornecia certos cartões de controle junto com o programa que era lido e executado pelo sistema operacional. A Fig. 1.3 mostra um exemplo de *deck* para um dos primeiros sistemas operacionais de amplo uso, o FMS (FORTRAN Monitor System), no IBM 709.

O sistema operacional lia o cartão *JOB e usava a informação nele contida para uso contábil. (O asterisco foi usado para identificar os cartões de controle, de modo que eles não fossem confundidos com os cartões de programa e dados). Mais tarde, ele lia o cartão *FORTRAN,

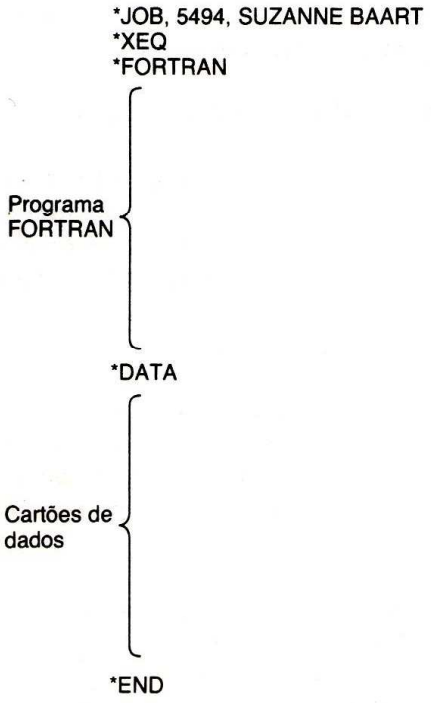


Fig. 1.3 Um exemplo de *job* para o sistema operacional FMS.

que era uma instrução para carregar o compilador FORTRAN a partir da fita magnética. O compilador então lia e compilava o programa escrito em FORTRAN. Quando o computador terminava, ele devolvia o controle ao sistema operacional, que então lia o cartão *DATA. Esta era uma instrução para executar o programa traduzido, utilizando como dados os cartões que se seguiam ao cartão *DATA.

Embora o sistema operacional fosse projetado para automatizar o trabalho do operador (daí o seu nome), foi também o primeiro passo no desenvolvimento de uma nova máquina virtual. O cartão *FORTRAN podia ser visto como uma instrução virtual "compilar programa". Do mesmo modo, o cartão *DATA podia ser considerado como uma instrução virtual "executar programa". Um nível com apenas duas instruções não era bem um nível, mas era um passo nessa direção.

Nos anos seguintes, os sistemas operacionais ficaram cada vez mais sofisticados. Novas instruções, facilidades e características foram adicionadas ao nível de máquina convencional, até que ele começou a adquirir a aparência de um novo nível. Algumas das instruções deste novo nível eram idênticas às instruções do nível de máquina convencional, mas outras, particularmente as instruções de entrada/saída, eram completamente diferentes. As novas instruções eram freqüentemente conhecidas como "macros de sistema operacional" ou "chamadas de supervisor" e estes termos são ainda utilizados.

Os sistemas operacionais se desenvolveram também de outras maneiras. Os primeiros liam *decks* de cartões e imprimiam a saída em uma impressora de linha. Esta organização ficou conhecida como **sistema em lote** (*batch*). Havia geralmente uma espera de várias horas entre o instante em que o programa era submetido e o momento em que os resultados ficavam prontos. Desenvolver *software* era difícil em tais circunstâncias.

No início da década de 60, pesquisadores do Dartmouth College, M.I.T. e de outros lugares desenvolveram sistemas operacionais que permitiam que (vários) programadores se comunicassem diretamente com o computador. Nestes sistemas, terminais remotos estavam conectados ao computador central por meio de linhas telefônicas. O programador podia digitar um programa e obter os resultados quase que imediatamente, em seu próprio escritório, ou em sua casa, ou em qualquer lugar onde se localizasse seu terminal. Estes sistemas eram, e ainda são, denominados **sistemas de tempo compartilhado** (*time-sharing systems*).

Nosso interesse em sistemas operacionais está nas partes que interpretam as instruções e nas características presentes no nível 3 e ausentes no nível 2, mais do que nos aspectos de compartilhamento do tempo. Embora não o enfatizemos, devemos lembrar que os sistemas operacionais fazem mais do que apenas interpretar programas de nível 3.

1.4 *HARDWARE, SOFTWARE* E MÁQUINAS MULTINÍVEL

Os programas escritos na linguagem de máquina de um computador (nível 1) podem ser executados diretamente pelos circuitos eletrônicos do computador (nível 0), sem quaisquer interpretadores ou tradutores intermediários. Estes circuitos eletrônicos, juntamente com a memória e dispositivos de entrada/saída, constituem o ***hardware*** do computador. O ***hardware*** é composto de objetos tangíveis — circuitos integrados, placas de circuito impresso, cabos, fontes de alimentação, memórias, leitoras de cartões, impressoras e terminais — em lugar de idéias abstratas, algoritmos ou instruções.

O ***software***, ao contrário, consiste em **algoritmos** (instruções detalhadas que dizem como fazer algo) e suas representações para o computador — ou seja, os programas. Os programas podem estar representados em cartões perfurados, fita magnética, filme fotográfico e outros meios, mas a essência do *software* está no conjunto de instruções que constitui os programas, não nos meios físicos sobre os quais eles estão gravados.

Uma forma intermediária entre o *hardware* e o *software* é o ***firmware***, que consiste no *software* embutido em dispositivos eletrônicos durante a fabricação. O *firmware* é utilizado quando se espera que os programas raramente ou nunca serão mudados, por exemplo, em brinquedos ou instrumentos. O *firmware* é também usado quando os programas não podem ser perdidos ao se acabar a alimentação (p. ex., quando a bateria da boneca se descarrega). Em muitos computadores o microprograma está em *firmware*.

Um tema central neste livro que sempre aparecerá é:

Hardware e software são logicamente equivalentes.

Qualquer operação efetuada pelo *software* pode também ser implementada diretamente em *hardware*, e qualquer instrução executada pelo *hardware* pode também ser simulada pelo *software*. A decisão de se colocar certas funções em *hardware* e outras em *software* baseia-se em fatores tais como: custo, velocidade, confiabilidade e freqüência esperada de alterações. Não há regras rígidas e diretas para se dizer que X deve ser implementado em *hardware* e Y deve ser programado explicitamente. Projetistas com objetivos diferentes podem, e muitas vezes o fazem, tomar diferentes decisões.

Nos primeiros computadores, a distinção entre *hardware* e *software* era clara. O *hardware* executava algumas instruções simples, tais como ADD (somar) e JUMP (saltar para), e tudo o mais era programado explicitamente. Se um programa precisasse multiplicar dois números, o programador tinha que escrever seu próprio procedimento de multiplicação ou pedir um emprestado à biblioteca. Com o passar do tempo, tornou-se claro aos projetistas de *hardware* que certas operações eram executadas com uma freqüência suficiente para justificar a construção de circuitos especiais para executá-las diretamente em *hardware* (para torná-las mais rápidas). Isto resultou em uma tendência de se levar as operações para os níveis inferiores. O que era previamente programado explicitamente no nível de máquina convencional era mais tarde encontrado abaixo dele, no *hardware*.

Com a chegada da microprogramação e de computadores multinível, a tendência oposta também ocorreu. Nos primeiros computadores, não havia dúvidas de que a instrução ADD era executada diretamente pelo *hardware*. Em um computador microprogramado, a instrução ADD do nível de máquina convencional era interpretada através de um microprograma sendo executado no nível situado mais ao fundo, e era executada como uma série de pequenos passos: buscar a instrução, descobrir seu tipo, localizar os dados a serem adicionados, buscar os dados da memória, executar a adição e armazenar o resultado. Este é um exemplo de função que moveu para cima, do nível de *hardware* para o de *microprograma*. Nós enfatizaremos mais uma vez que: não existem regras rígidas e diretas a respeito do que deve estar em *hardware* e do que deve estar em *software*.

Ao desenvolverem uma máquina multinível, os projetistas devem decidir o que colocar em cada nível. Esta é uma generalização do problema mencionado anteriormente, de se decidir o que colocar em *hardware* e o que colocar em *software*, o *hardware* sendo meramente de nível mais baixo. É interessante notar que algumas das características de certos computadores modernos são agora executadas pelo *hardware* ou pelo microprograma, mas que originalmente foram programadas explicitamente no nível de máquina convencional. Dentre elas estão incluídas:

1. Instruções para multiplicação e divisão de inteiros.
2. Instruções aritméticas de ponto-flutuante (veja Apêndice B).
3. Instruções aritméticas de dupla precisão (aritmética de números com duas vezes o número usual de algarismos significativos).
4. Instruções para chamar e retornar de procedimentos.
5. Instruções para acelerar laços (*looping*).
6. Instruções de contagem (somar 1 a uma variável).
7. Instruções para manipular cadeias de caracteres.
8. Aspectos de aceleração de computações envolvendo matrizes (indexação e endereçamento indireto).
9. Características para permitir que os programas fossem movidos na memória após terem começado a ser executados (facilidades de recolocação).
10. Relógios para temporizar programas.
11. Sistemas de interrupção que avisem o computador tão logo uma operação de entrada ou saída esteja completada.
12. A capacidade de parar um programa e iniciar outro utilizando poucas instruções (chaveamento de processos).

O objetivo da discussão é mostrar que a fronteira entre o *hardware* e o *software* é arbitrária e que varia constantemente. O *software* de hoje é o *hardware* de amanhã, e vice-versa. Além disso, os limites entre os vários níveis são fluidos. Do ponto de vista do programador, a maneira

como uma instrução está realmente implementada não é importante (exceto, talvez, com relação à velocidade). Alguém que estiver programado no nível de máquina convencional pode usar sua instrução de multiplicação como se esta fosse uma instrução de *hardware* sem se preocupar com isto, ou até mesmo estar ciente se ela é realmente uma instrução de *hardware* ou não. O *hardware* de alguém é o *software* de outrem.

O fato de o programador não estar ciente de como o nível que ele está utilizando é implementado leva ao conceito de projeto estruturado de máquinas. Um nível é muitas vezes chamado de máquina virtual porque o programador pensa nele como uma máquina física real, mesmo se ela não existir realmente. Estruturando uma máquina como uma série de níveis, os programadores que trabalham no nível *n* não precisam saber de todos os detalhes confusos dos níveis inferiores. Esta estruturação simplifica enormemente a produção de máquinas (virtuais) complexas.

1.5 MARCOS DO DESENVOLVIMENTO DA ARQUITETURA DE COMPUTADORES

Centenas de computadores de diferentes tipos têm sido projetados e construídos durante a evolução do computador digital moderno. A maioria já foi esquecida, mas alguns tiveram um impacto significativo nas idéias modernas. Nesta seção, daremos uma visão breve de alguns destes desenvolvimentos-chave históricos, para conseguir uma melhor compreensão de como chegamos até onde estamos agora. Nem é preciso dizer que esta seção apenas toca os pontos importantes e deixa muitos outros de lado. A Fig. 1.4 lista algumas das máquinas marcantes a serem discutidas nesta seção. Slater (1987) é um bom lugar para se encontrar material histórico adicional sobre pessoas que ajudaram a iniciar a era do computador.

1.5.1 A Geração Zero — Computadores Mecânicos (1642-1945)

A primeira pessoa a construir uma máquina de calcular foi o cientista francês Blaise Pascal (1623-1662), em cuja honra deu-se o nome à linguagem de programação Pascal. Este aparelho, construído em 1642, quando Pascal tinha apenas 19 anos, foi projetado para ajudar seu pai, um coletor de impostos para o governo francês. Era inteiramente mecânico, utilizava engrenagens e funcionava através de uma manivela operada manualmente.

A máquina de Pascal podia apenas subtrair e somar, porém 30 anos mais tarde o grande matemático alemão Barão Gottfried Wilhelm von Leibniz (1646-1716) construiu uma outra máquina mecânica que podia também multiplicar e dividir. De fato, Leibniz construiu o equivalente a uma calculadora de bolso de quatro funções, três séculos atrás.

Nada mais aconteceu durante 150 anos até que um professor de matemática na Universidade de Cambridge, Charles Babbage (1792-1871), inventor do velocímetro, projetou e construiu sua **máquina de diferenças**. Este dispositivo mecânico, que do mesmo modo que o de Pascal só podia somar e subtrair, foi projetado para calcular tabelas de números úteis à navegação naval. A máquina foi projetada para executar um algoritmo simples, o método das diferenças finitas utilizando polinômios. O aspecto mais interessante da máquina de diferenças era seu método de saída: ela perfurava os resultados numa placa de cobre com um buril de aço, desta maneira prenunciando os ulteriores meios de apenas uma escrita, tais como cartões perfurados e os primeiros discos óticos.

Embora a máquina de diferenças funcionasse razoavelmente bem, Babbage logo cansou-se dela, pois podia executar apenas um algoritmo. Ele começou então a gastar quantias crescentes de seu tempo e da fortuna de sua família (sem mencionar 17.000 libras esterlinas de dinheiro governamental) no projeto e construção de uma sucessora, denominada **máquina analítica**. A máquina analítica possuía quatro componentes: o armazenamento (memória), o engenho (unidade de cálculo), a seção de entrada (leitora de cartões perfurados) e a seção de saída (saída perfurada e impressa). O armazenamento consistia em 1.000 palavras de 50 dígitos decimais utilizadas para armazenar variáveis e resultados. O engenho podia aceitar operandos do armazenamento, somá-los, subtraí-los, multiplicá-los ou dividi-los, e retornar o resultado ao armazenamento. Tal como a máquina de diferenças, ela era inteiramente mecânica.

O grande avanço da máquina analítica era ser de uso geral. Ela lia instruções de cartões perfurados e as executava. Algumas instruções comandavam a máquina para buscar dois números no armazenamento, levá-los ao engenho, sofrerem a operação (p. ex., soma) e ter o resultado

Ano	Nome	Construído por	Comentários
1834	Máquina analítica	Babbage	Primeira tentativa de se construir um computador digital
1936	Z1	Zuse	Primeira máquina calculadora a relés a funcionar
1943	COLOSSUS	Governo britânico	Primeiro computador eletrônico
1944	Mark I	Aiken	Primeiro computador americano de uso geral
1946	ENIAC I	Eckert/Mauchley	A história do computador moderno começa aqui
1949	EDSAC	Wilkes	Primeiro computador de programa armazenado
1951	Whirlwind I	M.I.T.	Primeiro computador de tempo real
1951	UNIVAC I	Eckert/Mauchley	Primeiro computador vendido comercialmente
1952	IAS	von Neumann	A maioria das máquinas atuais utiliza este projeto
1960	PDP-1	DEC	Primeiro minicomputador (50 vendidos)
1961	1401	IBM	Máquina comercial de pequeno porte muito popular
1962	7094	IBM	Dominou a computação científica no início dos anos 60
1963	B5000	Burroughs	Primeira máquina projetada para uma linguagem de alto nível
1964	360	IBM	Primeira linha de produtos projetados como uma família
1964	6600	CDC	Primeira máquina com amplo uso de paralelismo internamente
1965	PDP-8	DEC	Primeiro minicomputador de venda em massa (50.000 vendidos)
1970	PDP-11	DEC	Minicomputadores que dominaram os anos 70
1974	8080	Intel	Primeira CPU de uso geral em uma pastilha
1974	CRAY-1	Cray	Primeiro supercomputador
1978	VAX	DEC	Primeiro supermini de 32 bits

Fig. 1.4 Alguns marcos importantes no desenvolvimento do computador digital moderno.

enviado de volta ao armazenamento. Outras instruções podiam testar um número e desviar condicionalmente, dependendo se positivo ou negativo. Perfurando um programa diferente nos cartões de entrada, era possível fazer com que a máquina analítica executasse cálculos diferentes, algo que não era verdade na máquina de diferenças.

Uma vez que a máquina analítica era programável numa linguagem de montagem simples, ela precisava de *software*. Para produzir este *software*, Babbage contratou uma jovem chamada Ada Augusta Lovelace, que era filha do famoso poeta britânico, Lord Byron. Ada Lovelace foi assim o(a) primeiro(a) programador(a) de computador do mundo. A moderna linguagem de programação Ada® foi assim denominada em sua homenagem.

Infelizmente, do mesmo modo que muitos projetistas modernos, Babbage nunca conseguiu depurar completamente o *hardware*. O problema era que ele precisava dos milhares de dentes, rodas e engrenagens produzidos a um grau de precisão que a tecnologia do século 19 era incapaz de prover. Apesar disso, suas idéias estavam muito à frente de seu tempo, e, até mesmo hoje, a maioria dos computadores modernos tem uma estrutura muito similar à máquina analítica, de modo que é justo dizer que Babbage foi o pai (avô) do computador digital moderno.

O próximo grande marco de desenvolvimento ocorreu nos anos 30, quando um estudante de engenharia alemão chamado Konrad Zuse construiu uma série de máquinas de calcular automáticas utilizando relés eletromagnéticos. Zuse não conhecia o trabalho de Babbage, e suas máquinas foram destruídas pelo bombardeio de Berlim pelos aliados em 1944, de forma que seu trabalho não teve qualquer influência nas máquinas posteriores. Mesmo assim, ele foi um dos pioneiros neste campo.

Logo depois, nos Estados Unidos, duas pessoas também projetaram calculadoras, John Atanasoff, no Iowa State College, e George Stibbitz, no Bell Labs. A máquina de Atanasoff era surpreen-

dentemente avançada para sua época. Ela utilizava aritmética binária e possuía capacitores para a memória, que eram periodicamente refrescados para evitar que se descarregassem, processo esse denominado por ele de "estimulação da memória". As pastilhas modernas de RAM dinâmica funcionam exatamente da mesma maneira. Infelizmente, a máquina nunca se tornou realmente operacional. De certa forma, Atanasoff foi como Babbage: um visionário derrotado pela inadequada tecnologia de *hardware* de seu tempo.

O computador de Stibbitz, embora mais primitivo do que o de Atanasoff, funcionava realmente. Stibbitz fez uma demonstração pública em uma conferência no Dartmouth College em 1940. Uma das pessoas da audiência era John Mauchley, um desconhecido professor de física da Universidade da Pennsylvania. Posteriormente, o mundo da computação ouviria mais sobre o Prof. Mauchley.

Enquanto Zuse, Stibbitz e Atanasoff estavam projetando calculadoras automáticas, um jovem chamado Howard Aiken estava realizando manualmente tediosos cálculos numéricos como parte de sua pesquisa de doutorado em Harvard. Após seu doutorado, Aiken reconheceu a importância dos cálculos feitos por máquina. Começou a ler sobre o assunto, descobriu o trabalho de Babbage e decidiu construir com relés o computador de uso geral que Babbage não conseguiu construir com rodas dentadas.

A primeira máquina de Aiken, o Mark I, foi terminado em Harvard, em 1944. Ele tinha 72 palavras de 23 dígitos decimais cada, e tinha um tempo de ciclo (isto é, de instrução) de 6 segundos. Para entrada e saída, utilizava-se fita de papel perfurado. Quando Aiken acabou de construir o sucessor, o Mark II, computadores com relés estavam obsoletos. A era eletrônica tinha começado.

1.5.2 A Primeira Geração — Válvulas (1945-1955)

O estímulo para o computador eletrônico foi a Segunda Guerra Mundial. Durante a primeira fase da guerra, os submarinos alemães estavam massacrando os navios britânicos. Ordens eram enviadas pelos almirantes alemães de Berlim para os submarinos por rádio, o que os ingleses podiam interceptar, e o fizeram. O problema era que estas mensagens eram criptografadas utilizando um aparelho denominado **ENIGMA**, cujo precursor, casualmente, foi projetado pelo inventor amador e ex-presidente dos EUA, Thomas Jefferson.

No início da guerra, a inteligência britânica conseguiu adquirir uma máquina ENIGMA da inteligência polonesa, que tinha sido roubada dos alemães. Entretanto, para decifrar uma mensagem codificada, era necessária uma imensa quantidade de cálculos, e, para que ela pudesse ser aproveitada, esta decodificação deveria ser feita o mais rapidamente possível. Para decodificar estas mensagens, o governo britânico criou um laboratório altamente secreto que construiu um computador eletrônico denominado COLOSSUS. O famoso matemático britânico Alan Turing ajudou a projetar esta máquina. O COLOSSUS ficou operacional em 1943, mas, como o governo britânico manteve praticamente todos os detalhes do projeto classificados como segredo militar durante 30 anos, a linha COLOSSUS nasceu morta. Vale apenas salientar que foi o primeiro computador eletrônico digital do mundo.

Além de destruir as máquinas de Zuse e estimular a construção do COLOSSUS, a guerra também afetou a computação nos Estados Unidos. O exército precisava de tabelas de alcance para calibragem de mira de sua artilharia pesada, e achava que calculá-las manualmente consumia muito tempo e era sujeito a erros.

John Mauchley, que conhecia o trabalho de Atanasoff e o de Stibbitz, sabia que o exército estava interessado em calculadoras mecânicas. Da mesma maneira que muitos cientistas da computação que vieram depois dele, Mauchley apresentou um pedido de auxílio ao exército para financiamento da construção de um computador eletrônico. A proposta foi aceita em 1943, e Mauchley e seu aluno de pós-graduação, J. Presper Eckert, construíram um computador eletrônico que eles denominaram **ENIAC (Electronic Numerical, Integrator And Computer**, ou seja, Computador e Integrador Numérico Eletrônico). Ele era constituído de 18.000 válvulas e 1.500 relés. O ENIAC pesava 30 toneladas e consumia 140 quilowatts de potência. Arquiteturalmente, a máquina possuía 20 registradores, cada um capaz de armazenar um número decimal de 10 dígitos. Sua programação era feita através de cerca de 6.000 chaves multiposicionais e da interconexão de um grande número de soquetes através de um verdadeiro emaranhado de cabos.

A máquina só ficou pronta em 1946, quando já era muito tarde para ter qualquer uso em seus objetivos originais. Entretanto, logo que a guerra acabou, Mauchley e Eckert obtiveram permissão para organizar um curso de verão para descrever o trabalho deles para seus colegas cientistas. Aquele curso de verão deu início a uma explosão de interesse na construção de grandes computadores digitais.

Após esse histórico curso de verão, muitos outros pesquisadores começaram a construir computadores eletrônicos. O primeiro computador operacional foi o EDSAC (1949), construído na Universidade de Cambridge, na Inglaterra, por Maurice Wilkes. Dentre outros, temos o JOHNIAC, na Rand Corporation, o ILLIAC, na University of Illinois, o MANIAC, no Los Alamos Laboratory, e o WEIZAC, no Weizmann Institute, em Israel.

Eckert e Mauchley começaram a construir uma nova máquina, o **EDVAC (Electronic Discrete Variable Automatic Computer**, ou seja, Computador Automático Eletrônico de Variáveis Discretas), mas este projeto foi seriamente afetado quando eles deixaram a Universidade da Pennsylvania para criar uma companhia, a Eckert-Mauchley Computer Corporation, em Philadelphia (o Vale do Silício não existia ainda). Após uma série de fusões, esta companhia tornou-se a atual Unisys Corporation.

Enquanto isso, uma das pessoas envolvidas no projeto ENIAC, John von Neumann, foi para o Instituto de Estudos Avançados de Princeton, para construir sua própria versão do EDVAC, a **máquina IAS.** Von Neumann era um gênio do tipo Leonardo da Vinci. Falava muitas línguas, era um especialista em ciências físicas e matemáticas, e tinha total lembrança de tudo que ouvia, via ou lia. Era capaz de citar de cor, literalmente, o texto de livros que ele havia lido anos antes. Quando ele se interessou por computadores, já era o mais eminente matemático do mundo.

Uma das coisas óbvias para ele era que a programação de computadores com um grande número de chaves e cabos era lenta, tediosa e inflexível. Ele começou a perceber que o programa poderia ser representado em forma digital na memória do computador, juntamente com os dados. Observou também que a desajeitada aritmética decimal utilizada pelo ENIAC, em que cada dígito era representado por 10 válvulas (uma ligada e nove desligadas), poderia ser substituída por uma aritmética binária paralela.

Seu projeto básico, agora conhecido como **máquina de von Neumann**, foi utilizado no EDSAC, o primeiro computador com programa armazenado, e é ainda a base de quase todos os computadores digitais, até mesmo hoje, quase meio século depois. Este projeto, e a máquina IAS, construída em colaboração com Herman Goldstine, tem tido tanta influência que vale uma breve descrição. Um esboço da arquitetura é dado na Fig. 1.5.

A máquina de von Neuman possuía cinco partes básicas: a memória, a unidade lógico-aritmética, a unidade de controle de programa e os equipamentos de entrada e saída. A memória consistia de 4096 palavras, cada palavra possuindo 40 bits (0 ou 1). Cada palavra armazenava duas instruções de 20 bits ou um inteiro de 39 bits com sinal. As instruções possuíam 8 bits dedicados a dizer o tipo da instrução, e 12 bits para especificar uma dentre 4096 palavras de memória.

Dentro da unidade lógico-aritmética, a precursora da atual **CPU (Central Processing Unit,** ou seja **(Unidade Central de Processamento)**, havia um registrador interno especial de 40 bits denominado **acumulador**. Uma instrução típica adicionava uma palavra de memória ao acumulador ou armazenava o acumulador na memória. A máquina não possuía aritmética de ponto-flutuante,

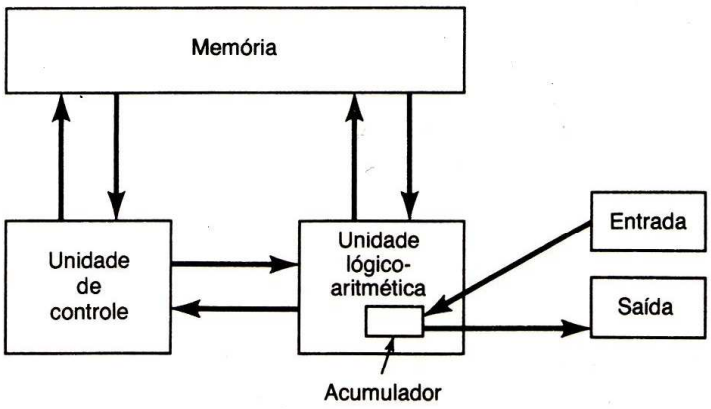


Fig. 1.5 A máquina original de von Neumann.

pois von Neumann achava que qualquer matemático competente deveria ser capaz de acompanhar de cabeça a posição do ponto decimal (na realidade, ponto binário).

Mais ou menos na mesma época em que von Neumann estava construindo a máquina IAS, pesquisadores do M.I.T estavam também construindo um computador. Ao contrário do IAS, ENIAC e outras máquinas deste tipo, que possuíam comprimentos longos de palavras e que visavam ao cálculo numérico pesado, a máquina do M.I.T., o Whirlwind I, possuía palavra de 16 bits e foi projetado para controle de tempo real. Este projeto levou à invenção da memória de núcleo, por Jay Forrester, e depois, eventualmente, ao primeiro minicomputador comercial.

Enquanto tudo isto acontecia, a IBM era uma companhia pequena engajada no ramo da fabricação de perfuradoras de cartões e de máquinas mecânicas de classificação de cartões. Embora a IBM fosse um dos financiadores de Aiken, ela não estava muito interessada em computadores até produzir o 701 em 1953, muito depois da companhia de Eckert e Mauchley já ser o número um do mercado comercial com seu computador UNIVAC. O 701 tinha 2K de palavras de 36 bits, com duas instruções por palavra. Foi a primeira de uma série de máquinas científicas que vieram a dominar a indústria em uma década. Três anos mais tarde surgiu o 704, que tinha 4K de memória de núcleo, instruções de 36 bits e *hardware* para ponto-flutuante. Em 1958, a IBM iniciou a produção de sua última máquina a válvulas, o 709, que era basicamente uma versão melhorada do 704.

1.5.3 A Segunda Geração — Transistores (1955-1965)

O transistor foi inventado no Bell Labs em 1948 por John Bardeen, Walter Brattain e William Shockley, pelo qual foram agraciados com o Prêmio Nobel de Física de 1956. Nos 10 anos seguintes, o transistor revolucionou os computadores, e no final dos anos 50 os computadores a válvula estavam obsoletos. O primeiro computador transistorizado foi construído no Lincoln Laboratory do M.I.T., uma máquina de 16 bits baseada no Whirlwind I. Foi denominada **TX-0** (Transistorized eXperimental computer 0, ou seja, computador transistorizado experimental 0), que visava meramente a ser um protótipo para testar o TX-2, uma versão melhorada.

O TX-2 não significou muito, mas um dos engenheiros que trabalhavam no Lincoln Laboratory, Kenneth Olsen, criou uma firma (DEC), em 1957, para fabricar uma máquina comercial muito parecida com o TX-0. Apenas quatro anos depois é que o PDP-1 apareceu, e este atraso aconteceu principalmente porque os investidores fundadores da DEC acreditavam firmemente que não havia mercado para computadores. Ao invés disso, a DEC vendia principalmente pequenas placas de circuitos.

Quando o PDP-1 finalmente apareceu em 1961, ele possuía 4K palavras de 18 bits e um tempo de ciclo de 5 microssegundos. Este desempenho era a metade daquele do IBM 7090, a máquina transistorizada sucessora do 709, e o mais rápido computador do mundo naquele tempo. O PDP-1 custava US$ 120.000, e o 7090 custava milhões. A DEC vendeu dezenas de PDP-1, e a indústria de minicomputadores nascia.

Um dos primeiros PDP-1 foi doado ao M.I.T., onde rapidamente atraiu a atenção de alguns desses jovens gênios tão comuns nesse instituto. Uma das muitas inovações que o PDP-1 trazia era um *display* visual (CRT) e a capacidade de se plotar pontos em qualquer posição de uma tela de 512 por 512. Em pouco tempo, os estudantes já tinham programado o PDP-1 para jogar guerra nas estrelas, e o mundo tinha assim seu primeiro videogame.

Poucos anos mais tarde, a DEC lançou o PDP-8, que era uma máquina de 12 bits, mas muito mais barata que o PDP-1 (US$ 16.000). O PDP-8 trouxe uma grande inovação: um barramento único, o *omnibus*, como mostrado na Fig. 1.6. Um **barramento** é um conjunto de fios paralelos utilizados para conectar os componentes de um computador. Esta arquitetura divergiu, em muito, da máquina IAS, com memória centralizada. Desde então, foi adotada por quase todos os computadores de pequeno porte. Eventualmente, a DEC vendeu 50.000 PDP-8, o que a tornou líder no mercado de minicomputadores.

Entrementes, a reação da IBM ao transistor foi construir uma versão transistorizada do 709, o 7090, como já dissemos antes, e mais tarde o 7094. O 7094 tinha um tempo de ciclo de 2 microssegundos e 32K palavras de 36 bits de memória de núcleo. O 7090 e o 7094 marcaram o fim das máquinas do tipo ENIAC, mas elas dominaram a computação científica durante vários anos na década de 60.

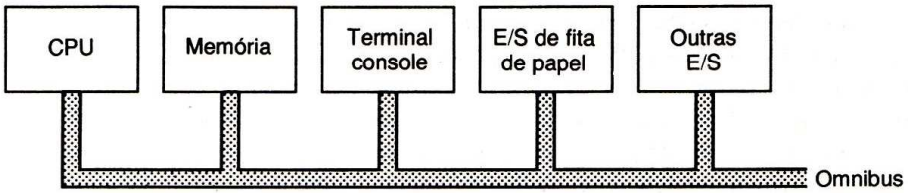


Fig. 1.6 O *omnibus* do PDP-8.

Ao mesmo tempo que a IBM tornou-se uma importante força na computação científica com o 7094, ela ganhou muito dinheiro vendendo uma pequena máquina de orientação comercial, denominada 1401. Esta máquina podia ler e escrever fitas magnéticas, ler e perfurar cartões, e imprimir quase tão rapidamente quanto o 7094, por uma fração do preço. Era terrível para aplicações científicas, mas perfeita para manter registros comerciais.

O 1401 era incomum no fato de não possuir registradores, ou mesmo um comprimento fixo de palavra. Possuía uma memória de 4K bytes de 8 bits. Cada byte continha um caractere de 6 bits, um bit administrativo e um bit usado para indicar fim-de-palavra. Uma instrução MOVE, por exemplo, tinha um endereço fonte e um endereço destino, e começava copiando bytes do fonte para o destino até atingir um com o bit fim-de-palavra ativado.

Em 1964, uma nova companhia iniciante, CDC, lançou o 6600, uma máquina que era quase uma ordem de grandeza mais rápida que o poderoso 7094. Foi amor à primeira vista entre os *number crunchers* (trituradores de números), e o CDC foi lançado assim para o sucesso. O segredo de sua velocidade, e a razão de ser muito mais rápido do que o 7094, era que dentro da CPU havia uma máquina altamente paralela. Ela possuía diversas unidades funcionais para fazer adições, outras para fazer multiplicações, e ainda uma outra para divisão, e todas elas podiam executar em paralelo. Embora fosse necessária uma programação cuidadosa para obter o máximo dela, com algum trabalho era possível ter 10 instruções sendo executadas simultaneamente.

Como se isto não bastasse, o 6600 tinha alguns pequenos computadores internos para ajudá-lo, uma espécie de Branca de Neve e os Sete Anões. Isto significava que a CPU podia gastar todo o seu tempo "triturando" números, deixando todos os detalhes de gerência de tarefas e de entrada/saída para computadores menores. O 6600 foi um marco importante na computação numérica.

Houve muitos outros computadores nesta geração, mas um sobressaiu-se por uma razão muito diferente, e vale mencionar: o Burroughs B5000. Os projetistas de máquinas como o PDP-1, 7094 e 6600 estavam todos completamente preocupados com o *hardware*, ou fazendo-o barato (DEC) ou rápido (IBM e CDC). O *software* era quase completamente irrelevante. Os projetistas do B5000 tomaram uma direção diferente. Eles construíram uma máquina com a intenção específica de programá-la em Algol 60, uma linguagem precursora do Pascal, e incluíram muitos aspectos no *hardware* para facilitar a tarefa do compilador. A idéia de que o *software* também importava nascia. Infelizmente, ela foi esquecida quase que imediatamente.

1.5.4 A Terceira Geração — Circuitos Integrados (1965-1980)

A invenção do circuito integrado possibilitou que dezenas de transistores fossem colocados em uma única pastilha. Este encapsulamento tornou possível construir computadores menores, mais rápidos e mais baratos que seus predecessores transistorizados. Alguns dos mais significativos computadores desta geração são descritos abaixo.

Por volta de 1964, a IBM era o fabricante líder de computadores, e tinha um grande problema com suas duas máquinas de grande sucesso, o 7094 e o 1401: eram totalmente incompatíveis entre si. Uma era uma processadora de números altamente veloz (*number cruncher*), que utilizava aritmética binária paralela sobre registradores de 36 bits, e a outra era uma excelente processadora de entrada/saída, que utilizava aritmética decimal serial sobre palavras de tamanho variável na memória. Muitos clientes possuíam ambos, e não gostavam da idéia de ter dois departamentos de programação com nada em comum.

Quando chegou o momento de substituir estas duas séries, a IBM deu um passo radical. Lançou uma única linha de produtos, o System/360, baseada em circuitos integrados, projetada

tanto para computação científica quanto comercial. O System/360 continha muitas inovações, a mais importante delas era que esta família, de cerca de meia dúzia de máquinas, possuía a mesma linguagem de montagem, com tamanhos e potências crescentes. Um cliente poderia substituir seu 1401 por um 360 Modelo 30, e seu 7094 por um 360 Modelo 75. O Modelo 75 era maior e mais rápido (e mais caro), mas o *software* escrito para um deles poderia, em princípio, ser executado no outro. Na prática, o *software* escrito para o modelo pequeno executaria em um modelo maior sem problemas, mas, quando mudássemos para uma máquina menor, o programa poderia não caber na memória. Todavia, este foi um grande avanço em relação à situação existente com o 7094 e o 1401. A idéia de família de máquinas foi aceita imediatamente, e poucos anos depois a maioria dos fabricantes de computadores possuía uma família de máquinas comuns apresentando uma grande variação de preços e desempenhos. Algumas características da família 360 inicial estão mostradas na Fig. 1.7. Outros modelos foram lançados mais tarde.

Uma outra importante inovação do 360 foi a **multiprogramação,** com vários programas presentes na memória simultaneamente, de forma que quando um estiver esperando pela entrada/saída ser completada, um outro poderia estar sendo executado.

O 360 também foi a primeira máquina que podia emular (simular) outros computadores. Os modelos menores podiam emular o 1401, e os maiores podiam emular o 7094, de forma que os clientes podiam continuar a executar seus antigos programas binários sem modificações, enquanto eram convertidos para o 360. Alguns modelos executavam os programas do 1401 tão mais rapidamente que o próprio, que muitos clientes nunca os converteram.

O 360 resolveu o dilema do binário-paralelo *versus* decimal serial com um compromisso: a máquina tinha 16 registradores de 32 bits para aritmética binária, mas sua memória era orientada para bytes, como aquela do 1401, e ela possuía instruções seriais do estilo 1401 para copiar registros de tamanhos variáveis na memória.

Uma outra característica importante do 360 era um enorme (para aquela época) espaço de endereçamento de 2^{24} bytes (16 megabytes). Com o custo de memória da ordem de vários dólares por byte naquela época, 16 megabytes parecia uma infinidade. Infelizmente, a série 360 foi seguida posteriormente pela série 370, série 4300, série 3080 e série 3090, todas utilizando exatamente a mesma arquitetura. Por volta da metade dos anos 80, o limite de 16 megabytes tornou-se um sério problema, e a IBM teve que abandonar parcialmente a compatibilidade quando adotou os endereços de 32 bits necessários para endereçar a nova memória de 2^{32} bytes.

Olhando para o passado, poder-se-ia argumentar que, uma vez que eles já tinham palavras e registradores de 32 bits, provavelmente poderiam ter tido endereços de 32 bits também, mas naquela época ninguém poderia imaginar uma máquina com 16 megabytes. Culpar a IBM por esta falta de visão é como culpar hoje um vendedor de computadores pessoais de ter apenas endereçamento de 32 bits, pois dentro de poucos anos todos os computadores pessoais necessitarão de mais de 4 gigabytes (um gigabyte é igual a 10^9 bytes), e então os endereços de 32 bits se tornarão intoleravelmente pequenos.

O mundo do minicomputador deu um enorme passo à frente na terceira geração com o lançamento, pela DEC, do PDP-11, um sucessor de 16 bits do PDP-8. De várias maneiras, era como se o PDP-11 fosse o irmão caçula da série 360, assim como o PDP-1 parecia ser o irmão caçula da série 7094. Tanto o 360 quanto o PDP-11 possuíam registradores orientados para palavras

	Modelo			
Propriedade	30	40	50	65
Desempenho relativo	1	3,5	10	21
Tempo de ciclo (ns)	1000	625	500	250
Memória máxima (K)	64	256	256	512
Bytes buscados por ciclo	1	2	4	16
Número máximo de canais de dados	3	3	4	6

Fig. 1.7 As ofertas iniciais da linha de produtos IBM 360.

e memória orientada para bytes, e ambos cobriam uma ampla faixa de relação custo/desempenho. O PDP-11 obteve um enorme sucesso, especialmente nas universidades, e manteve a liderança da DEC em relação aos outros fabricantes de minicomputadores.

1.5.5 A Quarta Geração — Computadores Pessoais e VLSI (1980-199?)

Nos anos 80, a VLSI (**Very Large Scale Integration,** ou seja, Integração em Escala Muito Grande) tornou possível colocar dezenas de milhares, depois centenas de milhares, e finalmente milhões de transistores em uma única pastilha. Este desenvolvimento levou a computadores menores e mais rápidos. Antes do PDP-1, os computadores eram tão grandes e caros que as companhias e universidades tinham que ter departamentos especiais, denominados **centros de computação,** para operá-los. Com o advento do minicomputador, um departamento podia comprar seu próprio computador. Por volta de 1980, os preços tinham caído tanto que era possível a uma pessoa física possuir seu próprio computador. A era do computador pessoal tinha começado.

Os computadores pessoais foram utilizados de uma maneira bastante diferente daquela dos grandes computadores. Eles eram usados para processamento de textos, planilhas e inúmeras aplicações altamente interativas que os computadores maiores não realizavam bem.

Atualmente, os computadores podem ser divididos grosseiramente em cinco categorias que se sobrepõem, como mostrado na Fig. 1.8. Elas se baseiam em tamanho físico, desempenho e áreas de aplicação. Na categoria de baixo, temos os computadores pessoais, máquinas de mesa que utilizam uma única pastilha processadora, geralmente dedicados a um único usuário. Eles são amplamente usados em escritórios, em ensino e para o uso doméstico.

Os minicomputadores são amplamente utilizados em aplicações de tempo real, por exemplo, para controle de tráfego aéreo ou automação industrial. Dizer precisamente em que consiste um minicomputador é difícil, pois muitas companhias fabricam produtos que consistem em um microprocessador de 16 ou 32 bits, uma certa quantidade de memória e algumas pastilhas de entrada/saída, tudo em uma única placa. Funcionalmente, tal placa equivale a um minicomputador tradicional, tal como o PDP-11.

O supermini é essencialmente um minicomputador muito grande, quase sempre baseado em um processador de 32 bits, e geralmente equipado com dezenas de megabytes de memória. Tais máquinas são usadas como sistemas departamentais de tempo compartilhado, como servidoras de arquivos (em rede), e muitas outras aplicações. Estas máquinas modernas são muito mais poderosas do que o IBM 360 Modelo 75, o *mainframe* mais poderoso do mundo quando foi lançado (1964).

Os *mainframes* tradicionais descendem de máquinas do tipo IBM 360 e CDC 6600. As diferenças reais entre um *mainframe* e um supermini estão na capacidade de entrada/saída e nas aplicações para as quais são usados. Um supermini típico pode ter um ou dois discos de 1 gigabyte, Um *mainframe* pode ter 100 deles. Os superminis são utilizados normalmente para aplicações interativas, enquanto a maioria dos *mainframes* é usada para grandes *jobs* em lote (*batch*) ou para processamento de transações, tais como bancos ou reservas de passagens aéreas, onde grandes bancos de dados são necessários.

Na extremidade superior do espectro estão os supercomputadores. Estas máquinas são especialmente projetadas para maximizar o número de **FLOPS (Floating-point Operations Per Second,**

Tipo	MIPs típicos	Megabytes típicos	Máquina exemplo	Exemplo de uso
Computador pessoal	1	1	IBM PS/2	Processamento de textos
Minicomputador	2	4	PDP-11/84	Controle de tempo real
Supermini	10	32	SUN-4	Servidora de arquivos (em rede)
Mainframe	30	128	IBM-3090/300	Bancos
Supercomputador	125	1024	Cray-2	Previsão de tempo

Fig. 1.8 Cinco tipos comuns de computadores.

ou seja, operações de ponto-flutuante por segundo). Qualquer coisa abaixo de 1 gigaflop/s não é considerada um supercomputador. Os supercomputadores possuem arquiteturas singulares altamente paralelas, a fim de atingir estas velocidades, e são apenas eficazes em uma pequena faixa de problemas.

Durante anos, os nomes supercomputador e Seymour Cray foram quase sinônimos. Cray projetou o CDC 6600 e seu sucessor, o 7600. Depois ele criou sua própria companhia, a Cray Research, para construir o Cray-1 e o Cray-2. Em 1989, Cray deixou-a para formar uma outra companhia para construir o Cray-3.

1.5.6 A Família Intel

Em 1968, a Intel Corporation foi criada para fabricar pastilhas de memória. Logo depois, ela foi contactada por um fabricante de calculadoras que queria uma CPU em uma única pastilha para a sua calculadora, e por um fabricante de terminais que queria um controlador em uma única pastilha para o seu terminal. A Intel produziu ambas as pastilhas, o 4004, uma CPU de 4 bits, e o 8008, uma CPU de 8 bits. Estas foram as primeiras CPUs numa única pastilha do mundo.

A Intel não esperava outros interessados além dos clientes originais, de maneira que estabeleceu uma linha de produção de baixo volume. Estavam errados. Houve um interesse tremendo, por isto começaram a projetar uma pastilha de CPU de uso geral, que resolvesse o problema do limite de 16K de memória do 8008 (imposto pelo número de pinos da pastilha). Este projeto resultou no 8080, uma pequena CPU de uso geral. Similarmente ao PDP-8, este produto tomou a indústria de assalto, e instantaneamente tornou-se um item de venda em massa. Porém, ao invés de vender milhares, como a DEC, a Intel vendeu milhões.

Dois anos mais tarde, em 1976, a Intel lançou o 8085, um 8080 encapsulado com alguns detalhes extras de entrada/saída. Depois surgiu o 8086, uma verdadeira CPU de 16 bits numa única pastilha. O 8086 foi projetado para ter uma certa semelhança com o 8080, mas não era completamente compatível com o 8080. O 8086 foi seguido pelo 8088, que possuía a mesma arquitetura que o 8086 e executava os mesmos programas, mas possuía um barramento de 8 bits, ao invés de um barramento de 16 bits, o que o tornava mais lento, porém mais barato que o 8086. Quando a IBM escolheu o 8088 para CPU do IBM PC original, esta pastilha tornou-se rapidamente o padrão da indústria de computadores pessoais.

Nos anos seguintes, a Intel lançou o 80186 e o 80188, essencialmente novas versões do 8086 e 8088, respectivamente, mas contendo também uma grande quantidade de circuitaria de entrada/saída. Nunca foram amplamente utilizados.

Nem o 8088 nem o 8086 podiam endereçar mais de 1 megabyte de memória. No início dos anos 80, isto se tornou um problema cada vez mais sério, por isso a Intel projetou o 80286, uma versão superior, compatível com o 8086. O conjunto básico de instruções era essencialmente o mesmo do 8086 e 8088, mas a organização da memória era muito diferente, e um tanto desajeitada, devido à necessidade de compatibilidade com as pastilhas anteriores. O 80286 foi utilizado no IBM PC/AT e nos modelos intermediários do PS/2. Do mesmo jeito que o 8088, obteve um grande sucesso.

O próximo passo lógico foi uma CPU verdadeiramente de 32 bits numa pastilha, o 80386. Como o 80286, este microprocessador era mais ou menos compatível com todos os anteriores, até o 8008, o que era de grande ajuda para aqueles para quem executar *software* antigo era importante, mas um incômodo para aqueles que preferiam uma arquitetura moderna, limpa e simples, sem os erros e a tecnologia do passado. Como o 80286, esta pastilha foi amplamente utilizada. O 80386SX é uma versão especial do 80386, projetado para ser compatível com o soquete do 80286 para prover um melhoramento parcial às máquinas 80286 existentes.

O 80486 é uma versão superior, compatível com o 80386. Todos os programas para o 80386 serão executados no 80486 sem modificações. Por isso, quase sempre no texto, quando nos referirmos ao 80386, o mesmo valerá para o 80486. A diferença básica entre o 80486 e o 80386 é a presença de um co-processador de ponto-flutuante, controlador de memória e 8K de memória cache na pastilha. Além disso, o 80486 é tipicamente duas a quatro vezes mais rápido que o 80386, e também mais adequado a sistemas multiprocessadores.

A evolução da linha de CPU da Intel reflete a evolução da indústria de computadores como

Nome	Ano	Largura dos registradores	Largura do barramento de dados	Espaço de endereçamento	Comentários
4004	1971	4	4	1K	Primeiro microprocessador em uma pastilha
8008	1972	8	8	16K	Primeiro microprocessador de 8 bits
8080	1974	8	8	64K	Primeira CPU de uso geral em uma pastilha
8085	1974	8	8	64K	8080 reencapsulado
8086	1978	16	16	1M	Primeira CPU de 16 bits em uma pastilha
8088	1980	16	8	1M	Processador utilizado no IBM PC
80186	1982	16	16	1M	8086 + suporte de entrada/saída em uma pastilha
80188	1982	16	16	1M	8088 + suporte de entrada/saída em uma pastilha
80286	1982	16	16	16M	Espaço de endereçamento aumentado para 16 megabytes
80386	1985	32	32	70T	Verdadeira CPU de 32 bits em uma pastilha
80386SX	1988	32	16	70T	80386 com um barramento de 80286
80486	1989	32	32	70T	Versão mais rápida do 80386

Fig. 1.9 A família Intel de CPUs. Observe que: K = Kilo (2^{10}), M = Mega (2^{20}), G = Giga (2^{30}) e T = Terra (2^{40}).

um todo. Em cerca de uma década e meia, fomos de uma CPU de 4 bits para uma CPU de 32 bits, com um fator de ganho de desempenho superior a 1000. Por outro lado, enquanto o 8086 contém 30.000 transistores, o 80486 possui mais de um milhão deles. A família Intel é mostrada na Fig. 1.9.

A série Intel de pastilhas constitui o coração de todos os computadores MS-DOS, portanto, é bastante importante estudá-los detalhadamente. Através deste livro utilizaremos repetidamente como exemplo esta família de CPUs, com ênfase no 8088, 80286 e 80386, os modelos mais importantes.

1.5.7 A Família Motorola

Logo após a Intel ter lançado o 8080, a Motorola, um fabricante de semicondutores rival, lançou o 6800. O 6800 era uma máquina de 8 bits comparável ao 8080. Foi bem recebido e amplamente utilizado como controlador embutido em equipamentos industriais no início dos anos 70. Foi seguido pelo 6809, que era compatível com o 6800, mas possuía detalhes extras que facilitavam a artimética de 16 bits.

Então, em 1979, a Motorola fez algo que poucas firmas tinham feito antes ou fizeram desde então: ela lançou uma pastilha completamente nova que não era compatível com o 6800 nem com o 6809. A idéia era que esta pastilha, o 68000, venceria a competição (com o 8086) e atrairia os engenheiros que desejassem um projeto limpo, em vez de um sobrecarregado pela compatibilidade com máquinas obsoletas.

O 68000 foi, na verdade, uma mudança radical em relação ao passado. Embora busque dados da memória, 16 bits de cada vez (em outras palavras, o barramento de dados possui 16 bits de largura), todos os registradores que o programador vê são de 32 bits de largura, e a máquina pode somar e subtrair (mas não multiplicar ou dividir) números de 32 bits em apenas uma instrução. Assim, o 68000 é um híbrido entre arquiteturas de 16 e 32 bits.

Como tal, constitui um bom teste de tornassol. Se você estiver tentando descobrir se alguém é uma pessoa de *hardware* ou de *software*, pergunte-lhe se o 68000 é uma máquina de 16 ou de 32 bits. Um engenheiro de *hardware* dirá 16, devido ao barramento de dados de 16 bits.

Um programador dirá 32, pois os registradores possuem todos 32 bits de largura. Tudo depende da perspectiva da pessoa.

O 68000 foi escolhido pelos projetistas do Macintosh, Atari, Amiga e outros computadores populares, devido à sua ruptura limpa com o passado e seu comprimento de palavra de 32 bits. De maneira geral, a pastilha fez muito sucesso, e deu início a uma família de pastilhas, do mesmo modo que a série Intel. Só que, ao contrário das pastilhas da Intel, os membros da família **680X0**, como são freqüentemente chamados, são muito similares do ponto de vista do programador, com apenas umas poucas novas instruções introduzidas em cada nova versão. São membros de uma família no sentido de que, quando uma nova instrução é acrescentada, todo o *software* existente continua sendo executado.

A segunda pastilha da família foi o 68008, que era idêntico ao 68000, exceto pelo uso de um barramento de dados de 8 bits para produtos simples. Ao contrário da versão Intel de 8 bits do 8086 (o 8088), o 68008 nunca foi amplamente utilizado.

Não demorou muito até aparecerem interessados em implementar sistemas operacionais sofisticados como o UNIX no 68000. Muitos destes sistemas têm memória virtual, técnica esta que permite aos programas endereçar mais memória do que o computador realmente tem. A memória virtual (que mais tarde estudaremos detalhadamente) funciona trocando automaticamente partes do programa da memória para o disco, na medida do necessário. O 68000 quase podia suportar memória virtual, mas não completamente. Havia alguns detalhes que faltaram na pastilha.

A Motorola resolveu este problema lançando a pastilha 68010, que possuía as características necessárias. Logo depois, ela lançou outra pastilha, o 68012, que era o mesmo que o 68010, exceto que possuía mais pinos de endereçamento, de forma que poderia endereçar 2 gigabytes de memória, ao invés de apenas 16 megabytes.

Estas duas pastilhas foram efetivamente eliminadas um ano mais tarde, quando a Motorola lançou o 68020, uma verdadeira pastilha de 32 bits, com um barramento de 32 bits e instruções de multiplicação e divisão de 32 bits. O 68020 foi um grande sucesso, e era o coração das principais estações de trabalho científicas e de engenharia, tais como aquelas fabricadas pela Sun Microsystems, Apollo e Hewlett-Packard. O sucessor do 68020 foi o 68030, que continha não apenas um 68020 completo, mas também uma unidade completa de gerenciamento de memória na mesma pastilha.

O 68040, como o 80486, contém uma CPU, um co-processador de ponto-flutuante, uma unidade de gerenciamento de memória e cache na mesma pastilha. Sendo aproximadamente iguais em complexidade, não é surpreendente que os dois contenham quase o mesmo número de transistores: 1,2 milhão para o 68040 e 1,16 milhão para o 80486. O 68040 executa o mesmo *software* que o 68030, de forma que nossa discussão da arquitetura 68030 neste livro também vale para o 68040. O 68040 se compara mais ou menos ao 80486 de muitas maneiras, tal como o 68030 se compara grosseiramente ao 80386 de muitas maneiras. Em dúvida, estas pastilhas continuarão a competir durante os próximos anos. Um sumário da família de pastilhas Motorola é mostrado na Fig. 1.10.

Nome	Ano	Largura dos registradores	Largura do barramento de dados	Espaço de endereçamento	Comentários
68000	1979	32	16	16M	Primeiro membro da família
68008	1982	32	8	4M	Pastilhas mais simples com barramento de 8 bits
68010	1983	32	16	16M	Suporta memória virtual
68012	1983	32	16	2G	Versão do 68010 com espaço de endereçamento grande
68020	1984	32	32	4G	CPU de 32 bits verdadeira
68030	1987	32	32	4G	Unidade de gerenciamento de memória na pastilha da CPU
68040	1989	32	32	4G	Versão mais rápida do 68030

Fig. 1.10 A família Motorola de CPUs.

1.6 VISÃO GERAL DESTE LIVRO

Este livro trata de computadores multinível (o que engloba quase todos os computadores modernos) e de como eles estão organizados. Examinaremos cinco níveis com considerável detalhe — a saber, o nível de lógica digital, o nível de microprogramação, o nível de máquina convencional, o nível de sistema operacional e o nível de linguagem de montagem. Alguns dos aspectos básicos a serem examinados são:

1. O projeto geral do nível (e por que ele foi projetado daquela maneira).
2. Os tipos de instruções disponíveis.
3. Os tipos de dados utilizados.
4. Os mecanismos disponíveis para alterar o fluxo de controle.
5. A organização e endereçamento da memória.
6. A relação entre o conjunto de instruções e a organização da memória.
7. O método pelo qual o nível é implementado.

O estudo destes tópicos, e de outros similares, denomina-se organização de computadores ou arquitetura de computadores, como já mencionamos.

Nós estamos principalmente interessados nos conceitos, mais do que em detalhes ou em matemática formal. Por isso, alguns dos exemplos dados serão bastante simplificados, para enfatizar as idéias principais, e não os detalhes.

Para prover alguma luz de como as idéias neste livro podem ser, e são, aplicadas na prática, utilizaremos as famílias Intel e Motorola como exemplos contínuos ao longo do livro. Estas duas famílias foram escolhidas por diversas razões. Primeiro, ambas são amplamente usadas e provavelmente o leitor tem acesso a pelo menos uma delas. Segundo, cada uma delas possui sua própria arquitetura, o que provê uma base de comparação e encoraja uma atitude de "quais são as alternativas?" Os livros que tratam de uma única máquina freqüentemente deixam o leitor com uma sensação de "foi revelado um verdadeiro projeto de máquina", o que é um absurdo, em vista dos muitos compromissos e decisões arbitrárias que os projetistas são forçados a assumir. Encorajamos você a estudar estes e todos os outros computadores com olho crítico e tentar entender por que as coisas são do jeito que são, e também como elas poderiam ter sido, em vez de simplesmente aceitá-las como são dadas.

Deve ficar claro, desde o início, que este não é um livro sobre como programar as pastilhas da Intel ou da Motorola. Estas máquinas serão utilizadas com objetivos ilustrativos onde apropriado, mas não temos a pretensão de sermos completos. Os leitores que desejarem um introdução completa de um deles devem consultar as publicações dos fabricantes.

O capítulo 2 é uma introdução aos componentes básicos de um computador — processadores, memórias e equipamentos de entrada/saída. Pretende-se dar uma visão geral da arquitetura do sistema.

Os capítulos 3, 4, 5, 6 e 7 tratam, cada um, de um nível específico dentre os mostrados na Fig. 1.2. Nosso tratamento é de baixo para cima (*bottom-up*), pois as máquinas têm sido tradicionalmente projetadas desta maneira. O projeto do nível k é, em grande parte, determinado pelas propriedades do nível k-1; assim é difícil entender qualquer nível, a menos que já se tenha uma boa compreensão do nível inferior que motivou este projeto. Também, é pedagogicamente sensato partir dos níveis inferiores mais simples para os níveis superiores mais complexos, em vez do contrário.

O capítulo 3 trata do nível de lógica digital, o verdadeiro *hardware* da máquina. Discute-se o que são portas e como elas são combinadas em circuitos úteis. Apresenta-se também a álgebra booleana, uma ferramenta de análise de circuitos digitais. São explicados os barramentos de computadores, descreve-se, em nível de pastilha, o funcionamento interno de um típico clone do PC.

O capítulo 4 apresenta os conceitos de microprogramação e arquitetura do nível de microprogramação, além de sua relação com a máquina convencional que ela suporta. A maior parte do capítulo consiste em um exemplo simples de máquina discutido em detalhe. O capítulo contém também discussões sobre o nível de microprogramação de algumas máquinas reais.

O capítulo 5 discute o nível de máquina convencional, aquele que a maioria dos vendedores

de computadores chama de linguagem de máquina. Primeiramente, examinaremos este nível de forma genérica e depois olharemos nossas máquinas-exemplo detalhadamente.

O capítulo 6 cobre algumas das instruções, organização de memória e mecanismos de controle presentes no nível de sistema operacional.

O capítulo 7 introduz o nível de linguagem de montagem. Como o nível de linguagem de montagem, ao contrário dos níveis abaixo dele, é geralmente implementado por tradução, em vez de interpretação, a ênfase aqui está mais no processo de tradução do que nos detalhes de quaisquer linguagens de montagem específicas. Tópicos relacionados como macros (que é, por si só, um tipo particular de técnica de tradução) e *linking* (ligação, que é realmente a última fase do processo de tradução) são também vistos.

O capítulo 8 apresenta diversos tópicos avançados. Em particular, as famílias Intel e Motorola são máquinas von Neumann bastante tradicionais. Uma fatia considerável da pesquisa atual está direcionada para alternativas à arquitetura von Neumann, e este capítulo foi incluído para dar uma visão deste assunto e para encorajar o leitor a investigá-lo mais. Dois tópicos são tratados detalhadamente: máquinas RISC e arquiteturas paralelas.

O capítulo 9 contém uma lista de sugestões de leitura, dispostas por assunto, e uma lista alfabética da literatura relacionada. É o capítulo mais importante do livro. Utilize-o.

Os apêndices contêm introduções breves à aritmética de precisão finita, incluindo números binários e números de ponto-flutuante.

PROBLEMAS

1. Explique cada um dos seguintes termos com suas próprias palavras:

   a. Tradutor
   b. Interpretador
   c. Máquina virtual.

2. Qual a diferença entre intrepretação e tradução?
3. É concebível que um compilador gere saída para o nível 1, em vez de para o nível 2? Discuta os prós e contras desta proposta.
4. Você consegue imaginar algum computador multinível em que o nível de dispositivos e o nível de lógica digital não sejam os mais inferiores? Explique.
5. Considere um computador com interpretadores idênticos nos níveis 1, 2 e 3. Um interpretador gasta n instruções para busca, exame e execução de uma instrução. Uma instrução de nível 1 leva k segundos de execução. Quanto tempo uma instrução dos níveis 2, 3 e 4 levará?
6. Considere um computador multinível em que todos os níveis diferem. Cada nível possui instruções m vezes mais poderosas do que aquelas do nível logo abaixo, ou seja, uma instrução do nível r pode fazer o trabalho de m instruções do nível $r - 1$. Se um programa de nível 1 requer k segundos de execução, quanto tempo programas equivalentes levariam nos níveis 2, 3 e 4, assumindo que n instruções de nível r são necessárias para interpretar uma única instrução $r + 1$?
7. Algumas instruções do nível de sistema operacional são idênticas às instruções de linguagem de máquina convencional. Estas instruções são executadas diretamente pelo microprograma, em vez de pelo sistema operacional. Em vista de sua resposta ao problema anterior, por que você acha que este é um exemplo disto?
8. Em que sentido *hardware* e *software* são equivalentes? E não-equivalentes?
9. Uma das conseqüências da idéia de von Neumann de armazenar o programa na memória é que os programas podem ser modificados, tais como os dados. Poderia dar um exemplo onde esta facilidade poderia ser utilizada? (Sugestão: Pense em aritmética de matrizes.)
10. A relação de desempenho do 360 Modelo 75 é 50 vezes a do 360 Modelo 30. Todavia, o tempo de ciclo é apenas cinco vezes mais rápido. A que se deve esta discrepância?
11. Dois projetos básicos de sistema estão mostrados nas Figs. 1.5 e 1.6. Descreva como a entrada/saída poderia ocorrer em cada sistema. Qual dos dois sistemas possui potencialmente melhor desempenho global?

2

ORGANIZAÇÃO DE SISTEMAS DE COMPUTADORES

Um computador digital consiste em um sistema interconectado de processadores, memórias e dispositivos de entrada/saída. Este capítulo é uma introdução a esses três componentes e suas interconexões, como base para um exame detalhado dos níveis específicos nos cinco capítulos seguintes. Processadores, memórias e entrada/saída são conceitos-chave e estarão presentes em cada nível, de modo que começaremos nosso estudo de arquitetura de computadores analisando um de cada vez.

2.1 PROCESSADORES

A organização de um computador simples orientado a barramento é mostrado na Fig. 2.1. A **unidade central de processamento (central processing unit — CPU)** é o "cérebro" do computador. Sua função é executar programas armazenados na memória principal, buscando suas instruções, examinando-as, e então executando uma após a outra. A CPU é composta por várias partes distintas. A unidade de controle é responsável pela busca de instruções da memória principal e determinação de seus tipos. A unidade lógica e aritmética faz operações tais como adições e E booleano, necessárias à execução das instruções. A CPU contém ainda uma memória pequena, de alta velocidade, usada para armazenar resultados temporários e certas informações de controle.

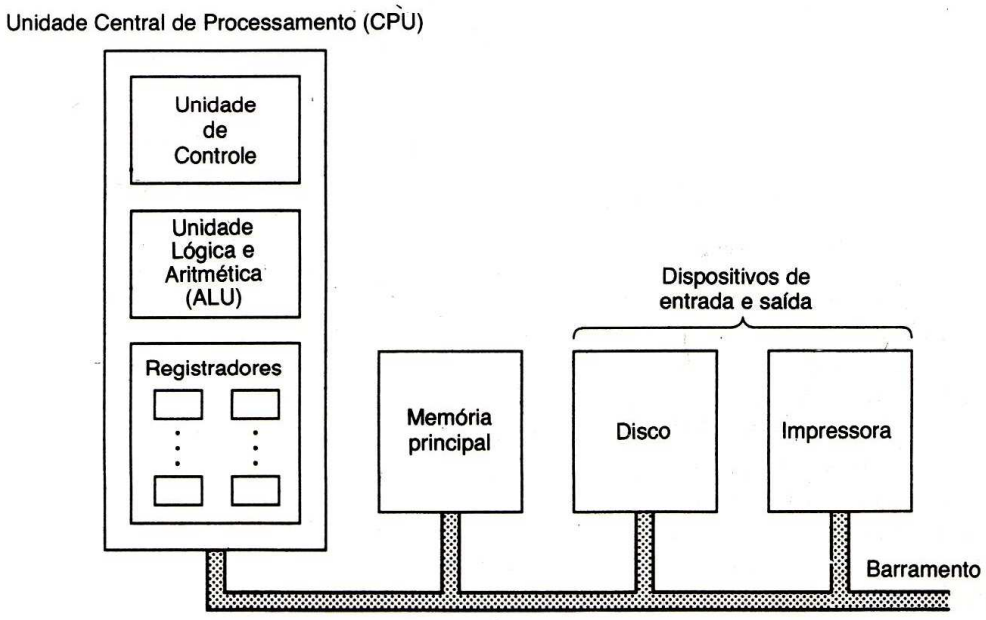


Fig. 2.1 A organização de um computador simples com uma CPU e dois dispositivos de E/S.

Esta memória é formada por um determinado número de *registradores*, cada um com uma dada função. O registrador mais importante é o **contador de programa (program counter — PC)**, que aponta para a próxima instrução a ser executada. O nome "contador de programa" é um tanto enganador, porque ele não tem nada a ver com a *contagem* de alguma coisa, mas o termo é universalmente usado. O **registrador de instrução (instruction register — IR)** é também importante. Ele contém a instrução que está sendo executada. A maioria dos computadores tem outros registradores, alguns deles disponíveis aos programadores de níveis 2 e 3, para armazenamento de resultados intermediários.

2.1.1 Execução de Instruções

A CPU executa a instrução em uma série de pequenos passos:

1. Busca a próxima instrução da memória para o registrador de instrução.
2. Atualiza o contador de programa para que ele aponte para a instrução seguinte.
3. Determina o tipo da instrução.
4. Se a instrução usa dados da memória, determina onde eles estão.
5. Busca os dados, se houver algum, para registradores internos da CPU.
6. Executa a instrução.
7. Armazena os resultados em locais apropriados.
8. Volta ao passo 1 para iniciar a execução da próxima instrução.

Esta seqüência de passos é freqüentemente referida como o ciclo **"busca-decodifica-executa"**. Ela é o centro da operação de todos os computadores.

Esta descrição de como uma CPU funciona parece muito com um programa escrito em português. A Fig. 2.2 mostra este programa informal reescrito como um procedimento Pascal. O fato de ser possível escrever um programa que pode imitar a função de uma CPU mostra que um programa não precisa ser executado pelo *hardware* de uma CPU, constituído por um emaranhado de circuitos eletrônicos. Em vez disso, um programa pode ser executado tendo-se um outro programa que busca, analisa e executa suas instruções. Um programa (como o da Fig. 2.2) que busca, analisa e executa as instruções de outro programa é chamado de interpretador, como mencionado no capítulo 1.

Esta equivalência entre processadores em *hardware* e interpretadores tem implicações importantes para a organização de computadores. Após ter sido especificada a linguagem de máquina *L* para um novo computador, o grupo projetista pode decidir se eles querem construir um processador para executar programas em *L* diretamente ou se eles querem escrever um interpretador. Se escolherem escrever um interpretador, também, precisam prover uma máquina para executá-lo. Como um interpretador quebra a instrução de sua máquina-alvo em pequenos passos, a máquina na qual o interpretador é executado pode freqüentemente ser muito mais simples que o processador para a máquina-alvo. Devido a razões econômicas, entre outras, os programas no nível de máquina convencional da maioria dos computadores modernos são executados por um interpretador em execução em uma máquina de nível 1 completamente diferente e muito mais primitiva, que chamamos nível de microprogramação.

A coleção de todas as instruções disponíveis ao programador em um nível é chamada **conjunto de instruções** daquele nível. O número de instruções de um conjunto de instruções varia de máquina para máquina e de nível para nível. Para o nível de máquina convencional, por exemplo, o tamanho do conjunto de instruções está tipicamente na faixa de 20 a 300. Um conjunto de instruções numeroso não é necessariamente melhor que um pequeno. De fato, o oposto tende a ser verdade. Um conjunto de instruções numeroso muitas vezes significa que as instruções

não são muito gerais. Os compiladores para linguagens de alto nível, como Ada, Modula 2 e Pascal, geralmente atuam melhor em máquinas com conjuntos de instruções pequenos e bem escolhidos do que em máquinas com conjuntos de instruções grandes e desajeitados. Máquinas com conjunto de instruções muito pequeno, chamadas máquinas **RISC**, serão discutidas no capítulo 8. Estas máquinas não usam microprogramação e são extremamente rápidas.

Certifique-se de compreender que o conjunto de instruções e a organização do nível de microprogramação são, de fato, o conjunto de instruções e a organização do *hardware* (CPU). O conjunto de instruções e a organização do nível de máquina convencional são, em contraste, determinados pelo microprograma, e não pelo *hardware*.

2.1.2 Organização da CPU

A organização interna de parte de uma CPU von Neumann clássica é mostrada na Fig. 2.3 em mais detalhe. Esta parte é chamada de **fluxo de dados** e consiste em registradores (tipicamente 1 a 16) e da **ALU (unidade lógica e aritmética)**. Os registradores alimentam os dois registradores de entrada da ALU, chamados de *A* e *B* na figura. Esses registradores mantêm as entradas da ALU enquanto ela está executando a operação.

```
type palavra =...;
     endereço =...;
     mem = array (0.. 4095) of palavra;
procedure interpretador (memória: mem; ac: palavra; EndereçoInicial: endereço)

(Este procedimento interpreta programas para uma máquina simples com 1 instrução
por palavra. A memória consiste em uma seqüência de palavras numeradas 0, 1,
..., 4095. A máquina tem um registrador chamado ac, usado para cálculos aritméticos.
A instrução ADD, por exemplo, soma uma palavra ao ac. O interpretador mantém
a execução até que o bit de execução é desligado por uma instrução HALT. O estado
do processo em execução nesta máquina consiste na memória, o contador de programa,
o bit de execução e o ac. O estado inicial é passado via parâmetros.)

var ContadordePrograma, LocalizaçãodoDado: endereço;
    RegistradordeInstrução, dado: palavra;
    DadoRequerido: boolean;
    TipodaInstrução: integer;
    BitdeExecução: 0..1;

begin
  ContadordePrograma:= EndereçoInicial;
  BitdeExecução:= 1;
  while BitdeExecução =1 do
    begin
      {Buscar a próxima instrução para o registrador de instrução}
      RegistradordeInstrução:= Memória [ContadordePrograma];

      {Avançar o contador de programa para apontar para a próxima instrução}
      ContadordePrograma:= ContadordePrograma + 1;

      {Decodificar a instrução e registrar seu tipo}
      DeterminaTipodeInstrução (RegistradordeInstrução, TipodaInstrução);

      {Localizar os dados usados pela instrução}
      AchaDado (TipodaInstrução, RegistradordeInstrução, LocalizaçãodoDado,
      DadoRequerido);

      {Buscar o dado na memória, se necessário}
      If DadoRequerido then dado:=memória [LocalizaçãodoDado];

      {Avançar o processo executando a instrução}
      execute (TipodeInstrução, dado, memória, ac, ContadordePrograma, BitdeInstrução)
    end
end;
```

Fig. 2.2 Um interpretador para um computador simples.

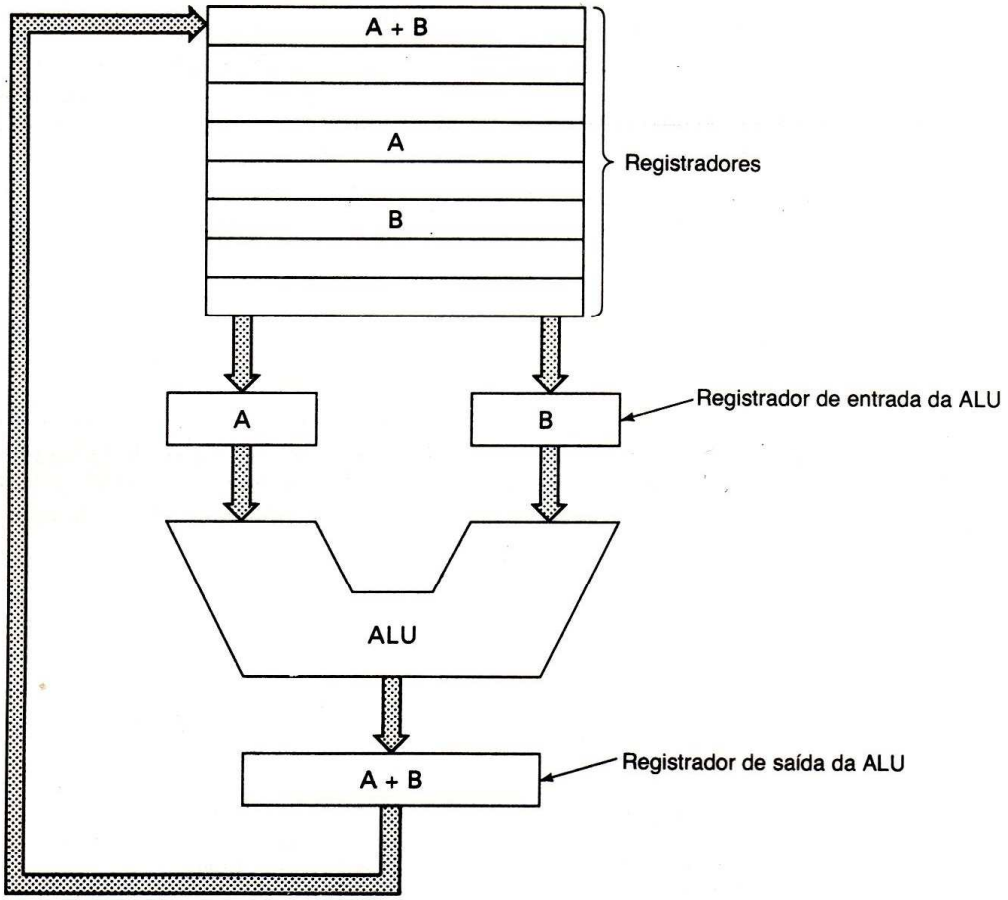


Fig. 2.3 O fluxo de dados para uma máquina von Neumann típica.

A própria ALU executa adição, subtração e outras operações simples com suas entradas, produzindo um resultado no seu registrador de saída. Este resultado pode ser armazenado de volta em um registrador e, de lá, de volta à memória, se desejado. O exemplo ilustra a adição.

As instruções podem ser divididas em três categorias: registrador-memória, registrador-registrador e memória-memória. Instruções registrador-memória permitem que palavras sejam buscadas da memória para registradores, onde podem ser usadas como entradas para a ALU em instruções seguintes, por exemplo. Uma instrução típica registrador-registrador busca dois operandos dos registradores, os traz para os registradores de entrada da ALU, executa alguma operação com eles e armazena o resultado de volta em um registrador. A operação do fluxo de dados é o coração da maioria dos computadores. Extrapolando, ela define o que a máquina pode faze Voltaremos a este importante tópico no capítulo 4.

2.1.3 Execução Paralela de Instruções

Desde os primórdios da computação, os projetistas tentam construir máquinas mais rápidas. Até certo ponto, as máquinas podem ser aceleradas simplesmente aumentando a velocidade do *hardware*. Entretanto, vários limites físicos começam a despontar no horizonte. Para começar, as leis da física dizem que nada pode viajar mais rápido que a luz, cuja velocidade é aproximadamente 30 cm/ns no vácuo, e 20 cm/ns em fio de cobre. Isto significa que, para construir um computador com tempo de instrução de 1 nanossegundo, a distância total que os sinais elétricos podem viajar dentro da CPU, para a memória, e de volta à CPU não pode ser maior que 20 centímetros. Logo, computadores muito rápidos precisam ser muito pequenos.

Infelizmente, computadores rápidos produzem mais calor que os lentos e a montagem do computador em um volume pequeno torna difícil a dissipação desse calor. Os supercomputadores

são, muitas vezes, submersos em fréon líquido, um refrigerante, para retirar o calor o mais rápido possível. Considerando tudo isso, produzir computadores cada vez mais rápidos está-se tornando cada vez mais difícil, e também cada vez mais caro.

Entretanto, existe outra abordagem. Em vez de uma única CPU de alta velocidade, é possível construir uma máquina com muitas ALUs mais lentas (e mais baratas) ou mesmo CPUs completas para se obter o mesmo poder computacional a um custo menor. Muita pesquisa tem sido dedicada à construção de tais máquinas paralelas. Nesta seção faremos uma breve introdução a algumas das técnicas utilizadas.

As máquinas paralelas podem ser divididas em três categorias (Flynn, 1972), baseando-se no número de fluxos de instruções e de dados que elas têm:

1. SISD: *Single Instruction, Single Data*
   fluxo único de instruções e de dados.

2. SIMD: *Single Instruction, Multiple Data*
   fluxo único de instruções e múltiplo de dados.

3. MIMD: *Multiple Instruction, Multiple Data*
   fluxo múltiplo de instruções e de dados.

A máquina tradicional de von Neumann é SISD. Ela tem apenas um fluxo de instruções (i. é, um programa), executado por uma única CPU, e uma memória contendo seus dados. A primeira instrução é buscada da memória e então executada. A seguir, a segunda instrução é buscada e executada.

Entretanto, mesmo com este modelo seqüencial, pode ser obtida uma limitada quantidade de paralelismo buscando-se e iniciando-se a próxima instrução antes que a corrente esteja terminada. O CDC 6600 e alguns de seus sucessores, por exemplo, têm múltiplas unidades funcionais (ALUs especializadas), cada qual podendo executar uma única operação em alta velocidade, como ilustrado na Fig. 2.4. Neste exemplo temos seis unidades funcionais, duas delas para cada uma das operações comuns de adição e multiplicação, uma para subtração e outra para divisão.

A idéia por trás deste projeto é que a unidade de controle busca uma instrução e a entrega para uma das unidades funcionais para execução. Enquanto isto, a unidade de controle busca a próxima instrução e a entrega para uma outra unidade funcional. Este processo continua até não poder mais, seja porque todas as unidades funcionais do tipo requerido estão ocupadas, ou porque o operando necessário está ainda sendo processado.

Esta estratégia geral obviamente se baseia no fato de que o tempo para executar uma operação é muito maior que o tempo para buscá-la, assim ela é usada nas operações de ponto-flutuante, que são complexas e lentas, em vez das operações com inteiros, que são simples e rápidas.

Uma variante desta idéia é dividir a execução de cada instrução em partes, como na linha de montagem de um carro. Na Fig. 2.5(a) vemos uma CPU composta por cinco unidades de processamento, *P1* a *P5*. Durante o primeiro intervalo de tempo, a primeira instrução é buscada da memória por *P1*, como mostra a Fig. 2.5(b). Durante o segundo intervalo de tempo, a primeira instrução passa por *P2* para análise, enquanto *P1* busca a próxima instrução. Em cada intervalo subseqüente, uma nova instrução é buscada por *P1* e as outras instruções são passadas para as outras unidades ao longo do caminho.

A organização da Fig. 2.5(a) é chamada de **máquina *pipeline***. Se cada passo (intervalo de tempo) é de n ns, cada instrução demorará $5n$ ns para ser executada. Entretanto, uma instrução completa é executada por *P5* a cada n ns, para um ganho de cinco vezes na velocidade. Note que embora seja empregado algum paralelismo interno, uma máquina *pipeline* é ainda uma máquina SISD, pois há apenas um programa e um conjunto de dados.

Máquinas SIMD, ao contrário, operam em múltiplos conjuntos de dados em paralelo. Uma aplicação típica para uma máquina SIMD é a previsão do tempo. Imagine o cálculo da temperatura média diária a partir de 24 médias horárias para muitos locais. Para cada local, exatamente o mesmo cálculo precisa ser feito, porém com dados diferentes.

Uma arquitetura apropriada para esta tarefa é uma **máquina vetorial**, mostrada na Fig. 2.6(a). O fluxo de dados aqui é similar ao da Fig. 2.3, exceto que em vez de se ter uma única variável

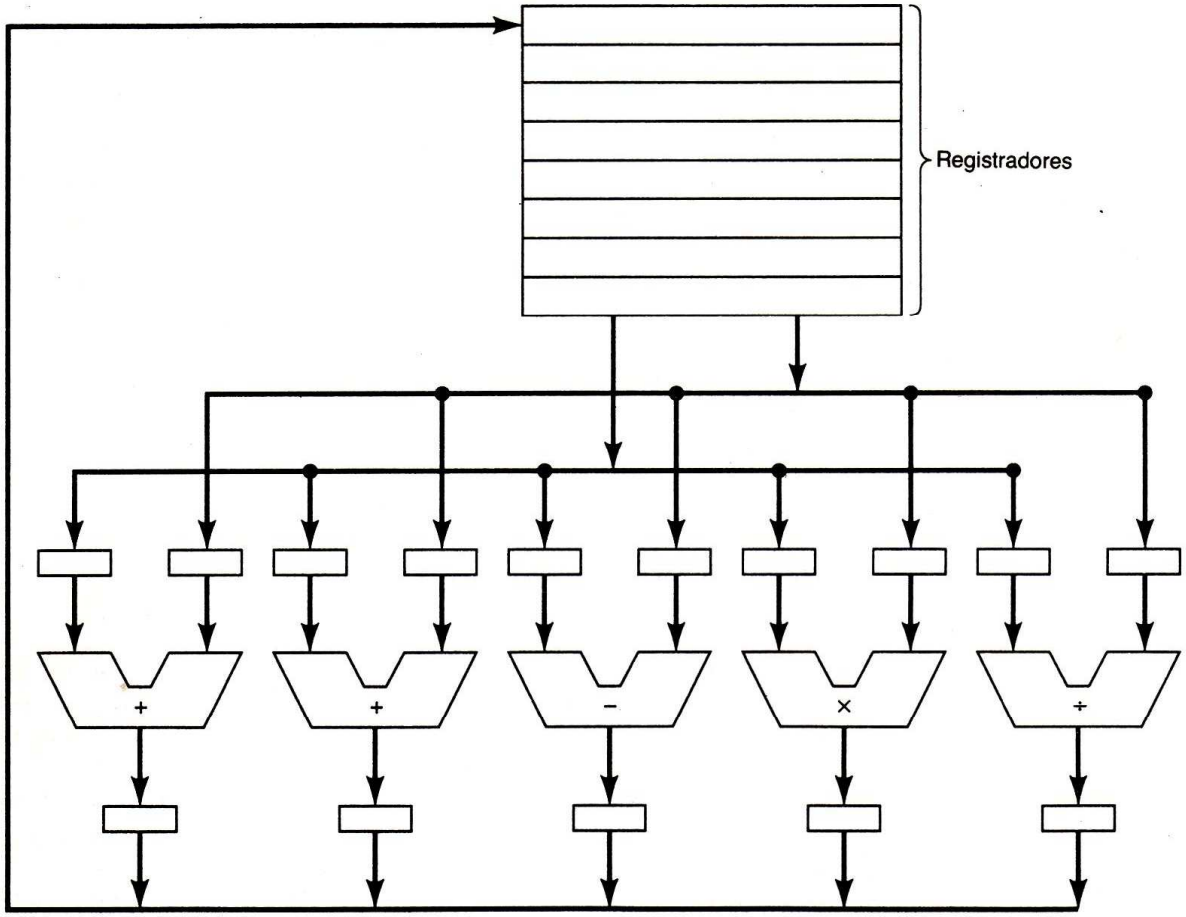


Fig. 2.4 Uma CPU com seis unidades funcionais que podem trabalhar em paralelo.

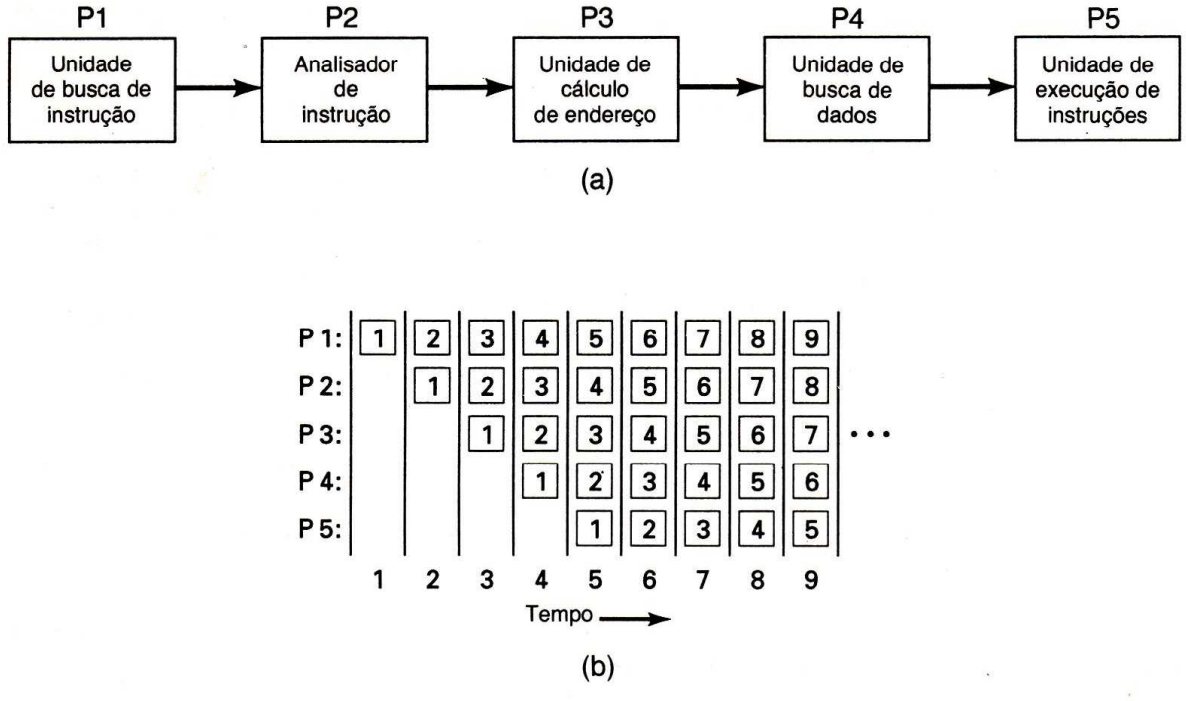


Fig. 2.5 (a) Uma máquina *pipeline* de cinco estágios. (b) O estado de cada unidade de processamento em função do tempo.

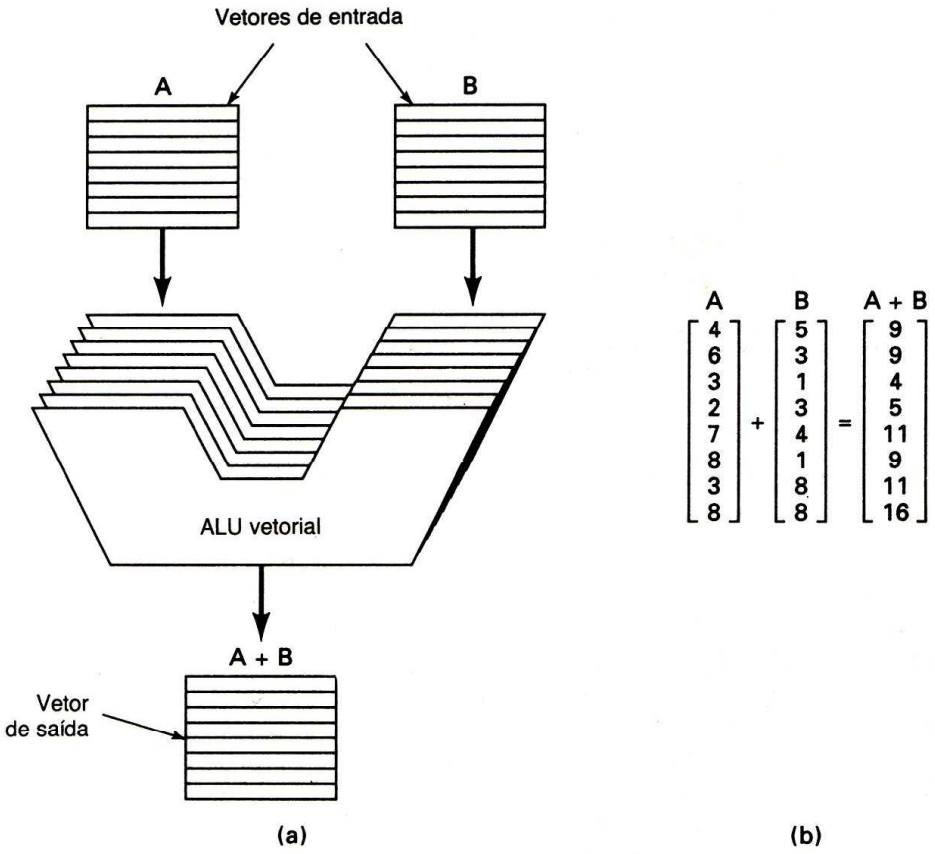


Fig. 2.6 (a) Uma ALU vetorial. (b) Um exemplo de soma de vetores.

para cada entrada da ALU, temos um vetor com n entradas. Similarmente, a ALU é na realidade uma ALU vetorial, capaz de realizar uma operação, tal como uma adição vetorial [mostrada na Fig. 2.6(b)], em dois vetores de entrada e produzindo um vetor de saída como resultado. Alguns supercomputadores têm arquitetura similar a esta.

Uma abordagem alternativa ao SIMD é o ***array processor***, cujo projeto pioneiro da Universidade de Illinois, o computador ILLIAC IV, é ilustrado na Fig. 2.7 (Hord, 1982). Esta arquitetura

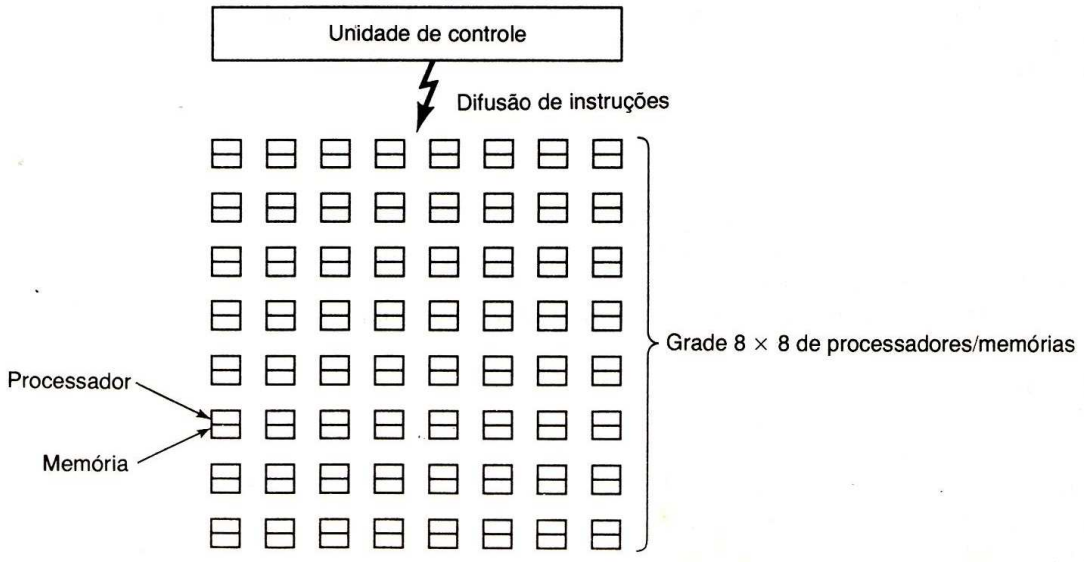


Fig. 2.7 Um *array processor* do tipo do ILLIAC IV.

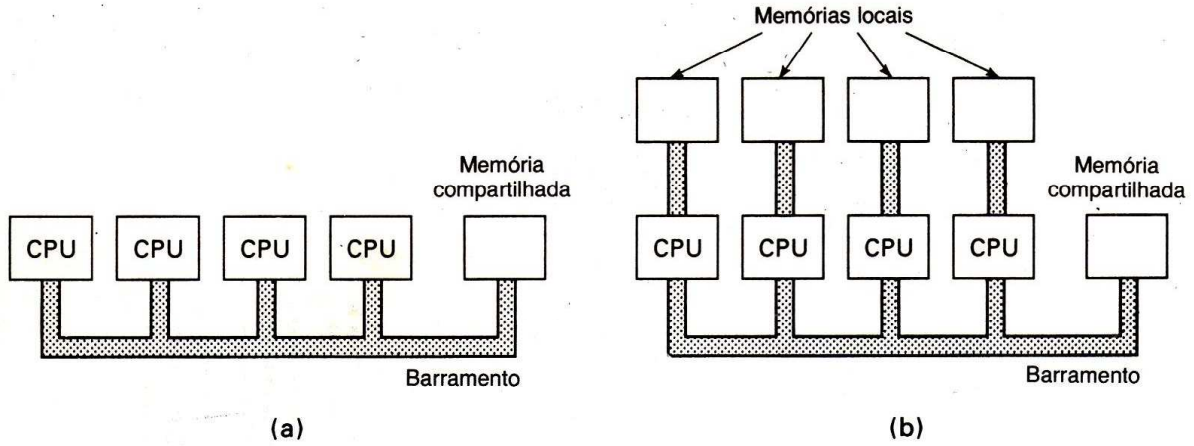


Fig. 2.8 (a) Um sistema multiprocessador básico. (b) Um sistema multiprocessador com memórias locais.

consiste em uma grade quadrada de processadores e memória. Uma única unidade de controle difunde as instruções, que são executadas paralelamente por todos os processadores, cada qual usando seu próprio dado de sua própria memória (carregada durante a fase de inicialização). O *array processor* é especialmente apropriado para cálculos matriciais.

A terceira categoria de Flynn é a MIMD, na qual CPUs diferentes executam programas diferentes, às vezes compartilhando alguma memória comum. Por exemplo, no sistema de reserva de passagens aéreas, reservas simultâneas múltiplas não prosseguem em paralelo, instrução por instrução, e assim temos fluxo múltiplo de instrução e fluxo múltiplo de dados. A Fig. 2.8(a) mostra um **sistema multiprocessador**, uma máquina MIMD usando memória compartilhada. Cada processador tem acesso à memória compartilhada através do barramento.

Não é necessária muita imaginação para perceber que um grande número de processadores rápidos tentando constantemente acessar a memória através do mesmo barramento irá resultar em conflitos. Projetistas de sistemas multiprocessadores trouxeram vários esquemas para reduzir esta contenção e aumentar o desempenho. Um projeto, mostrado na Fig. 2.8(b), dá a cada processador sua própria memória local, não acessível aos outros. Esta memória pode ser usada para o código do programa e aqueles dados que não precisam ser compartilhados. O acesso a esta memória privada não usa o barramento principal, reduzindo significativamente o tráfego no barramento.

Outros sistemas multiprocessadores usam não apenas um barramento, mas vários para reduzir a carga. Outros usam ainda uma técnica chamada ***cache***, que consiste em manter as palavras de memória freqüentemente referidas dentro de cada processador. O processo *cache* será discutido em detalhe no capítulo 4. Uma visão geral das arquiteturas multiprocessadoras é mostrada em Gajski e Pier, 1985.

2.2 MEMÓRIA

A **memória** é a parte do computador onde programas e dados são armazenados. Alguns cientistas da computação (especialmente os britânicos) usam o termo **armazenamento**, em vez de memória. Sem uma memória de onde os processadores podem ler e escrever informações, não haveria nenhum computador digital de programa armazenado.

2.2.1 Bits

A unidade básica de memória é o dígito binário, chamado **bit**. Um bit pode conter um 0 ou um 1. Ele é a unidade mais simples possível. (Um dispositivo capaz de armazenar somente zeros dificilmente poderia constituir a base de um sistema de memória. São necessários pelo menos dois valores.)

As pessoas freqüentemente dizem que os computadores utilizam aritmética binária porque

ela é "eficiente". O que elas querem dizer (entretanto, elas raramente o percebem) é que a informação digital pode ser armazenada através da distinção entre valores diferentes de alguma grandeza física contínua, tal como tensão ou corrente. Quanto mais valores devem ser distinguidos, menor é a separação entre valores adjacentes e menor é a confiabilidade da memória. O sistema de numeração binário requer apenas que dois valores sejam distintos. Conseqüentemente, ele é o método mais confiável para a codificação da informação digital. Se você não estiver familiarizado com números binários, veja o apêndice A.

Alguns computadores, como os grandes *mainframes* IBM, são divulgados como tendo tanto aritmética decimal quanto binária. Este truque é obtido utilizando quatro bits para armazenar um dígito decimal. Quatro bits provêem 16 combinações, sendo 10 para os dígitos de 0 a 9 e seis não utilizadas. O número 1944 é mostrado abaixo codificado em decimal e em binário puro, utilizando 16 bits em cada exemplo:

decimal: 0001 1001 0100 0100 binário: 0000011110011000

Dezesseis bits no formato decimal podem armazenar os números de 0 a 9999, dando apenas 10000 combinações, enquanto um número binário puro de 16 bits pode armazenar 65536 combinações diferentes. Por esta razão, as pessoas dizem que a forma binária é mais eficiente.

Considere, entretanto, o que aconteceria se algum jovem e brilhante eletricista inventasse um dispositivo eletrônico altamente confiável que pudesse armazenar diretamente os dígitos de 0 a 9, dividindo a região de 0 a 10 volts em 10 partes. Quatro destes dispositivos poderiam armazenar qualquer número decimal de 0 a 9999. Estes quatro dispositivos proveriam 10000 combinações. Eles poderiam ser usados para armazenar números binários usando apenas 0 e 1, e neste caso quatro deles poderiam armazenar apenas 16 combinações. Com tais dispositivos, o sistema decimal é obviamente mais eficiente.

2.2.2 Endereços de Memória

As memórias são compostas de um determinado número de **células** (ou **posições**), cada uma podendo armazenar uma parte da informação. Cada célula tem um número, chamado de seu **endereço,** pelo qual os programas podem referenciá-la. Se a memória tem n células, elas terão

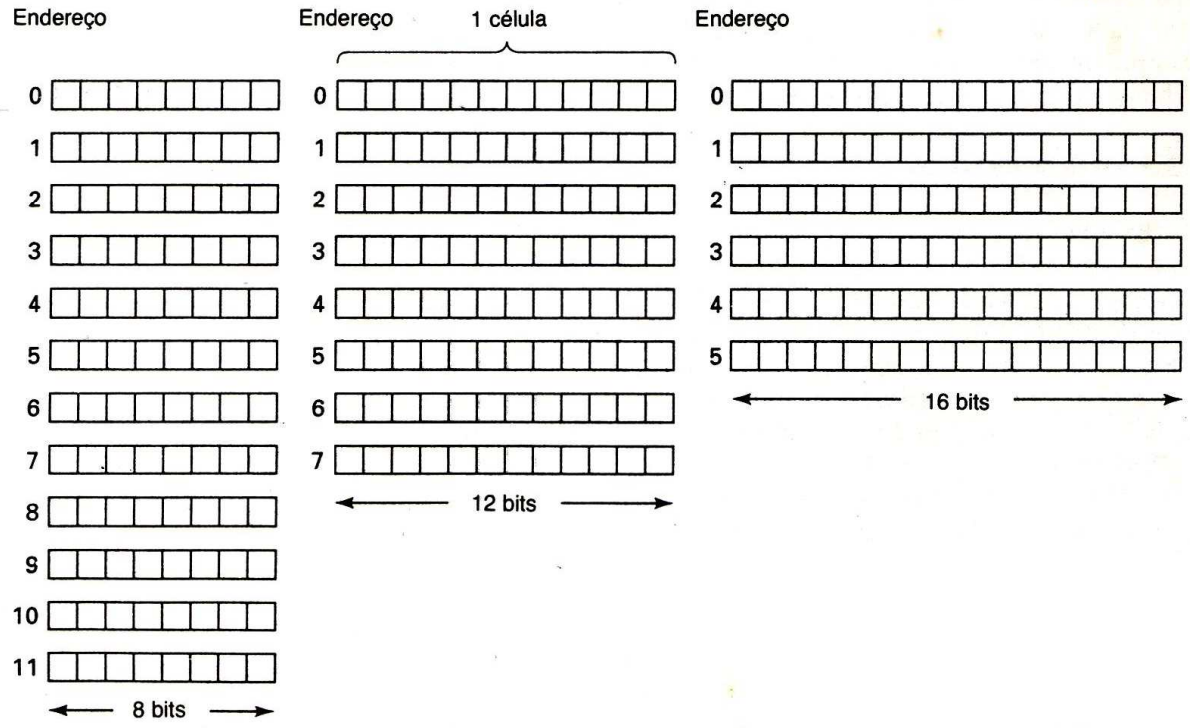


Fig. 2.9 Três formas de organizar uma memória de 96 bits.

endereços de 0 a n-1. Todas as células em uma memória possuem o mesmo número de bits. Se uma célula consiste em k bits, ela pode conter uma em 2^k diferentes combinações de bits. A Fig. 2.9 mostra três organizações diferentes para uma memória de 96 bits. Note que células adjacentes têm endereços consecutivos (por definição).

Os computadores que usam o sistema de numeração binário (incluindo aí a notação octal e a hexadecimal para números binários) também expressam os endereços de memória como números binários. Se um endereço possui m bits, o número máximo de células diretamente endereçáveis é 2^m. Por exemplo, o endereço usado para referenciar a memória da Fig. 2.9(a) necessitaria de no mínimo 4 bits para expressar um número de 0 a 11. Um endereço de três bits, entretanto, seria suficiente para as Figs. 2.9(b) e (c). O número de bits no endereço está relacionado ao número máximo de células endereçáveis, e é independente do número de bits por célula. Uma memória com 2^{12} células de 8 bits cada e uma memória de 2^{12} células de 60 bits cada necessitariam de um endereço de 12 bits.

O número de bits por célula para alguns computadores vendidos comercialmente é dado a seguir:

Burroughs B1700:	1 bit por célula
IBM PC:	8 bits por célula
DEC PDP-8:	12 bits por célula
IBM 1130:	16 bits por célula
DEC PDP-15:	18 bits por célula
XDS 940:	24 bits por célula
Electrologica X8:	27 bits por célula
XDS Sigma 9:	32 bits por célula
Honeywell 6180:	36 bits por célula
CDC 3600:	48 bits por célula
CDC Cyber:	60 bits por célula

O significado da célula é que ela é a menor unidade endereçável. Em anos recentes, a maioria dos fabricantes de computador padronizaram a célula de 8 bits, que é chamada **byte**. Bytes são agrupados em **palavras**. Um computador com uma palavra de 16 bits tem 2 bytes/palavra, enquanto que um computador com palavra de 32 bits tem 4 bytes/palavra. O significado de uma palavra é que a maioria das instruções operam em palavras inteiras, por exemplo somando duas palavras. Então uma máquina de 16 bits terá registradores de 16 bits e instruções que manipulam palavras de 16 bits, enquanto que uma máquina de 32 bits terá registradores de 32 bits e instruções para mover, somar, subtrair e outras que manipulam palavras de 32 bits.

2.2.3 Ordem de Bytes

Os bytes em uma palavra podem ser numerados da esquerda para a direita ou da direita para a esquerda. A princípio, parece que esta escolha não é importante, mas, como veremos brevemente, tem maiores implicações. A Fig. 2.10(a) mostra parte de uma memória de um computador de 32 bits cujos bytes são numerados da esquerda para a direita, como na família Motorola. A Fig. 2.10(b) mostra a representação análoga para um computador de 32 bits usando numeração da direita para a esquerda, como na família Intel. O primeiro sistema, onde a numeração começa no lado de alta ordem ("*big*"), é chamado de computador **"big endian"**, em contraste com o **"little endian"** da Fig. 2.10(b). Estes termos são devidos a Jonathan Swift, cujo livro *Aventuras de Gulliver* satirizou políticos que fizeram guerra por causa da disputa sobre se os ovos deviam ser quebrados do lado largo (*big end*) ou do lado estreito (*little end*). O termo foi usado pela primeira vez em arquitetura de computadores em um encantador artigo de Cohen (1981).

É importante entender que em ambos os sistemas um inteiro de 32 bits com o valor numérico de, digamos, 6 é representado pelos bits 110 nos três bits mais à direita (baixa ordem) da palavra e zeros nos 29 bits da esquerda. No esquema "*big endian*", esses bits estão no byte 3 (ou 7, ou 11, etc.), enquanto que no esquema "*little endian*" eles estão no byte 0 (ou 4, ou 8, etc.). Em ambos os casos, a palavra contendo este inteiro tem endereço 0. Se os computadores armazenassem apenas inteiros, não haveria nenhum problema. Entretanto, muitas aplicações requerem

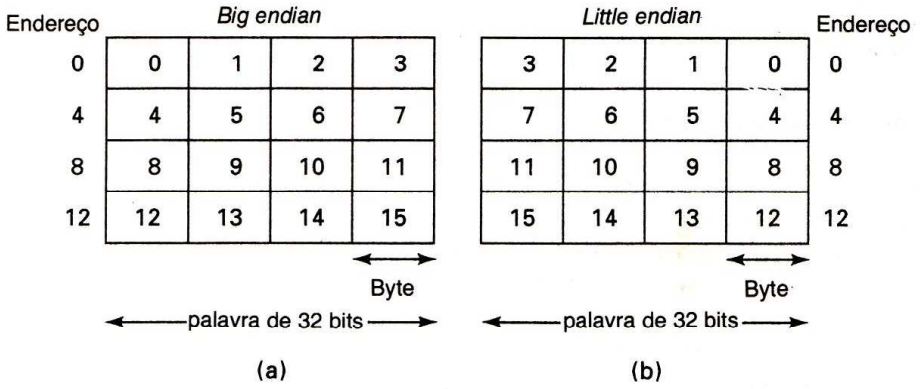


Fig. 2.10 (a) Memória *big endian*. (b) Memória *little endian*.

uma mistura de inteiros, cadeias de caracteres e outros tipos de dados. Considere, por exemplo, um simples registro pessoal consistindo em uma cadeia de caracteres (nome do empregado) e dois inteiros (idade e número do departamento). A cadeia de caracteres é terminada com um ou mais bytes contendo 0 para encher uma palavra. A representação "*big endian*" é mostrada na Fig. 2.11(a) e a representação "*little endian*" é mostrada na Fig. 2.11(b), para Jim Smith, 21 anos, departamento 260 ($1 \times 256 + 4 = 260$).

Ambas as representações são boas e internamente consistentes. Os problemas começam quando uma das máquinas tenta enviar o registro para a outra através de uma rede. Vamos assumir que o "*big endian*" envia o registro para o "*little endian*", um byte por vez, começando no byte 0 e terminando no byte 19. (Seremos otimistas e assumiremos que os bits dos bytes não estão invertidos pela transmissão, já que temos problemas suficientes do jeito que está.) Então o byte 0 do "*big endian*" vai para o byte 0 da memória do "*little endian*", e assim por diante, como mostrado na Fig. 2.11(c).

Quando o "*little endian*" tenta imprimir o nome, isso funciona, mas a idade aparece como 21×2^{24} e o departamento está tão confuso quanto. Esta situação ocorre porque a transmissão inverteu a ordem dos caracteres em uma palavra, como devia, mas também inverteu os bytes do inteiro, o que não podia. Uma solução óbvia é fazer o *software* inverter os bytes dentro de uma palavra após a cópia ter sido feita. Isto leva à Fig. 2.11(d), que acerta os dois inteiros mas converte a cadeia de caracteres para "MIJTIMS", com o "H" perdido no meio de nada. Esta inversão da cadeia de caracteres ocorre porque, quando está lendo a cadeia, o computador lê primeiro o byte 0 (um espaço), então o byte 1 (M), e assim por diante.

Não existe solução simples. Uma forma que funciona, mas é ineficiente, é incluir um cabeçalho em cada registro dizendo que tipo de dados se segue (cadeia de caracteres, inteiro ou outro) e qual o seu tamanho. Isto permite ao receptor realizar apenas as conversões necessárias. De

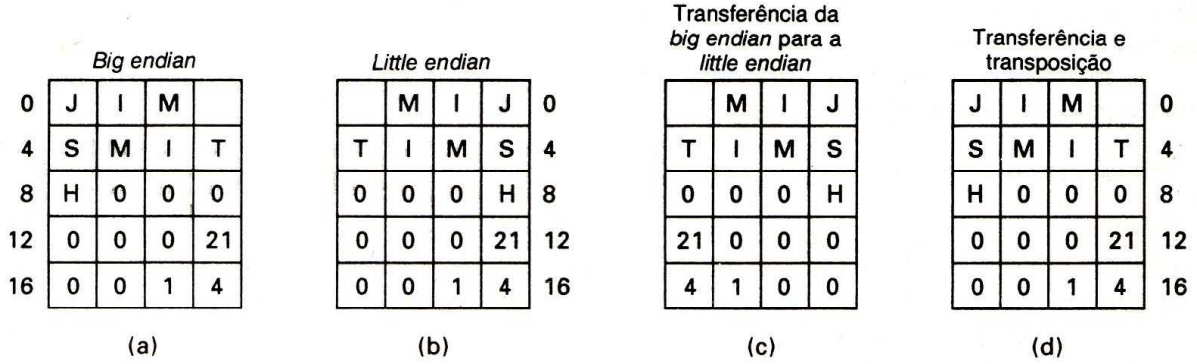


Fig. 2.11 (a) Um registro pessoal para uma máquina *big endian*. (2) O mesmo registro para uma máquina *little endian*. (c) O resultado da transferência do registro da *big endian* para a *little endian*. (d) O resultado de transposição de bytes de (c).

qualquer jeito, deve ficar claro que a falta de um padrão para a ordem dos bytes é um grande problema na troca de dados entre máquinas diferentes.

2.2.4 Códigos de Correção de Erros

As memórias dos computadores podem cometer erros ocasionalmente devido a picos de tensão na rede elétrica, entre outras causas. Para evitar tais erros, a maioria das memórias usa códigos de detecção e correção de erros. Quando estes códigos são utilizados, são adicionados bits extras a cada palavra da memória, de uma forma especial. Quando uma palavra é lida da memória, os bits extras são verificados para saber se ocorreu algum erro.

Para entender como os erros podem ser tratados, é necessário analisar o que é realmente um erro. Suponha que uma palavra de memória tenha m bits de dados aos quais adicionaremos r bits e redundância ou verificação. Seja o tamanho total igual a n (i. é, $n = m + r$). Uma unidade de n bits contendo m bits de dados e r de verificação é freqüentemente chamada de **palavra de código** de n bits.

Dadas duas palavras de código quaisquer, digamos, 10001001 e 10110001, é possível determinar quantos bits correspondentes diferem. Neste caso, diferem em 3 bits. Para determinar quantos bits diferem, simplesmente calcule o OU EXCLUSIVO booleano das duas palavras-código e conte o número de bits em 1 do resultado. O número de posições nas quais duas palavras de código diferem é chamado de **distância Hamming** (Hamming, 1950). Seu significado é que se duas palavras-código estão a uma distância Hamming d, serão necessários d erros simples para converter uma na outra. Por exemplo, as palavras código 11110001 e 00110000 estão a uma distância Hamming 3 porque são necessários 3 erros simples para converter uma na outra.

Com uma palavra de memória de m bits, todos os 2^m padrões de bits são válidos, mas devido ao modo que os bits de validação são calculados, somente 2^m das 2^n palavras-código são válidas. Se a leitura da memória retorna uma palavra-código inválida, o computador sabe que ocorreu um erro na memória. Dado o algoritmo para calcular os bits de verificação, é possível construir uma lista completa das palavras-código válidas, e, a partir desta lista, encontrar duas palavras-código cuja distância Hamming seja mínima. Esta é a distância Hamming do código completo.

As propriedades de detecção e correção de erro de um código dependem de sua distância Hamming. Para detectar d erros simples, você precisa de um código de distância $d + 1$, porque com tal código não é possível converter uma palavra-código válida em outra com d erros simples. Similarmente, para corrigir d erros simples você precisa de um código de distância $2d + 1$, porque assim as palavras-código estão tão distantes que mesmo com d mudanças a palavra original está ainda mais próxima que qualquer outra, e assim pode ser unicamente determinada.

Como exemplo simples de um código de detecção de erro, considere um código no qual um único **bit de paridade** é adicionado aos dados. O bit de paridade é escolhido para que o número de bits em 1 na palavra-código seja par (ou ímpar). Tal código tem distância 2, e qualquer erro em um bit produz uma palavra-código com paridade errada. Isto pode ser usado para detectar erros de um bit. Sempre que uma palavra com paridade errada é lida da memória, é sinalizada uma condição de erro e tomada uma ação especial. O programa não pode continuar, mas, pelo menos, nenhum resultado incorreto é calculado.

Como exemplo simples de um código de correção de erro, considere um código com apenas quatro palavras-código válidas:

0000000000, 0000011111, 1111100000 e 1111111111

Este código tem distância 5, o que significa que ele pode corrigir erros duplos. Se chega a palavra 0000000111, o receptor sabe que a palavra original deve ser 0000011111 (se não houve mais que dois erros). Se, entretanto, um erro triplo converte 0000000000 em 0000000111, o erro não será corrigido corretamente.

Imagine que desejamos projetar um código com m bits de dados e r bits de verificação que irá permitir a correção de todos os erros simples de um bit. Cada uma das 2^m palavras válidas de memória possui n palavras-código inválidas a uma distância 1 dela. Estas são produzidas invertendo sistematicamente cada um dos n bits da palavra-código de n bits. Então cada uma

Tamanho da palavra	Bits de verificação	Tamanho total	Percentual de *overhead*
8	4	12	50
16	5	21	31
32	6	38	19
64	7	71	11
128	8	136	6
256	9	265	4
512	10	522	2

Fig. 2.12 O número de bits de verificação necessários para o código de correção de erros simples.

das 2^m palavras de memória válidas requer $n + 1$ padrões dedicados a ela. Como o número total de padrões de bits é 2^n, devemos ter $(n + 1)2^m \leq 2^n$. Usando $n = m + r$, temos $(m + r + 1) \leq 2^r$. Dado m, isto fornece o limite inferior do número de bits de verificação necessários para corrigir erros em um bit. A Fig. 2.12 mostra o número de bits de verificação necessários para vários tamanhos de palavras de memória.

Este limite inferior teórico pode, na realidade, ser obtido usando um método devido a Richard Hamming (1950). Antes de dar o algoritmo de Hamming, analisemos uma representação gráfica simples que ilustra a idéia de um código de correção de erro para palavras de memória de 4 bits. O diagrama de Venn da Fig. 2.13(a) contém três círculos, A, B e C, que juntos formam sete regiões. Como exemplo, codifiquemos a palavra de memória de 4 bits 1100 nas regiões AB, ABC, AC, e BC, um bit por região (em ordem alfabética). Esta codificação é mostrada na Fig. 2.13(a).

A seguir adicionamos um bit de paridade a cada uma das três regiões vazias para produzir paridade par, como ilustrado na Fig. 2.13(b). Por definição, a soma dos bits nos três círculos, A, B e C, é agora um número par. No círculo A, temos os quatro números 0, 0, 1 e 1, que somam 2, um número par. No círculo B, os números são 1, 1, 0 e 0, que também somam 2, um número par. Finalmente, no círculo C, temos a mesma coisa. Neste exemplo todos os círculos são iguais, mas somas de 0 e 4 podem ser obtidas em outros exemplos. Esta figura corresponde a uma palavra-código de 4 bits de dados e 3 de paridade.

Suponha agora que o bit na região AC mude de 0 para 1, como mostrado na Fig. 2.13(c). O computador pode agora ver que os círculos A e C têm paridade errada (ímpar). A única mudança de um bit que os corrige é restaurar AC de volta para 0, corrigindo o erro. Desta forma, o computador pode corrigir automaticamente erros simples em um bit.

Vejamos agora como o algoritmo de Hamming pode ser usado para construir códigos de correção de erro para qualquer tamanho de palavra de memória. Em um código Hamming, r bits de paridade são adicionados a uma palavra de m bits, formando uma nova palavra de $m + r$ bits. Os bits são numerados começando de 1, e não 0, sendo o bit 1 o mais à esquerda

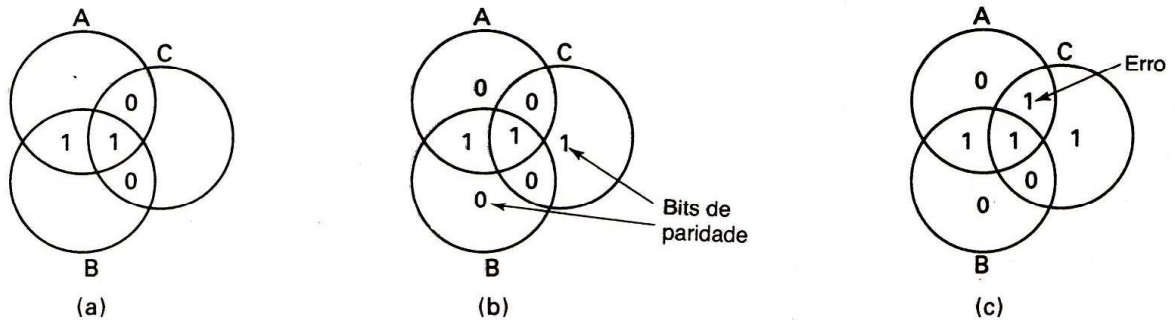


Fig. 2.13 (a) Codificação de 1100. (b) Adição de paridade par. (c) Erro em AC.

(de mais alta ordem). Todos os bits cujo número é uma potência de 2 são bits de paridade; os outros são de dados. Por exemplo, com uma palavra de 16 bits, são adicionados 5 bits de paridade. Os bits 1, 2, 4, 8 e 16 são de paridade, e os outros são de dados. Ao todo, a palavra de memória tem 21 bits (16 de dados e 5 de paridade). Usaremos (arbitrariamente) paridade par neste exemplo.

Cada bit de paridade verifica posições específicas de bit; o bit de paridade é estabelecido de forma que o número total de 1s nas posições verificadas seja par. As posições verificadas pelos bits de paridade são:

Bit 1 verifica os bits 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 e 21.

Bit 2 verifica os bits 2, 3, 6, 7, 10, 11, 14, 15, 18 e 19.

Bit 4 verifica os bits 4, 5, 6, 7, 12, 13, 14, 15, 20 e 21.

Bit 8 verifica os bits 8, 9, 10, 11, 12, 13, 14 e 15.

Bit 16 verifica os bits 16, 17, 18, 19, 20 e 21.

De forma geral, o bit b é verificado pelos bits b_1, b_2, ..., b_j, tal que $b_1 + b_2 + \ldots + b_j = b$. Por exemplo, o bit 5 é verificado pelos bits 1 e 4 porque 1 + 4 = 5. O bit 6 é verificado pelos bits 2 e 4 porque 2 + 4 = 6, e assim por diante.

A Fig. 2.14 mostra a construção de um código de Hamming para a palavra de memória de 16 bits 1111000010101110. A palavra código de 21 bits é 001011100000101101110. Para ver como a correção de erro funciona, considere o que aconteceria se o bit 5 fosse invertido por um ruído na fonte de alimentação. A nova palavra-código seria 001001100000101101110 em vez de 001011100000101101110. Os 5 bits de paridade são verificados, com os seguintes resultados:

Bit de paridade 1 incorreto (1, 3, 5, 7, 9, 11, 13, 15, 17, 19 e 21 contêm cinco 1s).

Bit de paridade 2 correto (2, 3, 6, 7, 10, 11, 14, 15, 18 e 19 contêm seis 1s).

Bit de paridade 4 incorreto (4, 5, 6, 7, 12, 13, 14, 15, 20 e 21 contêm cinco 1s).

Bit de paridade 8 correto (8, 9, 10, 11, 12, 13, 14 e 15 contêm dois 1s).

Bit de paridade 16 correto (16, 17, 18, 19, 20 e 21 contêm quatro 1s).

O número total de bits em 1 nos bits 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 e 21 deveria ser par porque está sendo utilizada paridade par. O bit incorreto deve ser um dos bits verificados pelo bit de paridade 1 — isto é, bit 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 ou 21. O bit de paridade 4 está errado, o que significa que um dos bits 4, 5, 6, 7, 12, 13, 14, 15, 20 ou 21 está errado. O erro precisa estar em um dos bits comuns a ambas as listas, isto é, 5, 7, 13, 15 ou 21. Entretanto, o bit 2 está correto, eliminando os bits 7 e 15. Similarmente, o bit 8 está correto, eliminando

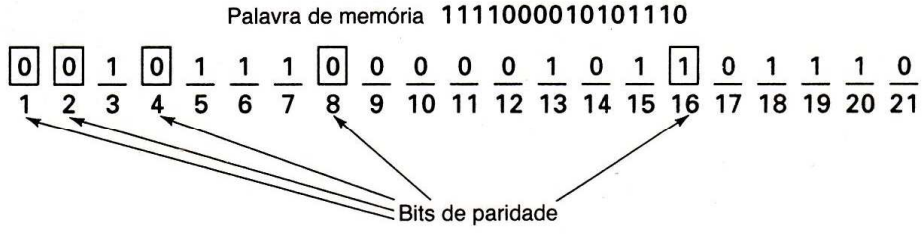


Fig. 2.14 Construção do código de Hamming para a palavra da memória 1111000010101110 com a adição de 5 bits para verificação aos 16 bits de dados.

o bit 13. Finalmente o bit 16 está correto, eliminando o bit 21. O único bit restante é o bit 5, que é o bit com erro. Como ele foi lido como 1, ele deve ser 0. Desta forma os erros podem ser corrigidos.

Um método simples para encontrar o bit incorreto é primeiro calcular todos os bits de paridade. Se todos estiverem corretos, não houve erro (ou houve mais de um). Então somam-se todos os bits de paridade incorretos, considerando 1 para o bit 1, 2 para o bit 2, 4 para o bit 4, e assim por diante. A soma resultante é a posição do bit incorreto. Por exemplo, se os bits de paridade 1 e 4 estão incorretos, mas os bits 2, 8 e 16 estão corretos, o bit 5 (1 + 4) está invertido.

2.2.5 Memória Secundária

Como toda palavra na memória principal precisa ser diretamente acessível em um intervalo de tempo muito curto, ela é relativamente cara. Conseqüentemente, a maioria dos computadores possui também memórias secundárias mais lentas, baratas e muito grandes. As memória secundárias são utilizadas para armazenar grande conjunto de dados, que a memória principal não suporta.

Fitas Magnéticas

Historicamente, a **fita magnética** foi o primeiro tipo de memória secundária. Uma unidade de fita de computador é análoga a um gravador de rolo convencional: uma fita de 2400 pés de comprimento é enrolada em um rolo alimentador, passa por uma cabeça de gravação e chega a um rolo receptor. Variando a corrente na cabeça de gravação, o computador pode gravar informações na fita em forma de pequenos pontos magnetizados.

A Fig. 2.15 mostra como a informação é organizada em uma fita magnética. Em um computador com bytes de 8 bits, cada *frame* contém 1 byte, além de um bit extra, redundante, chamado de bit de paridade, para aumentar a confiabilidade. Uma densidade típica de gravação é de 1600 *frames* (bytes) por polegada (representada por 1600 bpi), o que significa que a distância entre os *frames* é menor que 1/1000 da polegada. Outras densidades comuns são 800 e 6250 bpi. Após a unidade de fita terminar a gravação de um **registro físico** (uma seqüência de *frames*), ela deixa um espaço na fita, enquanto reduz a velocidade. Se o programa escreve pequenos registros físicos na fita, a maior parte da fita será gasta por esses espaços, denominados ***gaps***. A utilização da fita pode ser mantida alta gravando-se registros físicos bem maiores que o *gap*.

As fitas magnéticas são dispositivos de acesso seqüencial. Se a fita está posicionada no início, para ler o registro físico n é necessário ler antes os registros físicos de 1 a $n - 1$, um de cada vez. Se a informação desejada se encontra próxima ao fim da fita, o programa terá que ler quase a fita inteira, o que poderá levar vários minutos. Forçar a CPU, que pode executar milhões de instruções por segundo, a esperar 200 segundos enquanto a fita avança, é um desperdício. As fitas são mais adequadas quando os dados precisam ser acessados seqüencialmente.

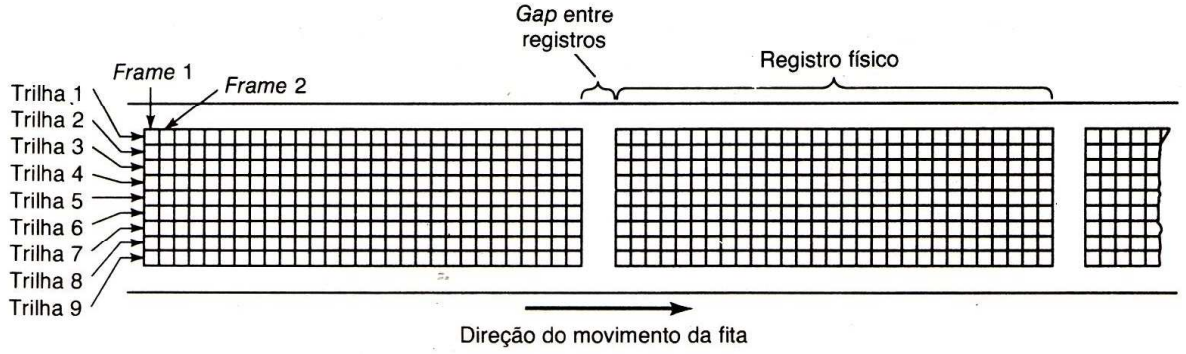


Fig. 2.15 A informação é gravada na fita magnética como uma seqüência de matrizes retangulares de bits.

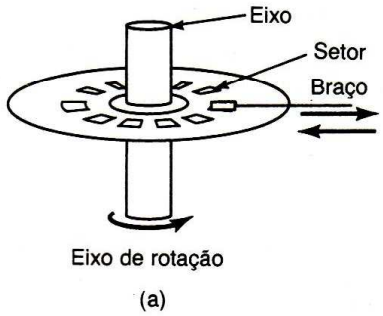
Discos Magnéticos

Um **disco** é um pedaço de metal, com diâmetro entre 5 e 10 polegadas, ao qual uma cobertura magnetizável foi aplicada no processo de fabricação, geralmente de ambos os lados. A informação é gravada em um certo número de círculos concêntricos chamados **trilhas** (veja a Fig. 2.16). Os discos possuem tipicamente entre 40 e algumas centenas de trilhas por superfície. Cada unidade de disco possui uma cabeça móvel que pode aproximar-se ou afastar-se do centro do disco. A cabeça é suficientemente pequena para ler ou gravar informações exatamente em uma trilha. A unidade de disco possui freqüentemente vários discos empilhados verticalmente, com uma separação de cerca de uma polegada. Em tal configuração, o braço terá uma cabeça próxima a cada superfície, e todas se movem juntas. A posição radial das cabeças (distância do eixo) é chamada **cilindro**. Uma unidade de disco com n pratos (e $2n$ faces) terá $2n$ cabeças, e portanto $2n$ trilhas por cilindro.

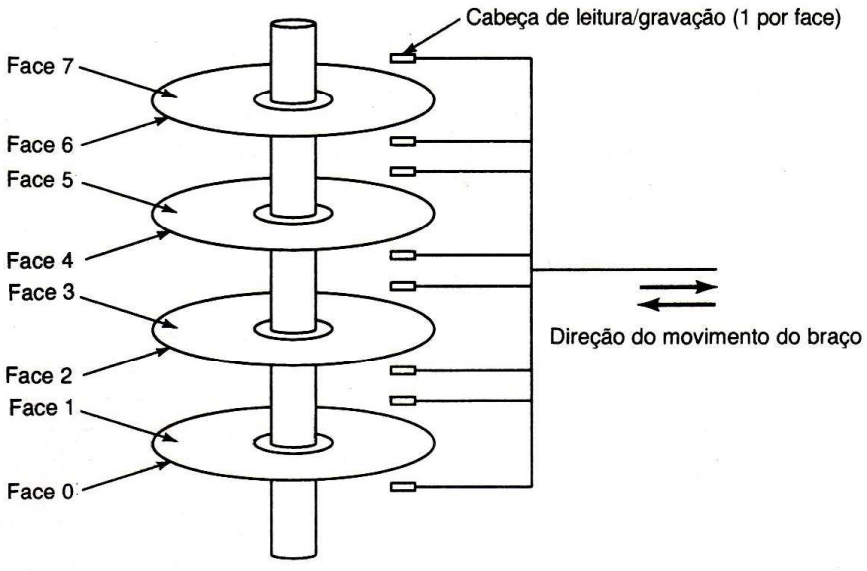
As trilhas são divididas em **setores**, e há normalmente entre 10 e 100 setores por trilha. Um setor consiste em um certo número de bytes, tipicamente 512.

Para especificar uma transferência, o programa deve fornecer as seguintes informações: o cilindro e a cabeça (ou face), que juntos definem uma única trilha, o número do setor onde começa a informação, o número de palavras a serem transferidas, o endereço na memória principal de onde a informação vem ou para onde ela vai, e se a informação é para ser lida do disco ou se é para ser escrita nele.

As transferências de informação no disco sempre começam no início de um setor, nunca



(a)



(b)

Fig. 2.16 (a) Um disco com um prato. (b) Um disco com quatro pratos.

no meio. Se uma transferência de múltiplos setores ultrapassa o limite de uma trilha dentro de um cilindro (por exemplo, da face 0 para a face 1 na mesma posição do braço), nenhum tempo é perdido porque a mudança de uma cabeça para outra é feita eletronicamente. Entretanto, se a transferência atravessa o limite de um cilindro, o tempo de uma rotação pode ser perdido no reposicionamento das cabeças no próximo cilindro e na espera pelo setor 0.

Se a cabeça está sobre um cilindro errado, ela precisa primeiramente ser movida. Este movimento é chamado **seek** (procura). Um *seek* gasta tipicamente 3 ms entre trilhas adjacentes, e de 20 a 100 ms para ir do cilindro mais interno para o mais externo. Uma vez que a cabeça está posicionada corretamente, o controlador precisa esperar até que o setor inicial passe sob a cabeça, antes de iniciar a transferência. O tempo gasto esperando pelo setor correto varia de 0, se o programa tiver sorte, até o tempo de uma rotação completa, se o setor tiver acabado de passar. Este tempo gasto é chamado **latência rotacional**. A maioria dos discos gira a 3600 rpm, dando uma latência máxima de 16,67 ms. O tempo total de acesso é igual ao tempo de *seek* mais a latência rotacional, mais o tempo de transferência. A informação é transferida a uma taxa de uma trilha por período de rotação.

Quase todos os computadores utilizam discos com vários pratos, como descrito anteriormente para armazenamento de dados. Eles são freqüentemente denominados **discos rígidos**. O tipo mais comum é o **disco winchester**, que é uma unidade selada (para evitar a contaminação com poeira). As cabeças em uma unidade winchester têm forma aerodinâmica e flutuam em um colchão de ar gerado pela rotação dos pratos. Suas capacidades variam de cerca de 20 megabytes em computadores pessoais a cerca de 10 gigabytes em grandes *mainframes*.

Discos Flexíveis

Com o advento dos computadores pessoais, foi necessária uma forma de distribuir *software*. A solução foi encontrada no **disquete** ou **disco flexível** (ou ainda *floppy disk*), um meio pequeno e removível, assim chamado porque os primeiros eram fisicamente flexíveis. O disquete foi realmente inventado pela IBM para gravar informações de manutenção nos *mainframes* para a equipe de serviço, mas foi rapidamente empregado pelos fabricantes de computadores pessoais como um meio conveniente para distribuir *software*.

Diferentemente dos discos winchester, onde as cabeças flutuam a poucos mícrons da superfície, as cabeças das unidades de disquetes realmente tocam os disquetes. Como resultado, tanto o meio quanto as cabeças desgastam comparativamente mais rápido. Para reduzir o desgaste, os computadores pessoais recolhem as cabeças e param a rotação enquanto a unidade não está lendo ou escrevendo. Conseqüentemente, quando o próximo comando de leitura ou escrita é dado, há um atraso de cerca de meio segundo enquanto o motor acelera.

Hoje em dia, dois tamanhos são bastante usados: 5 1/4 e 3 1/2 polegadas. Cada um destes tem sua versão de baixa e alta densidade. Os disquetes de 3 1/2 polegadas vêm em jaquetas rígidas para proteção, assim eles não são realmente "flexíveis". Como os discos de 3 1/2 polegadas armazenam mais dados e são mais bem protegidos, eles irão eventualmente substituir os de 5 1/4. Os parâmetros mais importantes dos quatro tipos são mostrados na Fig. 2.17.

Tamanho (polegadas)	5,25	5,25	3,5	3,5
Capacidade (bytes)	360K	1,2M	720K	1,44M
Trilhas	40	80	80	80
Sectores/trilha	9	15	9	18
Cabeças	2	2	2	2
Rotações/minuto	300	360	300	300
Taxa de transferência (kbps)	250	500	250	500
Tipo	Flexível	Flexível	Rígido	Rígido

Fig. 2.17 Comparação entre os quatro tipos comuns de disquete.

Discos Óticos

Nos últimos anos, discos óticos (em oposição aos magnéticos) tornaram-se disponíveis. Eles possuem densidades de gravação muito maiores que os discos magnéticos convencionais. Os discos óticos foram inicialmente desenvolvidos para gravar programas de televisão, mas eles podem ser submetidos a uma utilização mais estética como dispositivos de armazenamento de computadores. A televisão tem uma banda passante de 6 MHz, logo um disco de 1 hora tem a capacidade teórica de cerca de 10 gigabits. Na prática, os sistemas conseguem metade disto. (**Banda passante** é um termo de engenharia elétrica e se refere à capacidade de transferência de informação de um fio ou outro canal. Uma banda passante de 1 Hertz (Hz) é tipicamente, mas nem sempre, boa para a transmissão de um bit por segundo.)

Devido à sua enorme capacidade potencial, os discos óticos têm sido objeto de grande pesquisa e estão sofrendo uma evolução incrivelmente rápida. A primeira geração foi inventada pelo conglomerado eletrônico holandês Philips, e mais tarde desenvolvido em colaboração com a Sony. Esses discos se baseiam na mesma tecnologia dos *compact discs* usados em áudio, e são chamados **CD ROMs (Compact Disk Read Only Memory)**.

Um CD ROM é preparado utilizando um *laser* de alta potência para queimar furos de 1 mícron (10^{-6} metro) em um disco matriz. Desta matriz é feito um molde. Este molde é usado para estampar cópias em discos plásticos, do mesmo jeito que os discos fonográficos são feitos. Uma fina camada de alumínio é então depositada na superfície, seguida por uma camada plástica transparente para proteção. Os CD ROMs são lidos por dispositivos similares aos toca-discos CD, possuindo um detector que mede a energia refletida quando um *laser* de baixa potência incide na superfície. Os furos, chamados **pits**, e as áreas não queimadas entre os *pits*, chamadas **lands**, têm diferente refletividade, tornando possível a distinção entre *pits* e *lands*.

Esta tecnologia tem algumas conseqüências importantes. Inicialmente, como os CD ROMs são estampados ao invés de gravados como os disquetes convencionais, um maquinário totalmente automatizado pode produzi-los em massa a um custo muito baixo. Além disso, como o processo de estampar discos plásticos cobertos de alumínio não é muito preciso, a informação digital normalmente contém muitos erros.

O problema dos erros é atacado de duas formas. Primeiro, a cabeça de leitura da unidade contém um espelho de precisão acionado por um servomecanismo que é usado para rastrear a superfície, compensando as imperfeições de fabricação. Segundo, os dados são gravados em um método complexo chamado código de correção de erro **Reed-Solomon entrelaçado (crossinterleaved)**. Este método de codificação usa mais bits que um código Hamming, mas pode corrigir erros múltiplos.

Além disso, em vez de usar *pits* representando 0s e *lands* representando 1s (ou vice-versa), cada transição *pit-land* ou *land-pit* representa bits em 1. O intervalo entre duas transições diz quantos 0s estão presentes entre os bits em 1. Os dados são gravados em grupos de 24 bytes, sendo que cada byte é expandido de 8 para 14 bits, usando o código de Reed-Solomon. Três bits especiais são adicionados entre cada grupo e um byte de sincronismo é adicionado para formar um quadro. Um grupo de 98 quadros forma um bloco contendo 2K bytes de dados do usuário, que é a unidade básica endereçável. Embora este esquema seja complicado e gaste uma área considerável no disco, ele provê uma confiabilidade extremamente alta usando um meio de baixo custo.

Informações em um CD ROM são escritas como uma espiral única contínua, diferentemente dos discos magnéticos com suas trilhas e cilindros discretos. Este formato é ilustrado na Fig. 2.18. Cada CD ROM contém 270.000 blocos de dados, para uma capacidade total de 553 megabytes. Os dados são lidos a uma velocidade linear constante de 75 polegadas por segundo (sendo a velocidade de rotação reduzida à medida que a cabeça se move para fora), o que dá uma taxa de transferência de 153,60 kbytes/s.

Os CD ROMs são potencialmente úteis na distribuição de grandes bases de dados, especialmente aquelas que misturam textos e imagens. Uma biblioteca completa de leis, medicina ou literatura de 250 livros grossos cabe facilmente em um CD ROM, assim como uma enciclopédia totalmente ilustrada. Um curso por computador formado por milhares de *slides* coloridos, cada qual acompanhado por 10 segundos de narração, é outro candidato para CD ROM. Um mapa viário para um estado inteiro poderia ser a base de um sistema de navegação automobilístico,

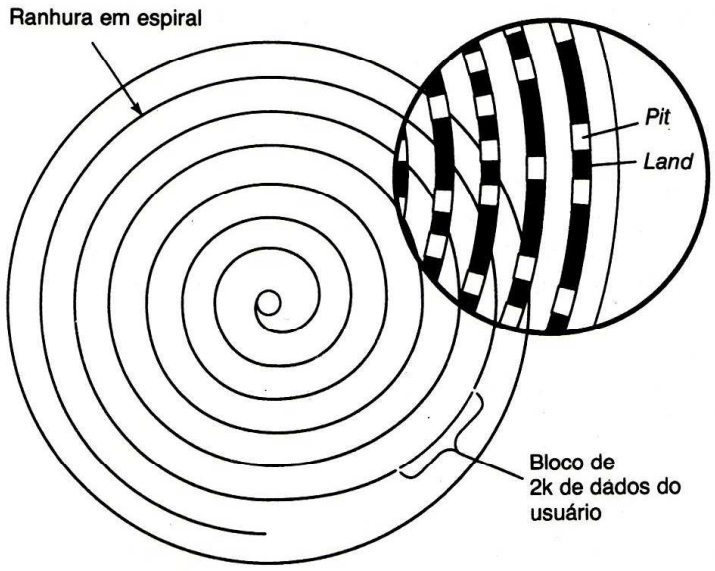


Fig. 2.18 Formato de um CD ROM.

com um sintetizador de voz guiando o motorista para o destino. As possibilidades são limitadas apenas pela nossa imaginação.

Apesar desse enorme potencial, os CD ROMs não são escrevíveis, o que limita sua utilidade como dispositivos de armazenamento de computadores. A intenção de se ter um meio escrevível conduziu à próxima etapa: os discos óticos **WORM (Write Once Read Many)**. Este dispositivo permite que os usuários escrevam informações nos discos óticos. Entretanto, uma vez que um *pit* foi queimado na superfície, ele não pode ser apagado. Tais discos são bons para arquivar dados, relatórios de auditoria contábil, e outras informações que são (semi)permanentes. Eles não são muito apropriados para guardar arquivos temporários de rascunho. Mas, dada a grande capacidade desses discos, o estilo descartável de apenas acrescentar arquivos temporários até que o disco esteja cheio, e então jogá-lo fora, é aceitável.

É claro que a existência de discos que só podem ser escritos uma vez provoca um impacto na forma em que os programas são escritos. A incapacidade de modificar arquivos sugere um tipo diferente de sistema de arquivos, no qual um arquivo é, na realidade, uma seqüência de **versões imutáveis**, onde nenhuma pode ser modificada, e cada uma substitui a anterior. Este modelo é um pouco diferente do modelo comum de regravação usado em discos magnéticos.

A terceira fase de evolução dos discos óticos é o meio ótico apagável. Estes usam tecnologia magneto-ótica. O disco plástico é coberto com ligas de metais tão exóticos, tais como térbio e gadolínio, que são poucas as pessoas que ouviram falar deles. Esses metais possuem a interessante propriedade de, a baixa temperatura, serem insensíveis aos campos magnéticos, mas, a alta temperatura, sua estrutura molecular se alinha com qualquer campo magnético presente.

Para usar esta propriedade para gravar informações, a cabeça da unidade contém um *laser* e um magneto. O *laser* dispara uma rajada de luz ultracurta no metal, aumentando sua temperatura instantaneamente, mas não esburacando a superfície. Ao mesmo tempo, o magneto está criando um campo em uma das duas direções. Quando o pulso de *laser* termina, o metal foi magnetizado em uma das direções possíveis, representando 0 ou 1. Esta informação pode ser lida da mesma maneira que no CD ROM, usando um *laser* muito mais fraco. O disco pode também ser apagado ou reescrito da mesma maneira que na primeira vez.

Discos óticos escrevíveis não irão substituir os discos winchester convencionais por algum tempo — provavelmente alguns anos, se por acaso isso ocorrer — por duas razões. Primeiro, seu tempo de *seek* é uma ordem de grandeza maior que o dos discos winchester. Depois, sua taxa de transferência é também uma ordem de grandeza mais baixa. O desempenho dos discos magnéticos é simplesmente muito melhor. Enquanto os discos óticos irão, sem dúvida, melhorar com o tempo, os discos magnéticos irão, provavelmente, melhorar tão rápido quanto, mantendo-se

na frente. Mesmo assim, para aplicações onde um grande número de memória removível é crítico, os discos óticos têm um futuro brilhante.

2.3 ENTRADA/SAÍDA

Antes de um computador trabalhar na solução de um problema, devemos fornecer-lhe o programa e os dados. Após encontrar a solução, o computador precisa comunicá-la aos seres humanos que propuseram o problema em primeiro lugar. O processo de fornecer informações ou obtê-las dos computadores é chamado entrada/saída, ou simplesmente **E/S**.

Nem toda entrada vem de pessoas e nem toda saída é destinada a pessoas. Um telescópio solar operado por computador pode obter seus dados de entrada diretamente dos instrumentos que observam o sol. Um computador que controla uma indústria química automatizada pode dirigir sua saída às máquinas distribuídas pela planta, que regulam a produção.

Duas organizações de E/S são comumente usadas nos computadores modernos. Grandes *mainframes* usam o projeto mostrado na Fig. 2.19. Neste projeto, um sistema de computação consiste em uma CPU (ou possivelmente múltiplas CPUs), uma memória, e um ou mais processadores de E/S, chamados de **canais de dados**. Todos os dispositivos de E/S são conectados aos canais.

Quando a CPU quer realizar E/S, ela carrega um programa especial em um dos canais, e diz ao canal para executá-lo. O canal manipula toda a E/S para e da memória principal, deixando a CPU livre para fazer outras coisas. Quando o canal termina, ele envia à CPU um sinal especial chamado **interrupção**, o que faz a CPU parar o que ela estava fazendo e dar atenção ao canal. A vantagem dessa organização é que ela permite que a CPU descarregue todo o trabalho de E/S para o canal. Desta forma, processamento e E/S podem ser feitos ao mesmo tempo.

Como os *mainframes* tipicamente geram uma tremenda carga de E/S, eles são usualmente equipados com pelo menos três barramentos. O barramento de memória permite aos canais ler e escrever palavras na memória. O barramento de E/S permite à CPU enviar comandos aos canais e permite que os canais interrompam a CPU. Finalmente, há um terceiro barramento que permite que a CPU acesse a memória sem usar os outros barramentos.

Computadores pessoais usam uma estrutura mais simples, como a ilustrada na Fig. 2.20. A maioria dessas máquinas tem uma caixa com uma placa de circuito impresso grande, no fundo,

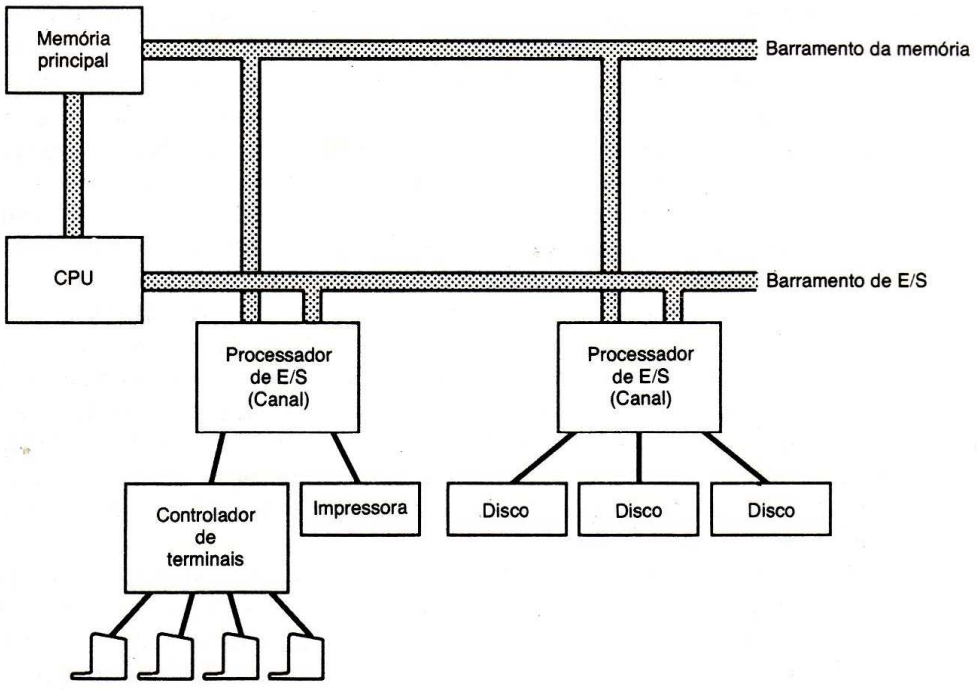


Fig. 2.19 Estrutura de E/S em um grande *mainframe*.

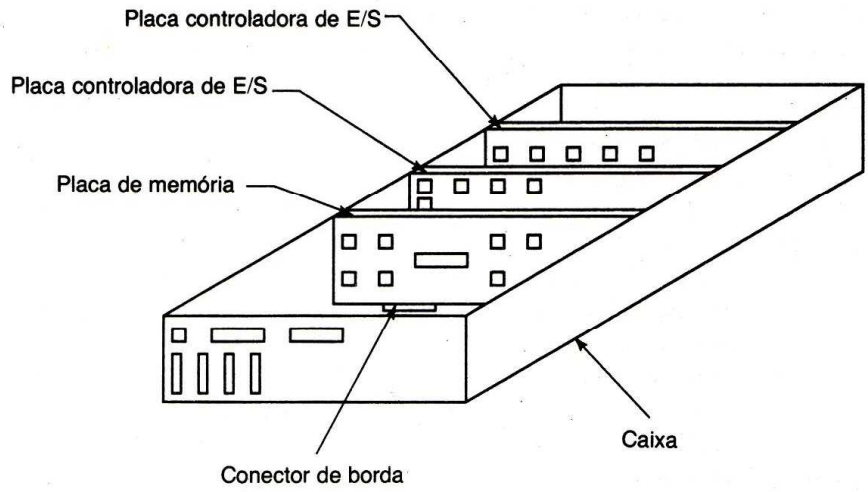


Fig. 2.20 Estrutura física de um computador pessoal.

chamada de ***motherboard*** (placa-mãe). A *motherboard* contém a pastilha de CPU, alguma memória e várias pastilhas de suporte. Ela também contém um barramento ao longo de sua extensão e soquetes nos quais conectores de borda de placas de memória adicional e de E/S podem ser inseridos. A estrutura de minicomputadores é similar, exceto que a CPU é também localizada em uma placa de expansão (*plug-in*), sendo que a *motherboard* se torna apenas um receptáculo passivo para outras placas, e neste caso é chamada ***backplane***.

A estrutura lógica de um computador pessoal é mostrada na Fig. 2.21. A maioria tem um barramento único usado para interligar a CPU, memória e dispositivos de E/S. Cada dispositivo de E/S consiste em duas partes: uma contendo a maior parte dos componentes eletrônicos, chamada **controlador,** e a outra contendo o próprio dispositivo de E/S, tal como uma unidade de disco. O controlador geralmente se situa numa placa de expansão, exceto para aqueles controladores não-opcionais (tais como o teclado), os quais normalmente estão localizados na *motherboard.* Embora o vídeo (monitor) não seja um opcional, o controlador de vídeo normalmente se localiza em uma placa de expansão, para permitir ao usuário escolher entre colorido, monocromático, de alta resolução, baixa resolução, e assim por diante. O controlador se conecta ao dispositivo por um cabo ligado a um conector na parte de trás da caixa.

A função de um controlador é controlar seu dispositivo de E/S e gerenciar os acessos ao barramento para ele. Quando um programa quer dados de um disco, ele manda um comando para o controlador de disco, que então envia *seeks* e outros comandos para o dispositivo. Quando

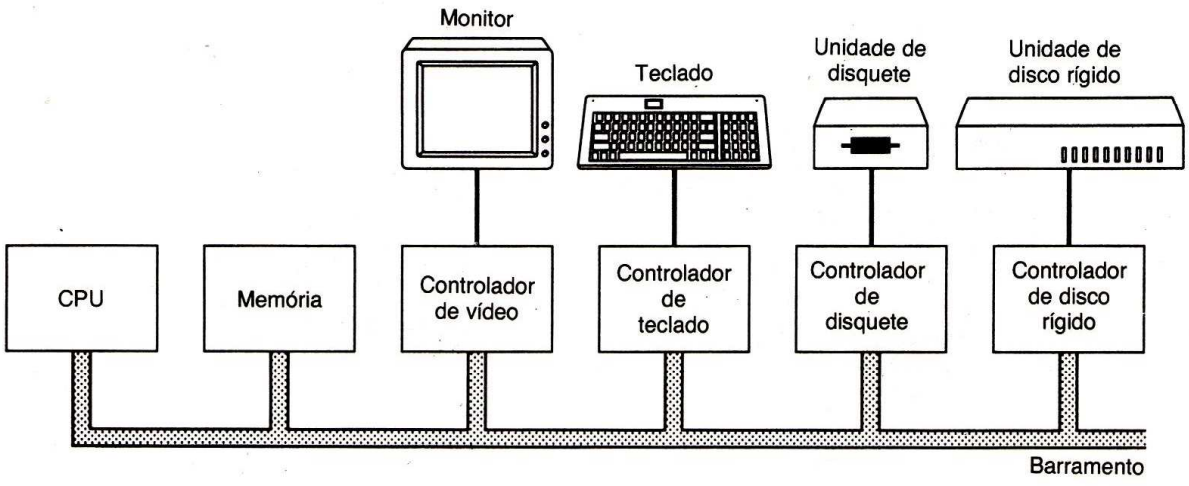


Fig. 2.21 Estrutura lógica de um computador pessoal.

a trilha e o setor desejados foram localizados, a unidade começa a soltar os dados como uma seqüência serial de bits para o controlador. É tarefa do controlador quebrar a seqüência de bits em palavras e escrever cada palavra montada na memória. Um controlador que lê ou escreve um bloco de dados na memória sem a intervenção da CPU é dito fazer acesso direto à memória, mais conhecido pela sigla **DMA** (***Direct Memory Access***).

O barramento não é apenas usado pelos controladores de E/S, mas também pela CPU para buscar instruções e dados. Que acontece se a CPU e um controlador querem usar o barramento ao mesmo tempo? A resposta é que uma pastilha chamada **árbitro de barramento** decide quem é o próximo a usar. Geralmente é dada a preferência aos dispositivos de E/S sobre a CPU, porque os discos e outros dispositivos com partes móveis não podem ser parados, e forçá-los a esperar causaria perda de dados. Quando não há E/S em execução, a CPU pode usar todos os ciclos de barramento para referenciar a memória. Entretanto, quando algum dispositivo de E/S está operando, tal dispositivo irá requerer o barramento e será atendido quando necessário. Este processo é chamado **roubo de ciclos** e reduz a velocidade do computador. Os *mainframes* têm múltiplos barramentos, como vimos, e assim não sofrem este problema.

Existem hoje muitos tipos de dispositivos de E/S. Alguns dos mais comuns são analisados a seguir.

2.3.1 Terminais

Os terminais de computador consistem em três elementos: o teclado, o monitor e algum circuito eletrônico para controlá-los. Existem diversos tipos de teclados. Nos mais baratos, cada tecla é simplesmente uma chave que fecha um contato elétrico quando pressionada. Os mais caros têm um ímã sob cada tecla que passa através de uma bobina quando pressionada, induzindo assim uma corrente que pode ser detectada. Vários outros métodos, tanto mecânicos quanto eletromecânicos, são também utilizados em alguns teclados.

O monitor é uma caixa contendo basicamente um **CRT (Tubo de Raios Catódicos)** e suas fontes de alimentação. O CRT contém um canhão que pode disparar um feixe de elétrons contra uma tela fosforescente na frente do tubo, como mostrado na Fig. 2.22(a). (Monitores coloridos possuem três canhões, para vermelho, verde e azul.) Durante a varredura horizontal, o feixe percorre a tela em cerca de 50 μs, traçando uma linha quase horizontal na tela. Então ele executa um retraço horizontal para voltar para o lado esquerdo e começar nova varredura. Um dispositivo como este, que produz uma imagem linha a linha, é chamado dispositivo ***raster***. Se o retraço horizontal não é instantâneo, ele é dito ***boustrophedonic*** (a maneira como um boi percorre um campo com arado: esquerda para a direita, depois direita para a esquerda; arados também não têm retraço instantâneo).

A varredura horizontal é controlada por uma tensão linearmente crescente aplicada às placas de deflexão horizontal dispostas dos lados esquerdo e direito do canhão eletrônico. O movimento

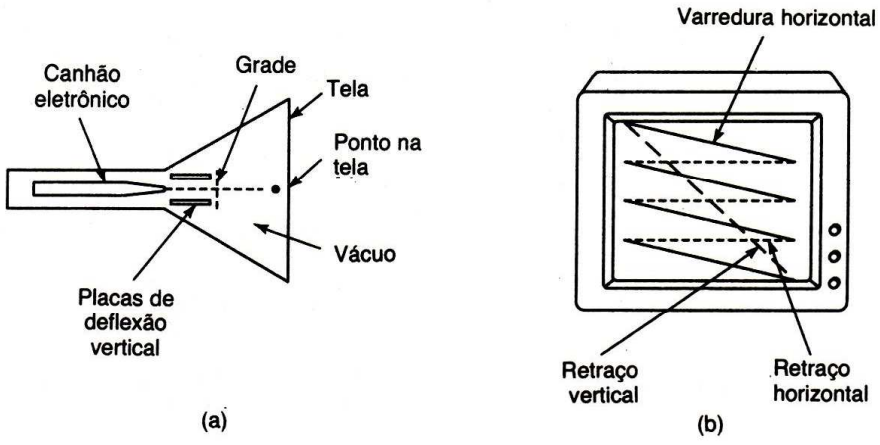


Fig. 2.22 (a) Seção de um CRT. (b) Padrão de varredura de um CRT.

vertical é controlado por uma tensão também linearmente crescente, porém lentamente aplicada às placas de deflexão vertical acima e abaixo do canhão. Após algo entre 400 e 1000 varreduras, as tensões nas placas de deflexão horizontal e vertical são rapidamente revertidas simultaneamente para voltar o feixe para o canto esquerdo superior. Toda a imagem de uma tela é normalmente reescrita entre 30 e 60 vezes por segundo. Os movimentos do feixe são mostrados na Fig. 2.22(b).

Para produzir um padrão de pontos na tela, existe uma grade dentro do CRT. Quando uma tensão positiva é aplicada à grade, os elétrons são acelerados, fazendo com que o feixe atinja a tela e a faça brilhar por um curto período de tempo. Quando uma tensão negativa é usada, os elétrons são repelidos, de modo que eles não passam através da grade e a tela não brilha. Assim, a tensão aplicada à grade faz com que o padrão de bits correspondente apareça na tela. Este mecanismo permite que um sinal elétrico seja convertido numa exibição visual.

Três tipos de terminais são comumente utilizados: terminais alfanuméricos mapeados em memória (*character-map*), terminais *bit-map* e terminais RS-232-C. Todos eles podem usar qualquer tipo de teclado, mas diferem na forma como o computador se comunica com eles e como a saída é tratada. Nas seções seguintes, descreveremos brevemente cada um deles.

Terminais Alfanuméricos Mapeados em Memória

A maioria dos computadores pessoais usa o esquema mostrado na Fig. 2.23 para mostrar a saída no monitor. (O teclado é tratado como um dispositivo completamente isolado.) Na placa controladora de vídeo existe uma porção de memória, chamada de **memória de vídeo**, assim como um circuito para acessar o barramento e gerar sinais de vídeo.

Para mostrar os caracteres, a CPU os copia para a memória de vídeo em bytes alternados. Associado a cada caractere, existe um **byte de atributo** que descreve como o caractere deve ser mostrado. Os atributos podem incluir cor, intensidade, piscante ou não, e assim por diante. Deste modo, uma tela de 25 por 80 caracteres requer 4000 bytes de memória de vídeo, 2000 para os caracteres e 2000 para os atributos. Algumas placas possuem mais memória, podendo armazenar múltiplas telas.

A tarefa da placa de vídeo é repetidamente buscar os caracteres da memória de vídeo e gerar o sinal necessário para acionar o monitor. Uma linha completa de caracteres é buscada de uma só vez para que as linhas de varredura possam ser calculadas. Este sinal é um sinal analógico de alta freqüência que controla a varredura do feixe de elétrons que desenha os caracteres na tela. Como a placa produz um sinal de vídeo, o monitor deve ficar a poucos metros do computador.

Terminais Bit-map

Uma variação desta idéia é ter a tela considerada não como uma matriz de 25 × 80 caracteres, mas sim uma matriz de pontos, chamados **pixels**. Cada pixel pode estar ligado ou desligado, e representa um bit de informação. Nos computadores pessoais, a tela pode conter tão pouco quanto 200 × 320 pixels, sendo comum encontrar 480 × 640. Nas estações de trabalho, a tela é tipicamente de 1024 × 1024 pixels. Terminais que usam mapeamento de bits em vez de mapea-

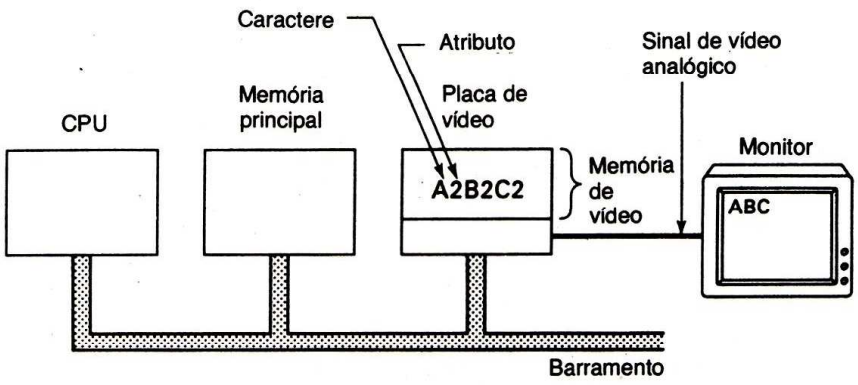


Fig. 2.23. Saída de um terminal em um computador pessoal.

mento de caracteres são chamados **terminais bit-map**. Várias placas de vídeo podem operar tanto como terminais *character-map* quanto *bit-map*, sob o controle de *software*.

A mesma idéia geral é usada como na Fig. 2.23, exceto que a memória de vídeo é vista como uma grande matriz de bits. O *software* pode estabelecer qualquer padrão desejado, e que é mostrado instantaneamente. Para desenhar caracteres, o *software* pode decidir alocar, por exemplo, um retângulo de 9 × 14 para cada caractere e preenchê-lo com os bits necessários para formar o caractere. Esta abordagem permite ao *software* criar *fonts* múltiplos e combiná-los à vontade. Tudo o que o *hardware* faz é mostrar a matriz de bits. A cor pode ser manipulada utilizando múltiplas matrizes, algumas vezes chamadas de **bit planes**. Com *n bit planes*, uma entre 2^n cores pode ser selecionada para cada pixel.

Terminais *bit-map* são geralmente utilizados para suportar a exibição de várias **janelas**. Uma janela é uma área na tela usada por um programa. Com múltiplas janelas é possível executar vários programas ao mesmo tempo, cada um mostrando seus resultados independentemente dos outros.

Apesar de os terminais *bit-map* serem bastante flexíveis, eles têm duas grandes desvantagens. Primeiro, eles requerem uma quantidade considerável de memória de vídeo. Uma tela de 1024 × 1024 requer um milhão de bits, ou 128 K bytes para monitor monocromático, e metade de um megabyte para um monitor colorido com 4 bits por pixel (comparado com 4 K para terminais *character-map*, tanto monocromático quanto colorido).

A segunda desvantagem é o desempenho. Para mostrar qualquer coisa, é necessária uma tremenda quantidade de cópia. Mover um caractere com caixa 9 × 14 para a tela requer a transferência de pelo menos 28 bytes, assim como todo o trabalho para inserir o caractere no lugar certo no mapa de bits. Pior ainda, rolar a tela geralmente significa copiar a memória inteira. Conseqüentemente, terminais *bit-map* precisam de CPUs muito rápidas para manipular os bits. Alternativamente, eles podem ser equipados com um *hardware* especial para gerenciar os pixels rapidamente.

Terminais RS-232-C

Dúzias de companhias fabricam computadores e centenas de companhias fabricam terminais. Para permitir que (quase) qualquer terminal possa ser usado com (quase) qualquer computador, um padrão de interface entre computador e terminal, chamado **RS-232-C**, foi desenvolvido. Qualquer terminal que suporte a interface RS-232-C pode ser conectado a qualquer computador que também suporte esta interface.

Os terminais RS-232-C têm um conector padronizado de 25 pinos. O padrão RS-232-C define o tamanho e a forma do conector, os níveis de tensão e o significado de cada um dos sinais nos pinos.

Quando um computador e um terminal estão longe um do outro, é freqüente o caso em que a única forma prática de conectá-los é através do sistema telefônico. Infelizmente, o sistema telefônico não é capaz de transmitir os sinais requeridos pelo padrão RS-232-C. Então, um dispositivo, chamado **modem** (**mo**dulador-**dem**odulador), precisa ser inserido entre o computador e o telefone, e também entre o terminal e o telefone, para realizar a conversão de sinais. Estudaremos os modems na próxima seção.

A Fig. 2.24 mostra a disposição do computador, modems e terminal quando a linha telefônica é utilizada. Quando o terminal está perto o bastante do computador para que possam ser ligados diretamente por fio, os modems não são utilizados. Porém os mesmos conectores e cabos do padrão RS-232-C são utilizados, apesar dos pinos relacionados ao controle de modems não serem necessários.

Para se comunicarem, o computador e o terminal contêm, cada um, uma pastilha chamada **UART (Universal Asynchronous Receiver Transmitter)**, assim como uma lógica para acessar o barramento. Para mostrar os caracteres, o computador busca o caractere de sua memória principal e o coloca na UART. Esta, por sua vez, o desloca através do cabo RS-232-C, bit a bit. Na verdade, a UART é um conversor paralelo para serial, uma vez que o caractere inteiro (1 byte) lhe é entregue de uma vez, e sua saída é de um bit por vez, numa taxa específica. As taxas mais comuns são de 110, 300, 1200, 2400, 9600 e 19200 bits/s.

No terminal, outra UART recebe os bits e reconstrói o caractere inteiro, que então é mostrado

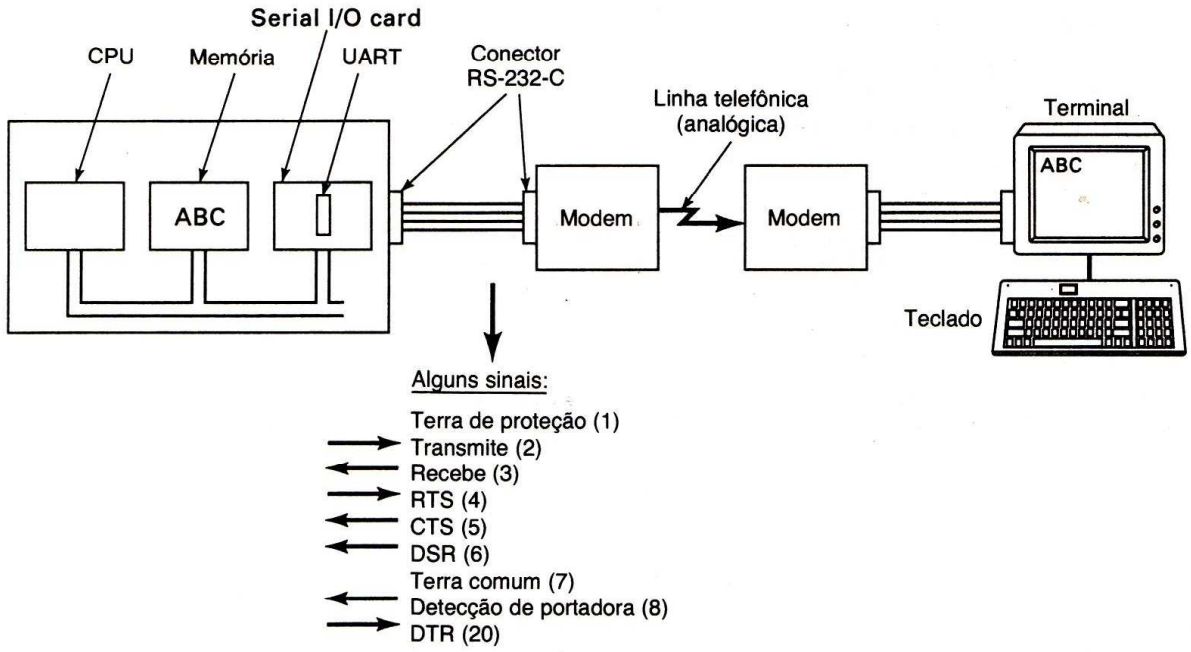


Fig. 2.24 Conexão de um terminal RS-232-C a um computador. Os números entre parênteses, na lista de sinais, são os números dos pinos.

na tela. A entrada vinda do teclado do terminal passa através do conversor paralelo para serial no terminal e é depois reconstruída pela UART no computador.

O padrão RS-232-C define 25 sinais, mas na prática somente alguns são utilizados (e a maioria deles é omitida quando o terminal está ligado diretamente ao computador, sem modems). Os pinos 2 e 3 são para transmissão e reação de dados, respectivamente. Cada um é dedicado a uma seqüência unidirecional de bits, em sentidos opostos. Quando o terminal ou o computador é ligado, ele ativa (coloca em 1) o sinal DTR (*Data Terminal Ready*) para informar o modem de que ele está ligado. Similarmente, o modem ativa o DSR (*Data Set Ready*) para indicar sua presença. Quando o terminal ou o computador deseja enviar dados, ele ativa o RTS (*Request To Send*) para pedir a permissão. Se o modem pode atender esta requisição, ele ativa o CTS (*Clear To Send*) como resposta. Outros pinos são usados para várias funções de *status*, testes e temporizações.

2.3.2 Modems

Com o crescimento do uso de computadores no passado, é comum que um computador precise comunicar-se com outro. Por exemplo, muitas pessoas possuem computadores pessoais em casa que elas utilizam para se comunicar com seus computadores no trabalho, com sistemas bancários, ou sistemas de boletins eletrônicos. Todas essas aplicações usam o telefone como meio de comunicação.

Entretanto, a linha telefônica comum não é apropriada para transmitir sinais de computadores, que geralmente representam um 1 como aproximadamente 5 volts e um 0 como 0 volt, como mostrado na Fig. 2.25(a). Sinais de dois níveis sofrem considerável distorção quando transmitidos através de um canal de voz da linha telefônica, levando assim a erros de transmissão. Um sinal de onda senoidal pura a uma freqüência de 1000 a 2000 Hz, chamado **portadora**, pode ser transmitido com relativamente pouca distorção, e este fato é explorado como a base da maioria dos sistemas de telecomunicação.

Como os pulsos de uma onda senoidal são completamente previsíveis, uma onda senoidal pura não transmite nenhuma informação. Entretanto, através da variação da amplitude, freqüência ou fase, uma seqüência de 1s e 0s pode ser transmitida, como mostrado na Fig. 2.25. Este processo

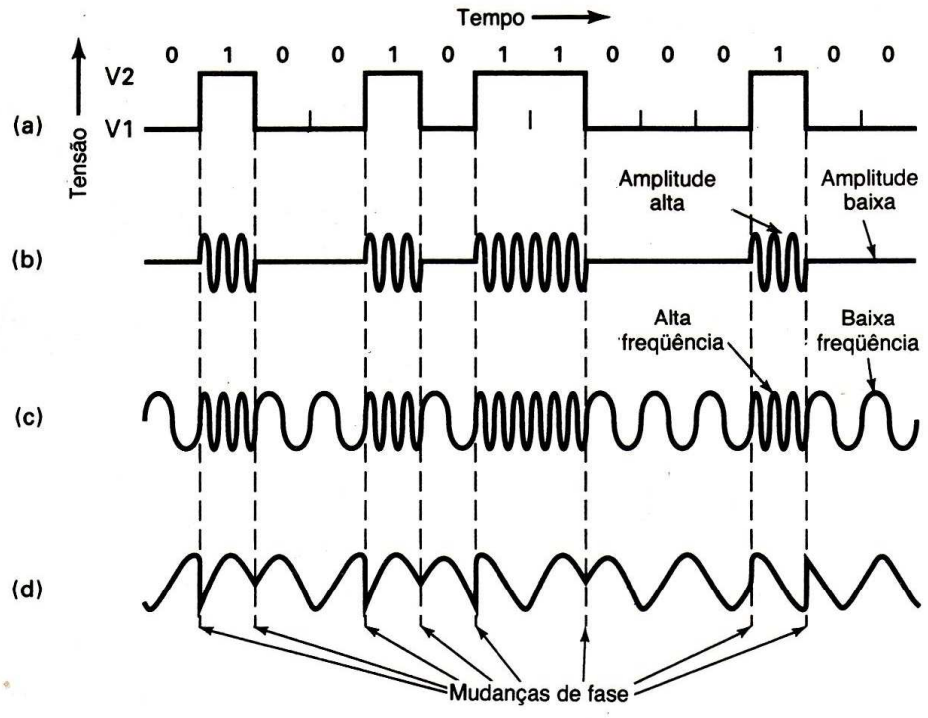


Fig. 2.25 Transmissão do número binário 01001011000100 em uma linha telefônica, bit a bit. (a) Sinal de dois níveis. (b) Modulação em amplitude. (c) Modulação em freqüência. (d) Modulação em fase.

é chamado **modulação**. Na **modulação em amplitude** [veja a Fig. 2.25(b)], dois níveis diferentes de tensão são utilizados para 0 e 1, respectivamente. Uma pessoa ouvindo dados digitais transmitidos a uma taxa muito baixa ouviria um ruído alto para um 1 e nenhum ruído para um 0.

Na modulação em freqüência [veja a Fig. 2.25(c)], o nível de tensão é constante, mas a freqüência da portadora é diferente para 1 e 0. Uma pessoa ouvindo dados digitais modulados em freqüência ouviria dois tons, correspondendo a 0 e 1. A modulação em freqüência é muitas vezes chamada ***frequency shift keying*** (chaveamento por deslocamento de freqüência, **FSK**).

Na **modulação em fase** simples [veja a Fig. 2.25(d)], a amplitude e a freqüência não variam, mas a fase da portadora é invertida 180 graus quando o dado muda de 0 para 1 ou de 1 para 0. Em sistemas mais sofisticados de modulação em fase, no início de cada intervalo indivisível de tempo, a fase da portadora é abruptamente deslocada de 45, 135, 225 ou 315 graus, para permitir dois bits por intervalo de tempo. Isto é chamado codificação **dibit** em fase. Outros esquemas para transmitir 3 ou mais bits por intervalo de tempo também existem. O número de intervalos de tempo é a **taxa de baud** (***baud rate***). Com 2 ou mais bits por intervalo, a taxa de bits será maior que o *baud rate*. Muitas pessoas confundem estes dois termos.

Se o dado a ser transmitido consiste em uma série de caracteres de 8 bits, seria desejável que houvesse uma conexão capaz de transmitir 8 bits simultaneamente — isto é, 8 pares de fios. Como as linhas telefônicas possuem apenas um canal, os bits devem ser enviados serialmente, um após o outro (ou em grupos de dois, se a codificação dibit estiver sendo utilizada). O dispositivo que aceita caracteres do computador na forma de sinal de dois níveis, um bit de cada vez, e transmite os bits em grupos de um ou dois, na forma modulada em amplitude, freqüência ou fase, é o modem.

O modem transmissor envia os bits individuais de um caractere em intervalos de tempo regularmente distribuídos. Por exemplo, 1200 *baud* significa que um sinal muda a cada 833 μs. Um segundo modem no lado receptor é usado para converter a portadora modulada para um número binário. Como os bits chegam ao receptor em intervalos regularmente espaçados, uma vez que o modem receptor determina o início de um caractere, seu relógio indica quando amostrar a linha para ler os valores dos bits individuais.

Transmissão Assíncrona e Síncrona

Dois métodos diferentes são usados para transmitir caracteres. Na **transmissão assíncrona**, o intervalo de tempo entre dois caracteres não é fixo, embora o intervalo entre dois bits consecutivos de um caractere seja fixo. Por exemplo, uma pessoa digitando em um terminal de um sistema de tempo compartilhado não irá teclar com uma velocidade uniforme, assim o intervalo entre dois caracteres sucessivos não será constante.

Essa variação na velocidade cria o problema de como o receptor pode reconhecer o primeiro bit de um caractere. Se os métodos de modulação da Fig. 2.25 são usados, não há forma de distinguir entre a ausência de dado e um bit em 0. Um caractere formado exclusivamente de 0s seria completamente invisível. Além disso, um caractere formado por um 1 seguido de sete 0s não poderia ser distinguido de um caractere formado de sete 0s seguido de um 1, porque o receptor não teria meios de dizer se o bit em 1 estava no início, meio ou fim do caractere.

Para permitir que o receptor reconheça o início de um caractere, um bit de partida (*start bit*) é transmitido exatamente antes de cada caractere. Para aumentar a confiabilidade, 1 ou 2 bits de parada (*stop bits*) são transmitidos exatamente após cada caractere. Normalmente, a linha é mantida no estado 1 quando não há dados sendo transmitidos, para permitir a detecção de um circuito interrompido. Assim, o *start bit* é 0. Os *stop bits* são então 1, para distingui-los do *start bits*. Entre o *start bit* e os *stop bits*, bits de dados são transmitidos em intervalos de tempo uniformemente espaçados. Um temporizador no modem receptor é disparado quando o *start bit* chega, permitindo ao modem dizer qual bit é qual. A comunicação assíncrona é ilustrada na Fig 2.26(a).

As taxas de bits usadas para transmissão assíncrona variam de 110 a 19200 bps. A 110 bps, são usados 2 *stop bits*, assim um caractere de 7 bits, mais um bit de paridade, mais um bit de *start*, mais dois bits de *stop*, dão 11 bits por caractere. Portanto, 110 bps correspondem a 10 caracteres/s. As taxas mais altas usam apenas um *stop bit*.

Na **transmissão síncrona**, a necessidade dos bits de *start* e *stop* é eliminada. O resultado é o aumento da velocidade de transmissão de dados. Comunicações síncronas ocorrem normalmente a taxas de 4800 bps, 9600 bps, ou ainda maiores. Neste método, uma vez que os modems estão sincronizados, eles continuam a enviar caracteres para manter o sincronismo, mesmo quando não há dados a transmitir. Um caractere especial é enviado quando não há dados. Nas transmissões síncronas, ao contrário das assíncronas, o intervalo entre dois caracteres sempre é exatamente o mesmo.

As transmissões síncronas requerem que os relógios do transmissor e do receptor permaneçam sincronizados por longos períodos de tempo, o que não ocorre na transmissão assíncrona porque

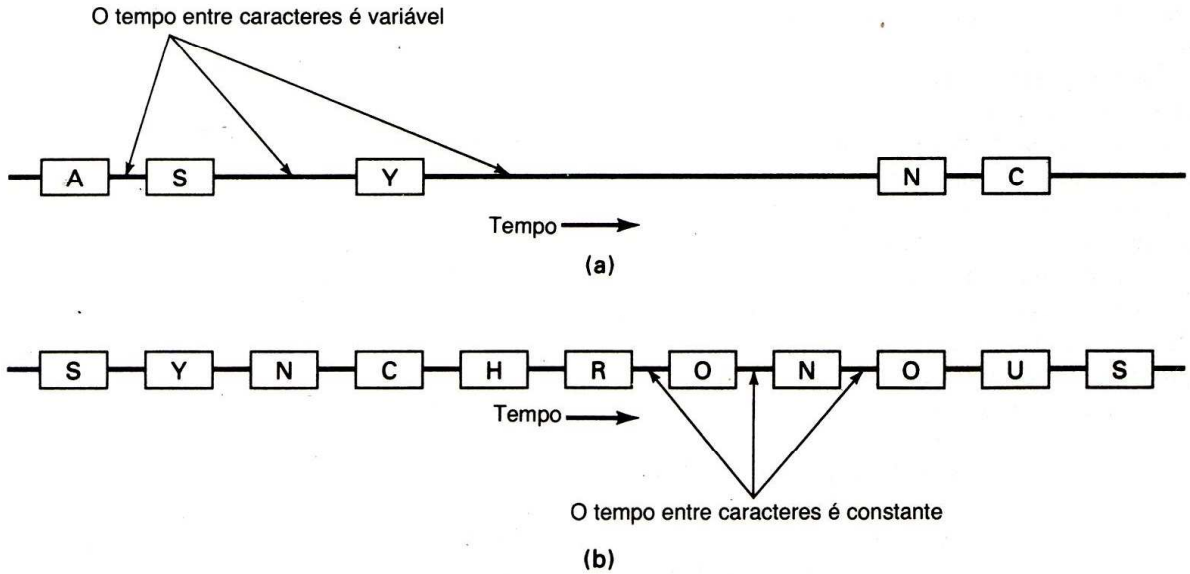


Fig. 2.26 Modos de transmissão. (a) Transmissão assíncrona. (b) Transmissão síncrona.

o início de cada caractere é explicitamente indicado por um *start bit*. O período de tempo durante o qual uma transmissão pode ser mantida sem ressincronizar as fases do receptor e do transmissor depende da estabilidade dos relógios. Tipicamente, os relógios são suficientemente estáveis para permitir a transmissão de blocos de milhares de caracteres sem ter que ressincronizar. Esses blocos, muitas vezes, usam códigos Hamming, ou outras técnicas, para detectar e corrigir erros de transmissão. A transmissão síncrona é ilustrada na Fig. 2.26(b).

Transmissão *Simplex, Half-duplex* e *Full-duplex*

Três modos de transmissão são utilizados com o propósito de comunicação: *simplex, half-duplex* e *full-duplex*. Uma linha ***simplex*** é capaz de transmitir dados em apenas uma direção. Isto não é devido a nenhuma propriedade dos fios, mas simplesmente porque de um lado há apenas um transmissor e, do outro, apenas um receptor. Esta configuração é raramente usada por computadores porque ela não provê nenhum meio de o receptor transmitir um sinal de reconhecimento para o transmissor, indicando que a mensagem foi recebida corretamente. A difusão de rádio e televisão são exemplos de transmissão *simplex*.

Uma linha ***half-duplex*** pode enviar e receber dados em ambas as direções, mas não simultaneamente. Durante qualquer transmissão, um modem é o transmissor e o outro é o receptor. Uma situação comum é um dispositivo *A*, agindo como transmissor, enviar uma série de caracteres a um dispositivo *B*, agindo como receptor. Então *A* e *B* trocam simultaneamente os papéis e *B* envia uma mensagem de volta a *A* especificando se os caracteres foram recebidos com ou sem erros. Se não houve erros de transmissão, *A* e *B* trocam novamente os papéis e *A* envia a próxima mensagem para *B*. Se houve erros, *A* transmite a mensagem deturpada de novo. A "conversa" entre transmissor e receptor a respeito do que fazer a seguir é chamada de **protocolo**. O tempo gasto para comutar uma linha *half-duplex* de uma direção para outra pode ser de muitos tempos de um caractere. Uma linha férrea é um exemplo de um canal *half-duplex*, porque ela pode lidar com tráfego em ambas as direções, mas não ao mesmo tempo.

Uma linha ***full-duplex*** pode, ao contrário, enviar e receber dados em ambas as direções simultaneamente. Conceitualmente, uma linha *full-duplex* é equivalente a duas linhas *simplex*, uma em cada direção. Como as duas transmissões podem ocorrer em paralelo, uma em cada direção, uma linha *full-duplex* pode transmitir mais informações que uma linha *half-duplex* com a mesma taxa. Além disto, linhas *full-duplex* não perdem tempo comutando direções.

2.3.3 O Mouse

Com o decorrer do tempo, os computadores estão sendo utilizados por pessoas com menos conhecimento de como eles funcionam. Os computadores da geração ENIAC eram utilizados somente por pessoas que os construíram. Na década de 50, os computadores só eram utilizados por programadores profissionais altamente especializados. Atualmente, os computadores são utilizados por pessoas que necessitam de algum trabalho realizado e não sabem (nem mesmo querem saber) muito sobre como eles funcionam ou como são programados.

A maioria destas pessoas acham que os comandos (freqüentemente criptográficos) digitados nos teclados não são amigáveis, se não hostis. Elas preferem sistemas nos quais o computador exibe menus na tela, e elas apontam o item desejado. O uso deste modelo requer um meio de apontar a tela. O meio mais comum de permitir que usuários apontem a tela é com um dispositivo chamado ***mouse***.

Um *mouse* é uma pequena caixa plástica que fica na mesa ao lado do terminal. Quando ele é movido ao longo da mesa, um pequeno apontador na tela se move também, permitindo que os usuários apontem os itens na tela. O *mouse* tem um, dois ou três botões na parte superior, para permitir que os usuários selecionem itens de menus. Muito sangue tem sido derramado como resultado das discussões sobre quantos botões um *mouse* deve ter. Usuários ingênuos preferem um (é difícil apertar o botão errado quando há apenas um), mas os mais sofisticados gostam do poder de muitos botões para fazer coisas fantásticas.

Existem três tipos de *mouse* atualmente: o mecânico, o ótico e o optomecânico. Um tipo de *mouse* mecânico possui duas rodas de borracha na parte de baixo, com seus eixos perpendiculares um ao outro, como ilustrado na Fig. 2.27. Quando o *mouse* é movimentado paralelamente ao

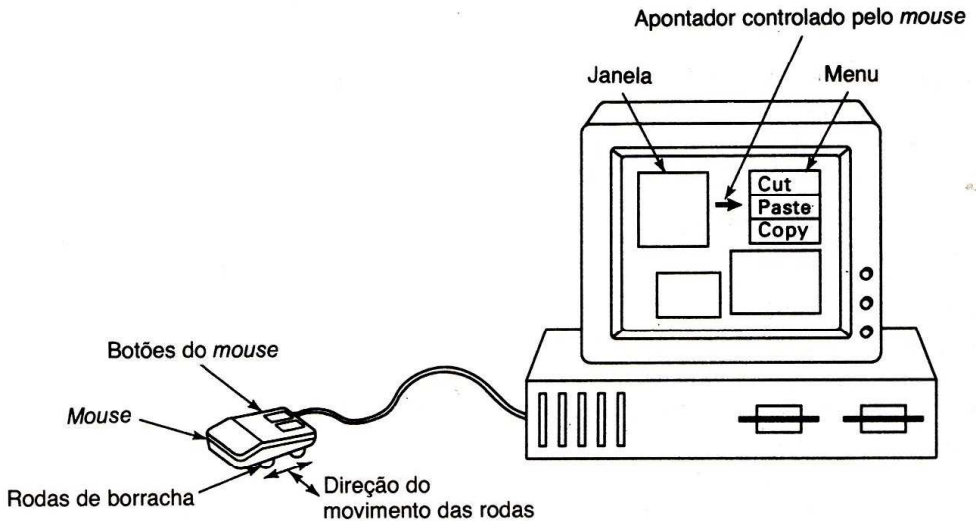


Fig. 2.27 O uso de um *mouse* com rodas para apontar itens no menu.

seu eixo principal, uma das rodas gira. Quando ele é movimentado perpendicularmente ao seu eixo principal, a outra gira. Cada roda aciona um resistor variável (potenciômetro). Medindo as variações de resistência, é possível saber quanto cada roda girou, e assim calcular quanto o *mouse* se moveu em cada direção. Outro tipo de *mouse* mecânico obtém o mesmo efeito usando uma esfera levemente saliente na parte inferior.

O segundo tipo é o *mouse* ótico. Este tipo não tem rodas ou esferas. Em vez disto, ele possui um **LED (*Light Emitting Diode*** — diodo emissor de luz) e um fotodetector na parte inferior. O *mouse* ótico é usado sobre uma placa especial de plástico contendo uma grade retangular de linhas pouco espaçadas. À medida que o *mouse* se movimenta sobre a grade, o fotodetector sente o passar das linhas, percebendo as mudanças na quantidade de luz refletida do LED. O circuito eletrônico dentro do *mouse* conta o número de linhas atravessadas em cada direção.

O terceiro tipo é o *mouse* optomecânico. Como o *mouse* mecânico de duas rodas, ele tem duas rodas montadas a 90 graus uma da outra. Entretanto, cada roda tem um LED no centro, uma série de fendas uniformemente espaçadas ao longo da circunferência da roda e um detector fora da roda. À medida que o *mouse* se movimenta, as rodas giram e pulsos de luz atingem os detectores sempre que uma fenda fica entre um LED e seu detector. O número de pulsos detectados é proporcional ao movimento.

Embora um *mouse* possa funcionar de diversas formas, um método comum é ele transmitir uma seqüência de três bytes para o computador a cada 100 ms. Geralmente, esses caracteres vêm através de uma linha RS-232-C, como se tivessem sido digitados em um teclado. O primeiro byte contém um número inteiro com sinal dizendo quantas unidades o *mouse* moveu no eixo x nos últimos 100 ms. O segundo fornece a mesma informação para o movimento no eixo y. O terceiro byte contém o estado atual dos botões do *mouse*. Algumas vezes são utilizados dois bytes para cada coordenada.

Um *software* de baixo nível no computador recebe esta informação e converte os movimentos relativos enviados pelo *mouse* em posições absolutas. Ele então mostra uma seta na tela na posição correspondente à posição do mouse. Quando a seta aponta o item desejado, o usuário aperta um botão do *mouse*, e o computador pode determinar qual item foi selecionado através do seu conhecimento da posição da seta na tela.

2.3.4 Impressoras

Embora terminais CRT possam ser suficientes para muitas aplicações interativas, para outras é necessário um registro permanente das saídas em papel. Para atender esta necessidade, muitos tipos de impressora têm sido desenvolvidos. Descreveremos brevemente a seguir os principais tipos de impressora e como eles funcionam.

Impressoras de Impacto

O tipo mais antigo de impressora, a impressora de impacto, funciona como um teletipo: um pedaço de metal ou plástico, com uma letra em alto-relevo, comprime uma fita tintada contra uma folha de papel, deixando uma imagem da letra no papel, como mostrado na Fig. 2.28(a). Nos computadores pessoais atuais, esta forma de impressão é usada nas impressoras tipo **margarida** (*daisy wheel*), assim chamadas porque elas possuem uma roda raiada como uma margarida, com os caracteres nas pétalas, como ilustrado na Fig. 2.28(b). Para imprimir um caractere, a impressora roda o caractere para a frente de um eletroímã, e então energiza o eletroímã, pressionando a letra contra a fita. Impressoras deste tipo produzem boa qualidade, especialmente quando usadas com fita carbonada, e atingem uma velocidade de 20 a 40 caracteres/s.

Grandes *mainframes* também usam impressoras de impacto, mas aqui as letras são gravadas em uma esteira de aço que contorna o papel. Uma impressora de 80 colunas terá 80 martelos, um para cada coluna. Uma linha é impressa instruindo cada martelo a bater logo que a letra apropriada estiver em frente a ele. A cada rotação da esteira, todos os caracteres irão aparecer em cada coluna, tornando possível imprimir a linha inteira. Essas impressoras podem geralmente imprimir uma página em poucos segundos ou menos.

Impressoras Matriciais

Outro tipo comum de impressora são as **impressoras matriciais**, nas quais a cabeça de impressão contém de 7 a 24 agulhas eletromagneticamente ativáveis e varrem cada linha impressa. Uma impressora barata deve ter sete agulhas para imprimir 80 caracteres, com uma caixa de 5 × 7, em cada linha. De fato, a linha impressa consiste em sete linhas horizontais, cada qual formada por 5 × 80 = 400 pontos. Cada ponto pode ser ou não impresso, dependendo dos caracteres a serem impressos. A Fig. 2.29(a) ilustra a letra "A" impressa com uma caixa de 5 × 7.

A qualidade de impressão pode ser aumentada através de duas técnicas: usando mais agulhas ou sobrepondo os pontos. A Fig. 2.29(b) mostra um "A" impresso usando 24 agulhas com pontos sobrepostos. Geralmente, são necessários múltiplos passos para cada linha para produzir pontos sobrepostos, e assim o aumento de qualidade implica reduzir a velocidade de impressão. Boas impressoras matriciais podem operar em vários modos, oferecendo diferentes compromissos entre qualidade de impressão e velocidade.

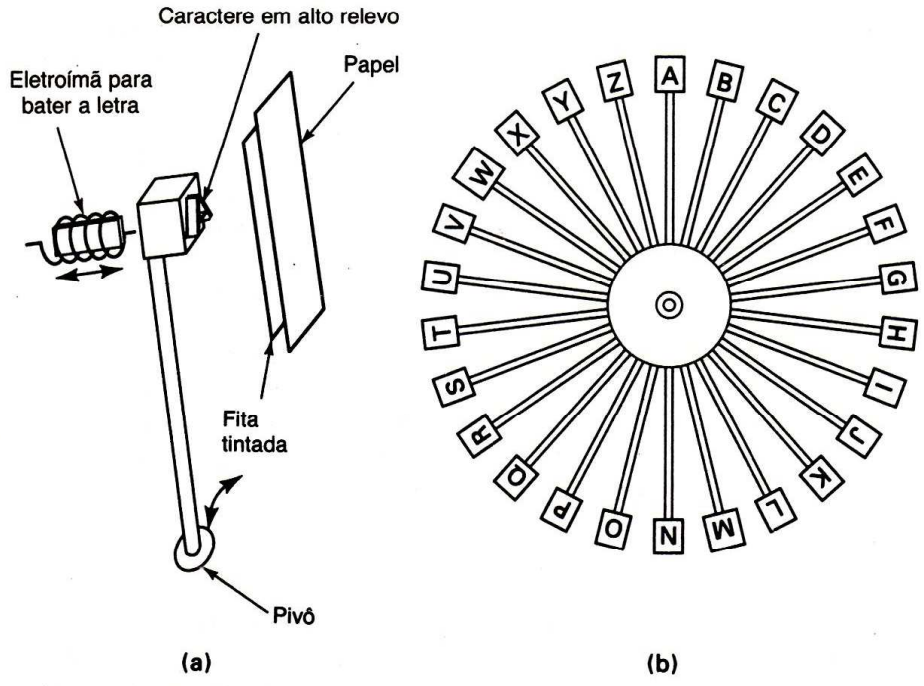


Fig. 2.28 (a) Seção de uma impressora de impacto. (b) Uma margarida.

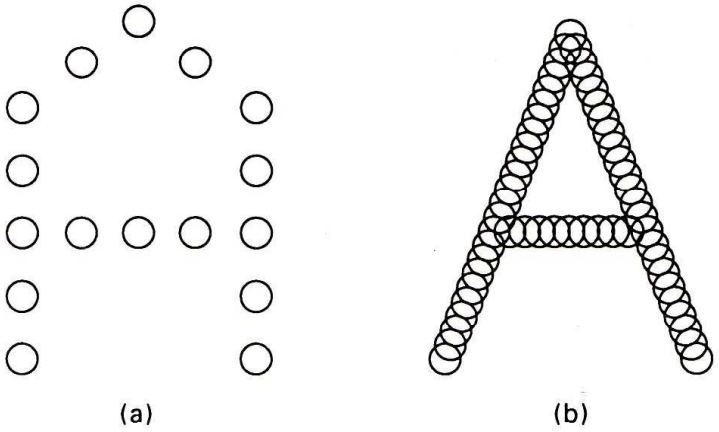


Fig. 2.29 (a) A letra "A" em uma matriz 5 × 7. (b) A letra "A" impressa com 24 agulhas sobrepostas.

Impressoras *Laser*

Provavelmente, o desenvolvimento mais excitante em impressão, desde que Johann Gutenberg inventou o tipo móvel no século XV, é a **impressora *laser***. Este dispositivo combina uma imagem de alta qualidade, excelente flexibilidade, boa velocidade e custo moderado em um único periférico. As impressoras *laser* usam praticamente a mesma tecnologia das máquinas fotocopiadoras, e provavelmente estarão disponíveis em breve em máquinas combinando copiadoras e impressoras.

A tecnologia básica é ilustrada na Fig. 2.30. O coração da impressora é um tambor rotativo de precisão. No início do ciclo de cada página, ele é carregado com cerca de 1000 volts e coberto com um material fotossensível. Então, a luz de um *laser* varre o comprimento do tambor de uma forma semelhante ao feixe eletrônico de um CRT, só que em vez de obter a deflexão horizontal utilizando tensão, um espelho octagonal rotativo é usado para varrer o tambor. O feixe de luz é modulado para produzir um padrão de pontos claros e escuros. Os pontos atingidos pelo feixe perdem sua carga elétrica.

Depois de uma linha de pontos ser desenhada, o tambor roda uma fração de grau para permitir o desenho da próxima linha. Eventualmente, a primeira linha de pontos atinge o *toner*, um reservatório de um pó preto eletrostaticamente sensível. O *toner* é atraído por aqueles pontos que ainda estão carregados, formando então a imagem visual daquela linha. Pouco tempo depois, o tambor coberto de *toner* é pressionado contra o papel, transferindo o pó preto para ele. O papel passa então através de rolos aquecidos que aderem o *toner* permanentemente ao papel,

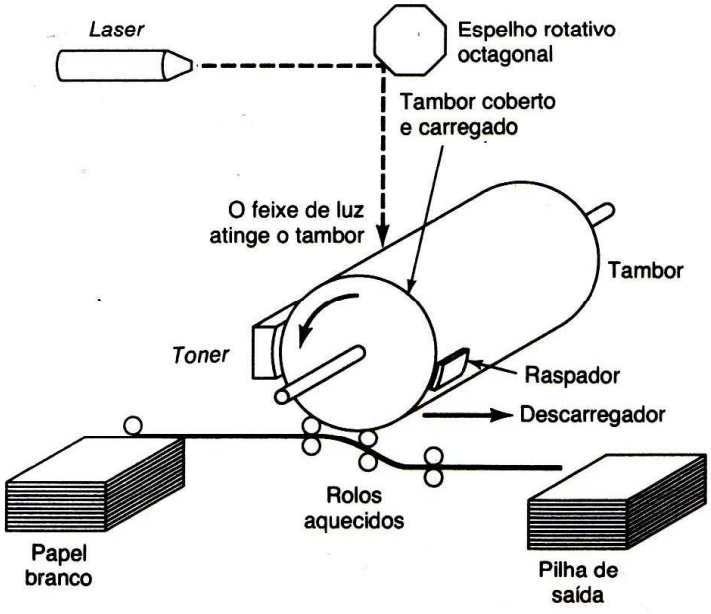


Fig. 2.30 Funcionamento em uma impressora *laser*.

fixando a imagem. Mais tarde em sua rotação, o tambor é descarregado e sofre uma limpeza de qualquer *toner* residual, preparando-se para ser carregado e coberto de novo para a próxima página.

Que este processo é uma combinação excessivamente complexa de física, química, engenharia mecânica e ótica, não precisa ser dito. Mesmo assim, montagens completas, chamadas **máquinas impressoras**, estão disponíveis por diversos vendedores. Os fabricantes de impressoras *laser* combinam essas máquinas impressoras com seus próprios circuitos e *software* para produzir uma impressora completa.

Um tipo ligeiramente diferente de máquina impressora que não usa um *laser* é também disponível. Neste tipo, uma linha de LEDs é montada ao longo do tambor (ou direciona para ele através de fibras óticas). Os LEDs são ligados e desligados por *software* para formar cada linha. Este projeto tem um projeto ótico mais simples, mas requer mais circuito para acionar todos os LEDs.

Existem duas abordagens diferentes para converter uma máquina de impressão em uma impressora completa. De um lado, o computador apenas fornece textos, com comandos para selecionar *fonts*, e talvez desenhar linhas, retângulos e círculos. Um computador dentro da impressora constrói o mapa de bits para cada página e o transfere para a máquina impressora. A vantagem desta abordagem é que a banda passante necessária entre a impressora e o computador pode ser muito baixa. A impressão de uma página de 50 linhas de 80 caracteres com um único *font* requer 4000 bytes de entrada. Uma impressora terminal funcionando a 12 páginas por minuto requer uma banda passante entre computador e impressora de 6400 bps, o que pode ser suprido por uma linha RS-232-C a 9600 baud. A desvantagem é a necessidade de um computador completo dentro da impressora. O computador deve ter cerca de um megabyte de memória para armazenar o mapa de bits para uma página de 8,5 por 11 polegadas, com uma densidade de 300 × 300 pontos por polegada quadrada.

Para evitar o gasto do computador e impressora, alguns sistemas usam o computador principal e sua memória para construir o mapa de bits e simplesmente transferem a imagem dos bits para a impressora. Este esquema torna a impressora muito barata, mas requer agora a transferência da imagem de um megabyte em 5 segundos, para uma taxa de transferência de 1,6 megabit/s. Caso contrário, a impressora pode tornar-se lenta se esta taxa não puder ser atingida.

Projetos intermediários entre esses dois extremos são também possíveis, mas, claramente, quando movemos para 600 pontos/polegada ou mais, é necessária uma considerável memória, além de maior poder computacional e maior banda passante. Infelizmente, sistemas tipográficos profissionais precisam de algo entre 1000 e 2000 pontos/polegada; assim, deveremos ter ainda que aumentar o desempenho no mínimo uma ordem de grandeza. Por um lado, a matriz para o original deste livro foi impressa usando uma impressora de LEDs com uma resolução efetiva de 450 pontos/polegada. Se você compará-lo com livros profissionalmente impressos, usando um microscópio de baixa potência, a diferença será bem aparente. Felizmente o conteúdo altamente intelectual sobrepõe a baixa resolução física.

A impressora *laser* fez aparecer uma nova indústria inteira, chamada **desktop publishing** (editoração eletrônica). O Macintosh da Apple, o IBM PC e outros computadores possuem *software* que permitem aos usuários digitar documentos e vê-los na tela do mesmo modo que eles serão na forma impressa. Este sistema é chamado geralmente **WYSIWYG** (***What You See Is What You Get***), pronunciado "*wiz e wig*". A maioria destes sistemas traduz o texto de entrada para uma linguagem intermediária chamada **PostScript**, e então o envia para a impressora. Na impressora existe um interpretador que converte *PostScript* em pixels e os imprime. *PostScript* é o padrão de fato para as impressoras *laser*.

2.3.5 Códigos de Caracteres

Cada computador possui um conjunto de caracteres que ele pode ler ou escrever. Este conjunto inclui, no mínimo, as 26 letras maiúsculas, os 10 dígitos de 0 a 9 e alguns sinais de pontuação, tais como espaço, ponto, hífen, vírgula, e o caractere de retorno de carro. Os conjuntos de caracteres mais sofisticados incluem letras maiúsculas e minúsculas, os 10 dígitos, uma grande variedade de pontuação, uma coleção de caracteres especiais úteis em matemática e no comércio, e algumas vezes letras gregas.

Caracteres de controle

0	NUL	Null	20	DLE	Data link escape
1	SOH	Start of heading	21	DC1	Device control 1
2	STX	Start of text	22	DC2	Device control 2
3	ETX	End of text	23	DC3	Device control 3
4	EOT	End of transmission	24	DC4	Device control 4
5	ENQ	Enquiry	25	NAK	Negative acknowledge
6	ACK	Acknowledge	26	SYN	Synchronous idle
7	BEL	Bell	27	ETB	End of transmission block
10	BS	Backspace	30	CAN	Cancel
11	HT	Horizontal tab	31	EM	End of medium
12	LF	Line feed	32	SUB	Substitute
13	VT	Vertical tab	33	ESC	Escape
14	FF	Form feed	34	FS	File separator
15	CR	Carriage return	35	GS	Group separator
16	SO	Shift out	36	RS	Record separator
17	SI	Shift in	37	US	Unit separator

40	(Space)	60	0	100	@	120	P	140		160	p
41	!	61	1	101	A	121	Q	141	a	161	q
42	"	62	2	102	B	122	R	142	b	162	r
43	#	63	3	103	C	123	S	143	c	163	s
44	$	64	4	104	D	124	T	144	d	164	t
45	%	65	5	105	E	125	U	145	e	165	u
46	&	66	6	106	F	126	V	146	f	166	v
47	'	67	7	107	G	127	W	147	g	167	w
50	(	70	8	110	H	130	X	150	h	170	x
51	)	71	9	111	I	131	Y	151	i	171	y
52	*	72	:	112	J	132	Z	152	j	172	z
53	+	73	;	113	K	133	[	153	k	173	{
54	'	74	<	114	L	134	\	154	l	174	|
55	−	75	=	115	M	135	]	155	m	175	}
56	.	76	>	116	N	136	^	156	n	176	~
57	/	77	?	117	O	137	—	157	o	177	(Delete)

Fig. 2.31 Código de caracteres ASCII (em octal).

Para transferir esses caracteres para o computador, a cada um é associado um número: por exemplo, a = 1, b = 2, ..., z = 26, + = 27, − = 28. O mapeamento dos caracteres em números inteiros é chamado de **código de caracteres**. Os computadores atuais usam códigos de 6, 7, 8 ou 9 bits. Um código de 6 bits permite apenas $2^6 = 64$ caracteres, a saber, 26 letras, 10 dígitos e outros 28 caracteres entre símbolos matemáticos e sinais de pontuação.

Para muitas aplicações, 64 caracteres não são suficientes, e, neste caso, um código de 7 ou 8 bits precisa ser usado. Um código de 7 bits permite até 128 caracteres. Um código de 7 bits amplamente usado é o chamado código **ASCII (American Standard Code for Information Interchange)**. A Fig. 2.9 mostra o código ASCII. Os códigos de 1 a 37 (octal) são caracteres de controle e não são imprimíveis. O código de 8 bits mais usado é o **EBCDIC** da IBM, usado em grandes *mainframes* IBM. Ele está obsoleto, mas, por razões de compatibilidade, a IBM continua utilizando-o.

2.4 RESUMO

Os sistemas de computadores são construídos a partir de três tipos de componentes: processadores, memórias e dispositivos de E/S. A tarefa de um processador é buscar instruções, uma de cada vez, da memória, decodificá-las e executá-las. O ciclo de busca-decodificação-execução pode sempre ser descrito por um algoritmo, e, de fato, é muitas vezes executado por um programa interpretador que é executado em um nível mais baixo.

Sistemas capazes de executar instruções em paralelo estão se tornando cada vez mais comuns. Computadores paralelos, do tipo SIMD, incluem os processadores vetoriais e os *array processors*. Os computadores MIMD são sistemas multiprocessadores que usam vários esquemas para interligar os processadores e memórias.

As memórias podem ser classificadas em principal e secundária. A memória principal é usada para conter o programa que está sendo executado. Seu tempo de acesso é pequeno — umas poucas centenas de nanossegundos no máximo — e independente do endereço acessado. A maioria das memórias são equipadas com códigos de correção de erro para aumentar sua confiabilidade. As memórias secundárias, ao contrário, possuem tempos de acesso muito maiores (milissegundos ou mais) e que são dependentes da posição do dado a ser lido ou escrito. As memórias secundárias mais comuns são fitas magnéticas, discos magnéticos e discos óticos.

Os dispositivos de E/S são usados para transferir informações de e para o computador. Como exemplo podemos citar os terminais, *mice* e impressoras *laser*. Em *mainframes*, os dispositivos de E/S são ligados a canais de dados. Em máquinas menores eles são acionados por controladores ligados diretamente ao barramento do sistema. Dispositivos de E/S comuns incluem os terminais, *mice*, modems e impressoras. A maioria dos dispositivos de E/S utilizam o código ASCII, exceto em *mainframes* da IBM, onde é usado o EBCDIC.

PROBLEMAS

1. Qual é o propósito do passo 2 na lista da seção 2.1.1? Que aconteceria se esse passo fosse omitido?

2. Um certo cálculo é altamente seqüencial — isto é, cada passo depende do anterior. Seria um *array processor* ou um processador *pipeline* o mais apropriado para este cálculo? Explique.

3. Para competir com a imprensa inventada mais recentemente, um mosteiro medieval decidiu produzir livros manuscritos em massa colocando um vasto número de escribas em um imenso *hall*. O monge chefe iria dizer a primeira palavra do livro a ser produzido e todos os escribas iriam copiá-la. Então, o monge diria a segunda palavra e os escribas a copiariam. Esse processo seria repetido até que todo o livro fosse ditado e copiado. Com qual sistema de processadores em paralelo, da seção 2.1.3, esse método mais se assemelha?

4. Estime quantos caracteres tem um livro típico, incluindo os espaços. Quantos bits seriam necessários para codificar um livro em ASCII com paridade? Quantas fitas de 1600 bpi seriam necessárias para armazenar uma biblioteca de 10^6 livros?

5. Estime a capacidade máxima de armazenamento do cérebro humano usando as seguintes suposições. Toda a memória é codificada usando moléculas de DNA. Uma molécula de DNA é uma seqüência linear de quatro nucleotídeos básicos *A*, *C*, *G* e *T*. Do peso médio de um nucleotídeo, cerca de 10^{-20} g, e um peso médio de um cérebro de 1500 g, deduza a capacidade de bits do cérebro para esta forma de codificação. *Nota:* Este cálculo é apenas um limite superior, porque o cérebro contém células que fazem outra função diferente de memória.

6. Quais das seguintes memórias são possíveis? Quais são razoáveis? Explique.

   a. endereço de 10 bits, 1024 células, células de 8 bits
   b. endereço de 10 bits, 1024 células, células de 12 bits
   c. endereço de 9 bits, 1024 células, células de 10 bits
   d. endereço de 11 bits, 1024 células, células de 10 bits
   e. endereço de 10 bits, 10 células, células de 1024 bits
   f. endereço de 1024 bits, 10 células, células de 10 bits

7. Os sociologistas podem obter três respostas possíveis para uma pergunta típica, tal como "Você acredita em Papai Noel?" — a saber, sim, não e nenhuma resposta. Com isto, a Companhia de Computação Sociomagnética decidiu construir um computador para processar os dados da pesquisa. Este computador tem uma memória trinária — isto é, cada byte (tryte?)

consiste em 8 trits, sendo que um trit pode conter um 0, 1 ou 2. Quantos trits são necessários para armazenar um número de 6 bits? Dê a expressão geral para o número de trits para representar n bits.

8. Certo computador pode ser equipado com 262.144 bytes de memória. Por que o fabricante escolheu um número tão peculiar, ao invés de um número fácil de lembrar como 250.000?

9. A taxa de transferência entre a CPU e sua memória é de ordens de grandeza maiores que a taxa de transferência de dispositivos de E/S mecânicos. Como esta diferença causa ineficiências?

10. Meça sua própria velocidade de leitura. Como sua taxa de dados se compara a uma linha telefônica de 300 bps?

11. Calcule a taxa de dados do olho humano usando as seguintes informações. O campo visual consiste em cerca de 10^6 elementos (pixels). Cada pixel pode ser reduzido a uma superposição das três cores primárias, cada uma com 64 intensidades. A resolução de tempo é 100 ms.

12. Quanto tempo é gasto para ser lido um disco de 800 cilindros, cada um com cinco trilhas de 32 setores? Primeiramente, todos os setores da trilha 0 são lidos começando do setor 0. Então, todos os setores da trilha 1 a partir do setor 0, e assim por diante. O tempo de rotação é de 20 ms, e um *seek* gasta 10 ms entre cilindros adjacentes e 50 ms para o pior caso. A comutação entre trilhas no mesmo cilindro pode ser feita instantaneamente.

13. Um computador tem um barramento com um ciclo de 250 ns, durante o qual ele pode ler ou escrever uma palavra de 32 bits na memória. O computador tem um disco que usa este barramento e que tem uma taxa de transferência de 4 Mbytes/s. A CPU normalmente busca e executa uma instrução de 32 bits a cada 250 ns. Em quanto o disco torna a CPU mais lenta?

14. O diretor de seu centro de computação local decidiu que o lugar apropriado para as perfuradoras de cartões, leitoras de cartões e equipamentos similares é o Museu Nacional. Entretanto, para tê-los em lugar seguro, ele decidiu salvar todos os *decks* de cartão existentes em uma fita magnética. Se o *gap* entre registros em uma fita de 1600 bpi é de 0,25 polegada, e 15 cartões de 80 colunas são gravados em um registro físico, quantas caixas de 2000 cartões serão gravadas em uma fita de 2400 pés?

15. Decodifique o seguinte texto binário ASCII: 1001001 0100000 1001100 1001111 1010110 1000101 0100000 1011001 1001111 1010101 0101110.

16. Projete um código de Hamming de paridade par para os dígitos de 0 a 9.

17. Projete um código para os dígitos de 0 a 9, cuja distância Hamming é 2.

18. Em um código de Hamming, alguns bits são "desperdiçados" no sentido de que eles são usados para verificação, e não para informação. Qual é a percentagem de bits desperdiçados para mensagens cujo comprimento total (dados + bits de verificação) é $2^n - 1$. Calcule numericamente esta expressão para valores de n de 3 a 10.

19. Erros de transmissão em uma linha telefônica geralmente ocorrem em rajadas, em vez de individualmente. Como o código básico de Hamming só pode corrigir erros simples em um caractere, ele não tem utilidade se o ruído deturpa n bits consecutivos. Projete um método de transmitir um texto ASCII em uma linha onde ruídos podem deturpar tantos quantos 100 bits consecutivos. Assuma que o intervalo mínimo entre duas rajadas de ruídos é de milhares de caracteres. *Sugestão:* Pense cuidadosamente na ordem de transmissão dos bits.

20. Quando um texto ASCII com paridade par é transmitido assincronamente a uma taxa de 10 caracteres por segundo em uma linha de 110 bps, qual a percentagem dos bits recebidos que realmente contêm dados (ao contrário de *overhead*)?

21. Certo terminal ASCII assíncrono verifica a paridade e imprime a 60 caracteres por segundo. Qual a velocidade que ele usa: 110, 300, 600 ou 1200 bps?

22. Um terminal *bit-map* possui uma tela de 1000 × 1000. A tela é redesenhada 50 vezes por segundo. Qual a duração do pulso correspondente a um pixel?

23. Um fabricante anuncia que seu terminal colorido *bit-map* pode mostrar 2^{24} cores diferentes. Porém o *hardware* tem apenas 1 byte para cada pixel. Como isto pode ser feito?

24. A Companhia de Modems Hi-Fi acabou de projetar um novo modem de modulação em freqüência, que usa 16 freqüências ao invés de 2. Cada segundo é dividido em n intervalos iguais de tempo, cada qual contendo um dos 16 tons possíveis. Quantos bits por segundo pode este modem transmitir, usando transmissão síncrona?

25. Quantos caracteres por segundo (7 bits + paridade) podem ser transmitidos através de uma linha de 2400 bps em cada um dos seguintes modos?

a. Síncrono
b. Assíncrono (1 *start bit* e 1 *stop bit*)

26. Escreva um procedimento *hamming* (*ascii*, *codificado*) que converte os 7 bits de baixa ordem de *ascii* em uma palavra-código inteira de 11 bits em *codificado*.

27. Escreva uma função *distância* (*código*, *n*, *k*) que recebe um arranjo *código*, de *n* caracteres de *k* bits cada, como entrada, e retorna a distância do conjunto de caracteres como saída.

4

O NÍVEL DE MICROPROGRAMAÇÃO

O limite entre *hardware* e *software* não é bem definido e, além disso, está constantemente se alterando. Os primeiros computadores tinham instruções para operações aritméticas, booleanas, deslocamentos, comparações, *loops,* entre outras, que eram executadas diretamente pelo *hardware.* Para cada instrução, um circuito específico de *hardware* estava presente para executá-la. Poder-se-ia, pelo menos em princípio, desparafusar o painel traseiro e apontar os componentes eletrônicos usados pela instrução de divisão.

Num computador multinível moderno, não é mais possível isolar os circuitos de divisão, porque não existem circuitos de divisão. Todas as instruções disponíveis no nível de máquina convencional (por exemplo, instruções aritméticas, booleanas, deslocamentos, comparações e *loops*) são executadas passo a passo por um interpretador executado no nível de microprogramação. O equivalente atual de olhar para os circuitos de divisão é obter uma listagem do microprograma e procurar pela parte que interpreta as instruções de divisão.

Embora programas em qualquer nível possam ser executados por um *software* interpretador, e embora este interpretador possa, também, ser executado por outro interpretador, essa hierarquia não pode prosseguir indefinidamente. No nível mais baixo, deve existir fisicamente uma máquina em *hardware,* com circuitos integrados, fontes de alimentação e objetos "sólidos" similares. Estes itens foram o tema do capítulo anterior. Neste capítulo, iremos estudar como os componentes eletrônicos são controlados pelo microprograma e como o microprograma interpreta o nível de máquina convencional. No capítulo 8, estudaremos uma classe de máquinas que não são microprogramadas (máquinas RISC).

Como a arquitetura do nível de microprogramação, chamada de **microarquitetura**, é definida pelo *hardware*, ela é normalmente primitiva e difícil de programar. As considerações de tempo são, por exemplo, freqüentemente importantes. Estas considerações levaram Rosin (1974) a definir microprogramação como "a implementação de sistemas esperançosamente razoáveis através da interpretação em máquinas não-razoáveis".

O nível de microprogramação tem uma função específica: executar interpretadores para outras máquinas virtuais (esperançosamente mais razoáveis). Esta meta de projeto leva naturalmente a uma organização altamente otimizada no sentido de buscar, examinar e executar instruções de máquina convencional e, em alguns casos, instruções mais sofisticadas. Os problemas e compromissos envolvidos na organização e projeto deste nível serão examinados neste capítulo.

Começaremos nosso estudo do nível de microprogramação revendo brevemente os blocos construtores básicos discutidos no capítulo 3, porque eles são parte da arquitetura do nível de microprogramação, e portanto de interesse para o microprogramador. (Um "microprogramador" é alguém que escreve microprogramas, e não um programador pequeno.) A seguir, chegamos ao âmago da questão: como instruções mais complexas podem ser construídas a partir de seqüências de instruções primitivas. Este tópico será discutido em detalhes e ilustrado por um extenso exemplo. Posteriormente, examinaremos os vários fatores que devem ser considerados ao se projetar o nível de microprogramação de um computador, para melhor compreender por que ele é do jeito que é. Olharemos também para as maneiras de melhorar o desempenho do computador. Final-

mente, examinaremos o nível de microprogramação de nossos dois exemplos correntes, as famílias Intel e Motorola.

4.1 REVISÃO DO NÍVEL DE LÓGICA DIGITAL

O trabalho do microprogramador é escrever um programa para controlar os registradores da máquina, seus barramentos, ALUs, memórias e outros componentes de *hardware*. Nós estudamos esses dispositivos no capítulo anterior; agora iremos apenas revisá-los brevemente para colocá-los em perspectiva. Após a revisão, também falaremos um pouco dos diferentes modos conceituais de encapsulamento dos componentes.

4.1.1 Registradores

Um registrador é um dispositivo capaz de armazenar informações. O nível de microprogramação sempre tem alguns registradores disponíveis para conter as informações necessárias para o processamento da instrução que está sendo correntemente interpretada. Conceitualmente, os registradores são o mesmo que memória principal, a diferença é que os registradores estão fisicamente localizados dentro do próprio processador, de modo que eles podem ser lidos e escritos mais rapidamente do que palavras na memória principal, que usualmente estão em pastilhas isoladas. Máquinas maiores e mais caras normalmente possuem mais registradores do que as menores e mais baratas, que precisam usar a memória principal para armazenar resultados intermediários. Em alguns computadores, um grupo de registradores, numerados 0, 1, 2, ..., n-1, é disponível ao nível de microprogramação, e é chamado **armazenamento local** ou **memória de rascunho.**

Um registrador pode ser caracterizado por um único número: quantos bits ele pode armazenar. A Fig. 4.1 mostra um registrador de 16 bits com a convenção de numeração de bits utilizada neste livro. A informação colocada num registrador permanece lá até que outra informação a substitua. O processo de ler a informação de um registrador não afeta o conteúdo do mesmo. Em outras palavras, quando um registrador é lido, uma cópia de seu conteúdo é feita e o original é deixado intacto no registrador.

4.1.2 Barramentos

Um barramento é uma coleção de fios usados para transmitir sinais em paralelo. Tipicamente, os barramentos são usados para permitir que o conteúdo de um registrador possa ser copiado para outro. Diferente dos barramentos de sistema que estudamos no capítulo 3, esses barramentos apenas conectam dois dispositivos e, portanto, não há necessidade de linhas de endereço e linhas de controle extensivo. As n linhas de dados e uma ou duas linhas de controle geralmente são suficientes. Barramentos são usados porque a transferência paralela de todos os bits de uma vez é muito mais rápida do que a transferência serial bit a bit.

Um barramento pode ser unidirecional ou bidirecional. Um barramento unidirecional só pode transferir dados em um sentido, enquanto que um barramento bidirecional pode transferir dados nos dois sentidos, mas não em ambos simultaneamente. Barramentos unidirecionais são tipicamente utilizados para conectar dois registradores, um dos quais é sempre a origem e o outro é sempre o destino. Barramentos bidirecionais são tipicamente utilizados quando qualquer um de um conjunto de registradores pode ser a origem e qualquer outro pode ser o destino.

Muitos dispositivos são capazes de se conectarem e de se desconectarem eletricamente dos barramentos aos quais estão fisicamente ligados. Estas conexões podem ser feitas ou desfeitas em nanossegundos. Um barramento cujos dispositivos possuem essa propriedade é chamado de

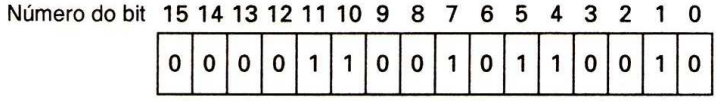


Fig. 4.1 Um registrador de 16 bits pode conter 16 bits de informação.

barramento ***tri-state***, pois cada linha pode ser 0,1 ou estar desconectada. Barramentos *tri-state* são comumente usados quando há muitos dispositivos ligados a um barramento, todos podendo potencialmente colocar informações neste último.

Na maioria das microarquiteturas, alguns registradores estão conectados a um ou mais barramentos de entrada e a um ou mais barramentos de saída. A Fig. 4.2(a) mostra um registrador de 8 bits conectado a um barramento de entrada e a um barramento de saída. O registrador consiste em oito *flip-flops* tipo D, cada um conectado ao barramento de saída através de um *buffer* não-inversor. Cada um armazena 1 bit. O registrador possui dois sinais de controle: CK (*Clock*, que na realidade significa "carrega registrador") e OE (*Output Enable*, habilitação da saída), ambos conectados a todos os *flip-flops*. Normalmente, ambos os sinais estão em seu estado quiescente, mas ocasionalmente eles podem ser ativados, fazendo com que a ação correspondente aconteça.

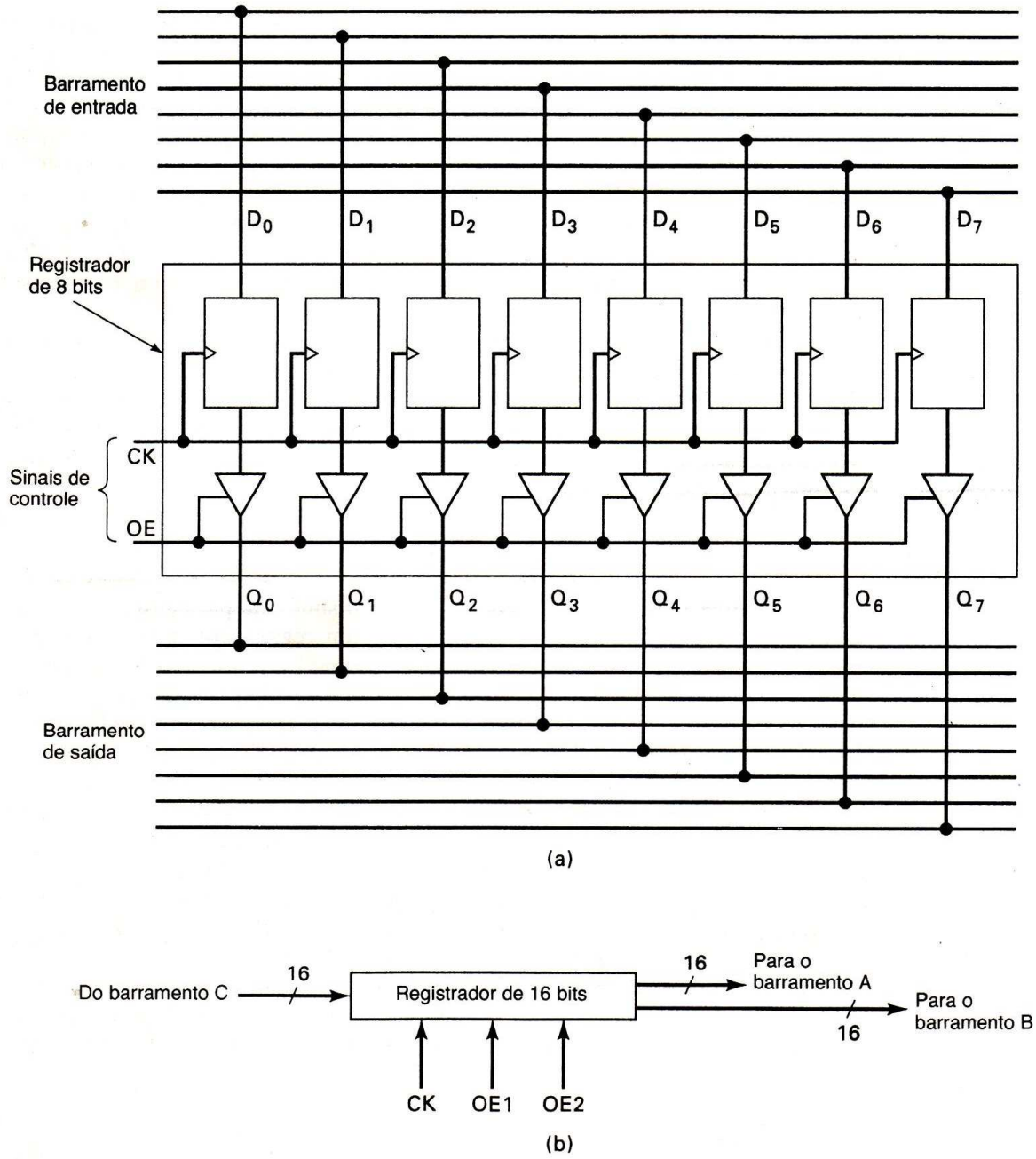


Fig. 4.2 (a) Detalhe de um registrador de 8 bits conectado a um barramento de entrada e um barramento de saída. (b) Representação simbólica de um registrador de 16 bits com um barramento de entrada e dois barramentos de saída.

Quando CK está desativado, o conteúdo do registrador não é afetado pelos sinais do barramento. Quando CK é ativado, o registrador é carregado a partir do barramento de entrada. Quando OE é desativado, o registrador é desconectado do barramento de saída e efetivamente deixa de existir em relação aos outros registradores do barramento. Quando OE é ativado, o conteúdo do registrador é colocado no barramento de saída.

Se outro registrador, *R*, tiver sua entrada conectada ao barramento de saída deste registrador, é possível transferir informação deste registrador para *R*. Para fazê-lo, OE deve ser ativado e mantido ativo por tempo suficiente para que a saída do barramento se torne estável. Então, a linha CK do registrador *R* deve ser ativada, carregando *R* a partir do barramento. A operação de habilitar a saída de um registrador em um barramento para que outro registrador possa obter o seu valor ocorre freqüentemente no nível de microprogramação, como veremos em breve. Como um segundo exemplo de registradores e barramentos, a Fig. 4.2(b) mostra um registrador de 16 bits com dois barramentos de saída, cada um controlado por um sinal OE diferente.

4.1.3 Multiplexadores e Decodificadores

Os circuitos que possuem uma ou mais linhas de entrada e calculam um ou mais valores de saída que são determinados unicamente pelas entradas atuais são chamados **circuitos combinatórios.** Dois dos mais importantes são os multiplexadores e os decodificadores. Um **multiplexador** tem 2^n entradas de dados (linhas individuais ou barramentos), uma saída de dados da mesma largura que as entradas e uma entrada de controle de n bits que seleciona uma das entradas e a direciona para a saída. A Fig. 4.3(a) mostra um multiplexador com dois barramentos como entrada. O sinal de controle de 1 bit seleciona *A* ou *B* como saída. A Fig. 3.12 mostra o circuito para um multiplexador de oito entradas.

O inverso de um multiplexador é um **demultiplexador**, que direciona sua entrada única para uma de suas 2^n saídas, dependendo do valor de suas n linhas de controle.

Outro importante circuito combinatório é o **decodificador,** que possui n linhas de entrada e 2^n linhas de saída, numeradas de 0 a $2^n - 1$. Se o número binário nas linhas de entrada é k, então a linha de saída k será 1 e todas as outras linhas de saída serão 0. Um decodificador sempre tem exatamente uma linha de saída em 1, com as restantes em 0. A Fig. 4.3(b) ilustra simbolicamente um decodificador de 4-para-16; a Fig. 3.14 mostra o circuito para um decodificador 3-para-8.

O inverso de um decodificador é um **codificador**, que possui 2^n entradas e n saídas. Somente uma linha de entrada pode estar em 1, e seu número, em binário, aparece na saída.

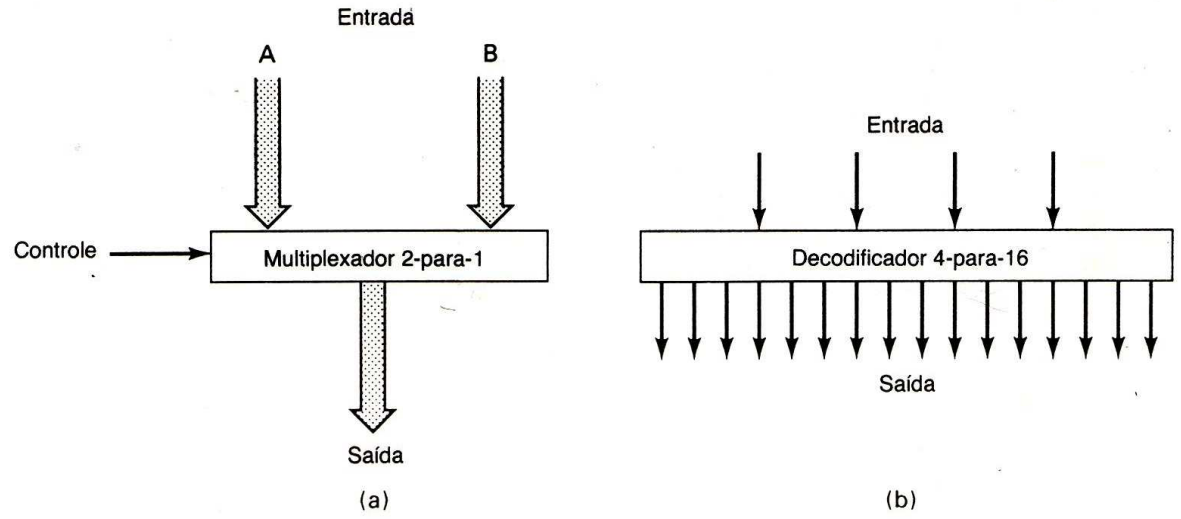


Fig. 4.3 (a) Um multiplexador 2-para-1. (b) Um decodificador 4-para-16.

4.1.4 ALUs e Deslocadores

Todo computador precisa, de alguma maneira, realizar operações aritméticas. O circuito mais simples é apenas um somador, que pega duas entradas de n bits e produz sua soma como saída. Um circuito aritmético mais geral é a **ALU** (*Arithmetic Logical Unit*), ou **Unidade Lógica e Aritmética.** Ela também possui duas entradas de dados e uma saída de dados, mas também possui algumas entradas e saídas de controle. A ALU da Fig. 4.4(a) tem dois bits de função, F_0 e F_1, que determinam qual a função que a ALU deve realizar. A ALU que usaremos em nosso exemplo pode calcular $A + B$, A AND B, A e também $\bar{A}$. As duas primeiras funções são auto-explicativas; a terceira simplesmente copia A para a saída; a quarta produz o inverso de A. A terceira função pode parecer inútil, mas veremos mais tarde para que ela é usada. A Fig. 3.20 mostra uma ALU de quatro funções que opera em um bit de dado. Repetindo este circuito n vezes e alimentando o vai-1 do bit i para o bit $i + 1$, podemos fazer um ALU de n bits que calcula AND, OR, NOT e adição.

Uma ALU pode também ter saídas de controle. Saídas típicas são linhas que estão em 1 quando a saída da ALU é negativa, quando é zero, quando existe um vai-1 do bit de mais alta ordem, ou quando ocorreu *overflow*. O exemplo da Fig. 4.4(a) tem duas saídas de controle: N, que indica que a saída da ALU é negativa, e Z, que indica que a saída da ALU é zero. O bit N é apenas uma cópia do bit de mais alta ordem da saída. O bit Z é o NOR de todos os bits de saída da ALU.

Embora algumas ALUs possam também realizar operações de deslocamento, na maioria das vezes é necessário ter uma unidade de deslocamento separada. Este circuito pode deslocar uma entrada multibit de 1 bit para a esquerda ou direita, ou, ainda, não realizar nenhum deslocamento. A Fig. 4.4(b) é o símbolo que usaremos para um deslocador; um circuito deslocador pode ser encontrado na Fig. 3.17.

4.1.5 Relógios (Clocks)

Os circuitos de computadores são normalmente acionados por um *relógio*, um dispositivo que emite uma seqüência periódica de pulsos. Estes pulsos definem os ciclos de máquina. Durante cada ciclo de máquina, ocorre alguma atividade básica, tal como a execução de uma microinstrução. É freqüentemente útil dividir o ciclo em subciclos, de modo que diferentes partes da microinstrução possam ser realizadas em uma ordem bem definida. Por exemplo, as entradas para a ALU devem ser colocadas disponíveis e se tornarem estáveis antes que a saída possa ser armazenada.

A Fig. 4.5(a) mostra um relógio simbólico com quatro saídas. A de cima é a saída primária; as outras três são derivadas dela pela inserção de diversos atrasos nas linhas de saída. O relógio primário mostrado na Fig. 4.5(b) (linha superior) tem uma largura de pulso igual a um quarto do período do ciclo. As outras três saídas são obtidas por atraso de uma, duas e três vezes

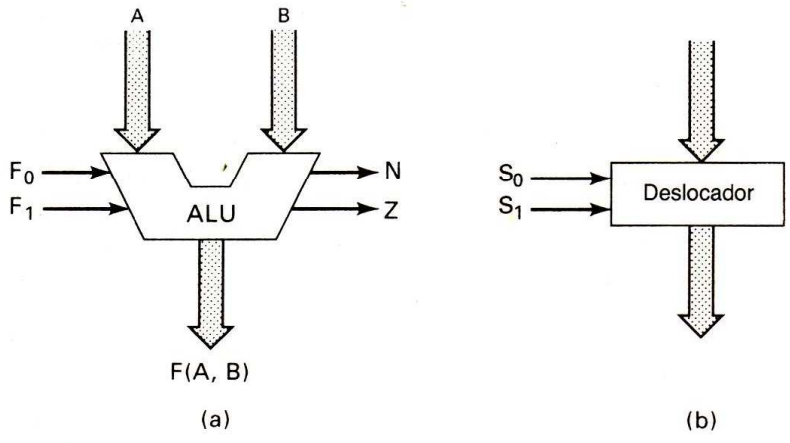


Fig. 4.4 (a) Uma ALU. (b) Um deslocador.

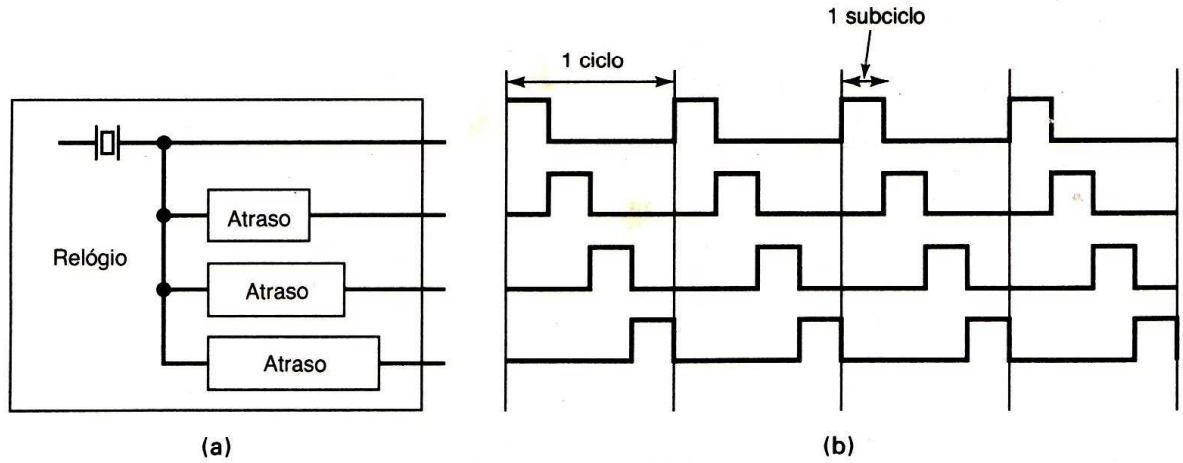


Fig. 4.5 (a) Um relógio com quatro saídas. (b) O diagrama de temporização de saída.

a largura do pulso. O resultado é um circuito que divide cada ciclo em quatro subciclos de mesma duração. Para mais detalhes, veja a Fig. 3.21.

Para provocar quatro transições diferentes que devem ocorrer em um ciclo em uma certa ordem, o projetista do computador pode fazer um AND do sinal habilitador de cada um com uma linha diferente do relógio. A transição ligada à linha de relógio primária ocorrerá primeiro, a transição ligada à linha de relógio com o menor atraso ocorrerá em segundo lugar, e assim por diante.

4.1.6 Memória Principal

Os processadores precisam ser capazes de ler dados da memória e escrever dados na memória. A maioria dos computadores possui um barramento de endereço, um barramento de dados e um barramento de controle para a comunicação entre a CPU e a memória. Para ler da memória, a CPU coloca um endereço de memória no barramento de endereço e estabelece os sinais de controle apropriados, ativando RD (READ), por exemplo. A memória, então, coloca o conteúdo requisitado no barramento de dados. Em alguns computadores, a leitura/escrita da memória é síncrona, isto é, a memória deve responder dentro de um tempo fixo. Em outros, a memória pode demorar quanto quiser, sinalizando a presença de dados usando uma linha de controle quando terminar.

Escritas na memória são feitas de modo similar. A CPU coloca o dado a ser escrito no barramento de dados e o endereço a ser armazenado no barramento de endereços, e então ativa WR (WRITE). (Uma alternativa para RD e WR é ter MREQ, que indica que se deseja uma requisição de memória, e RW, que distingue leitura de escrita.)

Um acesso à memória é quase sempre consideravelmente mais demorado que o tempo necessário para executar uma única microinstrução. Conseqüentemente, o microprograma deve manter os valores corretos nos barramentos de dados e endereços por várias microinstruções. Para simplificar esta tarefa, é muitas vezes conveniente ter-se dois registradores, o **MAR** (*Memory Address Register,* ou registrador de endereço de memória) e o **MBR** (*Memory Buffer Register,* ou registrador de dados de memória), que alimentam os barramentos de endereços e dados, respectivamente. Para os nossos propósitos, será conveniente organizar os barramentos como indicado na Fig. 4.6. Ambos os registradores ficam entre a CPU e o barramento do sistema. O barramento de endereços é unidirecional em ambos os lados e é carregado do lado da CPU quando a linha de controle é ativada. A saída para as linhas de endereço do sistema está sempre habilitada [ou possivelmente apenas durante as leituras e escritas, o que requer uma linha de habilitação de saída ativada pelo OR de RD e WR (não mostrado)]. A linha de controle de MBR faz com que o dado seja carregado do barramento de "Entrada de dados" no lado da CPU. A linha de "Saída de dados" está sempre habilitada. O barramento de dados do sistema é bidirecional, dando saída de MBR quando WR é ativado e carregando MBR quando RD é ativado.

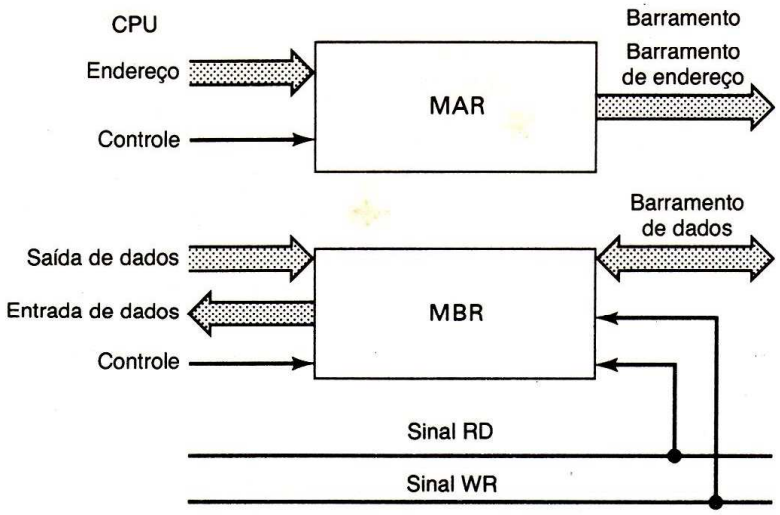


Fig. 4.6 Os registradores usados para acionar os barramentos de endereço e dados.

4.1.7 Encapsulamento de Componentes

Nas seções anteriores revimos vários circuitos que podem ser combinados para formar um computador. Estes circuitos são comercialmente disponíveis em várias formas conceitualmente diferentes. A maneira mais direta é em encapsulamento MSI (*Medium Scale Integration,* ou integração em média escala), com cada pastilha contendo um componente: por exemplo, um registrador, uma ALU ou um deslocador. Esta abordagem é ilustrada na Fig. 4.7(a). Os componentes são, então, interligados para formar o computador. Como existe uma grande variedade de pastilhas MSI de alta velocidade e de baixo custo disponíveis, muitos computadores foram construídos desse modo.

O principal inconveniente de construir um computador a partir de componentes MSI é o grande número de pastilhas necessárias que ocupam muitas placas, consomem muita energia e dissipam uma quantidade de calor significativa. Outra técnica é utilizar pastilhas **bit-slice.** Cada pastilha **bit-slice** tem, por exemplo, 1 bit para os registradores, ALU e outros componentes. A Fig. 3.20 mostra como uma ALU *bit-slice* de 1 bit é por dentro. Poderíamos facilmente estender esse projeto para adicionar, por exemplo, 16 registradores de 1 bit, um deslocador de 1 bit e outros componentes com largura de 1 bit. Pegando, digamos, 32 pastilhas desse tipo e colocando-as lado a lado, obtemos uma máquina com 16 registradores de 32 bits, uma ALU de 32 bits, um deslocador de 32 bits, e assim por diante.

Alternativamente, com apenas 16 pastilhas poderíamos construir uma máquina de 16 bits. A técnica de *bit-slice* fornece ao projetista a capacidade de compor facilmente uma máquina com qualquer largura de palavra. São amplamente disponíveis pastilhas *bit-slice* de 2 ou mesmo 4 bits. A Fig. 4.7(b) mostra uma máquina de 8 bits construída a partir de quatro unidades *bit-slice* de 2 bits. Em geral, a abordagem *bit-slice* requer menos pastilhas e muito menos tempo de projeto que a abordagem MSI, mas usualmente produz máquinas mais lentas.

Uma terceira técnica para combinar os componentes é colocar um processador completo em uma única pastilha [veja a Fig. 4.7(c)]. Enquanto isto obviamente reduz drasticamente o número de pastilhas (para uma pastilha), também tem algumas desvantagens. Para começar, a tecnologia necessária para encapsular grande número de componentes em uma pastilha é diferente da utilizada para pastilhas MSI ou *bit-slice* e normalmente produz máquinas mais lentas. Além disso, as tecnologias tanto de projeto quanto de fabricação são excessivamente complicadas e caras. Ao contrário, qualquer engenheiro eletrônico competente pode projetar um computador simples usando pastilhas MSI ou *bit-slice* sem muitos problemas. Do ponto de vista de um fabricante de computador que espera construir computadores por vários anos, provavelmente vale a pena dominar as tecnologias necessárias para a fabricação de processadores em uma única pastilha; para uma companhia que precisa apenas de uma máquina de uso específico, provavelmente não.

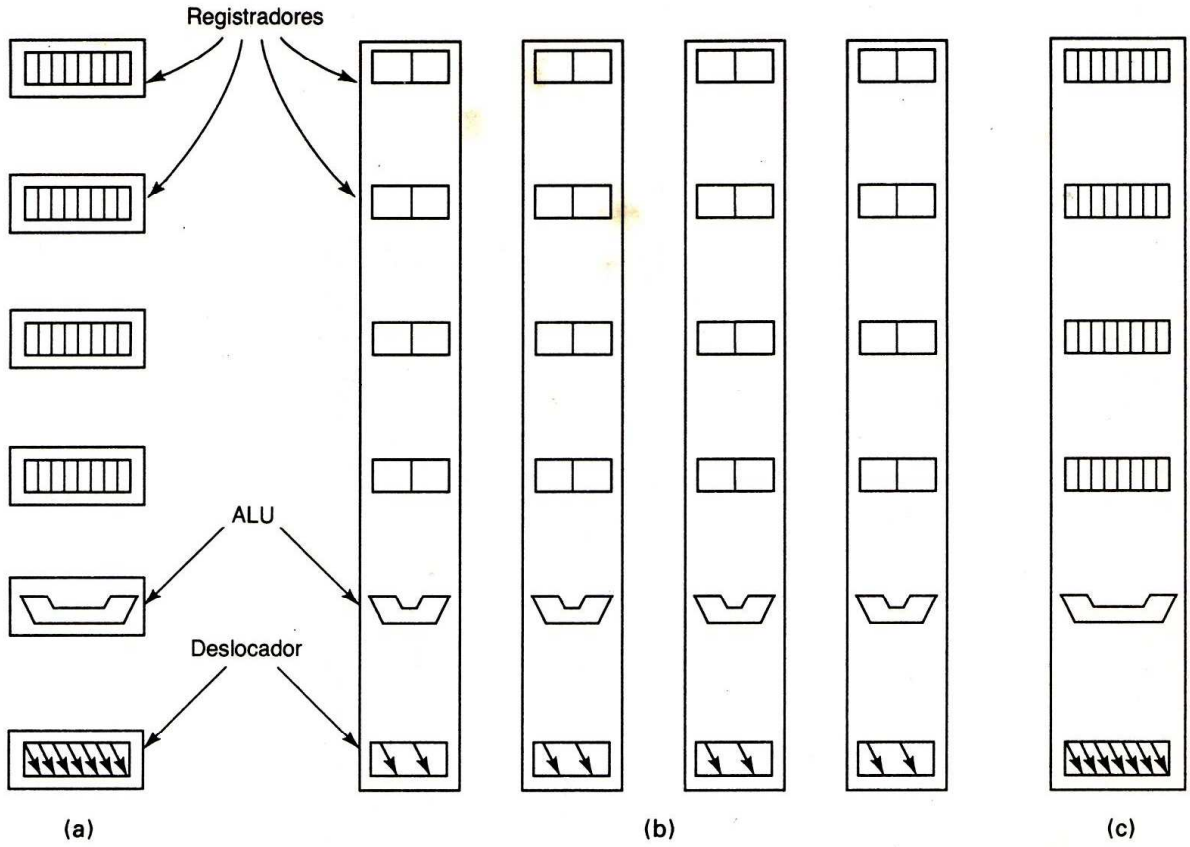


Fig. 4.7 Três formas de construir um computador. (a) Pastilhas MSI. (b) *Bit-slice*. (c) Pastilhas LSI.

Surgem as opções: usar um processador disponível comercialmente, contratar alguém para projetar e fabricar uma pastilha de uso específico, ou construí-la a partir de componentes MSI ou *bit-slice*.

4.2 UM EXEMPLO DE MICROARQUITETURA

Agora que já revisamos todos os componentes básicos com os quais se constrói o nível de microprogramação, é o momento de vermos como eles são conectados. Como, nesta área, os princípios gerais são poucos e dispersos, introduziremos o assunto através de um exemplo detalhado.

4.2.1 As Vias de Dados

As **vias de dados** de nossa microarquitetura exemplo estão mostradas na Fig. 4.8. (As vias de dados correspondem àquela parte da CPU que contém a ALU, suas entradas e suas saídas.) Elas possuem 16 registradores idênticos de 16 bits, denominados PC, AC, SP, e assim por diante, que formam uma memória de rascunho acessível apenas ao nível de microprogramação. Os registradores rotulados 0, +1 e −1 serão usados para armazenar as constantes indicadas; o significado dos nomes dos outros registradores será explicado mais tarde. (Na realidade, 0 não foi utilizado em nossos exemplos simples, mas provavelmente o seria em uma máquina mais complicada; de qualquer forma, foi incluído porque temos mais registradores do que podemos usar.) Cada registrador pode dar saída de seu conteúdo em um ou nos dois barramentos internos, o barramento A e o barramento B, e cada um pode ser carregado a partir de um terceiro barramento interno, o barramento C, como mostra a figura.

Os barramentos A e B alimentam uma ALU de largura de 16 bits, que pode realizar quatro funções: A + B, A AND B, A e NOT A. A função a ser executada é especificada pelas duas linhas de controle da ALU, F_0 e F_1. A ALU gera dois bits de *status*, com base na saída corrente

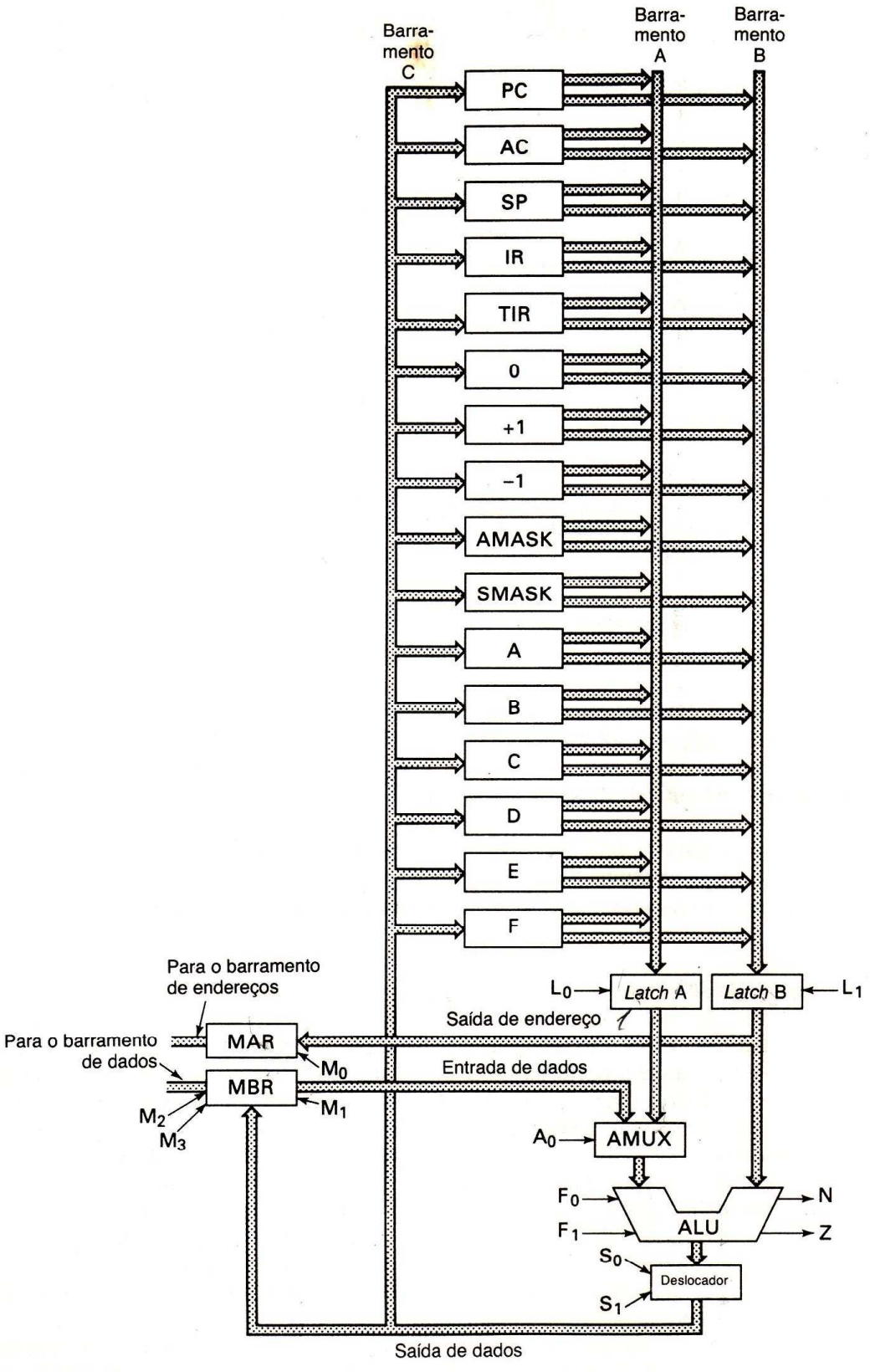


Fig. 4.8 As vias de dados da microarquitetura exemplo utilizada neste capítulo.

da ALU: N, que é ligado quando a saída da ALU é negativa, e Z, que é ligado quando a saída da ALU é zero.

A saída da ALU entra em um deslocador, que pode deslocá-la de 1 bit em qualquer direção, ou nenhuma. É possível executar um deslocamento de 2 bits à esquerda do conteúdo de um registrador, R, calculando R + R na ALU (que é um deslocamento de 1 bit para a esquerda) e então deslocando a soma de outro bit à esquerda utilizando o deslocador.

Nem o barramento A, nem o barramento B, alimenta a ALU diretamente. Em vez disso, cada um alimenta um *latch* (isto é, um registrador) que, por sua vez, alimenta a ALU. Os *latches* são necessários porque a ALU é um circuito combinatório — ela calcula continuamente a saída a partir das entradas correntes e do código de função. Esta organização pode causar problemas ao calcular, por exemplo, A := A + B. Como A está sendo armazenado, o valor no barramento A começa a se modificar, fazendo com que a saída da ALU, e portanto também o barramento C, se modifiquem. Conseqüentemente, um valor errado pode ser armazenado em A. Em outras palavras, na atribuição A := A + B, o A do lado direito é o valor original de A, e não alguma mistura bit a bit dos valores velho e novo. Inserindo *latches* nos barramentos A e B, podemos congelar lá os valores originais de A e B no início do ciclo, de modo que a ALU fica protegida de mudanças nos barramentos enquanto um novo valor está sendo armazenado na memória de rascunho (*scratchpad*). A carga dos *latches* é controlada por L_0 e L_1.

Vale a pena notar que nossa solução para este problema (isto é, a inserção de *latches* na frente da ALU) não é a única. Se todos os registradores fossem *flip-flops* em vez de *latches*, então um projeto de dois barramentos seria também possível, carregando os operandos nos barramentos A e B no início do ciclo e lendo o resultado de um dos barramentos mais tarde, no ciclo. Os compromissos entre os projetos de dois e três barramentos envolvem complexidade, paralelismo e quantidade de ligações. Um tratamento mais detalhado dessas considerações está fora do escopo deste livro.

Para a comunicação com a memória, incluímos um MAR e um MBR na microarquitetura. O MAR pode ser carregado a partir do *latch* B, em paralelo com a operação da ALU. A linha M_0 controla a carga do MAR. Nas escritas, o MBR pode ser carregado com a saída do deslocador, em paralelo, ou no lugar de uma escrita na memória de rascunho. M_1 controla a carga de MBR a partir da saída do deslocador. M_2 e M_3 controlam leituras e escritas da memória. Nas leituras, o dado lido da memória pode chegar à entrada esquerda da ALU através do multiplexador A, indicado por Amux na Fig. 4.8. A linha de controle, A_0, determina se é o *latch* A ou o MBR que deve alimentar a ALU. A microarquitetura da Fig. 4.8 é similar a vários *bit-slices* disponíveis comercialmente.

4.2.2 Microinstruções

Para controlar as vias de dados da Fig. 4.8, precisamos de 61 sinais. Estes podem ser divididos em nove grupos funcionais, como descrito abaixo.

16 sinais para controlar a carga do barramento A a partir da memória de rascunho
16 sinais para controlar a carga do barramento B a partir da memória de rascunho
16 sinais para controlar a carga da memória de rascunho a partir do barramento C
2 sinais para controlar os *latches* A e B
2 sinais para controlar a função da ALU
2 sinais para controlar o deslocador
4 sinais para controlar o MAR e o MBR
2 sinais para indicar leitura da memória e escrita na memória
1 sinal para controlar o Amux

Dados os valores dos 61 sinais, podemos realizar um ciclo nas vias de dados. Um ciclo consiste em colocar valores nos barramentos A e B, armazená-los nos dois *latches* de barramento, passar os valores através da ALU e do deslocador e, finalmente, armazenar o resultado na memória de rascunho e/ou MBR. Além disso, o MAR pode também ser carregado, e um ciclo de memória iniciado. Como uma primeira aproximação, podemos ter um registrador de controle de 61 bits, com um bit para cada sinal de controle. Um bit em 1 significa que o sinal está ativado, e em

0 significa que ele está desativado.

Entretanto, pelo preço de um pequeno aumento na circuitaria, podemos reduzir consideravelmente o número de bits necessário para controlar as vias de dados. Para começar, temos 16 bits para controlar a entrada do barramento A, o que permite 2^{16} combinações de registradores fonte. Apenas 16 dessas combinações são permitidas — ou seja, apenas um dos 16 registradores de cada vez. Portanto, podemos codificar a informação sobre o barramento A em 4 bits e usar um decodificador para gerar os 16 sinais de controle. O mesmo se aplica ao barramento B.

A situação é ligeiramente diferente para o barramento C. Em princípio, são possíveis armazenamentos simultâneos múltiplos na memória de rascunho, mas este aspecto virtualmente nunca é utilizado na prática, e a maioria das implementações não o provê. Assim, também codificaremos o controle do barramento C em 4 bits. Tendo economizado 3 × 12 = 36 bits, precisamos agora de 25 bits de controle para acionar as vias de dados. L_0 e L_1 sempre são necessários em instantes bastante determinados, de forma que são supridos pelo relógio, deixando-nos com 23 bits de controle. Um sinal adicional que não é estritamente necessário, mas que é freqüentemente útil, é um para habilitar/inibir o armazenamento do valor existente no barramento C na memória de rascunho. Em algumas situações, deseja-se meramente executar uma operação da ALU para gerar os sinais N e Z, mas não se quer armazenar o resultado. Com este bit extra, que denominaremos ENC (*ENable* C, ou habilita C), podemos indicar se o barramento C deve ser armazenado (ENC = 1) ou não (ENC = 0).

Neste ponto, podemos controlar as vias de dados com um número de 24 bits. Notemos agora que RD e WR podem ser utilizados para controlar a carga de MBR a partir do barramento de dados do sistema e para habilitar a saída de MBR para o barramento de dados do sistema, respectivamente. Esta observação reduz o número de sinais de controle independentes de 24 para 22.

O próximo passo no projeto da microarquitetura é inventar um formato de microinstrução contendo 22 bits. A Fig. 4.9 mostra um formato desse tipo, com dois campos adicionais, COND e ADDR, que serão descritos em breve. A microinstrução contém 13 campos, 11 dos quais são os seguintes:

AMUX — controla a entrada esquerda da ALU: $\bar{0}$ = *latch* A, 1 = MBR
ALU — função da ALU: 0 = A + B, 1 = A AND B, 2 = A, 3 = $\bar{A}$
SH — função do deslocador: 0 = nenhum deslocamento, 1 = à direita, 2 = à esquerda
MBR — carrega MBR a partir do deslocador: 0 = não carrega, 1 = carrega MBR
MAR — carrega MAR a partir do *latch* B: 0 = não carrega, 1 = carrega MAR
RD — requisita leitura de memória: 0 = nenhuma leitura, 1 = carrega MBR a partir da memória
WR — requisita escrita na memória: 0 = nenhuma escrita, 1 = escreve o conteúdo de MBR na memória
ENC — controla armazenamento na memória de rascunho: 0 = não armazena, 1 = armazena
C — seleciona registrador para armazenamento se ENC = 1: 0 = PC, 1 = AC, etc.
B — seleciona fonte do barramento B: 0 = PC, 1 = AC, etc.
A — seleciona fonte do barramento A: 0 = PC, 1 = AC, etc.

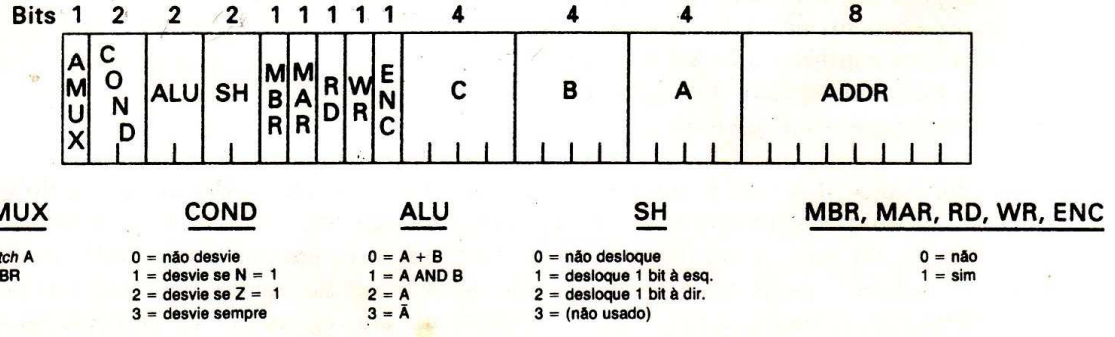


Fig. 4.9 O *layout* da microinstrução para controlar as vias de dados da Fig. 4.8.

A ordem dos campos é completamente arbitrária. Esta ordem foi escolhida para minimizar os cruzamentos de linhas na figura subseqüente. (Na realidade, este critério não é tão maluco como parece; cruzamentos de linhas em figuras geralmente correspondem a cruzamentos de fios em placas de circuito impresso ou em pastilhas, que causam problemas em projetos bidimensionais.)

4.2.3 Temporização de Microinstruções

Apesar de que nossa discussão de como uma microinstrução pode controlar as vias de dados durante um ciclo esteja quase completa, negligenciamos um assunto até agora: a temporização. Um ciclo básico de ALU consiste em utilizar os *latches* A e B, prover tempo para que a ALU e o deslocador executem seu trabalho e armazenar os resultados. É óbvio que esses eventos devem ocorrer nesta seqüência. Se tentarmos armazenar o valor existente no barramento C na memória de rascunho antes que os *latches* A e B sejam carregados, será armazenado lixo em vez de dados úteis. Para conseguir o seqüenciamento correto de eventos, vamos agora introduzir um relógio de quatro fases, isto é, um relógio com quatro subciclos, como o da Fig. 4.5. Os eventos chave durante cada um dos subciclos são:

1. Carregar a próxima microinstrução a ser executada em um registrador chamado **MIR** (*MicroInstruction Register*), o registrador de microinstrução.
2. Colocar os conteúdos dos registradores nos barramentos A e B e armazená-los nos *latches* A e B.
3. Agora que as entradas estão estáveis; dar tempo para que a ALU e o deslocador produzam uma saída estável, e carregar o MAR se requisitado.
4. Agora que a saída do deslocador está estável, armazenar o valor existente no barramento C na memória de rascunho e carregar o MBR, se for necessário.

A Fig. 4.10 é um diagrama de blocos detalhado da microarquitetura completa de nossa máquina exemplo. Pode parecer complicado inicialmente, mas vale a pena estudá-lo cuidadosamente. Ao compreender totalmente cada bloco e cada linha, estará bem encaminhado no entendimento do nível de microprogramação. O diagrama de blocos tem duas partes: as vias de dados, à esquerda, que já discutimos em detalhe, e a seção de controle, à direita, que veremos agora.

O maior e mais importante item na seção de controle da máquina é a **memória de controle.** Esta memória rápida especial é onde as microinstruções são mantidas. Em algumas máquinas, ela é memória somente para leitura; em outras, ela é memória de leitura e escrita. Em nosso exemplo, as microinstruções terão largura de 32 bits e o espaço de endereçamento das microinstruções consistirá em 256 palavras, de forma que a memória de controle ocupará um máximo de $256 \times 32 = 8.192$ bits.

Como qualquer outra memória, a memória de controle precisa de um MAR e de um MBR. Chamaremos o MAR de **MPC** (*MicroProgram Counter,* ou contador de microprograma), porque sua única função é apontar para a próxima microinstrução a ser executada. O MBR é apenas o **MIR,** como mencionado acima. Note que a memória de controle e a memória principal são completamente diferentes, a primeira guarda o microprograma, e a segunda, o programa na linguagem de máquina convencional.

A partir da Fig. 4.10 fica claro que a memória de controle tenta continuamente copiar a microinstrução endereçada pelo MPC para o MIR. Entretanto, o MIR somente é carregado durante o subciclo 1, como indicado pela linha tracejada vinda do relógio para ele. Durante os outros três subciclos ele não é afetado, independente do que ocorre com MPC.

Durante o subciclo 2, o MIR fica estável, e os vários campos começam a controlar as vias de dados. Em particular, A e B fazem com que os dados sejam colocados nos barramentos A e B. Os blocos decodificador A e decodificador B no diagrama de blocos provêem a decodificação de 4-para-16 de cada campo necessário para acionar as linhas OE1 e OE2 nos registradores [veja Fig. 4.2(b)]. O relógio ativa os *latches* A e B durante este subciclo, provendo entradas estáveis para a ALU durante o resto do ciclo. Enquanto os dados estão sendo colocados nos barramentos A e B, a unidade de incremento na seção de controle da máquina calcula MPC

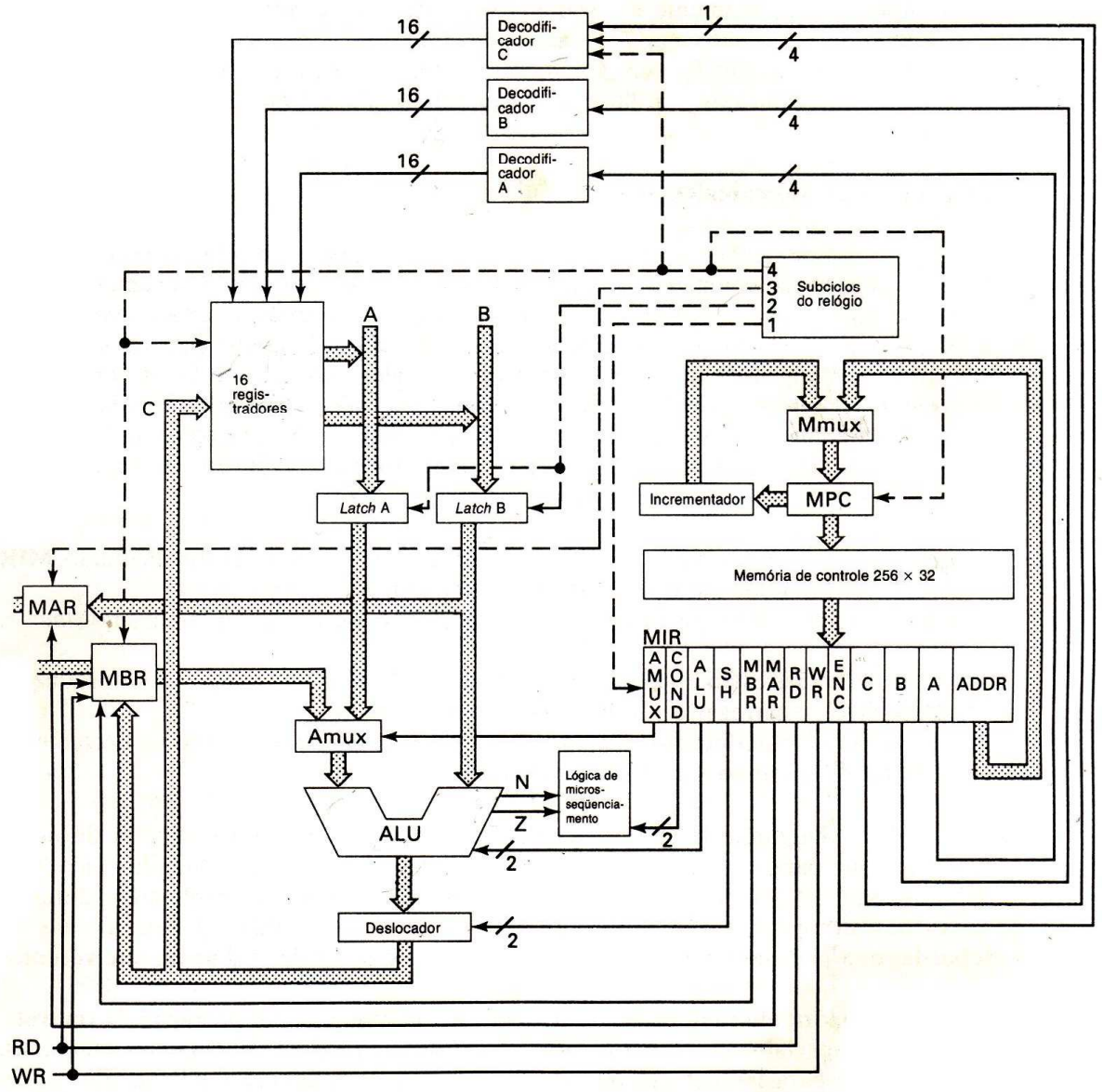


Fig. 4.10 O diagrama de blocos completo da nossa microarquitetura exemplo.

+ 1, preparando a carga da próxima microinstrução da seqüência durante o ciclo seguinte. Sobrepondo estas duas operações, a execução de instrução pode ser acelerada.

No terceiro subciclo, dá-se tempo à ALU e ao deslocador para a produção de resultados válidos. O campo AMUX da microinstrução determina a entrada esquerda da ALU; a entrada da direita é sempre o *latch* B. Embora a ALU seja um circuito combinatório, o tempo que ela gasta para calcular uma soma é determinado pelo tempo de propagação de vai-1, e não pelo atraso normal das portas. O tempo de propagação de vai-1 é proporcional ao número de bits da palavra. Enquanto a ALU e o deslocador estão calculando, o MAR é carregado a partir do barramento B, se o campo MAR da microinstrução for 1.

Durante o quarto e último subciclo, o valor existente no barramento C pode ser armazenado de volta na memória de rascunho e no MBR, dependendo de ENC e MBR. O bloco rotulado "decodificador C" pega ENC, a quarta linha do relógio, e o campo C da microinstrução como entradas e gera os 16 sinais de controle. Internamente, ele realiza uma decodificação de 4-para-16 do campo C e então faz um AND de cada um destes sinais com um sinal derivado do AND da linha do subciclo 4 com ENC. Assim, um registrador da memória de rascunho somente é carregado se prevalecerem três condições:

1. ENC = 1.
2. É o subciclo 4.
3. O registrador foi selecionado pelo campo C.

O MBR também é carregado durante o subciclo 4 se MBR = 1.

Os dois sinais que controlam a memória, RD e WR, são ativados se estiverem presentes em MIR. Na verdade, os campos correspondentes em MIR agem com *latches*.

4.2.4 Seqüenciamento de Microinstruções

A única consideração pendente é como a próxima microinstrução é escolhida. Embora algum tempo seja suficiente para buscar a próxima microinstrução da seqüência, algum mecanismo é necessário para permitir desvios condicionais no microprograma, para possibilitar que ele tome decisões. Por esta razão, deixamos dois campos em cada microinstrução: ADDR, que é o endereço da sucessora potencial da microinstrução corrente, e COND, que determina se a próxima microinstrução será buscada de MPC + 1 ou de ADDR. Toda microinstrução contém potencialmente um desvio condicional. Esta decisão foi tomada porque desvios condicionais são muito comuns em microprogramas, e, ao permitir que toda microinstrução tenha duas sucessoras possíveis, tornamo-las de execução mais rápida do que a alternativa de estabelecer alguma condição em uma microinstrução e depois testá-la na próxima. A maioria das microarquiteturas existentes usam nossa estratégia de uma forma ou outra.

A escolha da próxima microinstrução é determinada pelo bloco rotulado "Lógica de microsseqüenciamento" durante o subciclo 4, quando os sinais N e Z da saída da ALU são válidos. A saída deste bloco controla o multiplexador M (Mmux), que roteia MPC + 1 ou ADDR para MPC, de onde será buscada a próxima microinstrução. Fornecemos ao microprogramador quatro escolhas quanto à seleção da próxima microinstrução. A escolha é indicada fazendo o campo COND igual a:

0 = Não desvie; a próxima microinstrução será tomada de MPC + 1
1 = Desvie para ADDR se N = 1
2 = Desvie para ADDR se Z = 1
3 = Desvie para ADDR incondicionalmente

A lógica de microsseqüenciamento combina os dois bits da ALU, N e Z, e os dois bits de COND, L (*Left*) e R (*Right*), isto é, esquerda e direita, para gerar uma saída. O sinal correto é

$$\text{Mmux} = \bar{L}RN + L\bar{R}Z + LR = RN + LZ + LR$$

onde + significa INCLUSIVE OR. Em palavras, o sinal de controle para Mmux é 1 (roteando ADDR para MPC) se LR é 01_2 e N = 1, ou LR é 10_2 e Z = 1, ou LR é 11_2. Caso contrário, ele é 0, e a próxima microinstrução na seqüência é buscada. O circuito para a geração do sinal Mmux pode ser construído a partir de componentes SSI, como na Fig. 3.3(b), ou ser parte de uma PLA, como na Fig. 3.16.

Para tornar nossa máquina exemplo ligeiramente mais real, vamos assumir que um ciclo de memória principal tome mais tempo que uma microinstrução. Em particular, se uma microinstrução inicia uma leitura à memória principal, fazendo RD = 1, ela também deve ter RD = 1 na próxima microinstrução executada (que pode ou não estar localizada no próximo endereço da memória de controle). O dado torna-se disponível duas microinstruções após a leitura ter sido iniciada. Se o microprograma não tem nada mais útil a fazer na microinstrução seguinte à que iniciou a leitura de memória, a microinstrução simplesmente tem RD = 1 e é efetivamente desperdiçada. Do mesmo modo, uma escrita na memória também gasta dois tempos de microinstrução para completar.

4.3 UM EXEMPLO DE MACROARQUITETURA

Para continuar nosso exemplo do nível de microprogramação, vamos agora nos voltar para a arquitetura do nível de máquina convencional a ser suportada pelo interpretador em execução na máquina da Fig. 4.10. Por conveniência, vamos chamar a arquitetura da máquina de nível 2 ou 3 de **macroarquitetura,** em contraste com o nível 1, a microarquitetura. (Para os propósitos deste capítulo, vamos ignorar o nível 3, porque suas instruções são praticamente aquelas do nível 2, e as diferenças não são importantes aqui.) Similarmente, as instruções de nível 2 serão chamadas **macroinstruções**. Assim, pela duração deste capítulo, as instruções normais de ADD, MOVE e outras do nível de máquina convencional serão chamadas macroinstruções. (O motivo de repetir este comentário é que alguns montadores têm a facilidade de definir "macros" em tempo de montagem, que não são de maneira alguma relacionados ao que queremos dizer com macroinstruções.) Vamos algumas vezes nos referenciar à nossa máquina exemplo de nível 1 como Mic-1, e à máquina de nível 2 como Mac-1. Antes de descrever a Mac-1, entretanto, vamos digressar ligeiramente para motivar seu projeto.

4.3.1 Pilhas

Uma macroarquitetura moderna deve ser projetada com as necessidades das linguagens de alto nível em mente. Uma das questões mais importantes de projeto é o endereçamento. Para ilustrar o problema que deve ser resolvido, considere o programa Pascal da Fig. 4.11(a). O programa principal inicializa dois vetores, x e y, com valores tais que $x_k = k$ e $y_k = 2k + 1$. Então, ele calcula o produto escalar (também chamado de produto interno) dos dois vetores. Sempre que ele precisa multiplicar dois números positivos pequenos, ele chama a função *pmul.* (Imagine que o compilador é para um microcomputador e que somente implementa um subconjunto de Pascal, não incluindo o operador de multiplicação.)

Linguagens estruturadas em blocos, como Pascal, são normalmente implementadas de tal modo que, quando se sai de um procedimento ou função, a área de memória que estava sendo utilizada para variáveis locais é liberada. A maneira mais fácil para se conseguir isto é através de uma estrutura de dados denominada pilha. Uma **pilha** é um bloco contíguo de memória contendo alguns dados e um **apontador de pilha** (SP, de *Stack Pointer*) dizendo onde está o topo da pilha. O fundo da pilha está em um endereço fixo, e não mais nos interessará. A Fig. 4.12 (a) mostra uma pilha ocupando seis palavras de memória. O fundo da pilha está no endereço 4020, e o topo da pilha, para onde SP aponta, está no 4015. Nossas pilhas crescerão dos endereços de memória mais altos para os mais baixos, mas a escolha oposta é igualmente boa.

Muitas operações são definidas para pilhas. Duas das mais importantes são PUSH X e POP Y. PUSH avança o apontador da pilha (decrementando-o, no nosso exemplo) e então armazena X na posição de memória agora apontada por SP. PUSH aumenta o tamanho da pilha de um item. POP Y, em contraste, reduz o tamanho da pilha, armazenando o item do topo da pilha em Y, e então removendo-o através do incremento do apontador de pilha com o tamanho do item retirado. A Fig. 4.12(b) mostra como a pilha da Fig. 4.12(a) fica depois que uma palavra contendo 5 foi empilhada.

Outra operação que pode ser realizada numa pilha é avançar o apontador da pilha sem realmente inserir qualquer dado. Isto é normalmente feito na entrada de um procedimento ou função, para reservar espaço para as variáveis locais. A Fig. 4.13(a) mostra como a memória poderia ser alocada durante a execução do programa principal da Fig. 4.11. Arbitrariamente, assumimos que a memória consiste em 4096 palavras de 16 bits e que as palavras 4021 a 4095 são usadas pelo sistema operacional, e, portanto, não estão disponíveis para armazenar variáveis. A variável Pascal k é armazenada no endereço 4020. (Os endereços são dados em decimal.) O arranjo x requer 20 palavras, de 4000 a 4019. O arranjo y começa em 3980 para $y[1]$ e vai até 3999 para $y[20]$. Enquanto o programa principal estiver executando fora de *pmul,* SP tem o valor 3980, indicando que o topo da pilha está em 3980.

Quando o programa principal deseja chamar *pmul,* ele primeiro empilha os parâmetros da chamada, 2 e k, e então executa a instrução CALL, que empilha o endereço de retorno, de modo que *pmul* saberá para onde retornar quando ele terminar. Quando a rotina *pmul* começa

a execução, SP é 3977. A primeira coisa que ela faz é avançar o apontador da pilha de 2, para reservar duas palavras para suas próprias variáveis locais, *p* e *j*. Neste ponto, SP é 3975, como mostrado na Fig. 4.13(b). As cinco palavras no topo da pilha constituem a estrutura de pilha (*stack frame*) usada por *pmul*; elas serão liberadas quando ela terminar. As palavras 3979 e 3978 são rotuladas *a* e *b* porque esses são os nomes dos parâmetros formais de *pmul*, mas, naturalmente, elas contêm 2 e *k*, respectivamente.

A configuração da pilha após *pmul* ter retornado e *escalar* ser chamada é mostrada na Fig. 4.13(c). Quando *escalar* chama *pmul*, a pilha fica como mostrada na Fig. 4.13(d). Agora vem o problema. Que código o compilador precisa gerar para acessar as variáveis locais e os parâmetros de *pmul*? Se ele tentar ler *p* usando uma instrução como MOVE 3976,ALGUMLUGAR, *pmul* funcionará quando chamada do programa principal, mas não quando chamada de *escalar*. Similarmente, MOVE 3971,ALGUMLUGAR funcionará quando chamada de *escalar*, mas não quando chamada do programa principal. O que é realmente necessário é uma forma de dizer "busque a palavra uma posição acima do apontador de pilha corrente". Em outras palavras, o Mac-1

```
program ProdutoEscalar (output);

{Este programa inicializa dois vetores, x e y, de 20 elementos cada,
 então calcula o seu produto escalar:
    x[1] * y[1] + x[2] * y[2] + ... + x[20] * y[20]}

const max = 20;                                  {tamanho dos vetores}

type PequenoInt = 0..100;
     vet = array[1..max] of PequenoInt;

var k : integer;
    x, y: vec;

function pmul (a, b: PequenoInt): integer;
{Esta função multiplica seus dois parâmetros e retorna o produto.
 Ela realiza a multiplicação por adições sucessivas.}
var p, j: integer;
begin                                            {0: reserve espaço na pilha para p e j}
  if (a = 0) or (b = 0) then                     {1: se um dos dois for 0, resultado é o 0}
    pmul:= 0                                     {2: função retorna 0}
  else
    begin
    p := 0;                                      {3: inicializa p}
    for j := 1 to a do                           {4: some b a p a vezes}
      p := p + b;                                {5: faça a adição}
    pmul := p                                    {6: atribua resultado à função}
    end
end; {pmul}                                      {7: remova locais e retorne valor}

procedure escalar (var v: vet; var resp: integer);
{Calcula o produto escalar de v e x e o retorna em resp.}
var soma, i: integer;
begin                                            {8: reserve espaço na pilha para soma e i}
  soma := 0;                                     {9: soma vai acumular o produto escalar}
  for i:= 1 to max do                            {10: loop para todos os elementos}
    soma := soma + pmul (x[i], v[i]);            {11: acumule um termo}
  resp := soma                                   {12: copie o resultado para resp}
end; {escalar}                                   {13: remova soma e i e retorne}

begin                                            {14: reserve espaço para k, x e y}
  for k := 1 to max do                           {15: inicialização do loop}
    begin
      x[k] := k;                                 {16: inicialize x}
      y[k] := pmul (2, k) +1                     {17: inicialize y}
    end;
  escalar (y, k);                                {18: chame escalar}
  writeln (k)                                    {19: imprima os resultados}
end.
```

Fig. 4.11(a) Um programa Pascal para calcular um produto escalar.

```
        K = 4020        /DEFINA ALGUNS SÍMBOLOS
        X = 4000
        Y = 3980
        A = 4
        B = 3
        P = 1
        J = 0
        V = 5
        RESP = 4
        SOMA = 1
        I = 0

        JUMP    MAIN    /COMECE NO PROGRAMA PRINCIPAL
PMUL:   DESP    2       /0
        LODL    A       /1
        JNZE    ANOTZ   /DESVIE SE A <> 0
        LOCO    0       /2
        JUMP    DONE    /RETORNE 0
ANOTZ:  LODL    B       /AC := B
        JNZE    BNOTZ   /DESVIE SE B <> 0
        LOCO    0       /2
        JUMP    DONE    /RETORNE 0
BNOTZ:  LOCO    0       /3
        STOL    P       /P := 0
        LOCO    1       /4
        STOL    J       /J := 1
        LODL    A       /0 LOOP PODE SER EXECUTADO?
        JNEG    L2      /1 < 0, NÃO FAÇA O LOOP
        JZER    L2      /A = 0, NÃO FAÇA O LOOP
L1:     LODL    P       /5
        ADDL    B       /AC := P + B
        STOL    P       /P := P + B
        LOCO    1       /TESTE NO FINAL DO LOOP
        ADDL    J       /AC := J + 1
        STOL    J       /J := J + 1
        SUBL    A       /AC := J - A
        JNEG    L1      /DESVIE SE J < A
        JZER    L1      /DESVIE SE J = A
L2:     LODL    P       /6
DONE:   INSP    2       /7
        RETN            /RETORNE

ESCALAR: DESP   2       /8
        LOCO    0       /9
        STOL    SOMA    /SOMA := 0
        LOCO    1       /10
        STOL    I       /I := 1
L3:     LOCO    X-1     /11
        ADDL    I       /AC := X + I - 1
        PSHI            /EMPILHE X[I]
        LODL    V       /AC := ENDEREÇO DO VETOR
        ADDL    I       /AC := V + I
        SUBD    C1      /V COMEÇA EM 1, NÃO 0
        PSHI            /EMPILHE V[I]
        CALL    PMUL    /PMUL(X[I], V[I])
        INSP    2       /REMOVA PARÂMETROS
        ADDL    SOMA    /AC := SOMA + PMUL(...)
        STOL    SOMA    /SOMA := SOMA + PMUL(...)
        LOCO    1       /TESTE NO FINAL DO LOOP
        ADDL    I       /AC := I + 1
        STOL    I       /I := I + 1
        SUBD    C20     /AC := I - MAX
        JNEG    L3      /DESVIE SE I < MAX
        JZER    L3      /DESVIE SE I = MAX
        LODL    SOMA    /12
        PUSH            /EMPILHE SOMA
        LODL    RESP    /AC := ENDEREÇO DE RESP
        POPI            /RESP := SOMA
        INSP    2       /13
        RETN            /RETORNE

PRINCIPAL: DESP 41      /14
        LOCO    1       /15
        STOD    K       /K NÃO É LOCAL
L4:     LODD    K       /16
        PUSH            /EMPILHE K
        LOCO    X - 1   /AC := (ENDEREÇO DE X[1])-1
        ADD     K       /AC := X + K - 1
        POPI            /X[K] := K
        LOCO    2       /17
        PUSH            /PREPARE PMUL (2,...)
        LOOD    K       /PREPARE PMUL (2, K)
        PUSH            /AMBOS OS PARÂMETROS EMPILHADOS
        CALL    PMUL    /PMUL(2,K)
        INSP    2       /REMOVA PARÂMETROS
        ADDD    C1      /AC := 2 * K + 1
        PUSH            /PREPARE Y[K] := 2 * K + 1
        LOCO    Y - 1   /AC := (ENDEREÇO DE Y[1])-1
        ADDD    K       /AC := Y + K - 1
        POPI            /Y[K] := 2 * K + 1
        LOCO    1       /TESTE NO FINAL DO LOOP
        ADDD    K       /AC := K + 1
        STOD    K       /K := K + 1
        SUBD    C20     /AC := K - MAX
        JNEG    L4      /DESVIE SE K < 0
        JZER    L4      /DESVIE SE K = MAX
        LOCO    Y       /18
        PUSH            /EMPILHE O ENDEREÇO DE Y
        LOCO    K       /AC := ENDEREÇO DE K
        PUSH            /EMPILHE ELE TAMBÉM
        CALL    ESCALAR /CHAMADA DE PROCEDIMENTO
        INSP    2       /REMOVA PARÂMETROS
        LODD    K       /19
        PUSH            /PREPARE WRITELN(K)
        CALL    OUTNUM1 /ROTINA DE BIBLIOTECA
        INSP    1       /REMOVA PARAM
        CALL    STOP    /FIM DA TAREFA

C1:     1               /CONSTANTE 1
C20:    20              /CONSTANTE 20
```

Fig. 4.11(b) *ProdutoEscalar* em linguagem de montagem.

precisa de um modo de endereçamento que busque ou armazene uma palavra a uma distância conhecida relativa ao apontador de pilha (ou algum modo de endereçamento equivalente).

4.3.2 O Conjunto de Macroinstruções

Com esse modo de endereçamento em mente, estamos agora prontos para analisar a arquitetura do Mac-1. Basicamente, ela consiste em uma memória com 4096 palavras de 16 bits e três registradores visíveis ao programador de nível 2. Os registradores são o contador de programa, PC, o apontador de pilha, SP, e o acumulador, AC, que é usado para movimentação de dados, cálculos e outros propósitos. São providos três modos de endereçamento: direto, indireto e local. As instruções que usam o endereçamento direto contêm um endereço absoluto de memória de 12 bits nos 12 bits de baixa ordem. Tais instruções são úteis para acessar variáveis globais, tais como *x* na Fig. 4.11. O endereçamento indireto permite que o programador calcule um endereço de memória, coloque-o em AC e então leia ou escreva a palavra endereçada. Esta forma de endereçamento é muito genérica e é usada para acessar elementos de arranjos, entre outras coisas. O endereçamento local especifica uma distância a partir de SP e é usado para acessar variáveis locais, como acabamos de ver. Juntos, estes três modos provêem um sistema de endereçamento simples, mas adequado.

O conjunto de instruções do Mac-1 é mostrado na Fig. 4.14. Cada instrução contém um código de operação (*opcode*) e, algumas vezes, um endereço de memória ou uma constante.

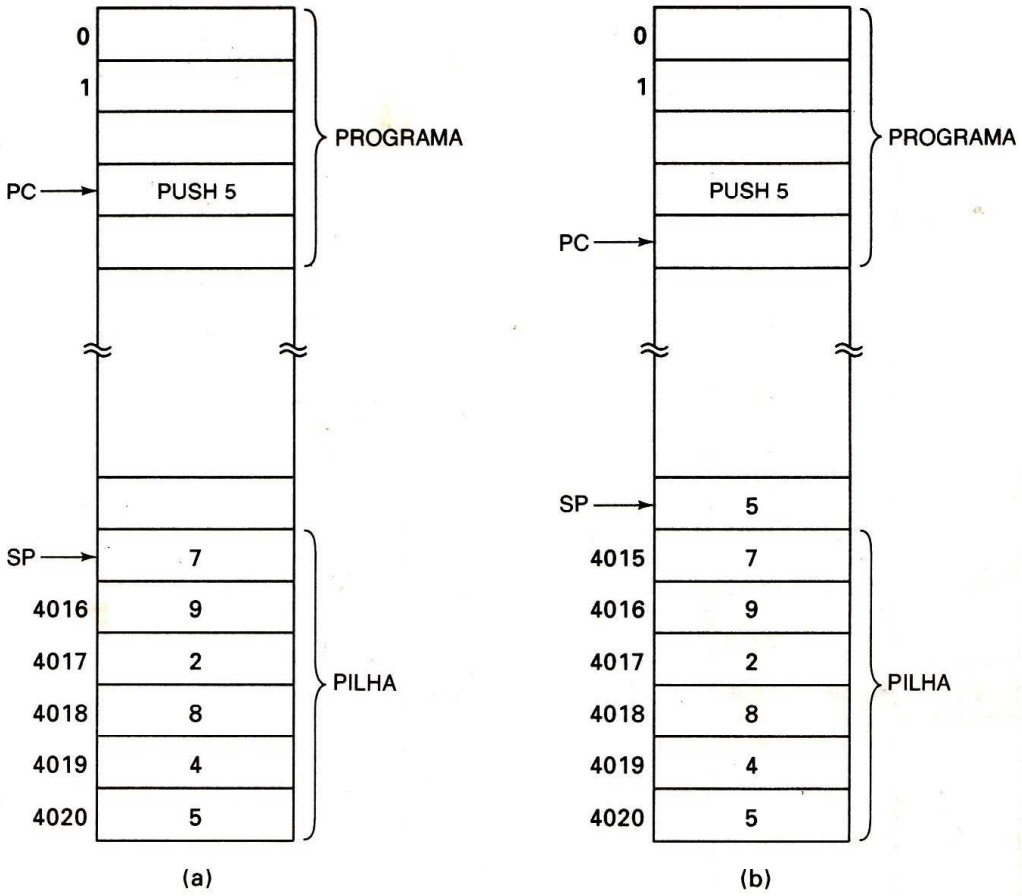


Fig. 4.12 (a) Uma pilha. (b) A pilha depois de empilhar 5.

A primeira coluna mostra o código binário da instrução. A segunda mostra seu mnemônico para a linguagem de montagem. A terceira mostra seu nome, e a quarta descreve o que ela faz, através de um trecho em Pascal. Nestes trechos, $m[x]$ se refere à palavra de memória x. Assim, LODD carrega o acumulador com a palavra de memória especificada em seus 12 bits de baixa ordem. LODD usa então endereçamento direto, enquanto LODL carrega o acumulador com uma palavra a uma distância x acima de SP, usando portanto o endereçamento local. LODD, STOD, ADDD e SUBD realizam quatro funções básicas usando endereçamento direto, e LODL, STOL, ADDL e SUBL realizam as mesmas funções usando endereçamento local.

Cinco instruções de desvio são fornecidas, um desvio incondicional (JUMP) e quatro condicionais (JPOS, JZER, JNEG e JNZE). JUMP sempre copia seus 12 bits de baixa ordem para o contador de programa, enquanto os outros quatro somente o fazem se a condição especificada se verificar.

LOCO carrega uma constante de 12 bits, na faixa de 0 a 4095 (inclusive), em AC. A operação inversa é POPI, que retira uma palavra da pilha e a armazena na posição de memória endereçada por AC. PUSH e POP são úteis na manipulação da pilha de várias maneiras. SWAP troca o conteúdo de AC com SP, o que é útil quando SP precisa ser aumentado ou diminuído de uma quantidade não conhecida no tempo de compilação. Ela é também útil para inicializar SP no início da execução. INSP e DESP são usados para alterar SP de valores conhecidos em tempo de compilação. Devido à falta de espaço de codificação, as distâncias aqui foram limitadas a 8 bits. Finalmente, CALL chama um procedimento, salvando o endereço de retorno na pilha, e RETN retorna de um procedimento, desempilhando o endereço de retorno e colocando-o em PC.

Até agora, nossa máquina não tem instruções de entrada/saída. Nem estamos a fim de acrescentar alguma agora. Ela não precisa delas. Em vez disso, a máquina usará E/S mapeada em memória. Uma leitura do endereço 4092 resultará numa palavra de 16 bits com o próximo caractere

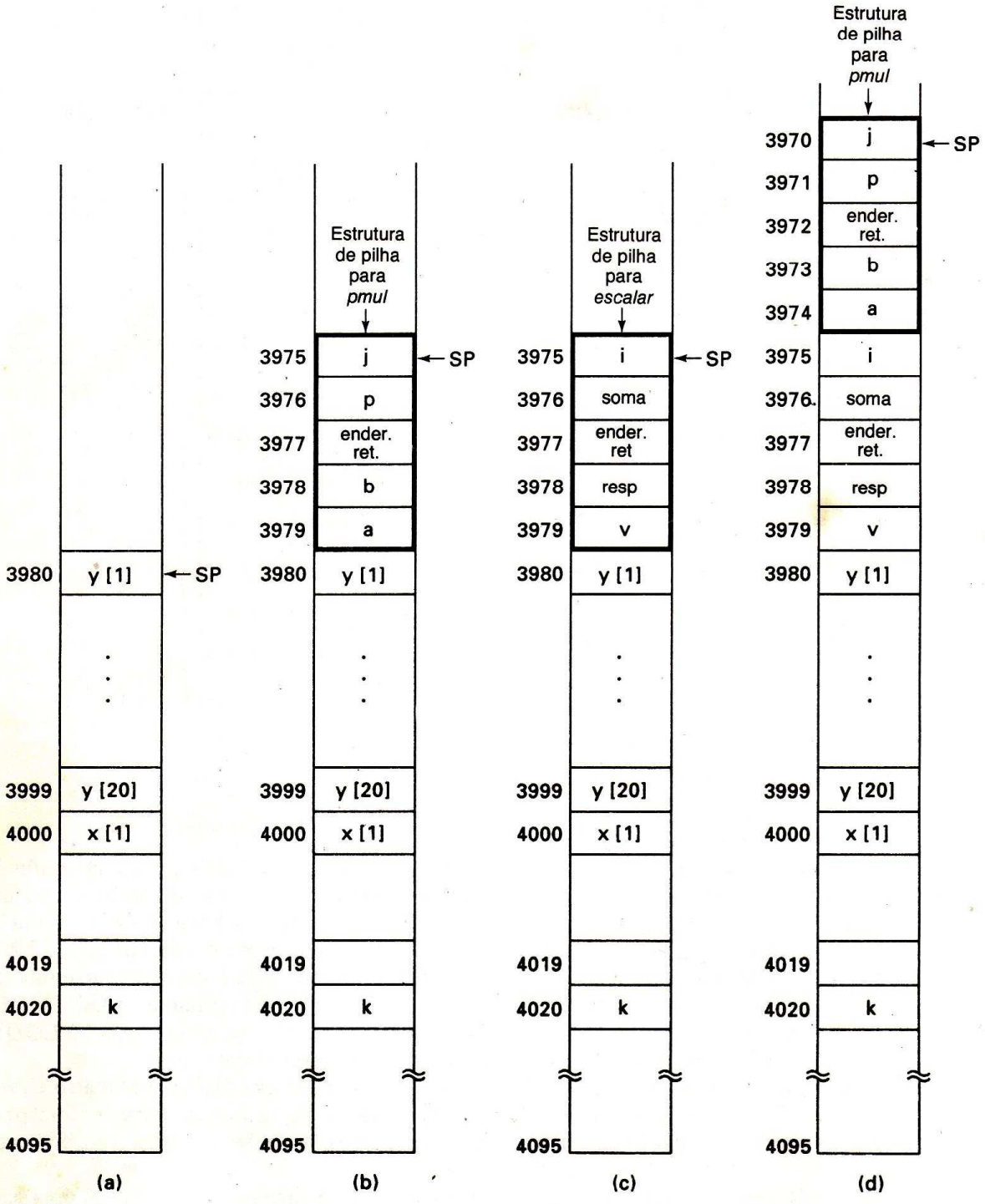


Fig. 4.13 Instantâneos da memória durante a execução de *ProdutoEscalar*. (a) Pilha durante a execução do programa principal. (b) Pilha durante a execução de *pmul*. (c) Pilha durante a execução de *escalar*. (d) Pilha durante a execução de *pmul*, quando chamado de *escalar*.

ASCII do dispositivo de entrada padrão nos 7 bits de baixa ordem e zeros nos 9 bits de alta ordem. Quando um caractere estiver disponível em 4092, o bit de alta ordem do registrador de *status* de entrada, 4093, será ativado. A leitura de 4092 limpa 4093. A rotina de entrada normalmente ficará num *loop* esperando que 4093 se torne negativo. Quando isso acontecer, a rotina de entrada carregará AC com 4092 e retornará.

A saída será feita usando um esquema similar. Uma escrita no endereço 4094 pegará os

Binário	Mnemônico	Instrução	Significado
0000xxxxxxxxxxx	LODD	Carrega direto	$ac := m[x]$
0001xxxxxxxxxxx	STOD	Armazena direto	$m[x] := ac$
0010xxxxxxxxxxx	ADDD	Adiciona direto	$ac := ac + m[x]$
0011xxxxxxxxxxx	SUBD	Subtrai direto	$ac := ac - m[x]$
0100xxxxxxxxxxx	JPOS	Desvia se positivo	**if** $ac \geq 0$ **then** $pc := x$
0101xxxxxxxxxxx	JZER	Desvia se zero	**if** $ac = 0$ **then** $pc := x$
0110xxxxxxxxxxx	JUMP	Desvia	$pc := x$
0111xxxxxxxxxxx	LOCO	Carega constante	$ac := x$ $(0 \leq x \leq 4095)$
1000xxxxxxxxxxx	LODL	Carga total	$ac := m[sp + x]$
1001xxxxxxxxxxx	STOL	Armazena local	$m[x + sp] := ac$
1010xxxxxxxxxxx	ADDL	Adiciona local	$ac := ac + m[sp + x]$
1011xxxxxxxxxxx	SUBL	Subtrai local	$ac := ac - m[sp + x]$
1100xxxxxxxxxxx	JNEG	Desvia se negativo	**if** $ac < 0$ **then** $pc := x$
1101xxxxxxxxxxx	JNZE	Desvia se não zero	**if** $ac \neq 0$ **then** $pc := x$
1110xxxxxxxxxxx	CALL	Chama procedimento	$sp := sp - 1; m[sp] := pc; pc := x$
1111000000000000	PSHI	Empilha indireto	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Desempilha indireto	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Coloca na pilha	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Retira da pilha	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Retorna	$pc := m[sp]; sp := sp + 1$
1111101000000000	SWAP	Troca ac, sp	$tmp := ac; ac := sp; sp := tmp$
11111100yyyyyyyy	INSP	Incrementa sp	$sp := sp + y$ $(0 \leq y \leq 255)$
11111110yyyyyyyy	DESP	Decrementa sp	$sp := sp - y$ $(0 \leq y \leq 255)$

xxxxxxxxxxx é um endereço de máquina de 12 bits; na coluna 4, ele é chamado de *x*.
yyyyyyyy é uma constante de 8 bits; na coluna 4, ele é chamado de *y*.

Fig. 4.14 O conjunto de instruções do Mac-1.

7 bits de baixa ordem da palavra escrita e os copiará para o dispositivo de saída padrão. O bit de alta ordem do registrador de *status* de saída, palavra 4095, será então desativado, voltando a estar ativo de novo quando o dispositivo de saída estiver pronto para aceitar um novo caractere. A entrada e saída padrões podem ser um teclado de terminal e um vídeo, ou uma leitura de cartões e uma impressora, ou alguma outra combinação.

Como um exemplo de como alguém programa usando este conjunto de instruções, veja a Fig. 4.11(b), que é o programa da Fig. 4.11(a) compilado para linguagem de montagem por um compilador que não faz nenhuma otimização. (Código otimizado faria com que o exemplo fosse difícil de acompanhar.) Os números 0 a 19 nos comentários, indicados por uma barra na linguagem de montagem, foram colocados para tornar mais fácil a ligação entre as duas metades da figura. OUTNUM1 e STOP são rotinas de biblioteca que realizam as funções óbvias (imprimir um número e parar a execução, respectivamente).

4.4 UM EXEMPLO DE MICROPROGRAMA

Tendo especificado em detalhes tanto a microarquitetura quanto a macroarquitetura, o problema restante é a implementação: Com que se parece um programa sendo executado na primeira

e interpretando a segunda, e como ele funciona? Antes que possamos responder a essas questões, devemos considerar cuidadosamente em que linguagem queremos fazer nosso microprograma.

4.4.1 A Microlinguagem de Montagem

Em princípio, poderíamos escrever microprogramas em binário, 32 bits por microinstrução. Programadores masoquistas poderiam até gostar disso; certamente, ninguém mais iria. Sendo assim, precisamos de uma linguagem simbólica na qual possamos expressar microprogramas. Uma notação possível é fazer com que o microprogramador especifique uma microinstrução por linha, identificando cada campo não-nulo e seu valor. Por exemplo, para somar AC e A, e armazenar o resultado em AC, poderíamos escrever

ENC = 1, C = 1, B = 1, A = 10

Muitas linguagens de microprogramação se parecem com isto. Entretanto, esta notação é terrível.

Uma idéia muito melhor é usar uma notação de linguagem de alto nível, mantendo o conceito básico de uma linha de fonte por microinstrução. Conceitualmente, poder-se-iam escrever microprogramas em uma linguagem de alto nível comum, mas como a eficiência é crucial em microprogramas, vamos nos ater à linguagem de montagem, que definimos como uma linguagem simbólica que tem um mapeamento um-para-um para instruções de máquina. Lembre-se de que uma ineficiência de 25% num microprograma torna toda a máquina 25% mais lenta. Vamos chamar nossa microlinguagem de montagem de alto nível de "MAL" (*Micro Assembly Language*), que, em português, pode significar "doente", como você poderá ficar se forçado a escrever muitos microprogramas complexos para máquinas idiossincráticas. Em MAL, armazenamentos nos 16 registradores de rascunho, MAR ou MBR são representados por comandos de assinalamento. Assim, o exemplo acima, em MAL, ficaria *ac* := *a* + *ac*. (Como nossa intenção é tornar MAL parecida com Pascal, adotaremos a convenção usual de Pascal de nomes em itálicos minúsculos para os identificadores.)

Para indicar o uso das funções 0, 1, 2 e 3 da ALU, podemos escrever, por exemplo,

ac := *a* + *ac*, *a* := *band*(*ir*, *smask*), *ac* := *a*, e *a* := *inv*(*a*)

respectivamente, onde *band* significa "AND booleano" e *inv* significa inverso. Deslocamentos podem ser representados pelas funções *lshift*, para deslocamentos para a esquerda, e *rshift*, para deslocamentos à direita, como em

tir := *lshift*(*tir* + *tir*)

que coloca *tir* em ambos os barramentos A e B, realiza a adição e desloca a soma 1 bit para a esquerda antes de armazená-la de volta em *tir*.

Desvios incondicionais podem ser manipulados com comandos **goto**; desvios condicionais podem testar *n* ou *z*, por exemplo:

if *n* **then goto** 27

Assinalamentos e desvios podem ser combinados na mesma linha. Entretanto, um pequeno problema aparece se desejarmos testar um registrador, mas não fazer um armazenamento. Como especificamos qual registrador deve ser testado? Para resolver este problema, introduzimos a pseudovariável *alu*, à qual pode ser assinalado um valor apenas para indicar o conteúdo da ALU. Por exemplo,

alu := *tir*; **if** *n* **then goto** 27

significa que *tir* deve passar pela ALU (código da ALU = 2), de modo que o seu bit de alta ordem possa ser testado. Note que o uso de *alu* significa que ENC = 0.

Para indicar leituras e escritas à memória, vamos apenas colocar *rd* e *wr* no programa fonte.

Comando	AMUX	COND	ALU	SH	MBR	MAR	RD	WD	ENC	C	B	A	ADDR
mar := pc; rd	0	0	2	0	0	1	1	0	0	0	0	0	00
rd	0	0	2	0	0	0	1	0	0	0	0	0	00
ir := mbr	1	0	2	0	0	0	0	0	1	3	0	0	00
pc := pc + 1	0	0	0	0	0	0	0	0	1	0	6	0	00
mar := ir; mbr := ac; wr	0	0	2	0	1	1	0	1	0	0	3	1	00
*alu :=*tir; **if** *n* **then goto** 15	0	1	2	0	0	0	0	0	0	0	0	4	15
ac := inv (mbr)	1	0	3	0	0	0	0	0	1	1	0	0	00
tir := lshift (tir); **if** *n* **then goto** 25	0	1	2	2	0	0	0	0	1	4	0	4	25
alu := ac; **if** *z* **then goto** 22	0	2	2	0	0	0	0	0	0	0	0	1	22
ac := band (ir, amask); **goto** 0	0	3	1	0	0	0	0	0	1	1	8	3	00
sp := sp + (−1); rd	0	0	0	0	0	0	1	0	1	2	2	7	00
tir := lshift (ir + ir); **if** *n* **then goto** 69	0	1	0	2	0	0	0	0	1	4	3	3	69

Fig. 4.15 Alguns comandos em MAL e as microinstruções correspondentes.

A ordem das várias partes do comando fonte é, em princípio, arbitrária, mas, para melhorar sua legibilidade, vamos tentar arranjá-las na ordem em que são executadas. A Fig. 4.15 mostra alguns exemplos de comandos MAL, junto com as microinstruções correspondentes.

4.4.2 Um Exemplo de Microprograma

Chegamos finalmente ao ponto em que podemos juntar todos os pedaços. A Fig. 4.16 é um microprograma que roda no Mic-1 e interpreta Mac-1. É um programa surpreendentemente curto — apenas 79 linhas. Agora a escolha dos nomes para os registradores de rascunho na Fig. 4.8 é óbvia: PC, AC e SP são usados para armazenar os três registradores de Mac-1. IR é o registrador de instrução e contém a macroinstrução que está sendo correntemente executada. TIR é uma cópia temporária de IR, usada para decodificar o código de operação (*opcode*). Os três registradores seguintes contêm as constantes indicadas. AMASK é a máscara de endereços, 007777 (octal), e é usado para separar *opcode* de bits de endereço. SMASK é a máscara de pilha, 000377 (octal), e é usado nas instruções INSP e DESP para isolar a distância de 8 bits. Os seis registradores restantes não têm função assinalada e podem ser usados como o microprogramador quiser.

Como todo interpretador, o microprograma da Fig. 4.16 tem um *loop* principal que busca, decodifica e executa instruções do programa que está sendo interpretado, neste caso, instruções de nível 2. Seu *loop* principal começa na linha 0, onde ele inicia buscando a macroinstrução em PC. Enquanto espera que a instrução chegue, o microprograma incrementa PC e continua a ativar o sinal de barramento RD. Quando ela chega, na linha 2, ela é armazenada em IR e, simultaneamente, o bit de alta ordem (bit 15) é testado. Se o bit 15 for 1, a decodificação continua na linha 28; caso contrário, ela continua na linha 3. Assumindo por enquanto que a instrução é um LODD, o bit 14 é testado na linha 3, e TIR é carregado com a instrução original deslocada de 2 bits para a esquerda, uma vez usando o somador e outra usando o deslocador. Note que o bit N de *status* da ALU é determinado pela saída da ALU, na qual o bit 14 é o de mais alta ordem, pois IR + IR desloca IR 1 bit para a esquerda. A saída do deslocador não afeta os bits de *status* da ALU.

Todas as instruções que têm 00 em seus dois bits de alta ordem eventualmente chegam à linha 4 para ter o bit 13 testado. As instruções que começam com 000 vão para a linha 5, a aquelas começando com 001 vão para a linha 11. A linha 5 é um exemplo de uma microinstrução

com ENC = 0; ela apenas testa TIR, mas não o altera. Dependendo do resultado deste teste, o código para LODD ou STOD é selecionado.

Para LODD, o microcódigo deve primeiro buscar a palavra endereçada diretamente carregando os 12 bits de baixa ordem de IR em MAR. Neste caso, os 4 bits de alta ordem são

```
 0: mar := pc; rd;                          {loop principal}
 1: pc := pc+ 1; rd;                        {incremente pc}
 2: ir := mbr; if n then goto 28;           {salva, decodifica mbr}
 3: tir := lshift (ir + ir); if n then goto 19;
 4: tir := lshift (tir); if n then goto 11; {000x ou 001x?}
 5: alu := tir; if n then goto 9;           {0000 ou 0001?}

 6: mar := ir; rd;                          {0000 = LODD}
 7: rd;
 8: ac := mbr; goto 0;

 9: mar := ir; mbr := ac; wr;               {0001 = STOD}
10: wr :goto 0;

11: alu := tir; if n then goto 15;          {0010 ou 0011?}

12: mar := ir; rd;                          {0010 = ADDD}
13: rd;
14: ac := mbr + ac; goto 0;

15: mar := ir; rd;                          {0011 = SUBD}
16: ac := ac + 1; rd;                       {Nota: x - y = x + 1 + not y}
17: a:= inv (mbr);
18: ac := ac + a; goto 0;

19: tir := lshift (tir); if n then goto 25; {010x ou 011x?}
20: alu := tir; if n then goto 23;          {0100 ou 0101?}

21: alu := ac; if n then goto 0;            {0100 = JPOS}
22: pc := band (ir, amask); goto 0;         {faça o desvio}

23: alu := ac; if z then goto 22;           {0101 = JZER}
24: goto 0;                                 {desvio falhou}

25: alu:= tir; if n then goto 27;           {0110 ou 0111?}
26: pc := band (ir, amask); goto 0;         {0110 = JUMP}

27: ac := band (ir, amask); goto 0;         {0111 = LOCO}

28: tir := lshift (ir + ir); if n then goto 40; {10xx ou 11xx?}
29: tir := lshift (tir); if n then goto 35;     {100× ou 101x?}
30: alu := tir; if n then goto 33;              {1000 ou 1001?}

31: a := ir + sp;                           {1000 = LODL}
32: mar := a; rd; goto 7;

33: a := ir + sp;                           {1001 = STOL}
34: mar := a; mbr := ac; wr; goto 10;

35: alu := tir; if n then goto 38;          {1010 ou 1011?}

36: a := ir + sp;                           {1010 = ADDL}
37: mar := a; rd; goto 13;

38: a := ir + sp;                           {1011 = SUBL}
39: mar := a; rd; goto16;
```

Fig. 4.16 O microprograma.

```
40: tir := lshift (tir); if n then goto 46;      {110x ou 111x?}
41: alu := tir; if n then goto 44;               {1100 ou 1101?}

42: alu := ac; if n then goto 22;                {1100 = JNEG}
43: goto 0;

44: alu := ac; if z then goto 0;                 {1101 = JNZE}
45: pc := band (ir, amask); then goto 0;

46: tir := lshift (tir); if n then goto 50;

47: sp := sp + (-1);                             {1110 = CALL}
48: mar := sp; mbr := pc; wr;
49: pc := band (ir, amask); wr; goto 0;

50: tir := lshift (tir); if n then goto 65;      {1111, examine endereço}
51: tir := lshift (tir); if n then goto 59;
52: alu := tir; if n then goto 56;

53: mar := ac; rd;                               {1111000 = PSHI}
54: sp := sp + (-1); rd;
55: mar := sp; wr; goto 10;

56: mar := sp; sp := sp + 1; rd;                 {1111001 = POPI}
57: rd;
58: mar := ac; wr; goto 10;

59: alu := tir; if n then goto 62;

60: sp := sp + (-1);                             {1111010 = PUSH}
61: mar := sp; mbr := ac; wr; goto 10;

62: mar := sp; sp := sp + 1; rd;                 {1111011 = POP}
63: rd;
64: ac:= mbr; goto 0;

65: tir := lshift (tir); if n then goto 73;
66: alu := tir; if n then goto 70;

67: mar := sp; sp := sp + 1; rd;                 {1111100 = RETN}
68: rd;
69: pc:= mbr; goto 0;

70: a := ac;                                     {1111101 = SWAP}
71: ac := sp;
72: sp := a; goto 0;

73: alu := tir; if n then goto 76;

74: a := band (ir, smask);                       {1111110 = INSP}
75: sp := sp + a; goto 0;

76: a := band (ir, smask);                       {1111111 = DESP}
77: a := inv (a);
78: a := a + 1; goto 75;
```

Fig. 4.16 (cont.)

todos zero, mas para STOD e outras instruções, não. Entretanto, como MAR só tem 12 bits de largura, os bits de *opcode* não afetam a escolha da palavra lida. Na linha 7, o microprograma não tem nada a fazer, e portanto apenas espera. Quando a palavra chega, ela é copiada para AC e o microprograma desvia para o início do *loop*. STOD, ADDD e SUBD são similares. O único ponto notável relativo a elas é como a subtração é feita. Ela usa o fato de que

$$x - y = x + (\bar{}y) = x + (\bar{y} + 1) = x + 1 + \bar{y}$$

em complemento de dois. A adição de 1 a AC é feita na linha 16, que de outra maneira seria desperdiçada como a linha 13.

O microcódigo para JPOS começa na linha 21. Se AC < 0, a condição de desvio falha e JPOS é terminado imediatamente através da volta ao *loop* principal. Se, entretanto, AC ≥ 0, os 12 bits de baixa ordem de IR são extraídos, fazendo um AND deles com a máscara 007777 e armazenando o resultado em PC. Não há nenhum custo extra para remover os bits de *opcode* aqui, e assim podemos também fazê-lo. Se isso tivesse custado uma microinstrução extra, no entanto, teríamos que analisar cuidadosamente para ver se a existência de lixo nos 4 bits de alta ordem de PC poderia causar problemas mais tarde.

Em certo sentido, JZER (linha 23) funciona de maneira oposta a JPOS. Com JPOS, se a condição for verificada, o desvio falha e o controle retorna ao *loop* principal. Com JZER. se a condição for verificada, o desvio é realizado. Como o código para realizar o desvio é o mesmo para todas as instruções de desvio, podemos economizar microcódigo simplesmente indo para a linha 22 sempre que possível. Este estilo de programação geralmente seria considerado grosseiro em um programa de aplicação, mas num microprograma não existem barreiras. Desempenho é tudo.

JUMP e LOCO são imediatas, de modo que a próxima rotina interessante a ser analisada é a para LODL. Primeiro, o endereço de memória absoluto a ser referenciado é calculado somando o deslocamento contido na instrução a SP. Então, a leitura de memória é iniciada. Como o resto do código é o mesmo para LODL e LODD, vamos apenas usar as linhas 7 e 8 para ambos. Isto não apenas economiza memória de controle sem perda na velocidade de execução, mas também significa menos rotinas para depurar. Código análogo é usado para STOL, ADDL e SUBL. O código para JNEG e JNZE é similar a JZER e JPOS, respectivamente (e não ao contrário). CALL primeiro decrementa SP, então empilha o endereço de retorno, e finalmente desvia para o procedimento. A linha 49 é quase idêntica à linha 22; se fosse exatamente a mesma, poderíamos ter eliminado a 49 colocando um desvio incondicional para 22 em 48. Infelizmente, precisamos continuar ativando WR por mais uma microinstrução.

Todas as macroinstruções restantes têm 1111 nos 4 bits de alta ordem, e então a decodificação dos "bits de endereço" é necessária para distingui-las. As rotinas reais são imediatas, de modo que não faremos mais comentários sobre elas.

4.4.3 Comentários sobre o Microprograma

Apesar de termos discutido o microprograma detalhadamente, vale a pena considerar mais alguns pontos. Na Fig. 4.16, incrementamos PC na linha 1. Isso poderia igualmente ter sido feito na linha 0, liberando a linha 1 para qualquer outra coisa enquanto esperava. Nesta máquina não há nada mais a fazer, mas numa máquina real o microprograma poderia usar esta oportunidade para verificar os dispositivos de E/S esperando serviço, refrescar RAM dinâmica, ou qualquer outra coisa.

Se deixarmos a linha 1 da maneira como está, entretanto, podemos acelerar a máquina modificando a linha 8 para

mar := *pc*; *ac* := *mbr*; *rd*; **goto** 1;

Em outras palavras, podemos iniciar a busca da próxima instrução antes de termos realmente concluído a corrente. Esta habilidade provê uma forma primitiva de *pipeline* de instruções. O mesmo artifício pode também ser aplicado às outras rotinas.

É claro que uma quantidade substancial do tempo de execução de cada macroinstrução é dedicada a decodificá-la bit a bit. Esta observação sugere que poderia ser útil carregar o MPC sob o controle do microprograma. Em muitos computadores existentes, a microarquitetura tem suporte de *hardware* para extrair *opcodes* de macroinstruções e colocá-los diretamente no MPC para realizar um desvio múltiplo. Se, por exemplo, pudéssemos deslocar IR de 9 bits para a direita, zerar os 9 bits superiores e colocar o número resultante em MPC, teríamos uma ramificação de 128 caminhos para as posições de 0 a 127. Cada uma dessas palavras conteria a primeira

microinstrução para a macroinstrução correspondente. Apesar de esta abordagem desperdiçar memória de controle, ela acelera bastante a máquina e, na prática, quase sempre é utilizada.

Não dissemos uma palavra sobre como a E/S é implementada. Nem temos que fazê-lo. Usando mapeamento de memória, a CPU não sabe a diferença entre endereços de memória verdadeira e registradores de dispositivos de E/S. O microprograma manipula leituras e escritas nas quatro palavras do topo do espaço de endereçamento do mesmo modo que qualquer outra leitura ou escrita.

4.4.4 Perspectiva

A hora parece apropriada para parar por um minuto e refletir sobre o que é mesmo a microprogramação. A idéia básica é começar com uma máquina com *hardware* simples. Em nosso exemplo, ela consiste em pouco mais que 22 registradores, uma pequena ROM para armazenamento do programa de controle, um somador, um incrementador, um deslocador e algum circuito combinatório para multiplexação, decodificação e seqüenciamento. Usando este *hardware*, fomos capazes de construir um *software* interpretador para executar as instruções de uma máquina de nível 2. Com o auxílio de um compilador, podemos traduzir programas de linguagens de alto nível em instruções de nível 2, e então interpretar estas instruções, uma a uma.

Assim, para executar um programa escrito em uma linguagem de alto nível, precisamos primeiro traduzi-lo para o nível 2, e então interpretar as instruções resultantes. O nível 2 efetivamente serve como interface entre o compilador e o interpretador. Embora, em princípio, o compilador pudesse gerar microcódigo diretamente, isto é complicado e desperdiça espaço. Cada uma de nossas macroinstruções ocupa uma palavra de 16 bits, enquanto que o microcódigo correspondente, excluindo a lógica de decodificação de instrução, requer aproximadamente quatro microinstruções de 32 bits, em média. Se fôssemos compilar diretamente para o nível 1, o armazenamento total requerido seria octuplicado. Além disso, a área adicional necessária é uma memória RAM de controle, que é muito mais cara devido à sua alta velocidade. O uso da memória principal para microcódigo não é desejável porque ele resulta em uma máquina lenta.

À luz desses exemplos concretos, deve ficar claro por que as máquinas são agora normalmente projetadas como uma série de níveis. Isto é feito para eficiência e simplicidade, pois cada nível trata um outro nível de abstração. O projetista do nível 0 se preocupa sobre como comprimir os últimos poucos nanossegundos da ALU usando algum novo algoritmo bacana para reduzir o tempo de propagação do vai-1. O microprogramador se preocupa sobre como obter o melhor uso de cada microinstrução, tipicamente explorando tanto do paralelismo inerente do *hardware* quanto possível. O projetista do conjunto de macroinstruções se preocupa sobre como prover uma interface que tanto o escritor de compilador quanto o microprogramador possam aprender a gostar e que seja eficiente ao mesmo tempo. Claramente, cada nível tem diferentes objetivos, problemas, técnicas e, em geral, um ponto de vista diferente em relação à máquina. Dividindo o problema de projeto total da máquina em vários subproblemas, podemos tentar dominar a complexidade inerente ao projeto de um computador moderno.

4.5 PROJETO DO NÍVEL DE MICROPROGRAMAÇÃO

Como quase todas as coisas em ciência da computação, o projeto da microarquitetura é cheio de compromissos. Nas seções seguintes, enfocaremos algumas considerações de projeto e seus respectivos compromissos.

4.5.1 Microprogramação Horizontal *versus* Vertical

Provavelmente, a decisão chave é quanta codificação deve ser colocada nas microinstruções. Se alguém fosse construir o Mic-1 em uma única pastilha VLSI, ele poderia ignorar as abstrações tais como registradores, ALU, e assim por diante, e visualizar apenas todas as portas lógicas. Para fazer a máquina funcionar, certos sinais são necessários, tais como os 16 sinais OE para controlar a saída dos registradores para o barramento A e os sinais que controlam a função da ALU. Quando examinamos a ALU internamente, vemos que os circuitos internos são, na

realidade, controlados por quatro linhas, e não duas, porque no canto inferior esquerdo da Fig. 3.20 se encontra um circuito decodificador de 2-para-4. Em resumo, para cada máquina, algum conjunto de *n* sinais de controle, aplicados nos lugares apropriados, pode fazer a máquina funcionar, sem nenhuma decodificação.

Este ponto de vista leva a um formato de microinstrução diferente: com *n* bits de largura, um bit para cada sinal de controle. As microinstruções projetadas de acordo com esse princípio são chamadas **horizontais** e representam um dos extremos de um espectro de possibilidades. Na outra extremidade do espectro se situam as microinstruções com um pequeno número de campos altamente codificados. Estas são denominadas **verticais**. Estes nomes vêm de como um artista poderia esboçar suas respectivas memórias de controle: projetos do tipo horizontal possuem um número relativamente pequeno de microinstruções largas; os do tipo vertical possuem muitas microinstruções estreitas.

Entre estes dois extremos se encontram muitos projetos mistos. Nossas microinstruções, por exemplo, possuem um número de bits, tais como MAR, MBR, RD, WR e AMUX, que controlam diretamente o funcionamento do *hardware*. De outro lado, os campos A, B, C e ALU requerem alguma lógica de decodificação antes que eles possam ser aplicados às portas lógicas individuais. Uma microinstrução extremamente vertical poderia ter apenas um *opcode*, que fosse meramente uma generalização de nosso campo ALU, e alguns operandos, como os nossos campos A, B e C. Em tal organização, *opcodes* seriam necessários para ler e escrever na memória principal, realizar microdesvios, e assim por diante, pois os campos que controlam estas funções em nossa máquina não estariam mais presentes.

Para tornar mais clara a distinção entre microinstruções horizontais e verticais, vamos agora reprojetar nosso exemplo de microarquitetura para usar microinstruções verticais. Cada microinstrução conterá agora três campos de 4 bits, totalizando 12 bits, contra os 32 da versão original. O primeiro campo é o *opcode*, OP, que diz o que a microinstrução faz. Os dois campos seguintes são dois registradores, R1 e R2. Para executar desvios, eles são combinados para formar um único campo de 8 bits, R. Uma microinstrução típica é ADD SP,AC, que significa que AC é somado a SP.

A lista completa dos códigos de operação das microinstruções para esta nova máquina, que chamaremos de Mic-2, é mostrada na Fig. 4.17. Desta lista fica claro que cada microinstrução realiza somente uma função: se ela realizar uma adição, ela não pode também deslocar, carregar o MAR, ou mesmo manter RD ativado. Com apenas 12 bits na microinstrução, só há lugar para especificar uma operação.

Agora precisamos refazer a Fig. 4.10 para refletir as novas microinstruções. O novo diagrama de blocos é mostrado na Fig. 4.18. A porção relativa às vias de dados, à esquerda, permanece idêntica à anterior. A maior parte do controle, à direita, também permanece a mesma. Precisamos ainda, especialmente, do MIR e de uma memória de controle (embora os novos tenham largura de 12 bits, em vez de 32 bits). Os tamanhos e funções do MPC, Mmux, incrementador, relógio e lógica de microsseqüenciamento são todos idênticos aos do projeto horizontal. Além disso, precisamos de decodificadores 4-para-16 para os campos R1 e R2, análogos aos para A, B e C na Fig. 4.10.

Três diferenças principais entre a Fig. 4.10 e a Fig. 4.18 são os blocos rotulados AND, NZ e decodificador OP. O AND é necessário porque o campo R1 agora alimenta tanto o barramento A quanto o C. Um problema surge porque o barramento A é carregado durante o subciclo 2, mas o barramento C não pode ser carregado na área de rascunho até que os *latches* A e B estejam estáveis, no subciclo 3. O bloco AND faz um AND dos 16 sinais decodificados com a linha do subciclo 4 proveniente do relógio e com um sinal proveniente do decodificador OP, que é equivalente ao antigo sinal ENC. O resultado é que os 16 sinais que carregam dados na área de rascunho são ativados nas mesmas condições anteriores.

O bloco NZ é um registrador de 2 bits que pode ser comandado para armazenar os sinais N e Z da ALU. Precisamos desta facilidade porque, no novo projeto, a ALU realizará o trabalho em uma microinstrução, mas os bits de *status* só serão testados a partir da microinstrução seguinte. Como a ALU não tem armazenamento interno, e N e Z são derivados da saída corrente (com N sendo o bit de alta ordem e Z o NOR de todos os bits de saída), ambos os sinais de *status* seriam perdidos depois de uma microinstrução se não fossem guardados em algum lugar.

O elemento chave da nova microarquitetura é o decodificador OP. Este bloco produz, a

Binário	Mnemônico	Instrução	Significado
0000	ADD	Adição	*r1* := *r1* + *r2*
0001	AND	AND booleano	*r1* := *r1* AND *r2*
0010	MOVE	Move registrador	*r1* := *r2*
0011	COMPL	Complemento	*r1* := *inv (r2)*
0100	LSHIFT	Deslocamento à esquerda	*r1* := *lshift(r2)*
0101	RSHIFT	Deslocamento à direita	*r1* := *rshift (r2)*
0110	GETMBR	Armazena MBR em registrador	*r1* := *mbr*
0111	TEST	Testa registrador	**if** *r2* < 0 **then** *n* := *true*; **if** *r2* = 0 **then** *z* := *true*
1000	BEGRD	Início de leitura	*mar* := *r1*; *rd*
1001	BEGWR	Início de escrita	*mar* := *r1*; *mbr* := *r2*; *wr*
1010	CONRD	Continua leitura	*rd*
1011	CONWR	Continua escrita	*wr*
1100		(Não utilizado)	
1101	NJUMP	Desvie se N = 1	**If** *n* **then goto** *r*
1110	ZJUMP	Desvie se Z = 1	**if** *z* **then goto** *r*
1111	UJUMP	Desvio incondicional	**goto** *r*

r = 16 * *r1* + *r2*

Fig. 4.17 Os *opcodes* do Mic-2.

partir do campo OPCODE de 4 bits, sinais que alimentam o bloco AND, a lógica de microsseqüenciamento, NZ, Amux, a ALU, o deslocador, MBR, MAR, RD e WR. A lógica de microsseqüenciamento, a ALU e o deslocador requerem, cada um, dois sinais idênticos aos do projeto anterior. Ao todo, o decodificador OP gera 13 sinais distintos, baseados nos 4 bits de alta ordem da microinstrução corrente.

Para cada um dos 16 códigos de operação de uma microinstrução, o projetista da máquina deve determinar quais dos 13 sinais vindos do decodificador OP estão ativados e quais não estão. Na verdade, deve ser gerada uma matriz binária de 16 × 13 que forneça o valor de cada linha de controle para cada um dos códigos de operação (*opcodes*). A matriz para o Mic-2 é mostrada na Fig. 4.19. As colunas estão rotuladas com os nomes dos sinais. Os sufixos L e H significam baixo (*Low*) e alto (*High*), respectivamente, e se aplicam somente aos dispositivos com duas linhas de controle: a ALU, o deslocador e a lógica de microsseqüenciamento.

Como um exemplo de um código de operação de uma microinstrução, considere BEGRD, que inicia uma leitura de memória. Ela utiliza a função 2 da ALU (selecionar o barramento A), de modo que ALUH = 1 e ALUL = 0. Ela também armazena o endereço em MAR e ativa o sinal RD. Todos os outros sinais de controle estão desativados. Considere agora os desvios. Para manter a compatibilidade com a lógica de microsseqüenciamento anterior, é preciso que o par MSLH MSLL seja 00 para não haver desvio, 01 para desviar se o bit N estiver ativo, 10 para desviar se o bit Z estiver ativo, e 11 para desviar incondicionalmente. (A compatibilidade atingiu proporções tão epidêmicas na indústria de computadores, que até as máquinas hipotéticas de livros-textos são compatíveis com suas predecessoras.) NJUMP gera 01, ZJUMP gera 10 e UJUMP gera 11. Todos os outros *opcodes* geram 00.

Agora vem a parte interessante. Como construir um circuito com quatro linhas de entrada (os 4 bits de *opcode*) e 13 linhas de saída (os 13 sinais de controle) que execute a função da Fig. 4.19? Resposta: utilize uma ou mais PLAs (ou ROMs). A Fig. 4.19 é, na verdade, uma maneira pouco peculiar de representar 13 tabelas verdade de quatro variáveis, uma por coluna, com o número da linha definindo implicitamente os valores das quatro variáveis. Assim, a questão

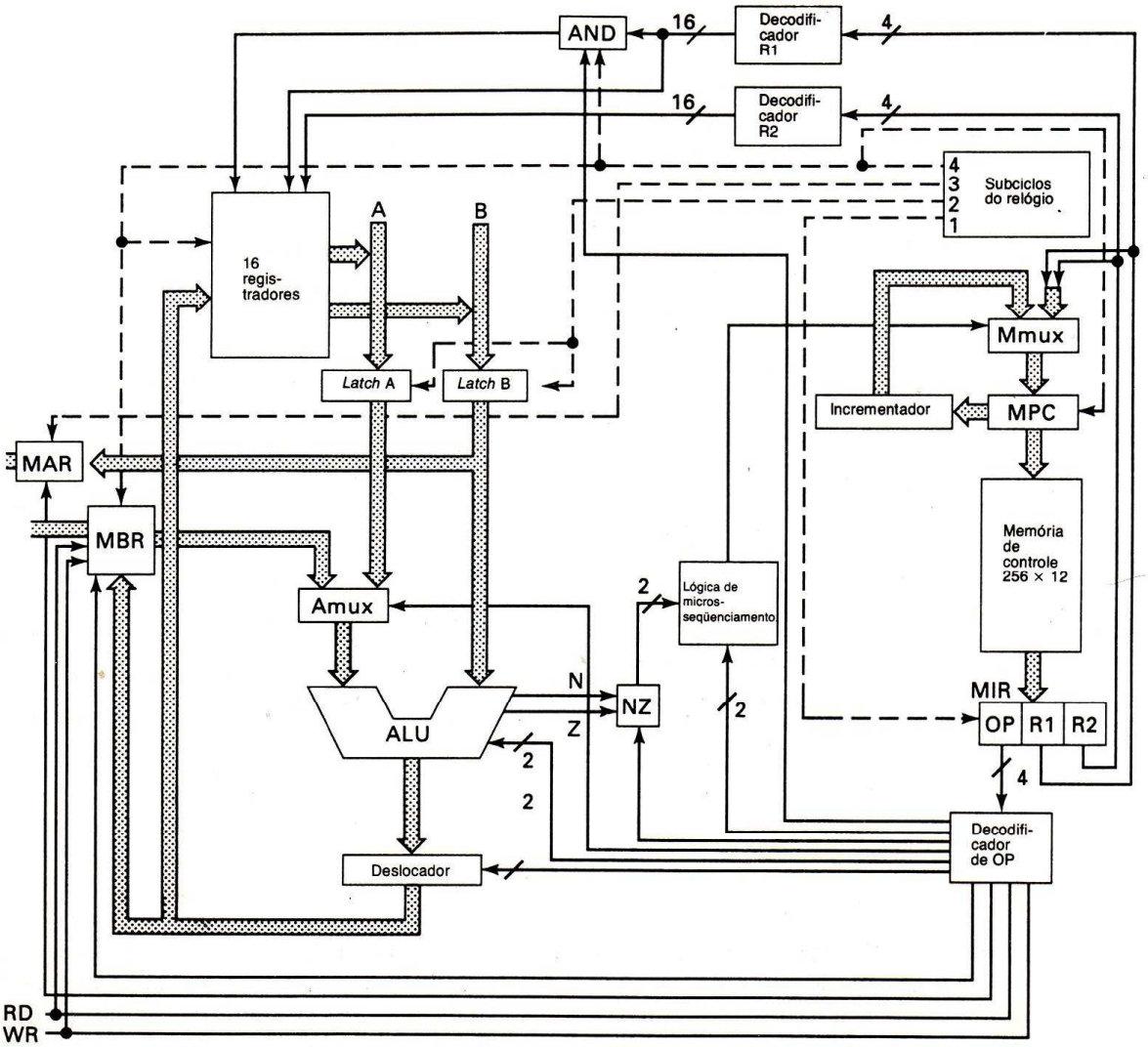


Fig. 4.18 Uma microarquitetura com microinstruções verticais.

de como construir o circuito se reduz à questão de como se implementa uma tabela verdade. A melhor forma é com uma PLA de quatro entradas e 13 saídas. Se isso não estiver disponível, três PLAs 74S330 de 12 entradas e seis saídas também podem ser usadas. Se denominarmos os quatro bits de *opcode* de A a D, do mais para o menos significativo, então algumas saídas serão:

$$\begin{aligned} \text{ALUL} &= \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}CD = \bar{A}\bar{B}D \\ \text{SHH} &= \bar{A}B\bar{C}\bar{D} \\ \text{MAR} &= A\bar{B}\bar{C}\bar{D} + A\bar{B}\bar{C}D = A\bar{B}\bar{C} \\ \text{MSLH} &= \text{ABC}\bar{D} + ABCD = ABC \end{aligned}$$

Somente 15 dos termos produto são gerados internamente, pois $AB\bar{C}\bar{D}$ não ocorre.

Agora, com o *hardware* completamente reprojetado, precisamos reescrever o microprograma. Ele é mostrado na Fig. 4.20. Os rótulos (*labels*) foram mantidos para tornar mais fácil a comparação dos dois microprogramas. A sintaxe também foi mantida. Poderíamos tê-lo escrito utilizando uma notação de linguagem de montagem típica — *assembly language* — (por exemplo, os *opcodes* da Fig. 4.17), mas em vez disso utilizamos a MAL de novo, pois ela é mais fácil de ser lida. Note que os comandos em MAL da forma *alu* := *reg* usam a microinstrução TEST para ativar os bits N e Z. Tenha certeza de que você entendeu a diferença entre o microprograma em

MEC / UFF / NDC / BEE
Biblioteca da Escola de Engenharia e

		Linhas de controle												
			ALUL		SHL		AMUX		MAR		RD		MSLH	
Opcode da microinstrução		ALUH	↓	SHH	↓	NZ	↓	AND	↓	MBR	↓	WR	↓	MSLL
0	ADD					+		+						
1	AND		+			+		+						
2	MOVE	+				+		+						
3	COMPL	+	+			+		+						
4	LSHIFT	+		+		+		+						
5	RSHIFT	+			+	+		+						
6	GETMBR	+				+	+	+						
7	TEST	+				+								
8	BEGRD	+							+		+			
9	BEGWR	+							+	+		+		
10	CONRD	+									+			
11	CONWR	+										+		
12														
13	NJUMP	+												+
14	ZJUMP	+											+	
15	UJUMP	+											+	+

Fig. 4.19 Os sinais de controle para cada *opcode* de microinstrução. Um mais significa que o sinal está ativo; um espaço significa que ele está desativado.

binário, como ele é carregado na memória de controle, e a versão em linguagem de montagem, que aparece no texto.

Em geral, este microprograma é mais simples que o primeiro, porque cada linha executa apenas uma operação. Como conseqüência, muitas das linhas do microprograma original foram divididas em duas, três ou mesmo quatro linhas neste. Outro aspecto do projeto vertical que aumenta o número de microinstruções é a falta de instruções de três endereços. Veja, por exemplo, as linhas 22 e 27 do primeiro microprograma.

O microprograma original utiliza 79 palavras de 32 bits, para um total de 2528 bits de armazenamento de controle. O segundo utiliza 160 palavras de 12 bits, para um total de 1920 bits. Essa diferença representa uma economia de 24% na memória de controle. Para um computador de uma única pastilha, ela também representa uma economia de 24% da área da pastilha necessária para a memória de controle, o que o torna mais fácil e mais barato para fabricar. A desvantagem de uma memória de controle menor é a necessidade de executar mais microinstruções para cada macroinstrução. Geralmente, isso torna a máquina mais lenta. Conseqüentemente, máquinas rápidas e caras tendem a ser horizontais, e máquinas lentas e baratas tendem a ser verticais.

A existência de microinstruções altamente codificadas, como em Mic-2, levanta algumas sérias questões filosóficas sobre o que é microprogramação. O conjunto de microinstruções da Fig. 4.17 poderia quase passar por um conjunto de instruções de uma *linguagem de máquina convencional* de um mini- ou microcomputador muito simples. O PDP-8, por exemplo, é um minicomputador com uma palavra de 12 bits cujas instruções não são muito mais poderosas que estas. Considerando que o "significado" das instruções está muito embutido no *hardware* (o circuito ou PLA que implementa o decodificador OP), poder-se-ia argumentar que o Mic-2 é, na realidade, uma máquina não microprogramada que, por acaso, tem um programa interpretador sendo executado nela para outra máquina. Se o microprograma para a máquina vertical estivesse armazenado na memória principal (como no IBM 370/145, por exemplo), então a distinção entre

```
0:  mar := pc; rd;
1:  rd;
    pc := pc + 1;
2:  ir := mbr;
    tir := lshift(ir);
    if n then goto 28;
3:  tir := lshift(tir);
    if n then goto 19;
4:  tir := lshift(tir);
    if n then goto 11;
5:  alu := tir;
    if n then goto 09;

6:  mar := ir; rd; {LODD}
7:  rd;
8:  ac := mbr;
    goto 0;

9:  mar := ir; mbr := ac; wr; {STOD}
10: wr;
    goto 0;

11: alu := tir;
    if n then goto 15;

12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
    ac := ac + a;
    goto 0;

15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
    a := inv(a);
18: ac := ac + a;
    goto 0;

19: tir := lshift(tir);
    if n then goto 25;
20: alu := tir;
    if n then goto 23;

21: alu := ac; {JPOS}
    if n then goto 0;
22: pc := ir;
    pc := band(pc, amask);
    goto 0;

23: alu := ac; {JZER}
    if z then goto 22;
24: goto 0;

25: alu := tir;
    if n then goto 27;
26: pc := ir; {JUMP}
    pc := band(pc, amask);
    goto 0;

27: ac := ir; {LOCO}
    ac := band(ac, amask);
    goto 0;

28: tir := lshift(tir);
    if n then goto 40;
29: tir := lshift(tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;

31: a := ir; {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0;

33: a := ir; {STOL}
    a := a + sp;
34: mar := a; mbr := ac; wr;
    wr;
    goto 0;

35: alu := tir;
    if n then goto 38;

36: a := ir; {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;

38: a := ir; {SUBL}
    a := a + sp;
39: mar := a; rd;
    rd;
    goto 99;

40: tir := lshift(tir);
    if n then goto 46;
41: alu := tir;
    if n then goto 44;

42: alu := ac; {JNEG}
    if n then goto 22;
43: goto 0;

44: alu := ac; {JNZE}
    if z then goto 0;
45: pc := ir;
    pc := band(pc, amask);
    goto 0;

46: tir := lshift(tir);
    if n then goto 50;

47: sp := sp + (-1); {CALL}
48: mar := sp; mbr := pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
```

Fig. 4.20 O microprograma para Mic-2.

```
50: tir := lshift(tir);
    if n then goto 65;
51: tir := lshift(tir);
    if n then goto 59;
52: alu := tir;
    if n then goto 56;

53: mar := ac; rd; {PSHI}
    rd;
54: sp := sp + (-1);
55: a := mbr;
    mar := sp; mbr := a; wr;
    wr;
    goto 0;

56: mar := sp; rd; {POPI}
57: rd;
    sp := sp + 1;

58: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
59: alu := tir;
    if n then goto 62;

60: sp := sp + (-1); {PUSH}
61: mar := sp; mbr := ac; wr;
    wr;
    goto 0;

62: mar := sp; rd; {POP}
63: rd;
    sp := sp + 1;

64: ac := mbr;
    goto 0;

65: tir := lshift(tir);
    if n then goto 73;
66: alu := tir;
    if n then goto 70;

67: mar := sp; rd; {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;

70: a := ac; {SWAP}
71: ac := sp;
72: sp := a;
    goto 0;

73: alu := tir;
    if n then goto 76;

74: a := ir; {INSP}
    a := band(a, smask);
75: sp := sp + a;
    goto 0;

76: a := ir; {DESP}
    a := band(a, smask);
77: a := inv(a);
78: a := a + 1;
    sp := sp + a;
    goto 0;
```

Fig. 4.20 (cont.)

uma máquina microprogramada altamente vertical e uma *hardwired* (através de interligações de fios) fica ainda mais difícil. Para um tratamento mais completo de codificação e paralelismo em microcódigo, veja Dasgupta (1979).

4.5.2 Nanoprogramação

Os projetos discutidos até agora têm tido duas memórias: a memória principal (usada para armazenar o programa de nível 2) e a memória de controle (usada para guardar o microprograma). Uma terceira memória, a **nanomemória**, torna possível interessantes compromissos entre microprogramação horizontal e vertical. Nanoprogramação é adequada quando muitas microinstruções ocorrem várias vezes. O microprograma da Fig. 4.16 não possui essa propriedade. A microinstrução que ocorre mais comumente é aquela que contém apenas *rd*, e isso ocorre somente cinco vezes.

A Fig. 4.21 ilustra o conceito de nanoprogramação. Na parte (a), é mostrado um microprograma de n microinstruções, cada uma com w bits de largura. Um total de nw bits de memória de controle é necessário para armazenar o microprograma. Suponha que um estudo cuidadoso do microprograma revelou que apenas m microinstruções diferentes das 2^w possibilidades são realmente usadas, com $m \ll n$. Uma nanomemória especial com m palavras de w bits poderia ser usada para armazenar cada uma das microinstruções. Cada microinstrução do programa original poderia então ser substituída pelo endereço da palavra da nanomemória que contém aquela microinstrução. Como só existem m palavras na nanomemória, a memória de controle precisaria ter uma largura de apenas $\log_2 m$ (arredondado para cima para um inteiro) bits, como mostrado na Fig. 4.21(b).

O microprograma é executado como se segue. A primeira palavra é buscada da memória

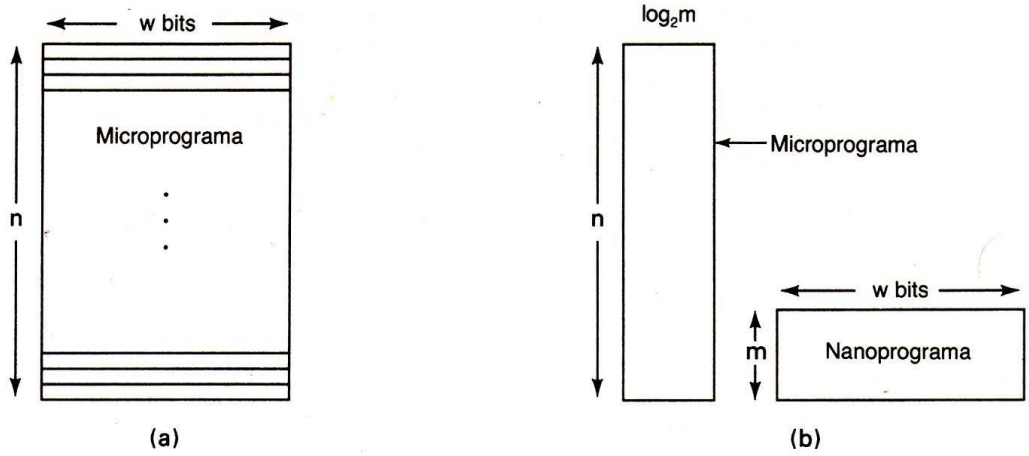


Fig. 4.21 (a) Um microprograma convencional. (b) O nanoprograma correspondente se apenas *m* microinstruções diferentes ocorrem no microprograma.

de controle. Ela é então usada para selecionar uma palavra da nanomemória, que é buscada e colocada no registrador de microinstrução. Os bits desse registrador são então usados para controlar as portas lógicas durante um ciclo. No final do ciclo, a próxima palavra é buscada da memória de controle, e o processo se repete.

Como um exemplo, suponha que o microprograma original tivesse 4096 × 100 bits, mas que existissem apenas 128 microinstruções diferentes. Uma nanomemória de 128 × 100 bits seria suficiente para armazenar todas as microinstruções necessárias. A memória de controle ficaria então com 4096 × 7 bits, com cada palavra apontando para uma nanoinstrução. A economia de memória, neste exemplo, é dada por:

$$\text{economia} = 4096 \times 100 - 4096 \times 7 - 128 \times 100 = 368.128 \text{ bits}$$

O preço que se paga pela economia de memória de controle é a execução mais lenta. A máquina com dois níveis de memória de controle funcionaria mais lentamente que a original, pois o ciclo de busca agora requer duas referências a memória, uma para a memória de controle e outra para a nanomemória. Estas duas buscas não podem ser sobrepostas.

Nanoprogramação é mais efetiva quando as mesmas microinstruções são altamente utilizadas. Se duas microinstruções que são quase a mesma pudessem ser contadas como sendo realmente a mesma, o microprograma conteria uma quantidade menor de microinstruções distintas, cada uma com uma freqüência de uso mais alta. Uma variante desta idéia básica permite exatamente isso. Palavras na nanomemória podem ser parametrizadas. Por exemplo, duas microinstruções podem diferir apenas no campo que diz qual registrador deve ser conectado a algum barramento. Colocando o número do registrador na memória de controle, em vez de na nanomemória (isto é, colocando zeros no campo de registrador das nanoinstruções), as duas microinstruções poderiam ambas apontar para a mesma nanopalavra. Quando a palavra é buscada e colocada no registrador de microinstrução, o campo de registrador é pego da memória de controle, em vez de da nanomemória. A montagem do registrador de microinstrução parcialmente a partir da memória de controle e parcialmente a partir da nanomemória obviamente requer *hardware* especial, mas este nada tem de especialmente complexo. É claro que o uso desta técnica aumenta a largura da memória de controle, que pode ser maior do que o ganho obtido por ter uma nanomemória menor.

4.5.3 Melhorando o Desempenho

Embora o objetivo da nanoprogramação seja reduzir o tamanho da memória de controle, mesmo ao preço de uma execução mais lenta, muito tempo e esforço foram gastos na direção inversa: tentando acelerar a execução, mesmo ao preço de mais memória de controle. Estes dois objetivos de projeto, aparentemente incompatíveis, visam a interesses comerciais diferentes

— produção de máquinas baratas e produção de máquinas rápidas. Nesta seção, enfocaremos algumas das maneiras como a microarquitetura direta de nossos exemplos pode ser modificada para fazer as máquinas trabalharem mais rapidamente.

Até agora, tacitamente assumimos que todos os quatro subciclos tinham a mesma duração. Ao mesmo tempo que esta abordagem simplifica, ela raramente resulta num desempenho ótimo, pois invariavelmente um dos quatro gasta mais tempo que os outros três. Ter todos os subciclos com a duração do pior caso torna a máquina mais lenta. A maneira de sanar esse problema é permitir que cada subciclo tenha uma duração independente dos outros. Desta forma, a duração de cada ciclo pode ser a quantidade de tempo necessária para realizar o trabalho, e não mais.

Embora seja este um passo na direção correta, ele é ainda muito conservador, pois a duração de cada subciclo ainda é determinada pelo pior caso daquele subciclo. Considere, por exemplo, o subciclo 3 de nossos exemplos. Se a operação da ALU é uma adição, a ALU provavelmente precisa de mais tempo (devido aos atrasos de propagação de vai-1) do que se a operação fosse meramente selecionar A. Se o tempo dedicado ao subciclo 3 é uma constante determinada quando a máquina é projetada, ele tem que ser o tempo de adição, e não o tempo de seleção de A. Uma estratégia alternativa é deixar que o tempo do subciclo seja determinado pela operação específica a ser realizada, de modo que ele seja tão curto quanto possível.

Uma vez que decidimos permitir que cada subciclo dependa da operação, precisamos encontrar uma maneira de implementar esta estratégia. Tornar o relógio mais lento ou acelerá-lo é tecnicamente difícil, de modo que, em vez disso, um relógio mestre com um período mais curto que o tempo do subciclo é usado. Cada subciclo dura então um determinado número de pulsos. Por exemplo, se o tempo da ALU puder variar entre 75 e 150 ns, um relógio mestre com período de 25 ns pode ser usado, com o subciclo consumido entre três e seis períodos.

Outra questão é: Como a máquina sabe qual a duração do subciclo? Duas técnicas podem ser utilizadas para respondê-la. Na primeira, a própria microinstrução contém um ou mais campos que fornecem explicitamente a temporização. A outra é deduzir a temporização baseando-se nos campos de operação (usando uma PLA), da mesma maneira que todos os outros sinais de controle foram gerados na máquina vertical. A primeira abordagem custa bits da memória de controle, e a segunda custa lógica de decodificação e tempo.

Outra maneira de obter melhor desempenho da máquina é aumentar a flexibilidade dos microdesvios condicionais. Como um exemplo, considere uma macroinstrução SKIP LESS, que compara AC com uma palavra de memória e salta a próxima macroinstrução se AC for numericamente menor. Fazer a comparação requer a subtração da palavra de memória de (uma cópia de) AC, que pode resultar num *overflow*, como ilustrado na Fig. 4.22.

Devido à potencialidade de *overflow*, não podemos dizer qual operando era menor simplesmente olhando para o bit de sinal do resultado. No quarto exemplo da Fig. 4.22, AC é menor que a memória, mas AC — memória é positivo. A condição correta a ser testada é N EXCLUSIVE OR V, onde V indica a presença ou ausência de *overflow*. (O *hardware* ativa o bit V sempre que o vem-1 para o bit de sinal difere do vai-1 do bit do sinal.)

Felizmente, a maioria das ALUs gera não apenas N e Z, mas também V e C (*carry*). Entretanto, se apenas quatro desvios condicionais estão disponíveis para o microprogramador, um para cada bit de NZVC, a macroinstrução SKIP LESS iria requerer muitas microinstruções. Para prover mais flexibilidade, muitas máquinas não possuem a capacidade de testar individualmente bits de *status* da ALU. Em vez disso, um único bit da microinstrução realiza um OR de NZVC com os 4 bits de baixa ordem do campo ADDR, e o desvio é feito. Alguns exemplos são dados na Fig. 4.23. Se os 4 bits de baixa ordem são todos iguais a 0, isso se torna um desvio condicional

AC	000100	000100	077777	100001	000010
Mem	000050	170000	177775	000010	100001
	000030	010100	100002	077771	100007
	N = 0	N = 0	N = 1	N = 0	N = 1
	V = 0	V = 0	V = 1	V = 1	V = 1

Fig. 4.22 Alguns exemplos de subtração de 16 bits em complemento de dois, com octal. Os bits N e V para cada resultado também são mostrados.

Campo ADDR	NZVC	Desviar para endereço
10000000	1001	10001001
10001000	1001	10001001
10001011	1001	10001011
10001011	1000	10001011
10001011	0000	10001011
10001111	0000	10001111
10001111	1100	10001111
10000000	1100	10001100

Fig. 4.23 Alguns exemplos de desvios de múltipla escolha baseados em NZVC.

de 16 possibilidades. Se forem todos iguais a 1, o desvio é do tipo incondicional para um endereço específico terminando em 1111.

Com esta facilidade, a instrução SKIP LESS fica muito mais fácil de ser interpretada. Escolhemos um campo ADDR terminando em 0101, por exemplo, 10000101 (binário), e executamos o desvio. As microinstruções em 10001101 e 10000111 lidam com desvios que têm sucesso, e aquelas em 10000101 e 10001111 lidam com desvios que falham. Nenhuma decodificação adicional é necessária. Poderíamos também ter utilizado um endereço base terminando em 0000 em vez de 0101, mas estes são de enorme valor, pois são os únicos disponíveis para desvios condicionais de 16 possibilidades. Portanto, eles não devem ser escolhidos despreocupadamente.

Deve estar claro que, com esse tipo de controle de seqüência de microinstrução, a localização das microinstruções na memória de controle torna-se uma verdadeira dor de cabeça. A primeira microinstrução executada depois de um desvio condicional de 16 possibilidades deve conter, ela própria, um desvio incondicional, pois a palavra que a segue (exceto para a última) já está em uso como um possível destino do desvio. Para onde cada microinstrução deve desviar? Certamente, não para um endereço da forma xxxx0000, pois eles são muito valiosos, mas todos os outros endereços, exceto aqueles da forma xxxx1111, também poderão ser necessários mais cedo ou mais tarde. A decisão deve ser feita cuidadosamente, para evitar esgotar algum tipo de endereço. Uma vez consumidos todos os endereços pares, por exemplo, não será mais possível testar o bit C, de modo que a escolha de endereços deve ser feita com muito cuidado.

4.5.4 *Pipelining*

Outra maneira de acelerar a máquina é construir o *hardware* a partir de várias unidades funcionais e colocá-las como numa linha de montagem. Na Fig. 2.5, vimos como um *pipeline* de cinco unidades funciona. Na Fig. 4.24, vemos que, se a instrução 1 é buscada no ciclo 1, ela será executada no ciclo 5. Similarmente, se a instrução 4 é buscada no ciclo (isto é, porção de tempo) 8, decodificada no ciclo 9, e assim por diante, ela será executada no ciclo 12. Sob condições ótimas, daí para frente uma instrução é executada a cada ciclo, dando uma taxa média de execução de uma instrução por ciclo, em vez de uma instrução a cada cinco ciclos.

Infelizmente, estudos mostraram que cerca de 30% de todas as instruções são desvios, e estes causam dano ao *pipeline*. Desvios podem ser classificados em três categorias: incondicionais, condicionais e *loop*. Um desvio incondicional diz ao computador para parar de buscar instruções consecutivamente, e ir para algum endereço específico. Um desvio condicional testa alguma condição e desvia se a condição é satisfeita. Um exemplo típico é uma instrução que testa algum registrador e desvia se o registrador contém zero. Se o registrador não contém zero, o desvio não é feito e o controle continua na seqüência corrente.

Instruções de *loop* tipicamente decrementam um contador de iteração, e então desviam de volta ao início do *loop* se ele é diferente de zero (isto é, ainda existem mais iterações a serem

	Ciclo												
	1	2	3	4	5	6	7	8	9	10	11		
Busca de instrução	1	2	B					4	5	6	7	8	9
Decodificação de instrução		1	2	B					4	5	6	7	8
Cálculo de endereço			1	2	B					4	5	6	7
Busca de operando				1	2	B					4	5	6
Execução					1	2	B					4	5

Fig. 4.24 Um *pipeline* de cinco estágios. Os números são usados para rotular as instruções. A instrução marcada *B* é um desvio condicional.

feitas). Instruções de *loop* são um caso especial importante de desvios condicionais, pois é conhecido *a priori* que elas quase sempre têm sucesso.

Considere o que acontece com o *pipeline* da Fig. 4.24 quando uma instrução de desvio condicional, marcada *B,* é encontrada. A próxima instrução a ser executada pode ser a que se segue ao desvio, mas pode também estar no endereço para o qual deve ser desviado o controle, chamado de **destino do desvio.** Como a unidade que busca a instrução não sabe qual até que o desvio seja executado, ela pára, e não pode ir adiante até que o desvio tenha sido executado. Conseqüentemente, o *pipeline* esvazia. Somente depois do ciclo 7 ter sido completado é que se sabe qual a instrução que vem a seguir.

O desvio efetivamente causou a perda de quatro ciclos, chamados de **penalidade do desvio.** Com uma entre cada três instruções sendo um desvio, fica claro que a perda de desempenho é substancial.

Grande quantidade de pesquisa tem tratado do problema de recuperar algum desse desempenho (DeRosa e Levy, 1987; McFarling e Hennessy, 1986; Hwu *et al.,* 1989; e Lilja, 1988). A coisa mais simples a fazer é assumir que o desvio não será realizado, e apenas continuar enchendo o *pipeline*, como se o desvio fosse uma simples instrução aritmética. Se acontecer que o desvio realmente não foi realizado, não perdemos nada. Se for, teremos que apagar as instruções atualmente no *pipeline*, algo chamado ***squashing***, e começar de novo.

Squashing causa problemas próprios. Em algumas máquinas, como um subproduto do cálculo de endereço, um registrador pode ser modificado. Se a instrução sendo apagada modificou um ou mais registradores, estes devem ser restaurados, o que significa que deve haver um mecanismo de guardar seus valores originais.

É instrutivo fazer um modelo simples da perda de desempenho.

Seja P_j a probabilidade de que uma instrução seja um desvio.

Seja P_t a probabilidade de que o desvio seja realizado.

Seja b a penalidade do desvio.

O tempo médio de execução (em ciclos) pode ser calculado como a soma ponderada dos dois casos: instruções comuns e instruções de desvio.

$$\text{tempo médio de instrução} = (1 - P_j)(1) + P_j[P_t(1 + b) + (1 - P_t)(1)]$$

Alguma álgebra revela que o tempo médio de instrução é $1 + bP_jP_t$. A eficiência da execução é então $1/(1 + bP_jP_t)$. Com $b = 4$, $P_j = 0{,}3$ e $P_t = 0{,}65$ (um valor típico medido), a máquina trabalha a menos de 60% de sua velocidade potencial.

O que se pode fazer para melhorar o desempenho? Para começar, se pudéssemos predizer de que maneira cada desvio seria realizado, poderíamos buscar a instrução adequada e eliminar a penalidade. Em nossa fórmula para a eficiência, podemos então substituir P_t por P_w, a probabilidade de adivinhar errado.

Dois tipos de previsão são possíveis: estática (tempo de compilação) e dinâmica (tempo de execução). Com previsão estática, o compilador faz uma suposição a respeito de cada instrução de desvio que ele gera. Com instruções de *loop*, por exemplo, adivinhar que ela irá desviar de volta ao início é o correto na maioria das vezes. Durante o teste de uma condição improvável, como uma chamada de sistema retornando um código de erro, a não realização do desvio é o que tem mais chance. Em muitos casos, instruções diferentes são usadas para esses casos, e somente olhando para o *opcode* temos uma pista substancial.

Um esquema mais elaborado é que os projetistas da máquina providenciem dois *opcodes* para cada tipo de desvio, e que o compilador use o primeiro se ele achar que o desvio será realizado, e o segundo, caso contrário. Alternativamente, para programas que são muito utilizados, o programa pode primeiro ser executado em um simulador, e o comportamento de cada desvio ser realmente guardado. Então, o programa binário pode ser alterado, substituindo cada desvio com o *opcode* apropriado (provavelmente será realizado/provavelmente não será realizado).

O outro método de previsão é o dinâmico. Durante a execução, o microprograma monta uma tabela dos endereços contendo desvios e mantém informações a respeito do comportamento de cada um. Este método tende a tornar a máquina mais lenta por causa da manutenção do registro, de modo que alguma ajuda de *hardware* pode ser necessária. Medições mostraram que a obtenção de 90% de acerto desta maneira não é difícil.

A previsão de desvios não é o único meio de ataque. Outro é tentar determinar de que maneira será o desvio antecipadamente no *pipeline*. Alguns testes, como desvios se igual, são muito mais fáceis de fazer do que desvios se menor que. O primeiro pode ser feito com um comparador, enquanto o último requer um ciclo completo pelas vias de dados para fazer uma subtração. Com esta técnica, sempre que um desvio for encontrado, o microprograma realiza uma rápida verificação num estágio inicial do *pipeline* para ver se ele pode resolver o desvio imediatamente. Se puder, ele sabe onde continuar buscando.

Compiladores podem fazer muito para ajudar. Por exemplo, quando o programador escreve um *loop* fazendo *i* variar de 1 a 10, o compilador pode testar *i* para ver se *i* é igual a 10, em vez de ver se ele é menor que 11, de modo que o microprograma pode realizar uma comparação em vez de uma subtração.

Para tratar aqueles desvios que não podem ser resolvidos antecipadamente, o compilador pode tentar encontrar alguma coisa útil para o computador fazer enquanto estiver esperando que o desvio seja executado. Considere a Fig. 4.25(a), um comando aritmético seguido por um teste. Um compilador com otimização muito inteligente poderia produzir código como na Fig. 4.25(b), que não é válido em Pascal, mas mostra a ordem dos eventos. Ele primeiro gera código para realizar o teste, depois realiza a operação aritmética. Depois que o desvio entrou no *pipeline*, muitas instruções comuns se seguem. Estas devem ser feitas de qualquer jeito, de modo que não há necessidade de fazer uma previsão e não há necessidade de fazer *squashing*. Para usar esta técnica efetivamente, os microprogramadores e os escritores de compiladores devem trabalhar juntos durante o projeto.

Se tudo o mais falhar, sempre há a possibilidade de seguir dois caminhos em paralelo. Isso requer dois *pipelines* no *hardware* e não elimina o problema de *squashing*. Ainda nos grandes computadores, onde a consideração é o desempenho e não o preço, este método é algumas vezes utilizado. É claro que, se qualquer caminho encontrar outro desvio pela frente antes que o primeiro seja resolvido, a vida fica ainda mais complicada. Ter algumas dúzias de *pipelines* por aí para tratar do pior caso provavelmente não é uma boa idéia.

Depois de toda essa teoria sobre o que se poderia fazer, vamos voltar à nossa máquina exemplo por um momento. Ela não tem unidades de *hardware* distintas para buscar, decodificar,

```
a := b + c;
if b < c
  then comando;
```

(a)

```
if b < c
a := b + c;
  then comando;
```

(b)

Fig. 4.25 (a) Um trecho em Pascal. (b) Como o compilador poderia tratá-lo.

e assim por diante, de modo que não é possível muito *pipeline*, mas, com pequenas mudanças no microprograma, podemos chegar a uma quantidade limitada de sobreposição entre busca e execução de microinstruções, que também é uma forma de *pipeline*.

Na Fig. 4.10, por exemplo, se a próxima microinstrução pudesse de alguma maneira ser buscada durante o subciclo 4, o subciclo 1 não seria mais necessário, e o relógio poderia apenas gerar pulsos para os subciclos 234234234... O problema principal ocorre no tratamento das microinstruções de desvios condicionais, como vimos. Se a máquina esperar até que as linhas de *status* da ALU estejam disponíveis antes de iniciar a busca da próxima microinstrução, é muito tarde: o ciclo praticamente já terminou e pouca sobreposição pode ser atingida. Em algumas máquinas, o problema é resolvido apenas através do uso dos sinais de *status* da ALU do ciclo anterior, que, é claro, devem ser armazenados para evitar que se percam. Estes valores estão disponíveis no início de cada microinstrução, de modo que a próxima busca pode começar assim que a busca corrente terminar, bem antes que a execução da microinstrução corrente se complete. Não é necessário dizer que essa maneira de fazer as coisas complica em muito a vida para o microprogramador.

Tecnicamente, é possível escrever microprogramas para tal máquina de um modo razoável. Por exemplo, para testar alguma palavra da área de rascunho e desviar se ela for negativa, o microprograma pode passar a palavra pela ALU numa microinstrução que *não* contém um desvio. A próxima microinstrução é uma *no-op* (nenhuma operação) com desvio condicional, isto é, não faz nada, a não ser testar os bits de *status* da ALU armazenados e desviar. O preço pago pela programação de uma forma racional é a duplicação do tempo de desvio, uma proposição não-desejável.

A única solução é pegar um vidro de aspirinas e tentar fazer o melhor possível nesta situação. Por exemplo, o microprogramador pode adivinhar para onde o desvio condicional usualmente vai e usar a segunda microinstrução para começar a fazer o trabalho que provavelmente será necessário. Infelizmente, se o desvio seguir outro caminho, será necessário fazer algum *squashing*.

Como um exemplo simples, vamos reescrever as linhas de 11 a 18 da Fig. 4.16 para uma máquina com sobreposição de busca e execução de microinstruções. Os resultados são mostrados na Fig. 4.26(a). Neste exemplo, tivemos sorte, pois ao executar a linha 11 nós sabemos que o próximo código será ADDD ou SUBD, e ambos começam com a mesma microinstrução. Assim, apenas colocamos a microinstrução comum na linha 12 para manter a máquina ocupada enquanto o desvio está sendo executado.

Um exemplo ligeiramente menos agradável é mostrado na Fig. 4.26(b). Na linha 52, sabemos que a próxima microinstrução será PSHI e POPI, mas infelizmente estas duas rotinas não possuem a mesma primeira microinstrução. Suponha, entretanto, que uma análise estatística dos programas para Mac-1 mostrou que PSHI é muito mais comum que POPI. Então, podemos proceder como mostrado na Fig. 4.26(b). Para PSHI, tudo está ótimo, mas para POPI, quando chegamos à linha 56, uma leitura de memória para a palavra errada já foi iniciada. Dependendo dos detalhes exatos do *hardware*, pode ou não ser possível abortar uma leitura de memória no meio sem causar danos. Se não pudermos abortá-la, vamos apenas esperar seu término e começar a leitura correta. Neste exemplo, POPI gasta 15 microinstruções, em vez de 13, como na Fig. 4.16. Mesmo assim, se o uso de sobreposição reduziu o tempo básico de microinstrução de até mesmo 15%, POPI será mais rápido agora do que antes.

```
11: alu := tir;
12: mar := ir; rd; if n then goto 16;

13: rd;
14: ac := mbr + ac; goto 0;

16: ac := ac + 1; rd;
17: a := inv(mbr);
18: ac := ac + a; goto 0;
```

(a)

```
52: alu := tir;
53: mar := ac; rd; if n then goto 56;
54: sp := sp + (-1); rd;
55: mar := sp; wr; goto 10;

56: rd;
57: mar := sp; sp := sp + 1; rd;
58: rd;
59: mar := ac; wr; goto 10;
```

(b)

Fig. 4.26 Dois exemplos envolvendo busca e execução sobrepostas de microinstruções.

A situação se torna muito mais complicada se desvios multidirecionais, que são formados pelo OR de NZVC com os bits de baixa ordem de ADDR, são utilizados em vez de testes de bits individuais. Além disso, em muitas máquinas a sobreposição de busca e execução de microinstruções é feita de tal maneira que, no início da execução da microinstrução n, a escolha da microinstrução $n + 1$ já foi feita, utilizando os bits de *status* da ALU previamente armazenados durante a microinstrução $n - 1$. Não vamos persistir mais nesse assunto aqui, mas pelo menos a motivação da definição de Rosin para microprogramação deve estar clara agora.

4.5.5 Memória *Cache*

Historicamente, as CPUs sempre foram mais rápidas que as memórias. Enquanto as memórias melhoraram, as CPUs também o fizeram, preservando assim a diferença. O que isso significa na prática é que depois que a CPU faz uma requisição à memória, ela deve permanecer inativa por um tempo substancial enquanto estiver esperando que a memória responda. Como vimos, é comum que a CPU estabeleça uma leitura à memória durante um ciclo de barramento e não obtenha o dado até dois ou três ciclos mais tarde, mesmo que não haja estados de espera.

Realmente, o problema não é tecnológico, mas econômico. Os engenheiros sabem como construir memórias tão rápidas quanto as CPUs, mas elas são tão caras que equipar um computador com um megabyte ou mais está fora de questão (exceto, talvez, para supercomputadores, onde o céu é o limite e preço não é objeção). Assim, a escolha cai para ter uma pequena quantidade de memória rápida ou uma grande quantidade de memória lenta. O que iríamos preferir é uma grande quantidade de memória rápida a um baixo preço.

São conhecidas técnicas bastante interessantes para combinar uma pequena quantidade de memória rápida com uma grande quantidade de memória lenta para obter (quase) a velocidade da memória rápida e a capacidade da grande memória a um preço moderado. A memória pequena e rápida é chamada ***cache*** (do francês *cacher*, que significa esconder) e está sob o controle do microprograma. Vamos descrever abaixo como *caches* são usados e como eles funcionam. Mais informações podem ser encontradas em Agarwal *et al.*, 1989; Farrens e Pleszkun, 1989; Kabakibo *et al.*, 1987; Kessler *et al.*, 1989; Pohm e Agarwal, 1983; Przybylski *et al.*, 1989; Smith, 1982; e Wang *et al.*, 1989.

Durante anos, as pessoas souberam que os programas não acessam suas memórias de maneira completamente aleatória. Se uma dada referência à memória é para o endereço A, é possível que a próxima referência à memória seja feita nas vizinhanças de A. Um exemplo simples é o próprio programa. Exceto pelos desvios e chamadas de procedimentos, as instruções são buscadas de posições consecutivas da memória. Além disso, a maior parte do tempo de execução de um programa é passada em *loops*, nos quais um número limitado de instruções é executado várias vezes. Similarmente, um programa de manipulação de matrizes provavelmente fará muitas referências à mesma matriz antes de tratar de outra coisa qualquer.

A observação de que referências à memória feitas em qualquer intervalo curto de tempo tendem a usar apenas uma pequena fração da memória total é chamado **princípio da localidade** e forma a base de todos os sistemas de *cache*. A idéia geral é que, quando uma palavra é referenciada, ela é trazida da grande memória lenta para o *cache*, de modo que, da próxima vez que for utilizada, ela pode ser acessada rapidamente. Uma organização comum da CPU, *cache* e memória principal está ilustrada na Fig. 4.27. Se uma palavra é lida ou escrita k vezes num intervalo curto, o computador precisará de 1 referência à memória lenta e $k - 1$ referências à memória rápida. Quanto maior for k, melhor o desempenho geral.

Podemos formalizar este cálculo introduzindo c, o tempo de acesso ao *cache*; m, o tempo de acesso à memória principal; e h, a **taxa de acerto** (*hit ratio*), que é a fração de todas as referências que podem ser satisfeitas pelo *cache*. Em nosso pequeno exemplo do parágrafo anterior, $h = (k - 1)/k$. Alguns autores também definem a **taxa de falha** (*miss ratio*), que é $1 - h$.

Com estas definições, podemos calcular o tempo médio de acesso como se segue:

$$\text{tempo médio de acesso} = c + (1 - h)m$$

À medida que $h \rightarrow 1$, todas as referências podem ser satisfeitas pelo *cache*, e o tempo de acesso se aproxima de c. Por outro lado, à medida que $h \rightarrow 0$, uma referência à memória é necessária

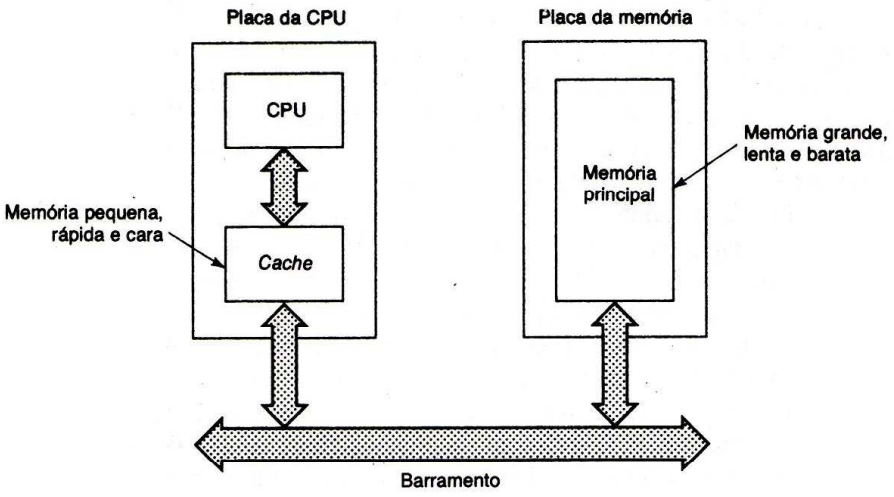


Fig. 4.27 O *cache* é usualmente localizado na placa de CPU.

toda vez, de modo que o tempo de acesso se aproxima de $c + m$, primeiro um tempo c para verificar o *cache* (sem sucesso), e então um tempo m para fazer a referência à memória. Em alguns sistemas, a referência à memória pode ser iniciada em paralelo com a procura no *cache*, de modo que, se ocorrer falha no *cache*, o ciclo de memória já começou. Entretanto, esta estratégia requer que a memória possa ser interrompida em seu trajeto se o *cache* contiver o dado, tornando isso mais complicado. O algoritmo básico para procurar coisas no *cache* e começar (ou interromper) a referência à memória principal dependendo do resultado da busca no *cache* é tratado pelo microprograma.

Duas organizações de *cache* fundamentalmente diferentes são usadas, junto com uma terceira forma híbrida das duas primeiras. Para todos os três tipos, a memória, assumindo ser de 2^m bytes, é (conceitualmente) dividida em blocos consecutivos de b bytes, para um total de $2^m/b$ blocos. Cada bloco tem um endereço que é um múltiplo de b. O tamanho do bloco, b, é normalmente uma potência de dois.

O primeiro tipo de *cache* é o **cache associativo,** um exemplo do qual está mostrado na Fig. 4.28. Ele consiste em um número de posições (**slots**) ou linhas, cada uma contendo um bloco e seu número de bloco, junto com um bit (*Válido*) dizendo se aquele *slot* está atualmente em uso ou não. O exemplo da Fig. 4.28 ilustra um *cache* com 1024 *slots* e uma memória com 2^{24} bytes divididos em 2^{22} blocos de 4 bytes. Num *cache* associativo, a ordem das entradas é aleatória.

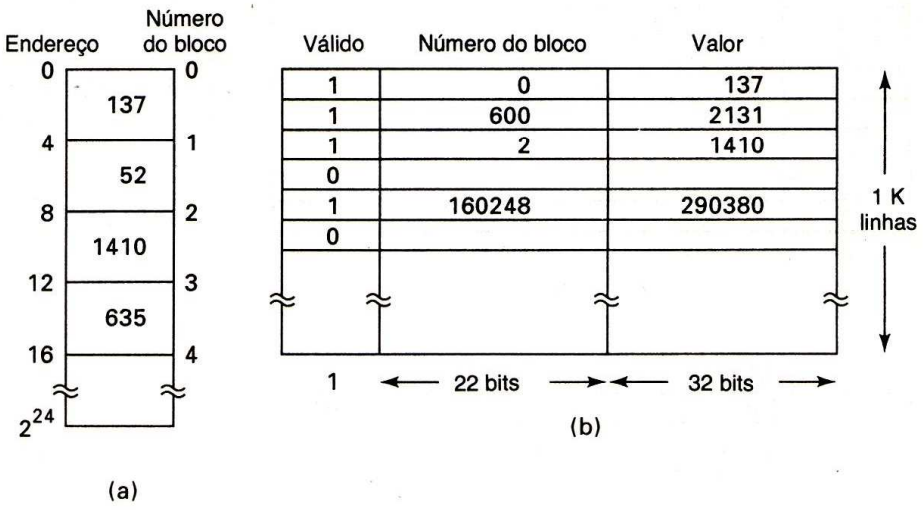


Fig. 4.28 Um exemplo de esquema de *cache*. (a) Memória com blocos de 4 bytes. (b) *Cache* associativo com 1024 linhas.

Quando o computador é inicializado, todos os bits *Válido* são colocados em 0, para indicar que nenhuma entrada do *cache* é válida. Suponha que a primeira instrução referencie a palavra de 32 bits do endereço 0. O microprograma vai verificar todas as entradas do *cache,* procurando por uma válida que contenha o bloco número 0. Não achando, ele vai então fazer uma requisição ao barramento para buscar a palavra 0 da memória e fazer uma entrada válida para o bloco número 0, contendo o conteúdo da palavra 0. Se esta palavra for necessária novamente, da próxima vez ela será pega do *cache,* eliminando a necessidade de uma operação de barramento.

À medida que o tempo passa, mais e mais entradas de *cache* serão marcadas como válidas. Se o programa usar menos que 1024 palavras de programa e dados, no final todo o programa e seus dados aparecerão no *cache,* e ele será executado a alta velocidade, sem fazer qualquer referência à memória através do barramento. Se mais do que 1024 palavras são necessárias, em algum ponto o *cache* estará cheio e uma entrada velha terá que ser descartada para deixar lugar para uma nova. Na prática, a decisão de qual deve ser jogada fora precisa ser feita muito rapidamente (nanossegundos). O VAX e muitas outras máquinas selecionam um *slot* aleatoriamente. Outros algoritmos possíveis são discutidos no capítulo 6, sob o título de memória virtual, onde o mesmo problema ocorre.

O aspecto do *cache* associativo que o distingue dos outros tipos de *caches* é que cada *slot* contém um número de bloco e sua entrada. Quando aparece um endereço de memória, o microprograma deve calcular o número do bloco relevante (fácil) e então procurar aquele número de bloco no *cache* (difícil). Para evitar uma pesquisa linear, o *cache* associativo tem um *hardware* especial que pode comparar todas as entradas com o número de bloco dado simultaneamente, em vez de um *loop* do microprograma. Este *hardware* torna o *cache* associativo caro.

Para reduzir o custo, um tipo diferente de *cache* foi inventado, o ***cache* com mapeamento direto.** Este tipo de *cache* evita a pesquisa colocando cada bloco num *slot* cujo número pode ser facilmente calculado a partir do número do bloco. Por exemplo, o número do *slot* pode ser o resto da divisão do número do bloco pelo número de *slots.* Com blocos de 4 bytes (isto é, 1 palavra) e 1024 slots, por exemplo, o número do *slot* para a palavra no endereço *A* é (*A*/4) *módulo* 1024. No exemplo da Fig. 4.29, vemos que as palavras em 0, 4096, 8192 e assim por diante são mapeadas no *slot* 0; as palavras em 4, 4100, 8196 e assim por diante são mapeadas no *slot* 1; e assim sucessivamente.

Enquanto que o *cache* com mapeamento direto elimina o problema de pesquisa, ele cria um novo problema — como dizer qual das muitas palavras que mapeiam num dado *slot* está atualmente ocupando aquele *slot.* De fato, criamos 1024 classes de equivalência, baseadas nos

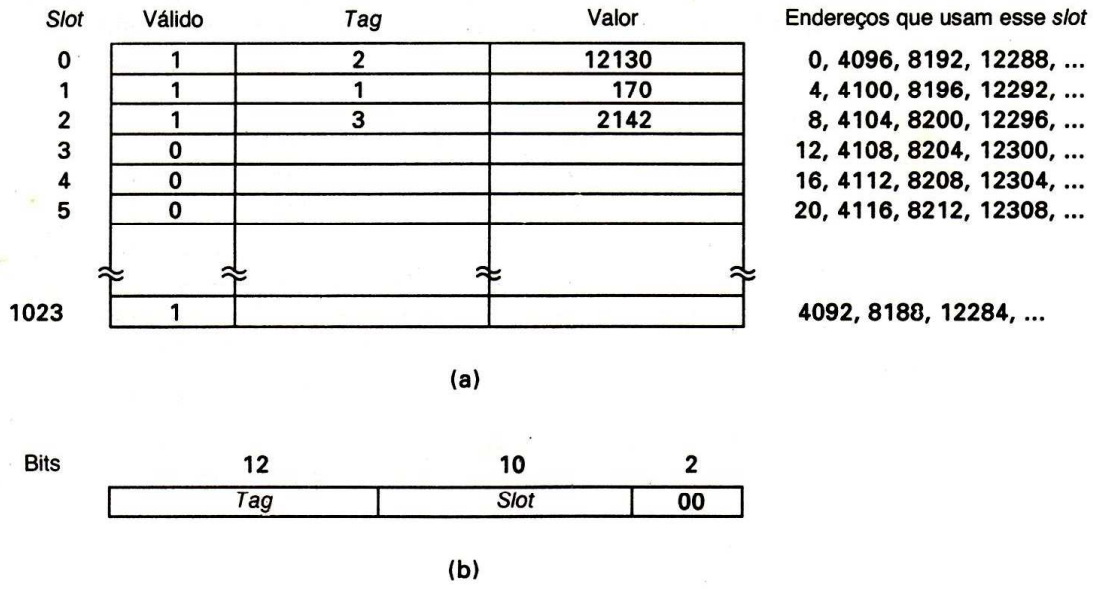


Fig. 4.29 (a) Um *cache* de mapeamento direto com 1024 *slots* de 4 bytes cada. (b) Cálculo do *slot* e *tag* a partir de um endereço de 24 bits.

números de bloco *módulo* o tamanho do *cache*. Neste exemplo, o *slot* 0 poderia conter qualquer uma das palavras 0, 4096, 8192 etc. A maneira de dizer qual está atualmente no *slot* é colocar parte do endereço no *cache*, no campo *Tag*. Este campo contém aquela parte do endereço que não pode ser calculada pelo número do *slot*.

Para tornar este ponto mais claro, considere uma instrução no endereço 8192 que move a palavra em 4100 para 12296. O número do bloco correspondente a 8192 é calculado dividindo-o por 4 (nosso tamanho de bloco exemplo) para obter 2048. Depois, o número do *slot* é calculado pegando 2048 módulo 1024, que é o mesmo que usar os 10 bits de baixa ordem de 2048. O número do *slot* é 0. Os 12 bits de cima contêm um 2, assim ele é o *Tag*. A Fig. 4.29(a) mostra o *cache* depois que todos os três endereços foram processados.

A Fig. 4.29(b) mostra como o endereço é dividido. Os dois bits de baixa ordem são sempre 0 (uma vez que o *cache* funciona com blocos inteiros, e estes são múltiplos do tamanho do bloco, 4 bytes neste exemplo). Depois vem o número do *slot* (10 bits) e finalmente o *Tag* (12 bits). É imediato construir o *hardware* que extrai diretamente o número do *slot* e *Tag* de qualquer endereço de memória.

O fato de que blocos múltiplos mapeiam no mesmo *slot* de *cache* pode causar problemas. Suponha que nossa instrução de movimentação mova 4100 para 12292, em vez de 12296. Esses dois endereços mapeiam no *slot* 1. Dependendo dos detalhes do microprograma, aquele que foi calculado por último terminaria no *cache*, com o outro expurgado. Em si mesmo, isso não é um desastre, mas degrada o desempenho do *cache* se acontecer que muitas palavras que estiverem sendo usadas mapeiem no mesmo *slot*. O objetivo, afinal de contas, é melhorar o desempenho.

A maneira de sair desta dificuldade é expandir o *cache* de mapeamento direto para ter mais do que uma entrada por *slot*. O PDP-11/70, por exemplo, tem duas entradas por *slot*. Um *cache* de mapeamento direto com múltiplas entradas por *slot* é chamado de ***cache* associativo por conjunto** e é ilustrado na Fig. 4.30.

Tanto o *cache* associativo quanto o de mapeamento direto são, de fato, casos especiais do *cache* associativo por conjunto. Se reduzirmos o número de *slots* para 1, todas as entradas estarão no mesmo *slot*, e devemos distingui-las inteiramente por seus *Tags*, uma vez que todas mapeiam no mesmo endereço. Esse caso é apenas um *cache* associativo. Se $n = 1$, voltamos ao *cache* de mapeamento direto puro, com uma entrada por *slot*.

Os *caches* associativos e de mapeamento direto têm diferentes vantagens e desvantagens. O *cache* de mapeamento direto é mais simples, mais barato para construir e tem um tempo de acesso mais rápido, uma vez que o *slot* apropriado pode ser encontrado simplesmente indexando o *cache* utilizando uma parte do endereço como índice. De outro lado, o *cache* associativo tem uma taxa de acerto maior para qualquer dado número de *slots*, pois nunca há conflitos. Não pode acontecer que k palavras importantes não possam ser colocadas no *cache* simultaneamente, porque elas têm a má sorte de mapear no mesmo *slot* do *cache*. Quando qualquer computador real está sendo projetado, uma simulação extensiva do *cache* sempre é feita para ver quanto de desempenho se obtém, e a que preço.

Além de determinar o número de *slots*, os projetistas devem também escolher um tamanho

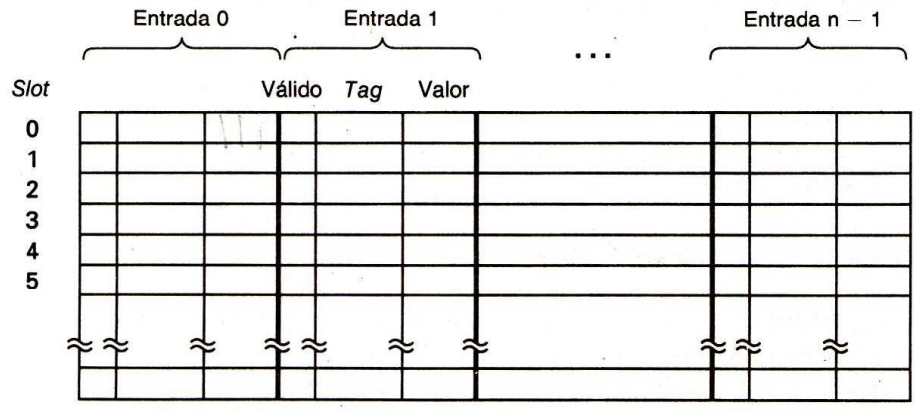


Fig. 4.30 Um *cache* associativo por conjunto com n entradas por *slot*.

de bloco. Nós temos usado uma palavra de 32 bits em nossos exemplos (para simplicidade), mas 2, 4, 8 ou mais palavras são também possíveis, e freqüentemente usadas. Uma vantagem de usar um tamanho de bloco maior é que existe menos *overhead* na busca de um bloco de oito palavras do que na busca de oito blocos de uma palavra, especialmente se o barramento permite transferência de blocos. Uma desvantagem é que nem todas as palavras podem ser necessárias, de modo que algumas das buscas podem ser desperdiçadas.

Outra importante consideração do projeto de *cache* é como manipular escritas. Duas estratégias são comumente usadas. Na primeira, chamada ***write through***, quando uma palavra é escrita no *cache*, ela é imediatamente escrita de volta à memória. Esta abordagem garante que as entradas do *cache* são sempre o mesmo que as entradas correspondentes na memória.

O outro sistema de escrita, chamado ***copy back***, não atualiza a memória quando o *cache* é alterado. Em vez disso, a memória somente é atualizada quando a entrada é expurgada do *cache* para permitir que outra entrada tome conta do *slot*. Quando *copy back* é usado, um bit é necessário em cada entrada do *cache*, dizendo se a entrada foi modificada desde que foi carregada para o *cache*.

Como todos os outros aspectos do projeto de *cache*, existem compromissos aqui. *Write through* obviamente causa mais tráfego de barramento que *copy back*. De outro lado, se a CPU começa uma transferência de E/S da memória para disco, e a memória não está correta (porque as palavras modificadas recentemente ainda não foram copiadas de volta à memória a partir do *cache*), dados incorretos serão escritos no disco. Este problema pode ser contornado, mas acrescenta complexidade ao sistema.

Se a razão de leituras para escritas for muito alta, pode ser mais simples usar *write through* e apenas aceitar o tráfego de barramento. Entretanto, se houver muitas escritas, pode ser melhor usar *copy back* e fazer com que o microprograma explicitamente expurgue o *cache* inteiro antes de começar uma operação de E/S.

Outra consideração de projeto a respeito de escritas é o que fazer se uma escrita causar uma falha no *cache*. Uma estratégia é buscar a palavra para o *cache*, e então atualizá-la, escrevendo-a de volta se *write through* estiver sendo utilizado. Uma estratégia diferente é escrever na memória diretamente, mas não buscar uma palavra para o *cache*, exceto nas leituras. Esta consideração é chamada **política de alocação de escrita.** Os dois métodos descritos aqui são utilizados.

À medida que a tecnologia de circuitos integrados evolui, está-se tornando possível colocar *caches* muito rápidos diretamente na pastilha da CPU. Estes *caches* são pequenos, devido à falta de área real na pastilha, de modo que é desejável ter *caches* de dois níveis, um na pastilha da CPU e um segundo na placa da CPU. Se uma palavra não puder ser encontrada no *cache* da pastilha, uma requisição é feita ao *cache* ao nível da placa. Se isso também falhar, a memória principal é usada.

Tendo considerado todas as coisas, o uso de *caches* é uma técnica importante para melhorar o desempenho do sistema. Quase todos os computadores de média e larga escala usam alguma forma de *cache*. Informações adicionais sobre *caches* podem ser encontradas em Hill, 1988; Przybylski *et al.*, 1988; Short e Levy, 1988; e Smith, 1986, 1987.

4.6. EXEMPLOS DO NÍVEL DE MICROPROGRAMAÇÃO

Nesta seção, vamos olhar o nível de microprogramação de nossos dois exemplos correntes, as famílias Intel e Motorola. Isso será, por necessidade, curto, porque máquinas reais são altamente complicadas. Em ambos os casos, examinaremos um dos membros da família mais simples, para evitar detalhes desnecessários.

4.6.1 A Microarquitetura do Intel 8088

A microarquitetura de toda CPU Intel é similar, porque todas elas evoluíram da primeira, o 8086 (McKevitt e Bayliss, 1979). O 8088 é virtualmente a mesma máquina que o 8086, exceto que contém uma interface de barramento de 8 bits em vez de uma interface de 16 bits. As vias de dados, a ALU e os registradores de rascunho são idênticos. Similarmente, o 80286 também não sofreu alterações em termos das vias de dados principais, porque, como seus antecessores,

ele também é internamente uma máquina de 16 bits. Entretanto, o 80286 consiste em quatro unidades funcionais independentes, como mostra a Fig. 3.40. O 80386 tem vias de dados de 32 bits e oito unidades funcionais, e é de alguma forma diferente dos outros.

Iremos nos concentrar no 8088 como o membro mais simples da família, mas os outros projetos são similares no espírito, mesmo diferindo em alguns dos detalhes. O 8088 usa uma mistura de microcódigo e lógica aleatória (circuitos de *hardware* especializados) para prover boa funcionalidade e, ao mesmo tempo, minimizar o tamanho do microcódigo. As microinstruções são verticais no formato, com campos multibits que especificam funções gerais, em vez de bits unitários que controlam portas individuais nas vias de dados.

O coração do 8088 são as vias de dados principais, ilustradas de forma simplificada na Fig. 4.31. As vias de dados têm duas porções, a parte inferior, na metade de baixo da figura, e a parte superior, na metade superior. As duas partes funcionam independentemente, em paralelo.

A parte inferior é comparável às vias de dados da maioria dos outros computadores, incluindo nossos exemplos. Ela contém uma ALU multifunção, que pega duas entradas, calcula alguma função simples com elas e produz uma saída. As entradas vêm de três registradores de 16 bits, TMPA, TMPB e TMPC, que são carregados antes do ciclo da ALU. A saída vai para o barramento da ALU, de onde ela pode ser roteada de volta a qualquer um dos três registradores de entrada, ou outro lugar nas vias de dados. A ALU pode realizar tanto operações de 8 bits quanto de 16 bits. Os códigos de condição resultantes de uma operação da ALU podem ser armazenados no PSW por controle do microprograma.

A parte inferior das vias de dados também contêm uma memória de rascunho de oito palavras, para AX, BX, CX, DX, SI, DI, BP e SP. Um registrador pode ser copiado da memória de rascunho para o barramento da ALU, e daí para um dos registradores temporários, onde ele pode ser utilizado como uma entrada da ALU. A saída da ALU pode ser roteada diretamente para a memória de rascunho, sem passar através de um registrador TMP.

O bloco marcado "cruzamento" entre a memória de rascunho e o barramento da ALU tem a capacidade de trocar bytes enquanto realiza cópias em qualquer direção. Esta característica foi necessária para acessar os registradores de 8 bits da memória de rascunho. Por exemplo, para somar AL e BH, AX pode ser copiado para TMPA. Depois, BX pode ser copiado para TMPB, trocando os bytes no processo, de modo que BH ocupe o byte de baixa ordem de TMPB e BL, o byte de alta ordem. Uma adição de 8 bits soma então AX e BH, cujo resultado pode ser então roteado de volta a AL ou BH, dependendo da instrução.

A parte superior das vias de dados trata de cálculo de endereços. Lembre-se de que, no 8088, um endereço de memória físico é formado pela adição de um endereço de 16 bits com o registrador de segmento apropriado para formar um endereço físico de 20 bits, que sai para o barramento de endereços. A parte superior contém sua própria memória de rascunho para registradores, para armazenar os quatro registradores de segmento, o contador de programa e mais dois registradores. Ela também guarda a fila de busca antecipada (*prefetch queue,* descrita abaixo).

A parte superior contém um somador que é usado para combinar distâncias (*offsets*) de 16 bits e registradores de segmento. Ela é ligada por fios de tal modo que os 4 bits de baixa ordem do *offset* vão diretamente para o barramento de endereços, sem passar pelo somador, enquanto os bits 4 a 15 do *offset* são somados aos bits 0 a 11 do registrador de segmento para formar os 16 bits superiores do barramento de endereço. Uma pequena ROM é provida para constantes comuns, por exemplo, para somar 1 ao contador de programa depois que um byte foi buscado da memória.

O 8088 tem *pipeline*. Quatro unidades estão presentes, para buscar instruções com antecedência, decodificar instruções, fazer os cálculos de endereço e executar as instruções. Os três últimos são relativamente convencionais, mas a unidade de *prefetch* é algo não-usual. Ela funciona completamente independente do resto do processador. Sempre que o barramento está inativo, a unidade de *prefetch* pede uma requisição à memória para ler o próximo byte da seqüência de instruções. Os bytes lidos são armazenados em uma fila, na memória de rascunho superior. Quando um novo byte de instrução é necessário, ele é pego da fila.

O tamanho da fila é um parâmetro de projeto interessante. Se for muito pequeno, a CPU irá freqüentemente parar, esperando por um byte da memória. Entretanto, ser muito grande também não é bom. A unidade de *prefetch* não sabe o que os bytes que ela busca significam;

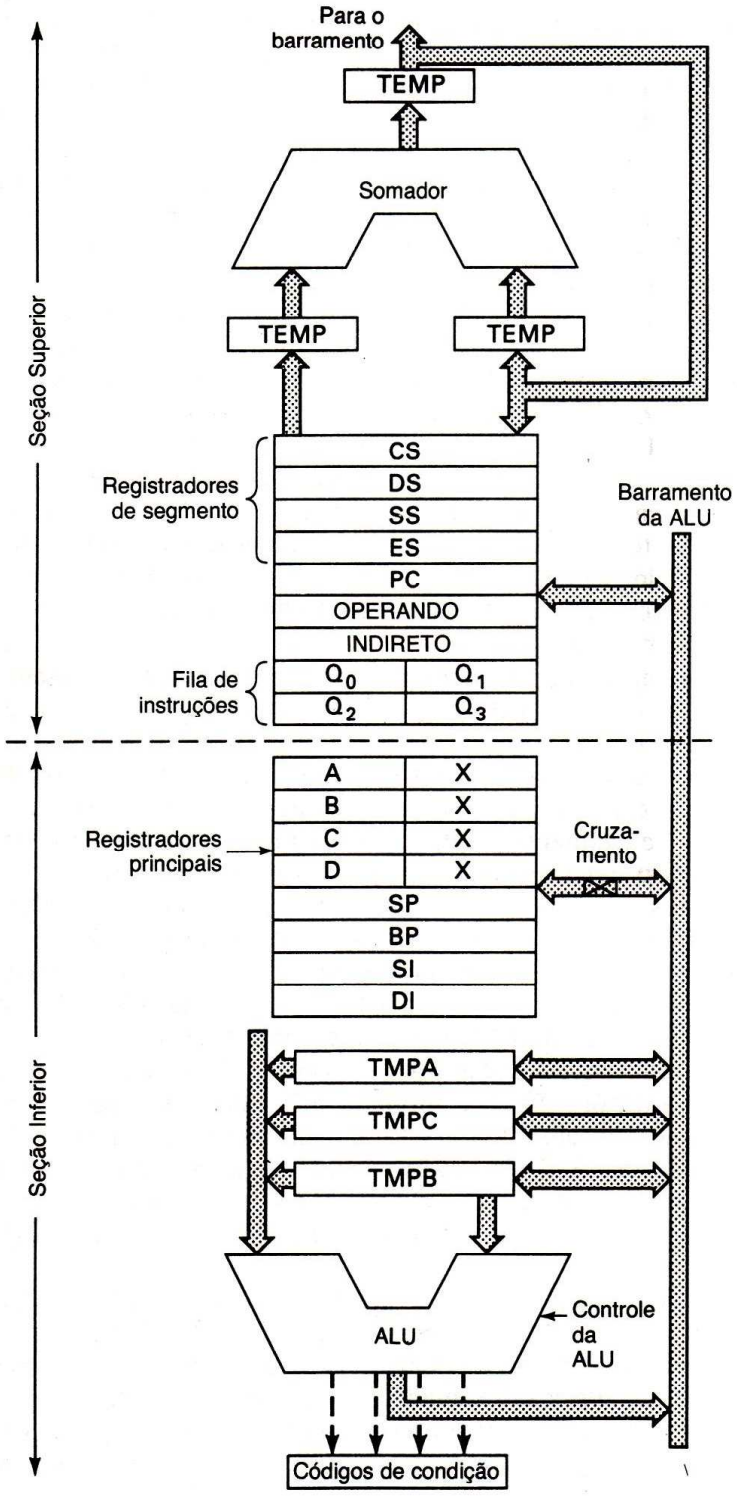


Fig. 4.31 Um diagrama altamente simplificado das vias de dados do 8088.

ela apenas busca o próximo, desde que haja espaço na fila. Em particular, depois de um desvio, mesmo um incondicional, ela continua a buscar bytes que não serão utilizados. Se a fila for muito grande, a unidade de *prefetch* vai desperdiçar substancial passagem pelo barramento buscando bytes depois de instruções de desvio. O 8086 tinha uma fila de 6 bytes, mas isso foi reduzido para 4 bytes no 8088. Além da largura do barramento, esta é a única diferença entre o 8088 e o 8086.

Além das vias de dados, o 8088 contém uma seção de controle que armazena o microprograma e controla as vias de dados (veja a Fig. 4.32). Quando uma nova instrução do nível de máquina convencional é iniciada, seu byte de *opcode* é tirado da fila de instruções e carregado no IR (*Instruction Register,* ou registrador de instruções).

A unidade de *hardware* denominada **decodificador de grupo** na Fig. 4.32 extrai informação de IR e a espalha pela máquina. Os registradores M e N pegam campos que são usados no cálculo dos endereços dos operandos fonte e destino. O registrador X guarda a informação de *opcode*, que é utilizada para dizer à ALU que função realizar. O decodificador de grupo também extrai o bit de byte/palavra do *opcode*. Este bit controla se as operações da ALU devem ser de largura de 8 ou 16 bits, e se as transferências de e para a memória de rascunhos são de 8 ou 16 bits.

As microinstruções têm largura de 21 bits. Um total de 504 delas é necessário, e elas são armazenadas em uma ROM de 504 × 21 na seção de controle. Quando uma instrução de máquina é buscada, uma PLA converte o *opcode* no endereço inicial do microcódigo que trata aquela instrução. Diferente de nossos exemplos, nos quais as instruções são inspecionadas bit a bit, nenhuma decodificação por *software* está presente. Ela é completamente realizada por *hardware*, para economizar tempo.

O microcódigo é dividido em ***bursts*** (seqüências) de até 16 microinstruções, cada *burst* manipulando uma ou mais instruções de máquina ou provendo um procedimento de serviços gerais, tal como cálculo de endereços. Dois tipos de microdesvios estão presentes: os curtos, para desviar dentro do *burst* corrente (endereço de 4 bits), e os longos, para desviar para qualquer lugar dentro do microprograma. Chamadas de microprocedimentos também existem e são similares a desvios longos, exceto que eles depositam seu endereço de retorno no registrador SR (*Subroutine Return,* ou retorno de sub-rotina). Microprocedimentos não podem ser aninhados. Ao todo, a ROM de 504 palavras contém cerca de 90 *bursts,* com uma média de 5 ou 6 palavras cada.

A estratégia geral para interpretar muitas das instruções aritméticas e lógicas é como se segue. Primeiro, um microprocedimento é chamado para realizar o cálculo do endereço. Este código usa os registradores M e N, que já contêm os campos de registradores fonte e destino,

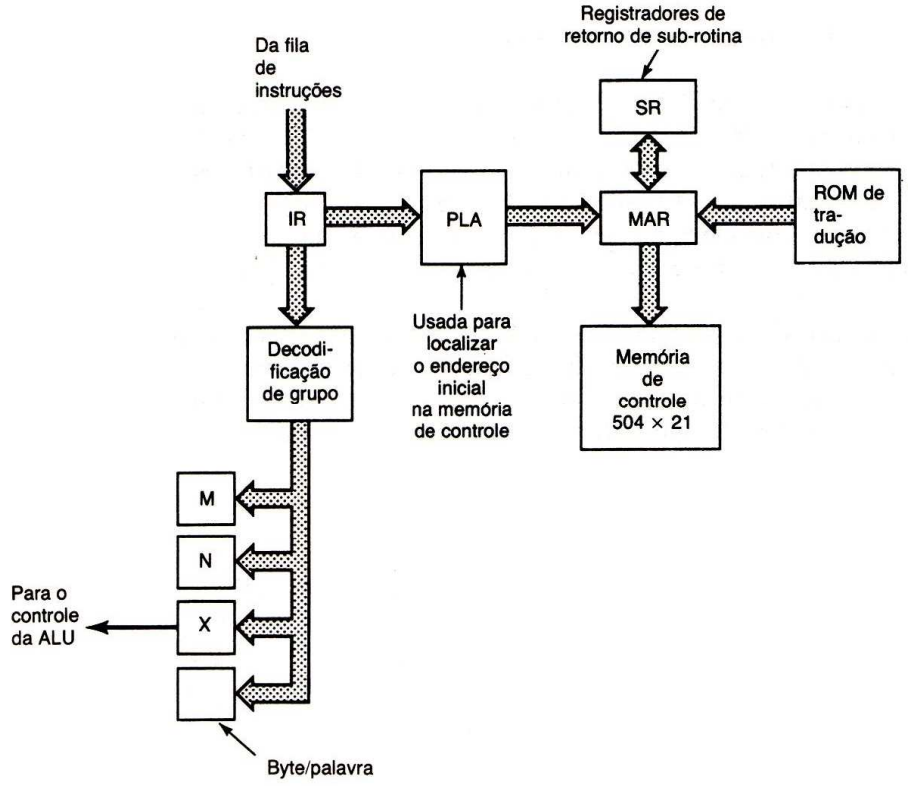


Fig. 4.32 Diagrama altamente simplificado da seção de controle do 8088.

quando apropriado. Quando o microprocedimento de cálculo de endereço retorna, os operandos estão disponíveis em posições fixas, e o registrador X contém o código da ALU para esta instrução de máquina (colocado lá pelo *hardware*). O procedimento principal agora executa um ciclo das vias de dados, com os operandos e o código de função da ALU vindos dos registradores. Ele não precisa nem mesmo saber que operação ele está realizando. Desta maneira, ADD, SUB, AND, OR e muitas outras instruções podem compartilhar o mesmo microcódigo, apesar de realizarem diferentes operações. Naturalmente, PUSH, CALL e JMP são completamente diferentes, e cada um requer seu próprio microcódigo.

Uma microinstrução típica do 8088 é mostrada na Fig. 4.33(a). Ela tem duas partes, que são executadas em paralelo. A parte da esquerda, consistindo em dois campos de 5 bits, permite que toda microinstrução realize uma transferência de registrador para registrador. Os 32 registradores que podem ser utilizados como fonte e destino são os registradores de 8 e 16 bits nas duas memórias de rascunho e os registradores temporários.

A parte da direita é uma microinstrução de 11 bits codificada verticalmente, consistindo nos campos de tipo, código da ALU, registrador e código de condição. Seis tipos de microinstrução estão presentes, como listado na Fig. 4.33(b). Eles são distinguidos pelo campo TIPO. Todas as microinstruções têm 10 bits para mover o conteúdo de um registrador fonte para um registrador destino, mas o formato dos 11 bits inferiores varia ligeiramente de tipo para tipo.

O código da ALU pode ordenar que uma função específica seja realizada, ou instruir a ALU para usar o código do registrador X. O campo de 3 bits de registrador fornece o operando, e o bit final diz se os códigos de condição devem ou não ser estabelecidos. Cada microinstrução é executada em um único ciclo de relógio, de modo que não é possível mover um valor da memória de rascunho para um registrador TMP, e então usar aquele registrador como um operando da ALU no mesmo ciclo.

Desvios longos e chamados de microprocedimentos criam um problema, na medida em que eles requerem mais bits para designar o destino do que está disponível na microinstrução. A solução é usar uma **ROM de tradução**, que mapeia endereços de 5 bits em endereços completos do microprograma. Desvios longos e chamadas usam endereços de 5 bits para indicar que destino eles querem. É claro que esta estratégia significa que apenas 32 endereços podem ser o destino de um desvio ou chamada, mas isso é suficiente.

4.6.2 A Microarquitetura do Motorola 68000

Como um segundo exemplo de microarquitetura de uma máquina real, vamos dar uma rápida olhada no Motorola 68000 (Stritter e Tredennick, 1978). O 68000 é uma pastilha ligeiramente maior que o 8088, contendo espaço pòtencial para 68000 transistores (daí o seu nome), embora apenas cerca de 40000 sejam utilizados, sendo o resto do espaço ocupado por fios e por aí vai.

Mencionamos a consideração de se o 68000 é uma máquina de 16 ou 32 bits antes. Esta questão aparece novamente na microarquitetura. As vias de dados principais são mostradas na Fig. 4.34.

Como pode ser visto, as "vias de dados" são, de fato, três vias de dados de largura de 16 bits separadas, cada uma das quais podendo operar independentemente das outras, e em paralelo. A parte da esquerda manipula os 16 bits superiores do cálculo de endereço. A parte do meio toma conta dos 16 bits inferiores do cálculo de endereço. A parte da direita realiza operações nos dados.

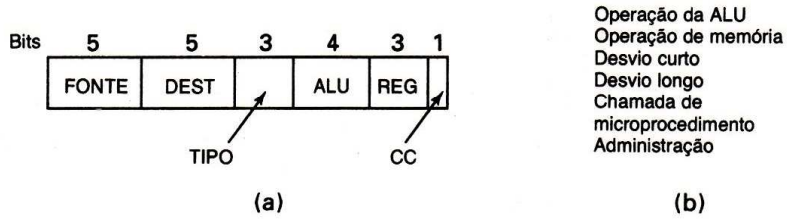


Fig. 4.33 (a) Formato da microinstrução da ALU do 8088. (b) Tipos de microinstrução.

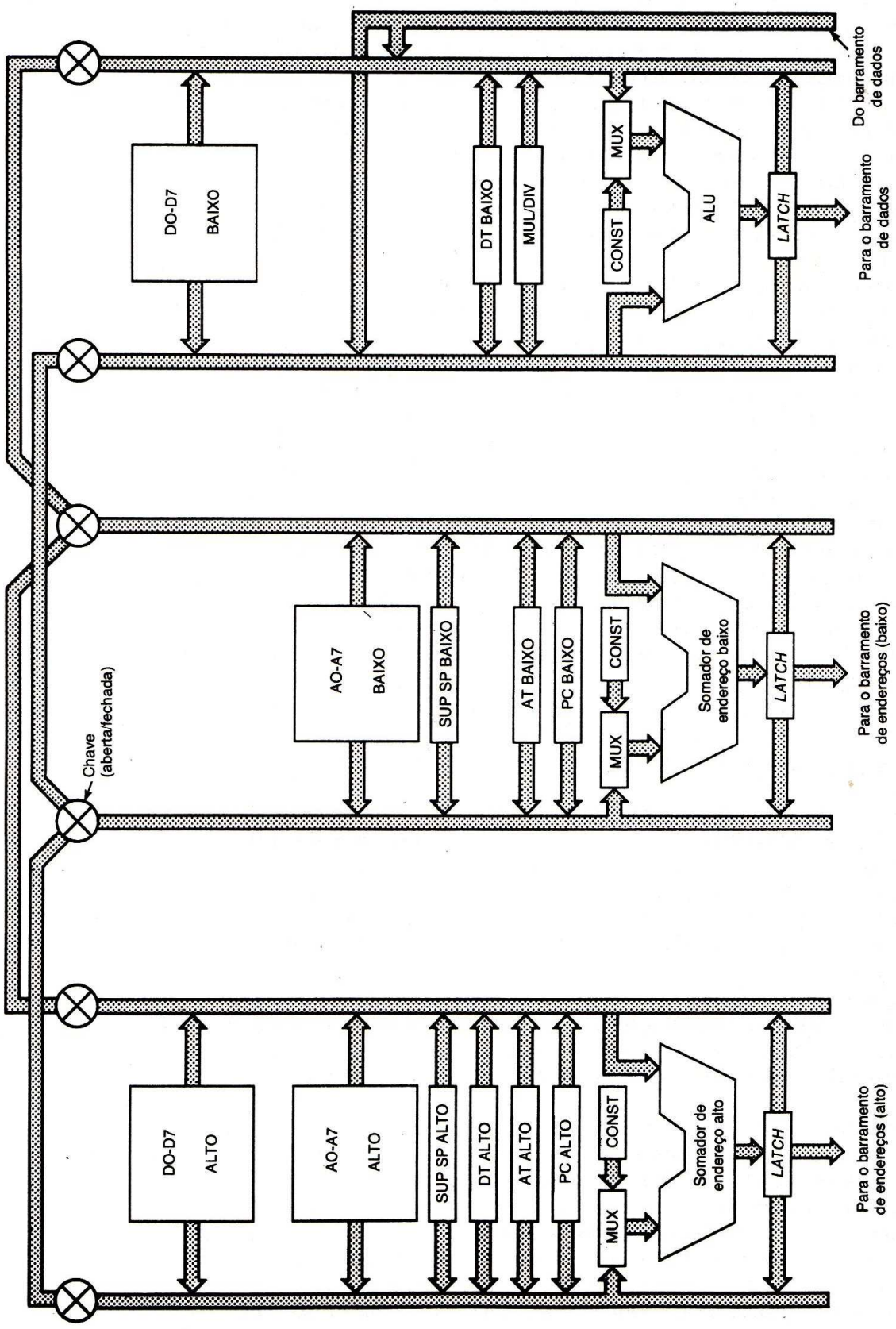


Fig. 4.34 Um diagrama altamente simplificado das vias de dados do 68000.

Cada parte tem dois barramentos que pegam dados de registradores e os alimentam no somador ou ALU. Eles também servem para trazer a saída da ALU de volta aos registradores. No topo de cada um dos seis barramentos existe uma chave, que pode aberta ou fechada por *software*. Abrindo a chave, os barramentos podem ser eletricamente conectados. Se todas as seis chaves estiverem abertas, por exemplo, os registradores da parte esquerda podem ser usados como operandos para a ALU, na parte direita.

Vamos dar uma olhada nos registradores, começando na esquerda. O bloco marcado D0-D7 ALTO contém os 16 bits superiores dos oito registradores de dados. O bloco debaixo dele contém os 16 bits superiores para os oito registradores de endereço. Depois, vem o apontador da pilha do modo supervisor, dois registradores de rascunho, DT (*Data Temporary*, ou temporário de dados) e AT (*Address Temporary*, ou temporário de endereços), e a metade superior do contador de programa. Todos esses 20 registradores podem ser colocados tanto no barramento da esquerda como no da direita, determinado pelo microprograma.

Ambos os barramentos levam a um somador de 16 bits. Este somador não é uma ALU completa, e pode apenas somar. Uma vez que ele é usado principalmente para cálculo de endereço, ele não precisa da capacidade de fazer qualquer outra coisa, e a construção de uma ALU completa teria custado mais área da pastilha. Note que a entrada esquerda do somador é um multiplexador que pode selecionar tanto o barramento da esquerda quanto a parte superior de uma ROM de constantes, novamente sob controle do microprograma.

A saída da ALU é armazenada em um *latch* e pode ser colocada em qualquer barramento, de modo que os resultados podem ser armazenados em registradores. Além disso, o *latch* pode fazer com que o barramento de endereço externo dê saída à metade superior de um endereço de memória de 32 bits.

A parte do meio é essencialmente o mesmo, exceto que ela não contém a memória de rascunho de dados nem o registrador DT. Naturalmente, sua ROM de constantes guarda os 16 bits de baixa ordem das constantes, e não os bits de alta ordem. A saída de seu *latch* pode acionar o endereço do barramento de baixa ordem. Ambas as partes podem ser habilitadas simultaneamente, de modo que um endereço completo de 32 bits pode ser colocado no barramento de memória de uma só vez.

A parte da direita é similar às outras duas, contendo a parte de baixa ordem da memória de rascunho de dados e a parte de baixa ordem de DT. Ela também tem uma ROM de constantes, embora esta esteja no outro barramento e contenha constantes diferentes. Ainda assim, a idéia é similar.

Três novas características aparecem aqui. Primeiro, o somador foi substituído por uma ALU de função completa, destinada a realizar funções aritméticas e booleanas nos registradores D. Como a ALU tem largura de apenas 16 bits, operações de 32 bits requerem dois ciclos de vias de dados. Em segundo lugar, um registrador de rascunho extra está presente. Ele é necessário para multiplicação e divisão. Em terceiro lugar, não apenas é possível enviar saída para o barramento (de dados), mas, diferente das outras duas partes, é também possível aceitar dados que estão chegando.

O que não é mostrado na figura, mas vale a pena mencionar, é um conjunto de três registradores de instrução para guardar instruções que acabaram de chegar. Três deles são necessários, porque o 68000 tem *pipeline*. O primeiro guarda a instrução que está sendo executada; o segundo guarda a instrução que está sendo decodificada; o terceiro guarda a instrução que está sendo buscada da memória.

Também não está mostrado o fato de que a interface para e do barramento de dados contém chaves *crossover* nas duas direções, como o 8088, de modo que bytes superior e inferior de uma palavra de 16 bits podem ser trocados quando entram ou saem do barramento de dados. Esta facilidade é necessária para endereçar e manipular bytes individuais.

O 68000 usa uma memória de controle de dois níveis, com um microprograma e um nanoprograma, como mostrado na Fig. 4.35. Os detalhes são alguma coisa diferente do exemplo dado no texto, mas o objetivo é o mesmo: reduzir o número de bits na memória de controle, fazendo com que o microprograma determine a seqüência de nanoinstruções executadas e com que as nanoinstruções realmente ativem as portas nas vias de dados. O microprograma consiste em 544 palavras de 17 bits, e o nanoprograma consiste em 336 palavras de 68 bits, para um total de 32.096 bits. Uma implementação com 544 palavras de 68 bits iria requerer 36.992 bits, uma

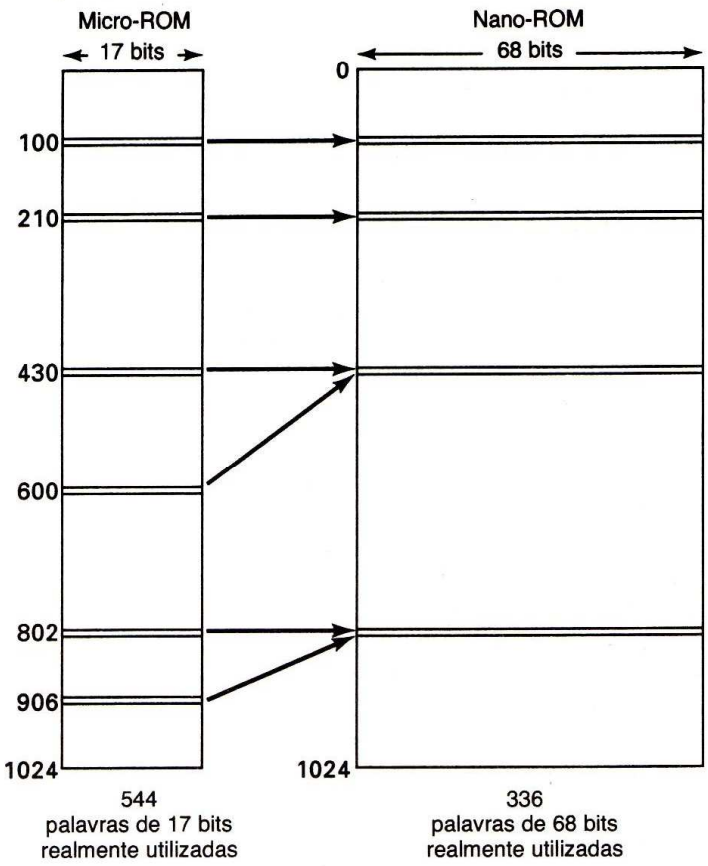


Fig. 4.35 O 68000 possui uma memória de controle de dois níveis, com uma micro-ROM e uma nano-ROM.

economia de 13%. Uma vez que esta pastilha estava definitivamente chegando ao limite do estado da arte quando foi introduzida, a economia foi considerada de valor. Para propósitos de comparação, o microprograma de 504 × 21 do 8088 tem 10.584 bits.

As ROMs para tanto o microprograma quanto o nanoprograma são endereçadas por números de 10 bits, permitindo até 1024 entradas em cada. Como mencionamos, somente 544 são necessárias na primeira, e somente 336 na segunda. Nos dois casos, as entradas são distribuídas ao longo do espaço de endereçamento de uma maneira muito cuidadosa. Na nanoprogramação convencional, o *hardware* primeiro busca a microinstrução, e esta microinstrução contém o endereço da nanoinstrução a utilizar. A nanoinstrução é então buscada e executada. O processo tem inerentemente dois estágios, uma vez que a nanoinstrução não pode ser buscada antes que a microinstrução esteja disponível.

Os projetistas do 68000 gostaram da idéia de economizar área de pastilha usando nanoprogramação, mas não gostaram da idéia de tornar a máquina mais lenta por ter que esperar até que a microinstrução estivesse disponível para começar a buscar a nanoinstrução. Eles usaram um truque para fazer tudo em um ciclo. Na maioria dos casos, a nanoinstrução correspondente à microinstrução do endereço *n* também está no endereço *n,* e então elas podem ser buscadas simultaneamente.

Nos casos em que a nanoinstrução é compartilhada por duas ou mais microinstruções, um transistor foi removido da circuitaria de decodificação da nanoinstrução para mapear dois ou mais endereços na mesma palavra. Por exemplo, o endereço 0000011111 pode ser mapeado no endereço 0000010111 tirando um transistor, de modo que as microinstruções dos endereços 31 e 23 usam a mesma nanoinstrução 23. É claro que uma escolha exata de qual instrução vai para onde fica crítica, mas os projetistas tiveram computadores para ajudá-los a montar o quebra-cabeça.

Dois formatos de microinstrução são usados no 68000. Os dois são mostrados na Fig. 4.36. As microinstruções não são executadas seqüencialmente, como no 8088. Em vez disso, cada microinstrução explicitamente diz qual é a sua sucessora. Este esquema é necessário, dada a necessidade de se terem microinstruções específicas mapeando em nanoinstruções específicas.

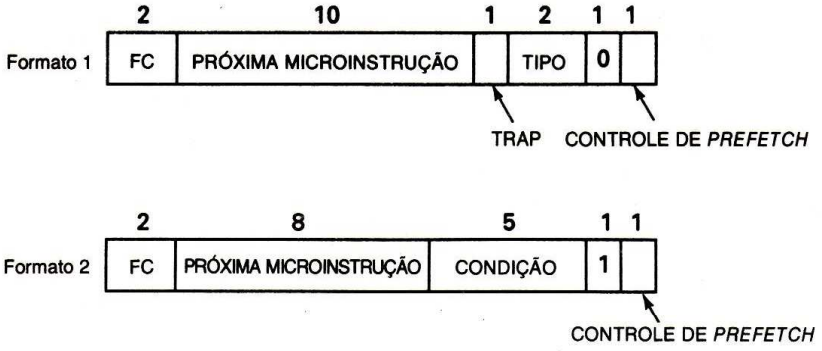


Fig. 4.36 Formatos de microinstrução do 68000.

O formato 1 é usado para todas as microinstruções, exceto os desvios condicionais. O formato 2 é usado para os desvios condicionais. Ambos têm um bit que é usado para começar a busca antecipada da próxima instrução, e ambos têm 2 bits do outro lado, FC, para dar saída a sinais para os pinos da pastilha, para dizer se uma referência à memória está no espaço de instruções, espaço de dados, reconhecimento de interrupção, ou nenhum dos acima. As microinstruções do formato 1 contêm alguns bits miscelâneos e um endereço de 10 bits dizendo de onde buscar a próxima microinstrução. As microinstruções do formato 2 contêm um campo de 5 bits que seleciona uma das 32 condições construídas para serem testadas, tais como os códigos de condição e bits no registrador de instrução. O resultado do teste produz 2 bits que são combinados com 8 bits da microinstrução para selecionar uma das quatro possíveis sucessoras.

As nanoinstruções de 68 bits são codificadas horizontalmente, com 38 campos distintos, a maioria deles de 1 ou dois bits. Os campos controlam o gatilho de todos os registradores e memórias de rascunho nos barramentos, a seleção de registradores, o controle das chaves, o estabelecimento do código de condição, a função da ALU, o uso das ROMs de constantes e dos multiplexadores e a operação da memória, que são o motivo pelo qual tantos bits são necessários.

4.7 RESUMO

No nível de microprogramação, a CPU consiste em dois componentes principais: as vias de dados e a seção de controle. As vias de dados consistem principalmente em uma memória de rascunho e em uma porção de ALU/deslocador. Um ciclo consiste em extrair alguns operandos da memória de rascunho, passá-los através da ALU/deslocador e possivelmente armazenar os resultados de volta à memória de rascunho.

A seção de controle possui a memória de controle, onde o microprograma é mantido. Cada microinstrução da memória de controle controla as portas nas vias de dados durante um microciclo. No exemplo dado, os microciclos foram divididos em subciclos controlados por um relógio. Durante o primeiro subciclo, a microinstrução é buscada da memória de controle e colocada em um registrador interno. No segundo subciclo, as entradas da ALU são colocadas em *latches* para mantê-las estáveis durante a microinstrução. No terceiro subciclo, a ALU e o deslocador realizam seu trabalho. Finalmente, no quarto subciclo, o resultado pode ser armazenado de volta à memória de rascunho, se for necessária para uso em uma microinstrução subseqüente.

O seqüenciamento de microinstruções pode ser mais bem descrito como bizarro. Poucas máquinas possuem um contador de programa explícito no nível de microprogramação. Em vez disso, as microinstruções geralmente contêm o endereço base para suas sucessoras. Este endereço base tipicamente passa por um OR com alguns bits de *status* para produzir o endereço final. Em muitas máquinas, o endereço base de uma microinstrução é combinado com bits de *status* resultantes da anterior para produzir um endereço de desvio que não tem efeito até uma instrução mais tarde.

As microinstruções podem ser organizadas horizontalmente, com um bit por sinal de controle, verticalmente, com alguns campos requerendo decodificação complexa, ou alguma mistura de horizontal com vertical. Organização horizontal leva a palavras longas e máquinas rápidas, altamente paralelas. Organização vertical leva a máquinas mais lentas e menores memórias de controle.

O desempenho pode ser melhorado usando várias técnicas. Uma delas é a nanoprogramação,

na qual o microprograma contém apontadores (curtos) para nanoinstruções (longas) que realmente controlam as portas. A nanoprogramação reduz a quantidade de espaço de pastilha destinado a ROMs, ao custo de execução mais lenta.

Outras maneiras de melhorar o desempenho são tempos de ciclo variáveis, desvios de múltiplos caminhos, *pipeline* e *cache*. Cada técnica traz consigo alguma complexidade aumentada, entretanto.

Finalmente, as microarquiteturas de duas pastilhas disponíveis comercialmente, o 8088 e o 68000, foram discutidas. O 8088 tem vias de dados tradicionais, melhoradas por uma seção superior para realizar cálculos com os registradores de segmento. Ele usa microinstruções verticais de 21 bits. O 68000 tem três vias de dados independentes de 16 bits, duas para os endereços e uma para os dados. Ele usa uma memória de controle de dois níveis com microinstruções de 17 bits e nanoinstruções horizontais de 68 bits.

PROBLEMAS

1. Descreva algumas das forças e fraquezas de microprogramação.

2. Traduza a função Pascal seguinte para a linguagem de montagem do Mac-1.

```
function min(i, j, k: PequenoInt): PequenoInt;
var m:PequenoInt;
begin
  if i < j then m := i else m := j;
  if k < m then m := k;
  min := m
end;
```

   O tipo *PequenoInt* é a mesma subfaixa do texto.

3. Traduza o comando Pascal

```
if i < j then goto 100;
```

   para a linguagem de montagem do Mac-1. Tanto i quanto j são inteiros de 16 bits em complemento de dois. Tome cuidado: se $i = 32767$ e $j = -10$, uma subtração dará um *overflow* não detectado.

4. Na Fig. 4.11(b), recarregamos K na linha rotulada L4. Os dois comandos precedentes garantem que K está em AC depois de tudo. Por que se preocupar em carregá-lo novamente?

5. Calcule o tempo de execução em microinstruções para cada instrução Mac-1 sendo executada no Mic-1.

6. Repita o problema anterior, mas agora para Mic-2.

7. Tente compilar o comando Pascal

```
n := a[i]
```

   diretamente para instruções Mic-1, passando por cima de todo o nível Mac-1. Discuta sua tentativa.

8. Considere uma possível revisão do Mic-1, na qual os dois campos de seqüenciamento, COND e ADDR, são removidos, produzindo microinstruções de 22 bits. Microdesvios são agora manipulados por microinstruções distintas de 22 bits. Os bits N e Z da ALU são também colocados em *latches*, para torná-los disponíveis para um microdesvio subseqüente. Compare o tamanho do programa Mic-1 original com aquele necessário para este novo projeto.

9. Os dois comandos Mic-1 seguintes executam exatamente a mesma função? Explique.

   a := *a* + *a*; **if** *n* **then goto** 0;
   a := *lshift*(*a*); **if** *n* **then goto** 0;

10. Sua única listagem de um microprograma chave Mic-1 foi acidentalmente colocada num alimentador automático de documentos para o picador de papel, em vez de para a fotocopiadora próxima a ele. Você terá agora que reconstruir o programa fonte a partir de um *dump* de memória. No meio da desmontagem, você passará pelo número binário

    11001000000100011001000000001000

    Reescreva-o em linguagem de montagem.

11. Quantos padrões de bits existem para a seguinte instrução Mic-1?

    ac := *mbr* + *a*; **if** *n* **then goto** 100;

12. Imagine que você trabalha para uma companhia que está vendendo uma máquina com Mic-1 como nível 1 e Mac-1 como nível 2. O vendedor de semicondutores que normalmente fornece suas pastilhas acabou de anunciar uma grande liquidação para acabar seu estoque de ALUs de complemento de um. Seu departamento de compras aproveitou a chance e comprou um carregamento inteiro delas. Será que o microprograma da Fig. 4.16 continuará a funcionar com uma ALU de complemento de um, em vez da de complemento de dois original? Na nova máquina, números negativos serão representados em complemento de um, é claro.

13. Se leituras e escritas à memória principal levassem três microinstruções em vez de duas no Mic-1, o tamanho do microprograma seria afetado, e, se for, de quanto?

14. O campo COND na palavra de Mic-1 tem 2 bits, o que fornece quatro códigos distintos. Se um desses códigos fosse necessário para alguma outra coisa, por exemplo, para testar o bit de *status* de *overflow* de uma ALU recentemente implementada, qual dos quatro códigos existentes você recomendaria sacrificar?

15. Proponha o acréscimo de uma facilidade simples ao Mic-1 para chamar procedimentos no nível micro. Alguma maneira deve ser encontrada para tanto chamar quanto retornar de microprocedimentos, que não podem ser aninhados (isto é, um microprocedimento não pode chamar outro microprocedimento). Você pode modificar a arquitetura do Mic-1, mas tente minimizar as alterações necessárias.

16. Microarquitetos se encontram constantemente com compromissos entre elegância (isto é, ortogonalidade de campos) e tamanho da memória de controle. Suponha que você tivesse o trabalho de modificar Mic-1 para adicionar um bit a COND ou ADDR, pegando um bit de um dos outros 11 campos. Faça uma sugestão de onde encontrar o bit sem enfraquecer muito a funcionalidade do Mic-1.

17. Existem lugares no microprograma para o Mic-2 onde o microprogramador foi forçado a escolher entre tempo e memória de uma maneira não requerida no microprograma para o Mic-1? Se tiver, dê um exemplo.

18. As microinstruções do Mic-2 são muito mais compactas que aquelas do Mic-1 (12 bits *versus* 32 bits). Você pode visualizar um Mic-3 com microinstruções ainda mais curtas? Discuta.

19. O presidente da Corporação de Memória Nano-Micro-Milli ficou tão impressionado com as vendas de suas nanomemórias, que ele considerou a adição de uma picomemória a sua linha de produtos. A picomemória seria referenciada pela nanomemória, da mesma maneira

que a nanomemória é referenciada pela memória de controle, isto é, cada nanopalavra conteria o endereço de uma picoinstrução, assim como a memória de controle contém endereços de nanoinstruções. Você é o Vice-Presidente Encarregado de Descartar Idéias Estúpidas. O que você acharia desta proposta?

20. Se fôssemos implementar o programa da Fig. 4.16 usando nanoprogramação, quantos bits a memória de controle, mais a nanomemória, iriam requerer juntas? Compare isto com o número de bits da memória de controle no projeto original.

21. Suponha que tivéssemos um *pipeline* de *n* estágios, em vez do *pipeline* de cinco estágios discutido no texto. Qual seria então a penalidade de desvio?

22. Considere uma máquina na qual 20% das instruções são desvios condicionais e outros 10% são desvios de *loop*. Os desvios condicionais podem ser previstos com 60% de acerto, e os desvios de *loop* podem ser previstos com 90% de acerto. A penalidade por adivinhar errado é de quatro ciclos. Não existe penalidade para desvios incondicionais ou para suposições corretas. Qual é a eficiência do *pipeline* desta máquina?

23. Na Fig. 4.29(b), qual é o endereço da palavra cujo conteúdo é 2131?

24. Um sistema de *cache* tem uma taxa de acerto de 95%, um tempo de acesso de 100 ns quando o dado for encontrado no *cache* e um tempo de acesso de 800 ns se o *cache* não contiver o dado. Qual é o tempo de acesso efetivo?

25. Um *cache* está sendo projetado para um computador com 2^{32} bytes de memória. O *cache* terá 2K *slots* e usará um bloco de 16 bytes. Calcule, tanto para um *cache* associativo quanto para um *cache* com mapeamento direto, quantos bytes o *cache* irá ocupar.

26. À primeira vista, a presença ou ausência de *cache* não parece ter nada a ver com a decisão de se mapear ou não os registradores de E/S no espaço de endereçamento de memória. Pense um pouco mais sobre isso e veja se você pode encontrar algum problema potencial. (*Sugestão*: se você não pode pensar em nenhum problema, continue pensando.)

27. O 68030 tem *caches* para instrução e dados na própria pastilha. Ele também tem um tipo especial de ciclo de barramento, lê-modifica-escreve, para ler e modificar uma palavra de memória em uma única ação indivisível. Existe alguma interação entre estas duas características? Explique.

28. Escreva um procedimento em linguagem de montagem Mac-1 para multiplicar dois inteiros com sinal, cujo produto caiba em 16 bits.

29. Escreva um programa de teste na linguagem de montagem Mac-1 para verificar se o microprograma está funcionando corretamente. Verifique cada instrução com uma série de operandos conhecidos para ver se os resultados conhecidos ocorrem. Tais programas são largamente utilizados pelo pessoal de reparos de computadores para tentar detectar falhas de *hardware*.

30. O fabricante do Mac-1 acabou de receber um telegrama de seu cliente mais importante. O telegrama diz: "Preciso ter nova instrução para deslocar AC *n* bits à esquerda, onde *n* são os 4 bits de baixa ordem da instrução. Preciso para amanhã de manhã, às 8 h. Envie o novo microprograma por telegrama. Urgência." Seu trabalho é implementar o microcódigo para o deslocamento.

31. Escreva um simulador de Mic-1 em sua linguagem de montagem preferida. O simulador deve ler um microprograma como uma série de linhas de entrada de 32 caracteres (0 e 1), seguida de uma linha em branco. Então, ele lê o programa Mac-1, 16 bits por linha, e o executa.